

Zhiqian Chen – Promotion Dossier

For ease of navigation, the table of contents is embedded as PDF Bookmarks. In Acrobat, these bookmarks are typically available as a tab and/or an icon to the left.

Table of Contents

Tab 1

- Application Letter
- Promotion and Tenure Form
- Initial Offer Letter

Tab 2

- Letter of Recommendation: Department Head
- Letter of Recommendation: Department Promotion & Tenure Committee
- Committee Report Form

Tab 3

- Curriculum Vitae

Tab 4

- Teaching Accomplishments

Tab 5

- Research Accomplishments

1 Current Research and Achievements

Tab 6

- Service Accomplishments

Tab 7

- Letters of Recommendation: External Reviewers
- Example Letter to Reviewers

Tab 8

- Response/Rebuttal to Recommendation

Tab 9

- Selected Publications

Application Letter

Dear Dr. Andy Perkins,

August 8, 2025

Enclosed you will find my application for promotion to the rank of Associate Professor (without tenure) in the Department of Computer Science and Engineering (CSE) at Mississippi State University (MSU). Since my appointment in August 2020, I have been committed to contributing meaningfully to the university's mission through high-impact research, effective teaching, and dedicated service. Over the past several years, I have greatly appreciated the opportunity to work alongside my colleagues and mentor talented students. I feel deeply connected to the MSU community and proud to call Starkville, Mississippi my home.

My research program is centered around machine learning on graph-structured data, with a focus on modeling dynamics, uncertainty, and inference over interdependent systems. I am especially interested in bridging spatial and spectral perspectives in graph neural networks, developing theory-informed architectures for sparse data, and applying these methods to real-world systems in agriculture, infrastructure, energy, and health. My work addresses both foundational algorithmic questions and societal needs, often in collaboration with government agencies such as NSF and USDA. Since joining MSU, I have secured or co-secured external funding totaling over \$11 million, with \$1.64 million attributed to my role as PI or Co-PI. Notably, I was awarded an NSF CAREER grant to pursue a long-term research agenda on cross-graph dynamics. Additional funded projects include an NSF CRII award, an NSF CIRC Planning grant for synergistic graph flow analytics, and USDA ARS collaborations on geospatial epidemiology and agricultural sensing. These awards reflect both the scholarly contributions and the translational potential of my research.

My publication record includes over 70 peer-reviewed articles across top-tier conferences and journals. Highlights include papers in AAAI, IJCAI, SDM, CIKM, ICDM, and CVPR, as well as journals such as ACM Computing Surveys, IEEE Access, *Frontiers in Big Data, and ACM Transactions on Knowledge Discovery from Data. I have also authored invited tutorials on graph learning theory at SDM, CVPR, SIAM MDS, and CIKM. My recent efforts explore how to unify spectral and spatial GNNs, enhance robustness and interpretability, and deploy graph models in applied domains such as microbiome analysis, power grid simulation, and low-cost sensing. Several of these papers have involved my MSU students as co-authors, and several others resulted from cross-disciplinary collaborations with engineering, agriculture, and biomedical researchers.

I am actively engaged in the broader research community. I have served as reviewer for conferences including NeurIPS, ICML, ICLR, AAAI, IJCAI, SIGIR, and KDD, and journals such as TKDE, TKDD, Scientific Reports, and Neurocomputing. I served as Co-Chair for the Poster Session at IEEE ICDM 2025 and the Big Data Cup at IEEE Big Data 2024. I am currently a Review Editor for *Frontiers in Big Data*, where I contribute to shaping the direction of the field. In addition to these roles, I have contributed to grant review through three NSF panels and regularly collaborate with researchers in academia and national labs.

In my teaching, I emphasize clear communication, rigorous thinking, and hands-on experience. I have regularly taught both undergraduate and graduate versions of CSE 4633/6633



MISSISSIPPI STATE
UNIVERSITY™

BAGLEY COLLEGE OF ENGINEERING
Department of Computer Science and Engineering
Mississippi State University
www.cse.msstate.edu

Artificial Intelligence and CSE 4693/6693/8673 Machine Learning. I developed two new graduate courses: CSE 8990 Graph Machine Learning and CSE 8990 Distributed Computing, and I redesigned the curriculum for CSE 8673 to incorporate recent advances in deep learning and representation learning. My classes integrate programming assignments, technical readings, and open-ended projects to prepare students for research and industry. My instruction has received consistently strong student feedback and peer evaluations. Beyond the classroom, I have mentored over 10 master's and 9 doctoral students, and guided multiple undergraduate researchers and high school students in summer research projects, many of which have resulted in co-authored publications or public presentations.

Within MSU, I have contributed through departmental and university service. I have served on the Graduate Studies Committee (2020–2024), Curriculum Committee (2024–2025), and Faculty Search Committee (2022–2023). I also co-advise the MSU AI Club and actively participate in mentoring efforts for undergraduates and underrepresented students. My mentoring approach emphasizes independence, collaboration, and scholarly growth. I have advised students across a range of topics, including graph optimization, anomaly detection, influence propagation, and explainable AI.

As recognition for my contributions, I have received several awards, including the Ottilie Schillig Special Teaching Award, Best Paper Awards at ACM SIGSPATIAL 2020 and GISTAM 2015, and an Outstanding Contribution Award from Toyota Research Institute of North America. These recognitions highlight my commitment to excellence in both research and education. I also received competitive travel awards from SIAM and the Southeastern Conference, which supported my involvement in professional development and outreach.

My dossier contains a detailed record of my research, teaching, and service accomplishments. I hope that you will find these contributions reflective of my continued growth as a scholar, educator, and colleague. I am enthusiastic about the opportunity to continue serving the department and advancing its mission through research, mentorship, and innovation. Thank you for your time and consideration of my application.
Sincerely,

Zhiqian Chen

Promotion and Tenure Form

Mississippi State University
Application for Promotion and/or Tenure

<i>Please check response(s) in both columns</i>	
TENURE:	PROMOTION:
☐ Mandatory tenure decision ☐ Not applicable (early promotion or professional track position or already possess tenure)	☐ Promotion to Instructor II ☐ Promotion to Instructor III ☐ Promotion to Associate Professor ☐ Promotion to Full Professor ☐ Not applicable (only tenure decision)

Faculty members eligible for consideration for promotion or tenure must provide the department head or appropriate official with all pertinent available information by **October 1**. The department head or other appropriate official has the responsibility to assist the faculty member in preparing for tenure or promotion review.

Materials to be provided in the applicant's dossier include:

- *1. Cover letter from the candidate requesting promotion and/or tenure.
- *2. Completed University Promotion and Tenure application form (this cover page and attached pages) with appropriate responses and associated documentation. This must include a summary sheet of teaching evaluations.
- *3. Complete up-to-date vita.
- *4. Copy of the initial offer letter and, if necessary, an additional letter detailing significant changes.
- *5. Letters from external reviewers (to be added by the department head). The department head should include a sample letter sent to external reviewers and biographical information about reviewers as appropriate.
- 6. All materials required by the academic unit's procedural guidelines.
- 7. All supporting documentation desired by the candidate.

*Only these items will be reviewed routinely above the college level. Items 1-7 must go to dept. head and dept. committee. Deans, college committees, and the Provost require items 1-5 but may also request items 6 and 7. Department heads and deans can use their discretion in sending forward any important information included in items 6 and 7. All department head, dean, and committee recommendations should be included in the package to the Provost.

Note: Please refer to the Faculty Handbook for information pertaining to the Promotion & Tenure process.

To apply and be considered for tenure requires that you be a citizen of the United States or be a permanent resident or have begun the permanent residence process (verification required) in order to be eligible for permanent employment in this country.

Are you a citizen or permanent resident of the United States: ☐ Yes ☐ No

If No, have you applied for permanent residency: ☐ Yes ☐ No (Date process initiated (if Yes): _____)

Name of Applicant: **Zhiqian Chen**

Present rank: **Assistant Professor**

Date of appointment at current rank: **August 16, 2020**

College/School: **Bagley College of Engineering** Department: **Computer Science and Engineering**

Department Head: **Professor Shahram Rahimi**

Preferred Mailing Address (Include City and Zip Code): **107 Mimosa Dr, Starkville, MS 39759**

Initial rank at MSU with date of appointment: **August 16, 2020**

Tenure track date of appointment: **August 16, 2020** Years of transferred service (if applicable): _____

Advanced Degrees with Dates: **PhD in CS 2020, MS in SE 2013, BS in SE 2009 (see CV for details)**

Salary Funding (%): E&G: _____ MSU Research Unit: _____ Extension: _____ Other: _____

All other information contained in the attached application is correct, to the best of my knowledge.

Date: **April 1, 2025** Signed: _____

Faculty Member

I. Current Fall semester responsibilities:

A. Current instruction

	<u>Course number</u>	<u>Title</u>	<u>Credit hours</u>	<u>Number of students</u>
1. Undergraduate:	CSE 4633	Artificial Intelligence	3	34
	CSE 4693	Intro to Machine Learning	3	15
2. Graduate:	CSE 8673	Machine Learning	3	25
	CSE 6693	Intro to Machine Learning	3	12
	CSE 6633	Artificial Intelligence	3	17
	CSE 8990	Special Topic in CS	3	16
3. Advisees:	Undergraduate: 10	Master's/Specialist: 10	Doctoral: 9	Postdoctoral ____
4. Non-credit educational programs (documented, non-credit instruction/teaching with student assessment).				
a. Adult audiences:				
b. Youth audiences:				
c. Professional and technical audiences:				
d. In-service trainings/assistance:				

B. Current or on-going research/creative/performance activities

I am elaborating on only those current/ongoing research activities and accomplishments since starting my position at Mississippi State University in August 2020.

C. Current service/administrative assignments

1. Public service and off-campus professional service activities (non-assessment activities, such as guest lectures and presentations, external committee/board memberships, business/industry/stakeholder advisement, etc. with dates, organizations, & places):

- National Science Foundation (NSF) Review Panel, Program: CISE, 2021, 2023, and 2024
- Reviewer, International Conference on Machine Learning (ICML), 2021-2024
- Reviewer, International Conference on Learning Representations (ICLR), 2022-2024
- Reviewer, Neural Information Processing System (NeurIPS), 2020-2025
- Reviewer, AAAI conference on Artificial Intelligence (AAAI), 2020-2025
- Reviewer, International Joint Conference on Artificial Intelligence (IJCAI), 2022-2025
- Reviewer, ACM SIG on Knowledge Discovery and Data Mining (KDD), 2020-2025
- Reviewer, ACM SIG on Information Retrieval (SIGIR), 2020-2023
- Reviewer, Knowledge-Based Systems, 2021
- Reviewer, IEEE Transactions on Knowledge and Data Engineering (TKDE), 2019-2020
- Reviewer, ACM Transactions on Knowledge Discovery from Data (TKDD), 2021, 2024
- Reviewer, Neurocomputing by Elsevier, 2019, 2020, 2021
- Reviewer Scientific Reports by Nature, 2021

2. Professional association service, as offices held, etc.:

- Review Editor, Editorial Board, Frontiers in Big Data - Data Science
- IEEE Big Data 2024, Co-chair of Big Data Cup
- IEEE ICDM 2025, Co-chair of Poster Session

3. University and departmental committee and administrative accomplishments:

- Department of Computer Science and Engineering, Graduate Committee, 2020-2024.
- Department of Computer Science and Engineering, Curriculum Committee, 2024-2025.
- Department of Computer Science and Engineering, Faculty Search Committee, 2022- 2023.
- Co-advisor, MSU AI Club

D. Other

II. Activities since last promotion (or initial appointment for tenure):

A. Teaching

- Evidence of quality of instruction, both credit and non-credit (check items submitted):
(The faculty member should provide material describing their teaching activities and documentation supporting effectiveness. This material must include a summary statement of student survey responses and may include any of the following or any other items deemed appropriate:

- ☒ peer evaluations (internal or external),
- ☒ course syllabi and exams,
- ☐ non-credit education program plans with assessment,
- ☐ non-credit education program outcomes and impacts,
- ☐ student input in the form of letters, emails, faculty nominations, etc.,
- ☒ recordings of teaching sessions, graduate student theses and dissertations, and other materials demonstrating teaching effectiveness.)

2.

Number of Students Supervised	Major Professor	Minor Professor
Undergraduate Students		
Undergraduate Research	10	
Clinical Interns & Residents		
Master's Students	12	
Specialist Students		
Doctoral Students	10	
Postdoctoral Students		
Visiting Scientist		

- Courses initiated or innovations instituted:

- I designed and offered a new course CSE 8990 Graph Machine Learning
- I designed and offered a new course CSE 4963/6693 Intro to Machine Learning
- I have redesigned CSE 8673 Machine Learning

- Non-credit educational programs initiated or instituted (documented, non-credit instruction/teaching with student assessment, such as certification programs, short courses, workshops, in-service trainings, workshops, etc.):

- Other (academic advisement may be described here or as service):

Mentoring Undergraduate Students 10

Prathyusha Mustiyala

Undergraduate Student, Department of Computer Science and Engineering, Mississippi State University

Summary: Prathyusha is one of top students in the department (Senior Class Rank 752/8671; Shackouls Honors College), and has working in a funded UG research program.

Ethan Rogers

Undergraduate Student, Department of Aerospace a, Mississippi State University

Summary: Ethan is one of top students in the Aerospace Engineering (Junior Class Rank, 1/3917; Shackouls Honors College), and has working on a core research top in our lab as intern format.

Aalok Uprety

Undergraduate Student, Department of Computer Science and Engineering, Mississippi State University

Summary: Aalok had been working as student worker for 2 semesters, and complete many excellent coding working as expected

Reid Sewell

Undergraduate Student, Department of Computer Science and Engineering, Mississippi State University

Summary: Reid worked as a student worker for one summer in NSF REU project, effectively leading communication between a 4-student group and myself.

Jason Weeks

Undergraduate Student, Department of Computer Science and Engineering, Mississippi State University

Summary: Jason has collaborated with me on multiple projects including NSF REU project (quantum machine learning) and VSV disease spreading (USDA), consistently demonstrating excellent work capacity.

Andrew McBride

Undergraduate Student, Department of Computer Science and Engineering, Mississippi State University

Summary: Andrew has been working with me in 1 NSF REU project and efficiently complete assigned task.

Mason Fisher

Undergraduate Student, Department of Computer Science and Engineering, Mississippi State University

Summary: Mason has worked on visualization of multilayer network system.

Kevin Ho

Undergraduate Student, Department of Computer Science and Engineering, Mississippi State University

Summary: Kevin participated in the MS State UG research program, studying graph dynamics in systems like power networks and transportation. He also completed a poster presentation at an undergraduate research symposium.

Ben Moore

Undergraduate Student, Department of Computer Science and Engineering, Mississippi State University

Summary: Ben developed a visualization system for a multilayer network and explored using supra-matrix methodology to represent these systems.

Jack Maloney:

Undergraduate Student, Department of Computer Science and Engineering, Mississippi State University

Summary: Jack worked on a UG research program to enhance comprehension of deep neural networks by conceptualizing them as multilayer graphs. The main aim of this study is to examine the correlation between the performance of a neural network and its geometric characteristics. Jack completed a poster presentation at research symposium.

M.S. Graduate Advising — Major Professor**Ramyasri Veerapaneni**

Department of Computer Science and Engineering, Mississippi State University

Course-Only Option, Comprehensive Oral Exam

Ramyasri completed a research project in the last year on graph learning for genetics, and get a accepted paper at IEEE EMBC 2024 as second author

Defended: April 2023

Samuel Prabhakar

Department of Computer Science and Engineering, Mississippi State University

Thesis option, *Learning Braess Paradox with Graph Neural Network*

Samuel's thesis research impressed the committee during the defense.

Defended: April 2024

Piero Bracamonte

Department of Computer Science and Engineering, Mississippi State University

Course-Only Option, Comprehensive Oral Exam

Defended: April 2023

Suman Adhikari

Department of Computer Science and Engineering, Mississippi State University
 Course-Only Option, Comprehensive Oral Exam
 Defended: May 2023

Jogi, Rajeev

Department of Computer Science and Engineering, Mississippi State University
 Course-Only Option, Comprehensive Oral Exam
 Defended: Dec 2021

Fang, Yiheng

Department of Computer Science and Engineering, Mississippi State University
 Course-Only Option, Comprehensive Oral Exam
 Defended: Feb 2022

Aymane Jouhari

Department of Computer Science and Engineering, Mississippi State University
 Project Option, Comprehensive Oral Exam
 Defended: Nov 2024

Matthew Rester

Department of Computer Science and Engineering, Mississippi State University
 Course-Only Option, Comprehensive Oral Exam
 Defended: April 2025

Henry Smith

Department of Computer Science and Engineering, Mississippi State University
 Course-Only Option, Comprehensive Oral Exam
 Defended: May 2025

Pavan Adapa

Department of Computer Science and Engineering, Mississippi State University
 Thesis Option, Comprehensive Oral Exam
 Anticipated Defense: Dec 2025

Sai Gade

Department of Computer Science and Engineering, Mississippi State University
 Thesis Option, Comprehensive Oral Exam
 Anticipated Defense: Dec 2025

Marouane Benbrahim

Department of Computer Science and Engineering, Mississippi State University
 Thesis Option, Comprehensive Oral Exam
 Anticipated Defense: Dec 2025

PhD Graduate Advising — Major Professor

Zonghan Zhang

Department of Computer Science and Engineering, Mississippi State University
Information Maximization
 Anticipated defense: December 2025

Josh Waldbieser

Department of Computer Science and Engineering, Mississippi State University
Plant Disease Spreading and Controlling
 Anticipated defense: 2029

Rocker D'Antonio

Department of Computer Science and Engineering, Mississippi State University
 Anticipated defense: 2028

George Anim

Financial Network Modeling

Department of Computer Science and Engineering, Mississippi State University
Anticipated defense: Dec 2027

Zijian Zhang

Information Maximization

Department of Computer Science and Engineering, Mississippi State University
Anticipated defense: Dec 2028

Jiashan Wu

Department of Computer Science and Engineering, Mississippi State University
Anticipated defense: Dec 2028

Gaoyuan Du

Department of Computer Science and Engineering, Mississippi State University
Anticipated defense: May 2030

Tamanna Rashme

Department of Computer Science and Engineering, Mississippi State University
Anticipated defense: May 2027

Naidu Singampalli

Department of Computer Science and Engineering, Mississippi State University
Anticipated defense: May 2030

Xinyuan Chen

Department of Computer Science and Engineering, Mississippi State University
Anticipated defense: May 2030

M.S./PhD Graduate Advising — Committee Member

Napier, Kollin (krn65)

Department of Computer Science and Engineering, Mississippi State University
An analysis of text-based machine learning models for vulnerability detection
Defended: Feb 27, 2023

Somayeh Bakhtiari Ramezani (sb3182)

Department of Computer Science and Engineering, Mississippi State University
Efficiency and Explainability of Motif Discovery Algorithms Over Long Temporal Data Captured From Dynamic Systems
Defended: July 26, 2023

B. Research, creative endeavor, or performances

1. Publications, performances or creative activities:

(For books, indicate date of publication and publisher; for articles, indicate refereed journals; for art shows, indicate judged competition; for musical shows, attach copies of programs; for reports, indicate those done for in-house use.)

1. [Conference] Fu, Kaiqun; Ji, Taoran; Self, Nathan; **Chen, Zhiqian**; Lu, Chang-Tien; , 2021, A hierarchical attention graph convolutional network for traffic incident impact forecasting, 2021 IEEE International Conference on Big Data (Big Data), Pages 1619-1624, IEEE
2. [Conference] Zhang, Xuchao; **Chen, Zhiqian**; Zhao, Liang; Boedihardjo, Arnold P; Lu, Chang-Tien; , 2017, Traces: Generating twitter stories via shared subspace and temporal smoothness, 2017 IEEE International Conference on Big Data (Big Data), Pages 1688-1693, IEEE
3. [Conference] He, Jianfeng; Zhang, Xuchao; Lei, Shuo; **Chen, Zhiqian**; Chen, Fanglan; Alhamadani, Abdulaziz; Xiao, Bei; Lu, Chang-Tien; , 2020, Towards more accurate uncertainty estimation in text classification, Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Pages 8362-8372
4. [Conference] Zhang, Zonghan; Biswas, Subhodip; Chen, Fanglan; Fu, Kaiqun; Ji, Taoran; Lu, Chang-Tien; Ramakrishnan, Naren; **Chen, Zhiqian**; , 2022, Blocking influence at collective level with hard constraints (student abstract), Proceedings of the AAAI Conference on Artificial Intelligence, Volume 36(11), Pages 13115-13116
5. [Journal] Ding, Zhao; **Chen, Zhiqian**; Ma, Tianyi; Lu, Chang-Tien; Ma, Wenhui; Shaw, Leon; , 2020, Predicting the hydrogen release ability of LiBH₄-based mixtures by ensemble machine learning, Energy Storage Materials, Volume 27, Pages 466-477, Elsevier

6. [Conference] Ji, Taoran; **Chen, Zhiqian**; Self, Nathan; Fu, Kaiqun; Lu, Chang-Tien; Ramakrishnan, Naren; , 2019, Patent citation dynamics modeling via multi-attention recurrent networks, Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-19)
7. [Journal] Lin, Beiyu; Jia, Xiaowei; **Chen, Zhiqian**; , 2021, SDM 21 Tutorial: Studying Spread Patterns of COVID-19 based on Spatiotemporal Data, SIAM Data Mining 2021
8. [Journal] Li, Guoming; Hui, Xue; **Chen, Zhiqian**; Chesser Jr, Gary D; Zhao, Yang; , 2021, Development and evaluation of a method to detect broilers continuously walking around feeder as an indication of restricted feeding behaviors, Computers and electronics in agriculture, Volume 181, Pages 105982, Elsevier
9. [Journal] **Chen, Zhiqian**; Zhang, Lei; Kolhe, Gaurav; Kamali, Hadi Mardani; Rafatirad, Setareh; Pudukotai Dinakarrao, Sai Manoj; Homayoun, Houman; Lu, Chang-Tien; Zhao, Liang; , 2021, Deep graph learning for circuit deobfuscation, Frontiers in big Data, Volume 4, Pages 608286, Frontiers Media SA
10. [Conference] Chen, Fanglan; **Chen, Zhiqian**; Biswas, Subhodip; Lei, Shuo; Ramakrishnan, Naren; Lu, Chang-Tien; , 2020, Graph convolutional networks with kalman filtering for traffic prediction, Proceedings of the 28th international conference on advances in geographic information systems, Pages 135-138
11. [Conference] Wang, Jason; Fu, Kaiqun; **Chen, Zhiqian**; Lu, Chang-Tien; , 2022, Augmentation of Chinese Character Representations with Compositional Graph Learning (Student Abstract), Proceedings of the AAAI Conference on Artificial Intelligence, Volume 36(11), Pages 13075-13076
12. [Conference] Fu, Kaiqun; **Chen, Zhiqian**; Lu, Chang-Tien; , 2018, Streetnet: preference learning with convolutional neural network on urban crime perception, Proceedings of the 26th ACM SIGSPATIAL international conference on advances in geographic information systems, Pages 269-278
13. [Journal] Biswas, Subhodip; Chen, Fanglan; **Chen, Zhiqian**; Lu, Chang-Tien; Ramakrishnan, Naren; , 2023, Memetic algorithms for spatial partitioning problems, ACM Transactions on Spatial Algorithms and Systems, Volume 9(1), Pages 1-31, ACM New York, NY
14. [Journal] Li, Guoming; Huang, Yanbo; **Chen, Zhiqian**; Chesser Jr, Gary D; Purswell, Joseph L; Linhoss, John; Zhao, Yang; , 2021, Practices and applications of convolutional neural network-based computer vision systems in animal farming: A review, Sensors, Volume 21(4), Pages 1492, MDPI
15. [Conference] Zhang, Xuchao; Zhao, Liang; **Chen, Zhiqian**; Boedihardjo, Arnold P; Dai, Jing; Lu, Chang-Tien; , 2017, Trendi: Tracking stories in news and microblogs via emerging, evolving and fading topics, 2017 IEEE International Conference on Big Data (Big Data), Pages 1590-1599, IEEE
16. [Conference] Biswas, Subhodip; Chen, Fanglan; Sistrunk, Andreea; Muthiah, Sathappan; **Chen, Zhiqian**; Self, Nathan; Lu, Chang-Tien; Ramakrishnan, Naren; , 2020, Geospatial clustering for balanced and proximal schools, Proceedings of the AAAI Conference on Artificial Intelligence, Volume 34(9), Pages 13358-13365
17. [Journal] Shukla, Manu; **Chen, Zhiqian**; Lu, Chang-Tien; , 2018, DIMPL: a distributed in-memory drone flight path builder system, Journal of Big Data, Volume 5(1), Pages 24, Springer International Publishing Cham
18. [Journal] Wang, Bingsheng; **Chen, Zhiqian**; Boedihardjo, Arnold P; Lu, Chang-Tien; , 2018, Virtual metering: An efficient water disaggregation algorithm via nonintrusive load monitoring, ACM Transactions on Intelligent Systems and Technology (TIST), Volume 9(4), Pages 1-30, ACM New York, NY, USA
19. [Conference] Biswas, Subhodip; Chen, Fanglan; **Chen, Zhiqian**; Lu, Chang-Tien; Ramakrishnan, Naren; , 2020, Incorporating domain knowledge into Memetic Algorithms for solving Spatial Optimization problems, Proceedings of the 28th International Conference on Advances in Geographic Information Systems, Pages 25-35
20. [Conference] **Chen, Zhiqian**; Feng, Wenya; , 2013, Detecting impolite crawler by using time series analysis, 2013 IEEE 25th International Conference on Tools with Artificial Intelligence, Pages 123-126, IEEE
21. [Conference] **Chen, Zhiqian**; Wu, Chih-Wei; Lu, Yen-Cheng; Lerch, Alexander; Lu, Chang-Tien; , 2017, Learning to fuse music genres with generative adversarial dual learning, 2017 IEEE International Conference on Data Mining (ICDM), Pages 817-822, IEEE
22. [Conference] Zhang, Lei; **Chen, Zhiqian**; Lu, Chang-Tien; Zhao, Liang; , 2022, From “Dynamics on Graphs” to “Dynamics of Graphs”: An Adaptive Echo-State Network Solution (Student Abstract), Proceedings of the AAAI Conference on Artificial Intelligence, Volume 36(11), Pages 13111-13112
23. [Conference] Biswas, Subhodip; Chen, Fanglan; **Chen, Zhiqian**; Sistrunk, Andreea; Self, Nathan; Lu, Chang-Tien; Ramakrishnan, Naren; , 2019, REGAL: A Regionalization framework for school boundaries, Proceedings of the 27th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, Pages 544-547
24. [Conference] **Chen, Zhiqian**; Zhang, Xuchao; Boedihardjo, Arnold P; Dai, Jing; Lu, Chang-Tien; , 2017, Multimodal storytelling via generative adversarial imitation learning, Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI-17)
25. [Conference] Ji, Taoran; Self, Nathan; Fu, Kaiqun; **Chen, Zhiqian**; Ramakrishnan, Naren; Lu, Chang-Tien; , 2021, Dynamic multi-context attention networks for citation forecasting of scientific publications, Proceedings of the AAAI Conference on Artificial Intelligence, Volume 35(9), Pages 7953-7960
26. [Journal] Ding, Zhao; Li, Shaoyuan; Zhou, Yang; **Chen, Zhiqian**; Yang, Weijie; Ma, Wenhui; Shaw, Leon; , 2020, LiBH4 for hydrogen storage-new perspectives, Nano Materials Science, Volume 2(2), Pages 109-119, Elsevier
27. [Conference] Zhang, Xuchao; **Chen, Zhiqian**; Zhong, Weisheng; Boedihardjo, Arnold P; Lu, Chang-Tien; , 2016, Storytelling in heterogeneous twitter entity network based on hierarchical cluster routing, 2016 IEEE International Conference on Big Data (Big Data), Pages 1522-1531, IEEE

28. [Conference] **Chen, Zhiqian**; Kolhe, Gaurav; Rafatirad, Setareh; Lu, Chang-Tien; PD, Sai Manoj; Homayoun, Houman; Zhao, Liang; , 2020, Estimating the circuit de-obfuscation runtime based on graph deep learning, 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE), Pages 358-363, IEEE
29. [Journal] Zhang, Ying; He, Xingfeng; **Chen, Zhiqian**; Bai, Qiang; Nolan, Adelaide M; Roberts, Charles A; Banerjee, Debasish; Matsunaga, Tomoya; Mo, Yifei; Ling, Chen; , 2019, Unsupervised discovery of solid-state lithium ion conductors, Nature communications, Volume 10(1), Pages 5260, Nature Publishing Group UK London
30. [Conference] **Chen, Zhiqian**; Chen, Feng; Lai, Rongjie; Zhang, Xuchao; Lu, Chang-Tien; , 2018, Rational neural networks for approximating graph convolution operator on jump discontinuities, 2018 IEEE International Conference on Data Mining (ICDM), Pages 59-68, IEEE
31. [Conference] Shukla, Manu; Chen, Ziqian; Lu, Chang-Tien; , 2015, DIFPL: Distributed Drone Flight Path Builder System, 2015 1st International Conference on Geographical Information Systems Theory, Applications and Management (GISTAM), Pages 1-10, IEEE
32. [Conference] Roy, Padmaksha; Sarkar, Shailik; Biswas, Subhodip; Chen, Fanglan; **Chen, Zhiqian**; Ramakrishnan, Naren; Lu, Chang-Tien; , 2021, Deep diffusion-based forecasting of COVID-19 by incorporating network-level mobility information, Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, Pages 168-175
33. [Journal] Rafatirad, Setareh; Homayoun, Houman; **Chen, Zhiqian**; Pudukotai Dinakarrao, SM; , 2022, Machine learning for computer scientists and data analysts, nan
34. [Journal] Rafatirad, Setareh; Homayoun, Houman; **Chen, Zhiqian**; Pudukotai Dinakarrao, Sai Manoj; , 2022, Graph learning, Machine Learning for Computer Scientists and Data Analysts: From an Applied Perspective, Pages 277-304, Springer International Publishing Cham
35. [Conference] Meng, Guangyu; Jiang, Qisheng; Fu, Kaiqun; Lin, Beiyu; Lu, Chang-Tien; Chen, Zhiqian; , 2022, Early forecasting of the impact of traffic accidents using a single shot observation, Proceedings of the 2022 SIAM International Conference on Data Mining (SDM), Pages 100-108, Society for Industrial and Applied Mathematics
36. [Book] Rafatirad, Setareh; Homayoun, Houman; **Chen, Zhiqian**; Pudukotai Dinakarrao, Sai Manoj; , 2022, Online Learning, Machine Learning for Computer Scientists and Data Analysts: From an Applied Perspective, Pages 235-256, Springer International Publishing Cham
37. [Conference] Chen, Fanglan; Biswas, Subhodip; **Chen, Zhiqian**; Lei, Shuo; Ramakrishnan, Naren; Lu, Chang-Tien; , 2023, Exploring Tradeoffs in Automated School Redistricting: Computational and Ethical Perspectives, Proceedings of the AAAI Conference on Artificial Intelligence, Volume 37(13), Pages 15912-15920.
38. [Journal] **Chen, Zhiqian**; Chen, Fanglan; Zhang, Lei; Ji, Taoran; Fu, Kaiqun; Zhao, Liang; Chen, Feng; Wu, Lingfei; Aggarwal, Charu; Lu, Chang-Tien; , 2023, Bridging the gap between spatial and spectral domains: A unified framework for graph neural networks, ACM Computing Surveys, Volume 56(5), Pages 42, Association for Computing Machinery
39. [Conference] Zhang, Zonghan; **Chen, Zhiqian**; , 2023, Understanding influence maximization via higher-order decomposition, Proceedings of the 2023 SIAM International Conference on Data Mining (SDM), Pages 766-774, Society for Industrial and Applied Mathematics
40. [Conference] Pillai, Nisha; Nanduri, Bindu; Rothrock, Michael J; **Chen, Zhiqian**; Ramkumar, Mahalingam; , 2023, Towards optimal microbiome to inhibit multidrug resistance, 2023 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology (CIBCB), Pages 1-9, IEEE
41. [Journal] Payandeh, Amirreza; Baghaei, Kourosh T; Fayyazsanavi, Pooya; Ramezani, Somayeh Bakhtiari; **Chen, Zhiqian**; Rahimi, Shahram; , 2023, Deep representation learning: Fundamentals, technologies, applications, and open challenges, IEEE Access, Volume 11, Pages 137621-137659, IEEE
42. [Journal] **Chen, Zhiqian**; , 2020, Graph Neural Networks: Techniques and Applications, nan, Virginia Tech
43. [Journal] Zhang, Lei; **Chen, Zhiqian**; Lu, Chang-Tien; Zhao, Liang; , 2023, Fast and adaptive dynamics-on-graphs to dynamics-of-graphs translation, Frontiers in big Data, Volume 6, Pages 1274135, Frontiers Media SA
44. [Conference] Zhang, Lei; Zhang, Qisheng; **Chen, Zhiqian**; Sun, Yanshen; Lu, Chang-Tien; Zhao, Liang; , 2023, Infinitely Deep Graph Transformation Networks, 2023 IEEE International Conference on Data Mining (ICDM), Pages 778-787, IEEE Computer Society
45. [Journal] Zhang, Zonghan; **Chen, Zhiqian**; , 2023, Accelerating Simulation-Based Influence Maximization via Bayesian Optimization, openreview.net
46. [Journal] Ji, Taoran; Self, Nathan; Fu, Kaiqun; **Chen, Zhiqian**; Ramakrishnan, Naren; Lu, Chang-Tien; , 2024, Citation Forecasting with Multi-Context Attention-Aided Dependency Modeling, ACM Transactions on Knowledge Discovery from Data, Volume 18(6), Pages 1-23, ACM New York, NY
47. [Journal] Zhang, Zijian; Zhang, Zonghan; **Chen, Zhiqian**; , 2024, FlowGPT: How Long Can LLMs Trace Back and Predict the Trends of Graph Dynamics?, SouthNLP
48. [Conference] Yuan, Zirui; Shao, Minglai; **Chen, Zhiqian**; , 2024, Graph bayesian optimization for multiplex influence maximization, Proceedings of the AAAI Conference on Artificial Intelligence, Volume 38(20), Pages 22475-22483
49. [Conference] Zhang, Zonghan; Zhang, Zijian; **Chen, Zhiqian**; , 2024, Multiple-source localization from a single-snapshot observation using graph Bayesian optimization, Proceedings of the AAAI Conference on Artificial Intelligence, Volume 38(20), Pages 22538-22546
50. [Conference] Shang, Rui; Chen, Siji; **Chen, Zhiqian**; Lu, Chang-Tien; , 2024, GraphNILM: A Graph Neural Network for Energy Disaggregation, Pacific-Asia Conference on Knowledge Discovery and Data Mining, Pages 431-443, Springer Nature Singapore Singapore

51. [Conference] Pillai, Nisha; Nanduri, Bindu; Rothrock, Michael J; **Chen, Zhiqian**; Ramkumar, Mahalingam; , 2024, Bayesian-Guided Generation of Synthetic Microbiomes with Minimized Pathogenicity, 2024 46th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), Pages 1-7, IEEE
52. [Journal] Napier, Kollin; Bhowmik, Tanmay; **Chen, Zhiqian**; , 2024, Explaining poor performance of text-based machine learning models for vulnerability detection, Empirical Software Engineering, Volume 29(5), Pages 113, Springer US
53. [Conference] Zhao, Yaya; Zhao, Kaiqi; **Chen, Zhiqian**; Zhang, Yuanyuan; Du, Yalei; Lu, Xiaoling; , 2024, A Graph-based Representation Framework for Trajectory Recovery via Spatiotemporal Interval-Informed Seq2Seq, Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24), Pages 2588-2597, <https://doi.org/10.24963/ijcai.2024/286>
54. [Conference] **Chen, Zhiqian**; Zhang, Lei; Zhao, Liang; , 2024, Unifying Spectral and Spatial Graph Neural Networks, Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, Pages 5511-5513
55. [Conference] Pillai, Nisha; Gireesan, Ganga; Rothrock Jr, Michael James; Nanduri, Bindu; **Chen, Zhiqian**; Ramkumar, Mahalingam; , 2024, Explainable Chickenomics: Advancing Poultry AI with Interpretability, Plant and Animal Genome Conference/PAG 31 (January 12-17, 2024), PAG
56. [Conference] ALMutairi, Mariam; AlKulaib, Lulwah; Wang, Shengkun; **Chen, Zhiqian**; ALMutairi, Youssif; Alenazi, Thamer M; Luther, Kurt; Lu, Chang-Tien; , 2024, Fhirviz: Multi-agent platform for fhir visualization to advance healthcare analytics, Proceedings of the 15th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics, Pages 1-7
57. [Conference] Zhang, Zonghan; Veerapaneni, Ramyasri; Ayoola, Moses; Das, Athish Ram; **Chen, Zhiqian**; Nanduri, Bindu; Ramkumar, Mahalingam; , 2024, Leveraging Graph Neural Networks for MIC Prediction in Antimicrobial Resistance Studies, 2024 46th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), Pages 1-4, IEEE
58. [Conference] Zhang, Zijian; Zhang, Zonghan; **Chen, Zhiqian**; , 2024, Neural Tangent Bayesian Optimization for Accurate and Efficient Influence Maximization, 2024 IEEE 36th International Conference on Tools with Artificial Intelligence (ICTAI), Pages 806-813, IEEE
59. [Journal] Zhang, Lei; Zhang, Qisheng; **Chen, Zhiqian**; Sun, Yanshen; Lu, Chang-Tien; Zhao, Liang; , 2025, Implicit graph neural network for deep graph transformation: L. Zhang et al., Knowledge and Information Systems, Pages 1-30, Springer London London
60. [Journal] Saleem, Adil; Shaw, Leon L; **Chen, Zhiqian**; Lai, Wei; , 2025, State-of-the-Art Machine Learning Technology for Sustainable Lithium Battery Cathode Design: A Perspective, Advanced Energy Materials, Pages 2405300, Wiley Online Library
61. [Conference] Zhang, Lei, **Chen, Zhiqian**, Chang-Tien Lu, and Liang Zhao. "Network Interdiction Goes Neural." In Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2, pp. 3774-3785. 2025.

2. Professional papers read; indicate whether invited, refereed, or volunteered.
Cite organization, date, and title:

3. Grants for research or study:

Proposals submitted since last promotion and total dollar amount: ____; \$ 27,116,150.00

1. **NSF**, TRTech-PGR: Developing Interpretable Artificial Intelligence Tools for Protein Posttranslational Modification Identification and Functional Characterization in Rice, Co-PI, \$1,452,030, submitted on 4/9/2021
2. **NSF**, RI:Small: Robust and Quantifiable Interpretation for Graph Learning, PI, \$499,999, submitted on 5/7/2021
3. **William T. Grant Foundation**, Automated Ethically Aware Redistricting in School Zone Planning by Geometric Deep Learning, PI, \$349,988.00, Submitted 7/7/2021
4. **NSF**, Interpretable Influence Propagating and Blocking on Graphs, PI, \$174,004, submitted on 9/20/2021
5. **NSF**, EAGER: Automating Large-Scale Cyberbullying Analysis: Bridging Social Science Theories and Machine Learning, PI, \$299,992, submitted on 12/13/2021
6. **NSF**, Understanding Human Behavior Through 3D Spatial Environment, PI, \$399,995, submitted on submitted on 1/18/2022
7. **NSF**, CAREER: Interpretable Coupled Dynamics on Multi-layer Networks, PI, \$492,085.00, submitted on 7/27/2022
8. **NSF**, EPSCoR Research Infrastructure Improvement (RII) Track-4:EPSCoR Research Fellows, PI, \$290,498.00, submitted on 4/6/ 2022
9. **NIH**, Multi-model and Multi-channel Dynamics Analysis to Understand the Association of Sleep Patterns for ADHD Prediction in Children, Co-PI, \$51,660.00, submitted on 4/10/2022
10. **NSF**, Creating Intelligent Solutions with Machine Learning and Computer Vision: Smart Computational Thinking as A New Pathway to AI Careers for Diverse Students (Smart CT), Co-PI, \$1,299,554.00, submitted on 8/11/2023
11. **NSF**, REU supplementary fund, PI, \$1, 400, submitted on 9/25/2023
12. **NSF**, CIRC: Planning-C: Synergistic Graph Flow Analytics: An Integrated Infrastructure for Bridging Complexity, Fragmentation, and Interdisciplinary Gaps, PI, \$99,998, submitted on 9/6/2023

13. **NIH**, Sleep Pattern Mining for ADHD Prediction: A Novel Machine Learning Framework to Understand the Association of Sleep Patterns to Pred, Co-PI, \$218,541, submitted on 2/23/2024
14. **Amazon**, FLOWGPT : A Generative Model for Dynamic Graph Modeling and Analysis, PI, \$70,000, submitted on 11/13/2023
15. **NIH**, Multi-model and Multi-channel Dynamics Analysis to Understand the Association of Sleep Patterns for ADHD Prediction Consider Health Disparities, \$52,280.00, submitted on 1/16/2024
16. **Amazon**, Foundation Models for Temporal Graph Flows, No cash fund, AWS credit only, submitted on 3/6/24
17. **NSF**, CISE MSI: RDP: III: Advancing Hate Speech Detection: A Hybrid Approach Using Large Language Models and Graph Neural Networks, university-PI, \$298,376.00, submitted on 4/24/2024
18. **NSF**, Theme 2: AI Institute for Discovery of Extreme Advanced Materials (IDEAL), Co-PI, \$500,001.00, submitted on 4/30/2024
19. **NSF**, CHIRRP: EAGER: Innovative modeling and assessment of urban flood vulnerability, Co-PI, \$298,376.00, submitted on 6/9/2024
20. **NSF**, Planning: CRISES: Center for Urban Water Equity and Sustainability, Co-PI, \$98,662, submitted on 7/1/2024
21. **NSF**, CAREER: From Fragmentation to Integration: Advancing Cross-Graph Dynamics in Interdependent Networks, PI, \$517,594, submitted on 7/25/2024
22. **NSF**, Towards a Unified and Interpretable Dynamical Networks, lead-PI, \$300,096.00, submitted on 10/8/2024
23. **NSF**, DMREF: DFT- and AI-Accelerated Discoveries of Solid Electrolytes for a Clean Energy Society, university-PI, submitted on \$731,861
24. **NSF**, CISE MSI: RPEP: AF: Advancing Hurricane Forecasting: Integrating Machine Learning, Computational Fluid Dynamics, and Data Assimilation, university-PI, \$279,984.00, submitted on 1/30/2025
25. **NSF**, EPSCoR Research Fellows: NSF: Towards a Unified Framework for Cross-Graph Dynamics, PI, \$299,894.00, submitted on 4/7/2025
26. **IES**, Agentic GenAI Center for Improving Postsecondary Instruction, Co-PI, \$9,508,347.00, submitted on 3/14/2025
27. **NIH**, Collaborative Artificial Intelligence Towards Practical High-Risk Head and Neck Cancer Automated Evaluation and Treatment Planning, Co-PI, \$3,392,010.00, submitted on 7/1/2025
28. **NSF**, Class Copilot: A Human-AI Approach to Improving Undergraduate Mathematics Learning, Co-PI, \$399,673, submitted on 1/15/2025
29. **NSF**, E-RISE RII: Mississippi Research and Education Consortium for Adopting and Advancing AI-Powered Optimization Across Fields through Enhancing Chip Technology and Quantum Computing, Co-PI, \$880,000, submitted on 8/2024
30. **NSF**, HNDs-R: Big Data and the Evolution of COVID-19 Misinformation, Co-PI, \$399,312, submitted on 8/15/2025
31. **NIH**, Analyzing Sleep Patterns to Assess the Association Between Sleep Behaviors and ADHD Risk in Children, Co-PI, \$566,385.00, submitted on 6/24/2025
32. **DoE**, EPSCoR: EPSCoR---Collaboration for new paradigm in advanced computation using quantum information science, Co-PI, \$2,894,954, submitted on 1/28/2025

Proposals funded (cite source, title of project, role [PI, etc.], \$ amount, dates):

Personal Credits: \$ 1,640,222. / Total External: \$ 11,171,037.

1. **NSF CAREER**: From Fragmentation to Integration: Advancing Cross-Graph Dynamics in Interdependent Networks (#2443266)
Sole PI, \$517,594 (100% credit), 06/2025–05/2030
2. **NSF CISE MSI: RCBP: III**: Advancing Speech Detection: A Hybrid Approach Using Large Language Models and Graph Neural Networks (#2431176)
Co-PI/University PI, \$400,000 (33% credit, MS State share \$259,948.00), 1/2025–12/2026
3. **NSF CIRC**: Planning-C: Synergistic Graph Flow Analytics: An Integrated Infrastructure for Bridging Complexity, Fragmentation, and Interdisciplinary Gaps (#2345921)
PI, \$99,998 (60% credit), 7/2024–6/2025
4. **NSF CRII**: Interpretable Influence Propagating and Blocking on Graphs (#2153369)
Sole PI, \$174,004 (100% credit), 5/2022–11/2024
5. **USDA [ARS]** Developing Detection and Modeling Tools for the Geospatial and Environmental Epidemiology of Animal Disease (#58-6064-3-017)
Co-PI, \$3,073,602 (5% credit), 10/2023–08/2028
6. **USDA [ARS]** Advancing Agricultural Research through High Performance Computing (#58-0200-0-002)
Co-PI, \$5,690,689 (5% credit), 10/2022–08/2024
7. **NSF [EDU] ITEST**: Learning to Create Intelligent Solutions with Machine Learning and Computer Vision: A Pathway to AI Careers for Diverse High School Students (#2342574)
Co-PI, \$1,192,951 (25% credit), 09/2024–08/2027
8. **NSF [III] REU Supplementary**
PI, \$14,000 (100% credit), 11/2023–11/2024
9. **USDA [ARS] Summer Internship Project**
PI, \$8,199 (100% credit), 5/2024–4/2025

Total Internal: \$19,094.5

1. **CTL Ottilie Schillig Special Teaching Award**, \$2,994.50

2. **INTER-INS** PI, Global Development Seed Grant Award, International Institute \$6,000
3. **BAGLEY** PI, Working Group in Graph AI, Bagley College of Engineering \$4,100
4. **ORED** PI/Co-PI, Undergrad Research Program, ORED, 2022/2024/2025 (Sole PI, \$2,000), 2023 (Co-PI, \$1,500)

4. Other:

C. Service

1. Public service, non-assessment activities such as guest lectures and presentations, external committee/board memberships, business/industry/stakeholder advisement, etc. (with dates, organizations, places):

- **Tutorial**
Unifying Spectral and Spatial Graph Neural Network
SDM 2025, May 3, D.C., U.S. (Tutorial page at SDM)
- **Tutorial**
Unifying Spectral and Spatial Graph Neural Network
CIKM 2024, Oct 21, Boise, Idaho, U.S. (CIKM tutorial list)
- **Tutorial**
Unifying Spectral and Spatial Graph Neural Network
SIAM Math & Data Science 2024, Oct 23, Atlanta, GA, U.S. (Tutorial page at MDS)
- **Tutorial**
Unifying Spectral and Spatial Graph Neural Network
CVPR 2024, June 18, Seattle, Washington, U.S. (Tutorial page at CVPR)
- **Tutorial**
Studying spread patterns of COVID-19 based on spatiotemporal data
SIAM Data Mining 2021, April, Online

2. Professional association service (offices held, journals edited, etc.):

- Department of Computer Science and Engineering, Graduate Committee, 2020-2024.
- Department of Computer Science and Engineering, Curriculum Committee, 2024-2025.
- Department of Computer Science and Engineering, Faculty Search Committee, 2022- 2023.
- Co-advisor, MSU AI Club

3. University service (committees, administrative accomplishments, etc.):

- Review Editor, Editorial Board, *Frontiers in Big Data - Data Science*
- IEEE Big Data 2024, Co-chair of Big Data Cup
- IEEE ICDM 2025, Co-chair of Poster Session

4. Other (academic advisement may be described here or as teaching):

III. **Awards and distinctions (title, date, organization):**

- Best Paper Award at ACM SIGSPATIAL 2020
- Best Paper Award at GISTAM 2015
- Outstanding Contribution Award, 2016, Toyota Research Institute, North America
- Otilie Schillig Special Teaching Award, 2025, Center for Teaching and Learning
- Editor's Choice Article, MDPI
- SIAM Data Mining, 2022, early career travel award
- SEC travel award, 2023

IV. **Memberships in learned and professional societies. Society, dates of membership, and offices held:**

- Association for Computing Machinery, ACM, International. (2016, 2018)
- Society for Industrial and Applied Mathematics (2021-2023)
- Institute of Electrical and Electronics Engineers (2021-2024)

V. **Previous academic ranks, institutions, and dates:**

- Assistant Professor, Mississippi State University, August 16, 2020 - present

VI. **Non-academic positions held prior to appointment at MSU:**

**VII. Summary listing of all required and supporting documentation (items 6 and 7 on the cover of the application form).
This listing should be less than one page in length.**

- Promotion and Tenure Form
- Initial Offer Letter
- Curriculum Vitae
- Table of Contents
 - Research Accomplishments
 - Teaching Accomplishments
 - Service Accomplishments
- Selected Publications

Department Head's Recommendation for Promotion or Tenure
(Cite the following information and sign.)

1. Name of candidate:_____ Present rank:_____
2. Recommended for promotion to the rank of:_____
(Or not recommended):_____
3. Recommended for tenure: Yes/No/NA

Assessment and evaluation by department head: strong points that warrant promotion should be listed, with documentation wherever possible; stress such items as teaching and advising of students, research accomplishments, and university and community service. Please avoid platitudes or general, subjective opinions. It would be useful, too, to comment upon the quality of personal relationships of the candidates with peers, superiors, and any who may report to them, as well as upon their professional performance. Finally, consider the candidate in relation to what you picture as the ideal candidate for this recommended position rather than in relation to other members of your department. In situations where "demonstrated excellence" is required, please provide various supporting evidence such as peer evaluations, reviews of publications, letters of commendation, student survey responses, or any other relevant measures of excellence. Attach relevant departmental committee recommendations.

Date_____ Signed:_____

Department Head

Dean's Recommendation for Promotion or Tenure
(Cite the following information and sign.)

1. Name of candidate:_____ Present rank:_____
2. Recommended for promotion to the rank of:_____
(Or not recommended):_____
3. Recommended for tenure: Yes/No/NA

Recommendation: Use materials provided by the candidate and department head, as appropriate, but please indicate your evaluation of the candidate's performance to date and prospects for the future. Avoid general, subjective opinions, stress obvious strong points, and indicate where further development may be expected. Attach relevant college/school committee recommendations.

Date_____ Signed:_____

Dean

Initial Offer Letter

Letter of Recommendation: Department Head

Letter of Recommendation: Department Promotion & Tenure Committee

Committee Report Form

Curriculum Vitae

Zhiqian Chen

zchen@cse.msstate.edu

imczq.com

(662) 325-1518

Research Interest: Fundamental principles of networked system dynamics by integrating machine learning, network science, and dynamical systems. My collaborations span diverse applications, including Social network, Transportation, Epidemiology, Brain Disease, Materials and Circuit.

Academic Positions

Mississippi State University, Assistant Professor	2020 – present
Virginia Tech, Research Assistant	2014 – 2020

Education

Ph.D., Computer Science Virginia Tech, Falls Church, Virginia, United States	2014 – 2020
M.E., Software Engineering Peking University, China	2010 – 2013
B.E., Software Engineering Huazhong University of Science & Technology, China	2005 – 2009

External Funding

Personal Credits: \$ 1,640,222. / Total External:\$ 11,171,037.

1. From Fragmentation to Integration: Advancing Cross-Graph Dynamics in Interdependent Networks
Role: PI (**sole investigator**)
Funding Agency: National Science Foundation (NSF)
Program: CAREER
Award Number: 2443266
Amount Awarded: \$517,594 (100% credit)
Duration: 06/2025 – 05/2030
2. RCBP: III: Advancing Speech Detection: A Hybrid Approach Using Large Language Models and Graph Neural Networks
Role: Co-PI / University PI
Funding Agency: National Science Foundation (NSF)
Program: CISE MSI
Award Number: 2431176
Amount Awarded: \$400,000 (33% credit, MS State share \$259,948)
Duration: 01/2025 – 12/2026
3. Planning-C: Synergistic Graph Flow Analytics: An Integrated Infrastructure for Bridging Complexity, Fragmentation, and Interdisciplinary Gaps
Role: PI
Funding Agency: National Science Foundation (NSF)
Program: CIRC
Award Number: 2345921
Amount Awarded: \$99,998 (60% credit)
Duration: 07/2024 – 06/2025
4. Interpretable Influence Propagating and Blocking on Graphs
Role: PI (**sole investigator**)
Funding Agency: National Science Foundation (NSF)
Program: CRII
Award Number: 2153369
Amount Awarded: \$174,004 (100% credit)
Duration: 05/2022 – 11/2024
5. Developing Detection and Modeling Tools for the Geospatial and Environmental Epidemiology of Animal Disease
Role: Co-PI
Funding Agency: USDA Agricultural Research Service (ARS)
Award Number: 58-6064-3-017

Amount Awarded: \$3,073,602 (5% credit)

Duration: 10/2023 – 08/2028

6. Advancing Agricultural Research through High Performance Computing

Role: Co-PI

Funding Agency: USDA Agricultural Research Service (ARS)

Award Number: 58-0200-0-002

Amount Awarded: \$5,690,689 (5% credit)

Duration: 10/2022 – 08/2024

7. Learning to Create Intelligent Solutions with Machine Learning and Computer Vision: A Pathway to AI Careers for Diverse High School Students

Role: Co-PI

Funding Agency: National Science Foundation (NSF)

Program: EDU ITEEST

Award Number: 2342574

Amount Awarded: \$1,192,951 (25% credit)

Duration: 09/2024 – 08/2027

8. REU Supplementary

Role: PI

Funding Agency: National Science Foundation (NSF)

Program: III

Amount Awarded: \$14,000 (100% credit)

Duration: 11/2023 – 11/2024

9. Summer Internship Project

Role: PI

Funding Agency: USDA Agricultural Research Service (ARS)

Amount Awarded: \$8,199 (100% credit)

Duration: 05/2024 – 04/2025

Internal Funding

Total: \$19,094.5

1. Otilie Schillig Special Teaching Award
Role: PI
Funding Agency: Center for Teaching and Learning (CTL)
Amount Awarded: \$2,994.50 (100% credit)
Duration: 2025-2026
2. Global Development Seed Grant Award
Role: PI
Funding Agency: International Institute
Amount Awarded: \$6,000 (100% credit)
Duration: 2024
3. Working Group in Graph AI
Role: PI
Funding Agency: Bagley College of Engineering
Amount Awarded: \$4,100 (100% credit)
Duration: 2023-2024
4. Undergraduate Research Program
Role: PI / Co-PI
Funding Agency: Office of Research and Economic Development (ORED)
Amount Awarded: 2022/2024/2025 (Sole PI, \$2,000), 2023 (Co-PI, \$1,500)
Duration: 2022 – 2025

Peer-Reviewed Conference Publications

1. Zhang, Lei, **Chen, Zhiqian**, Chang-Tien Lu, and Liang Zhao. "Network Interdiction Goes Neural." In Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2, pp. 3774-3785. 2025.
2. Yuan, Zirui; Shao, Minglai; **Chen, Zhiqian**; , 2024, Graph bayesian optimization for multiplex influence maximization, Proceedings of the AAAI Conference on Artificial Intelligence, Volume 38(20), Pages 22475-22483
3. Zhang, Zonghan; Zhang, Zijian; **Chen, Zhiqian**; , 2024, Multiple-source localization from a single-snapshot observation using graph Bayesian optimization, Proceedings of the AAAI Conference on Artificial Intelligence, Volume 38(20), Pages 22538-22546
4. Shang, Rui; Chen, Siji; **Chen, Zhiqian**; Lu, Chang-Tien; , 2024, GraphNILM: A Graph Neural Network for Energy Disaggregation, Pacific-Asia Conference on Knowledge Discovery and Data Mining, Pages 431-443, Springer Nature Singapore Singapore

5. Pillai, Nisha; Nanduri, Bindu; Rothrock, Michael J; **Chen, Zhiqian**; Ramkumar, Mahalingam; , 2024, Bayesian-Guided Generation of Synthetic Microbiomes with Minimized Pathogenicity, 2024 46th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), Pages 1-7, IEEE
6. Zhao, Yaya; Zhao, Kaiqi; **Chen, Zhiqian**; Zhang, Yuanyuan; Du, Yalei; Lu, Xiaoling; , 2024, A Graph-based Representation Framework for Trajectory Recovery via Spatiotemporal Interval-Informed Seq2Seq, Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24), Pages 2588-2597
7. **Chen, Zhiqian**; Zhang, Lei; Zhao, Liang; , 2024, Unifying Spectral and Spatial Graph Neural Networks, Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, Pages 5511-5513
8. Pillai, Nisha; Gireesan, Ganga; Rothrock Jr, Michael James; Nanduri, Bindu; **Chen, Zhiqian**; Ramkumar, Mahalingam; , 2024, Explainable Chickenomics: Advancing Poultry AI with Interpretability, Plant and Animal Genome Conference/PAG 31 (January 12-17, 2024), PAG
9. ALMutairi, Mariam; AlKulaib, Lulwah; Wang, Shengkun; **Chen, Zhiqian**; ALMutairi, Youssif; Alenazi, Thamer M; Luther, Kurt; Lu, Chang-Tien; , 2024, Fhirviz: Multi-agent platform for fhir visualization to advance healthcare analytics, Proceedings of the 15th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics, Pages 1-7
10. Zhang, Zonghan; Veerapaneni, Ramyasri; Ayoola, Moses; Das, Athish Ram; **Chen, Zhiqian**; Nanduri, Bindu; Ramkumar, Mahalingam; , 2024, Leveraging Graph Neural Networks for MIC Prediction in Antimicrobial Resistance Studies, 2024 46th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), Pages 1-4, IEEE
11. Zhang, Zijian; Zhang, Zonghan; **Chen, Zhiqian**; , 2024, Neural Tangent Bayesian Optimization for Accurate and Efficient Influence Maximization, 2024 IEEE 36th International Conference on Tools with Artificial Intelligence (ICTAI), Pages 806-813, IEEE
12. Zhang, Lei; Zhang, Qisheng; **Chen, Zhiqian**; Sun, Yanshen; Lu, Chang-Tien; Zhao, Liang; , 2023, Infinitely Deep Graph Transformation Networks, 2023 IEEE International Conference on Data Mining (ICDM), Pages 778-787, IEEE Computer Society
13. Chen, Fanglan; Biswas, Subhodip; **Chen, Zhiqian**; Lei, Shuo; Ramakrishnan, Naren; Lu, Chang-Tien 2023 Exploring Tradeoffs in Automated School Redistricting Computational and Ethical Perspectives Proceedings of the AAAI Conference on Artificial Intelligence Volume 37(13) Pages 15912–15920
14. Zhang, Zonghan; **Chen, Zhiqian**; , 2023, Understanding influence maximization via higher-order decomposition, Proceedings of the 2023 SIAM International Conference on Data Mining (SDM), Pages 766-774, Society for Industrial and Applied Mathematics

15. Pillai, Nisha; Nanduri, Bindu; Rothrock, Michael J; **Chen, Zhiqian**; Ramkumar, Mahalingam; , 2023, Towards optimal microbiome to inhibit multidrug resistance, 2023 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology (CIBCB), Pages 1-9, IEEE
16. Fu, Kaiqun; Ji, Taoran; Self, Nathan; **Chen, Zhiqian**; Lu, Chang-Tien; , 2021, A hierarchical attention graph convolutional network for traffic incident impact forecasting, 2021 IEEE International Conference on Big Data (Big Data), Pages 1619-1624, IEEE
17. Ji, Taoran; Self, Nathan; Fu, Kaiqun; **Chen, Zhiqian**; Ramakrishnan, Naren; Lu, Chang-Tien; , 2021, Dynamic multi-context attention networks for citation forecasting of scientific publications, Proceedings of the AAAI Conference on Artificial Intelligence, Volume 35(9), Pages 7953-7960
18. Roy, Padmaksha; Sarkar, Shailik; Biswas, Subhodip; Chen, Fanglan; **Chen, Zhiqian**; Ramakrishnan, Naren; Lu, Chang-Tien; , 2021, Deep diffusion-based forecasting of COVID-19 by incorporating network-level mobility information, Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, Pages 168-175
19. Wang, Jason; Fu, Kaiqun; **Chen, Zhiqian**; Lu, Chang-Tien; , 2022, Augmentation of Chinese Character Representations with Compositional Graph Learning (Student Abstract), Proceedings of the AAAI Conference on Artificial Intelligence, Volume 36(11), Pages 13075-13076
20. Zhang, Lei; **Chen, Zhiqian**; Lu, Chang-Tien; Zhao, Liang; , 2022, From “Dynamics on Graphs” to “Dynamics of Graphs”: An Adaptive Echo-State Network Solution (Student Abstract), Proceedings of the AAAI Conference on Artificial Intelligence, Volume 36(11), Pages 13111-13112
21. Meng, Guangyu; Jiang, Qisheng; Fu, Kaiqun; Lin, Beiyu; Lu, Chang-Tien; Chen, Zhiqian; , 2022, Early forecasting of the impact of traffic accidents using a single shot observation, Proceedings of the 2022 SIAM International Conference on Data Mining (SDM), Pages 100-108, Society for Industrial and Applied Mathematics
22. Zhang, Zonghan; Biswas, Subhodip; Chen, Fanglan; Fu, Kaiqun; Ji, Taoran; Lu, Chang-Tien; Ramakrishnan, Naren; **Chen, Zhiqian**; , 2022, Blocking influence at collective level with hard constraints (student abstract), Proceedings of the AAAI Conference on Artificial Intelligence, Volume 36(11), Pages 13115-13116
23. Chen, Fanglan; **Chen, Zhiqian**; Biswas, Subhodip; Lei, Shuo; Ramakrishnan, Naren; Lu, Chang-Tien; , 2020, Graph convolutional networks with kalman filtering for traffic prediction, Proceedings of the 28th international conference on advances in geographic information systems, Pages 135-138
24. Biswas, Subhodip; Chen, Fanglan; **Chen, Zhiqian**; Lu, Chang-Tien; Ramakrishnan, Naren; , 2020, Incorporating domain knowledge into Memetic Algorithms for solving

Spatial Optimization problems, Proceedings of the 28th International Conference on Advances in Geographic Information Systems, Pages 25-35

25. Chen, **Chen, Zhiqian**; Kolhe, Gaurav; Rafatirad, Setareh; Lu, Chang-Tien; PD, Sai Manoj; Homayoun, Houman; Zhao, Liang; , 2020, Estimating the circuit de-obfuscation runtime based on graph deep learning, 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE), Pages 358-363, IEEE
26. Biswas, Subhodip; Chen, Fanglan; Sistrunk, Andreea; Muthiah, Sathappan; **Chen, Zhiqian**; Self, Nathan; Lu, Chang-Tien; Ramakrishnan, Naren 2020 Geospatial clustering for balanced and proximal schools Proceedings of the AAAI Conference on Artificial Intelligence Volume 34(9) Pages 13358–13365
27. He, Jianfeng; Zhang, Xuchao; Lei, Shuo; **Chen, Zhiqian**; Chen, Fanglan; Alhamadani, Abdulaziz; Xiao, Bei; Lu, Chang-Tien; , 2020, Towards more accurate uncertainty estimation in text classification, Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Pages 8362-8372
28. Ji, Taoran; **Chen, Zhiqian**; Self, Nathan; Fu, Kaiqun; Lu, Chang-Tien; Ramakrishnan, Naren; , 2019, Patent citation dynamics modeling via multi-attention recurrent networks, Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-19)
29. Biswas, Subhodip; Chen, Fanglan; **Chen, Zhiqian**; Sistrunk, Andreea; Self, Nathan; Lu, Chang-Tien; Ramakrishnan, Naren; , 2019, REGAL: A Regionalization framework for school boundaries, Proceedings of the 27th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, Pages 544-547
30. Fu, Kaiqun; **Chen, Zhiqian**; Lu, Chang-Tien; , 2018, Streetnet: preference learning with convolutional neural network on urban crime perception, Proceedings of the 26th ACM SIGSPATIAL international conference on advances in geographic information systems, Pages 269-278
31. Chen, **Chen, Zhiqian**; Chen, Feng; Lai, Rongjie; Zhang, Xuchao; Lu, Chang-Tien; , 2018, Rational neural networks for approximating graph convolution operator on jump discontinuities, 2018 IEEE International Conference on Data Mining (ICDM), Pages 59-68, IEEE
32. Zhang, Xuchao; **Chen, Zhiqian**; Zhao, Liang; Boedihardjo, Arnold P; Lu, Chang-Tien; , 2017, Traces: Generating twitter stories via shared subspace and temporal smoothness, 2017 IEEE International Conference on Big Data (Big Data), Pages 1688-1693, IEEE
33. Zhang, Xuchao; Zhao, Liang; **Chen, Zhiqian**; Boedihardjo, Arnold P; Dai, Jing; Lu, Chang-Tien; , 2017, Trendi: Tracking stories in news and microblogs via emerging, evolving and fading topics, 2017 IEEE International Conference on Big Data (Big Data), Pages 1590-1599, IEEE

34. Chen, **Chen, Zhiqian**; Zhang, Xuchao; Boedihardjo, Arnold P; Dai, Jing; Lu, Chang-Tien; , 2017, Multimodal storytelling via generative adversarial imitation learning, Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI-17)
35. **Chen, Zhiqian**; Wu, Chih-Wei; Lu, Yen-Cheng; Lerch, Alexander; Lu, Chang-Tien; , 2017, Learning to fuse music genres with generative adversarial dual learning, 2017 IEEE International Conference on Data Mining (ICDM), Pages 817-822, IEEE
36. Zhang, Xuchao; **Chen, Zhiqian**; Zhong, Weisheng; Boedihardjo, Arnold P; Lu, Chang-Tien; , 2016, Storytelling in heterogeneous twitter entity network based on hierarchical cluster routing, 2016 IEEE International Conference on Big Data (Big Data), Pages 1522-1531, IEEE
37. Shukla, Manu; Chen, Ziqian; Lu, Chang-Tien; , 2015, DIFPL: Distributed Drone Flight Path Builder System, 2015 1st International Conference on Geographical Information Systems Theory, Applications and Management (GISTAM), Pages 1-10, IEEE
38. **Chen, Zhiqian**; Feng, Wenya; , 2013, Detecting impolite crawler by using time series analysis, 2013 IEEE 25th International Conference on Tools with Artificial Intelligence, Pages 123-126, IEEE

Peer-Reviewed Journal Publications

1. Zhang, Lei; Zhang, Qisheng; **Chen, Zhiqian**; Sun, Yanshen; Lu, Chang-Tien; Zhao, Liang; , 2025, Implicit graph neural network for deep graph transformation: L. Zhang et al., Knowledge and Information Systems, Pages 1-30, Springer London London
2. Saleem, Adil; Shaw, Leon L; **Chen, Zhiqian**; Lai, Wei; , 2025, State-of-the-Art Machine Learning Technology for Sustainable Lithium Battery Cathode Design: A Perspective, Advanced Energy Materials, Pages 2405300, Wiley Online Library
3. Ji, Taoran; Self, Nathan; Fu, Kaiqun; **Chen, Zhiqian**; Ramakrishnan, Naren; Lu, Chang-Tien; , 2024, Citation Forecasting with Multi-Context Attention-Aided Dependency Modeling, ACM Transactions on Knowledge Discovery from Data, Volume 18(6), Pages 1-23, ACM New York, NY
4. Zhang, Zijian; Zhang, Zonghan; **Chen, Zhiqian**; , 2024, FlowGPT: How Long Can LLMs Trace Back and Predict the Trends of Graph Dynamics?, SouthNLP
5. Napier, Kollin; Bhowmik, Tanmay; **Chen, Zhiqian**; , 2024, Explaining poor performance of text-based machine learning models for vulnerability detection, Empirical Software Engineering, Volume 29(5), Pages 113, Springer US

6. Zhang, Lei; **Chen, Zhiqian**; Lu, Chang-Tien; Zhao, Liang; , 2023, Fast and adaptive dynamics-on-graphs to dynamics-of-graphs translation, *Frontiers in big Data*, Volume 6, Pages 1274135, Frontiers Media SA
7. Zhang, Zonghan; **Chen, Zhiqian**; , 2023, Accelerating Simulation-Based Influence Maximization via Bayesian Optimization, *openreview.net*
8. Chen, **Chen, Zhiqian**; Chen, Fanglan; Zhang, Lei; Ji, Taoran; Fu, Kaiqun; Zhao, Liang; Chen, Feng; Wu, Lingfei; Aggarwal, Charu; Lu, Chang-Tien; , 2023, Bridging the gap between spatial and spectral domains: A unified framework for graph neural networks, *ACM Computing Surveys*, Volume 56(5), Pages 42, Association for Computing Machinery
9. Payandeh, Amirreza; Baghaei, Kourosh T; Fayyazsanavi, Pooya; Ramezani, Somayeh Bakhtiari; **Chen, Zhiqian**; Rahimi, Shahram; , 2023, Deep representation learning: Fundamentals, technologies, applications, and open challenges, *IEEE Access*, Volume 11, Pages 137621-137659, IEEE
10. Biswas, Subhodip; Chen, Fanglan; **Chen, Zhiqian**; Lu, Chang-Tien; Ramakrishnan, Naren; , 2023, Memetic algorithms for spatial partitioning problems, *ACM Transactions on Spatial Algorithms and Systems*, Volume 9(1), Pages 1-31, ACM New York, NY
11. Rafatirad, Setareh; Homayoun, Houman; **Chen, Zhiqian**; Pudukotai Dinakarrao, SM; , 2022, Machine learning for computer scientists and data analysts, nan
12. Rafatirad, Setareh; Homayoun, Houman; **Chen, Zhiqian**; Pudukotai Dinakarrao, Sai Manoj; , 2022, Graph learning, *Machine Learning for Computer Scientists and Data Analysts: From an Applied Perspective*, Pages 277-304, Springer International Publishing Cham
13. Ding, Zhao; **Chen, Zhiqian**; Ma, Tianyi; Lu, Chang-Tien; Ma, Wenhui; Shaw, Leon; , 2020, Predicting the hydrogen release ability of LiBH₄-based mixtures by ensemble machine learning, *Energy Storage Materials*, Volume 27, Pages 466-477, Elsevier
14. Ding, Zhao; Li, Shaoyuan; Zhou, Yang; **Chen, Zhiqian**; Yang, Weijie; Ma, Wenhui; Shaw, Leon; , 2020, LiBH₄ for hydrogen storage-new perspectives, *Nano Materials Science*, Volume 2(2), Pages 109-119, Elsevier
15. Zhang, Ying; He, Xingfeng; **Chen, Zhiqian**; Bai, Qiang; Nolan, Adelaide M; Roberts, Charles A; Banerjee, Debasish; Matsunaga, Tomoya; Mo, Yifei; Ling, Chen; , 2019, Unsupervised discovery of solid-state lithium ion conductors, *Nature communications*, Volume 10(1), Pages 5260, Nature Publishing Group UK London
16. Shukla, Manu; **Chen, Zhiqian**; Lu, Chang-Tien; , 2018, DIMPL: a distributed in-memory drone flight path builder system, *Journal of Big Data*, Volume 5(1), Pages 24, Springer International Publishing Cham

17. Wang, Bingsheng; **Chen, Zhiqian**; Boedihardjo, Arnold P; Lu, Chang-Tien; , 2018, Virtual metering: An efficient water disaggregation algorithm via nonintrusive load monitoring, ACM Transactions on Intelligent Systems and Technology (TIST), Volume 9(4), Pages 1-30, ACM New York, NY, USA
18. Li, Guoming; Hui, Xue; **Chen, Zhiqian**; Chesser Jr, Gary D; Zhao, Yang; , 2021, Development and evaluation of a method to detect broilers continuously walking around feeder as an indication of restricted feeding behaviors, Computers and electronics in agriculture, Volume 181, Pages 105982, Elsevier
19. Li, Guoming; Huang, Yanbo; **Chen, Zhiqian**; Chesser Jr, Gary D; Purswell, Joseph L; Linhoss, John; Zhao, Yang; , 2021, Practices and applications of convolutional neural network-based computer vision systems in animal farming: A review, Sensors, Volume 21(4), Pages 1492, MDPI
20. Chen, **Chen, Zhiqian**; Zhang, Lei; Kolhe, Gaurav; Kamali, Hadi Mardani; Rafatirad, Setareh; Pudukotai Dinakarrao, Sai Manoj; Homayoun, Houman; Lu, Chang-Tien; Zhao, Liang; , 2021, Deep graph learning for circuit deobfuscation, Frontiers in big Data, Volume 4, Pages 608286, Frontiers Media SA

Book

1. Rafatirad, Setareh; Homayoun, Houman; **Chen, Zhiqian**; Pudukotai Dinakarrao, Sai Manoj; , 2022, Online Learning, Machine Learning for Computer Scientists and Data Analysts: From an Applied Perspective, Pages 235-256, Springer International Publishing Cham.

Awards

1. Best Paper Award at ACM SIGSPATIAL
Year: 2020
Organization: ACM SIGSPATIAL
Note: Awards Recipients
2. Best Paper Award at GISTAM
Year: 2015
Organization: International Conference on Geographical Information Systems Theory, Applications and Management (GISTAM)
Note: Awards Recipients

3. Outstanding Contribution Award
Year: 2016
Organization: Toyota Research Institute, North America
4. Otilie Schillig Special Teaching Award
Year: 2025
Organization: Center for Teaching and Learning (CTL)
Note: Award Recipients
5. Editor's Choice Article
Organization: MDPI
Note: Editor Choices
6. Top 50 Chemistry & Materials Sciences Articles in Nature Communications
Year: 2019
Organization: Nature Communications
Note: Top Paper List
7. Excellent Reviewer
Year: 2024
Organization: IEEE Transactions on Network Science and Engineering
8. Excellent Reviewer (Top 20%)
Year: 2025
Organization: ACM SIGKDD

Conference and Tutorial Talks

1. Tutorial: Unifying Spectral and Spatial Graph Neural Network
Event: SIAM Data Mining 2025
Date: May 3, 2025
Location: Washington, D.C., U.S.
2. Tutorial: Unifying Spectral and Spatial Graph Neural Network
Event: CIKM 2024
Date: October 21, 2024
Location: Boise, Idaho, U.S.
3. Tutorial: Unifying Spectral and Spatial Graph Neural Network
Event: SIAM Math & Data Science 2024
Date: October 23, 2024
Location: Atlanta, Georgia, U.S.

4. Tutorial: Unifying Spectral and Spatial Graph Neural Network
Event: CVPR 2024
Date: June 18, 2024
Location: Seattle, Washington, U.S.
5. Paper: Early forecasting of the impact of traffic accidents using a single shot observation
Event: SIAM Data Mining 2022
Date: April 2022
Location: Minneapolis, Minnesota, U.S.
6. Tutorial: Studying spread patterns of COVID-19 based on spatiotemporal data
Event: SIAM Data Mining 2021
Date: April 2021
Location: Online

Service

1. BigData Cup Challenges Co-Chair at IEEE BigData 2024,
Note: BigData 2024 Organization Committee
2. Poster Co-Chair at IEEE ICDM 2025
Note: BigData 2024 Organization Committee
3. Panelist: NSF Panel
Years: 2021, 2023, 2024
4. Editorial Board Member:
Frontiers in Big Data – Data Science
Frontiers in Big Data – Data Mining and Management
Frontier Topic Editor (2022–present)
5. Reviewer:
International Conference on Machine Learning (ICML), 2021–2024
International Conference on Learning Representations (ICLR), 2022–2024
Neural Information Processing Systems (NeurIPS), 2020–2024
AAAI Conference on Artificial Intelligence (AAAI), 2021–2024
International Joint Conference on Artificial Intelligence (IJCAI), 2022–2024
ACM SIG on Knowledge Discovery and Data Mining (KDD), 2020–2024
ACM SIG on Information Retrieval (SIGIR), 2020, 2022, 2023
IEEE Transactions on Knowledge and Data Engineering (TKDE), 2020
ACM Transactions on Knowledge Discovery from Data (TKDD), 2021, 2024

6. Advisor: Mississippi State University AI Club

Departmental, University, and Community Service

1. CSE Graduate Studies Committee, 2020-2024.
2. Faculty Search Committee, 2023 - 2024.
3. Engineering Day (E-Day) Speaker, 2023, 2024. Delivered two 15-minute presentations aimed at encouraging students to enroll in computer science.

Student Advising

Serving as Major Professor or main research advisor

1. Ph.D. Students (current): Zonghan Zhang, Zijian Zhang, Tamanna Rashme, Amin George, Josh Waldbieser, Jiashan Wu, Rocker D'Antonio, Gaoyuan Du, Xinyuan Chen, Naidu Sigampalli
2. M.S. Students (current): Marouane Benbrahim , Sai Harika Gade, Pavan Dharma Adapa
3. Undergraduate Students (current): Kevin Ho
4. Alumni — Graduate: Samuel Prabhakar, Rajeev Jogi, Piero Bracamonte, Suman Adhikari, Ramyasri Veerapaneni, Aymane Jouhari, Jason Weeks, Matthew Rester, Henry Smith
5. Alumni — Undergraduate: Ben Moore, Jack Maloney, Ethan Rogers, Prathyusha Mustiyala, Andrew McBride, Aalok Uprety, Mason Fisher, Reid Sewell

Teaching

1. Undergraduate Course: CSE 4633/6633 Artificial Intelligence
2. Undergraduate Course: CSE 4693/6693 Introduction to Machine Learning
3. Graduate Course: CSE 8673 Machine Learning
4. Graduate Course: CSE 8990 Graph Machine Learning

Teaching Accomplishments

Contents

- Highlights
- Teaching Philosophy
- Student Teaching Evaluations
 - Numerical Student Feedback
 - Free-Form Student Feedback
- Curriculum Development
- Sample Syllabus

Highlights

- PARTICIPATE THE ORGANIZATION OF **Machine Learning** COURSES AND TEACHING MOST OF THEM IN THE DEPARTMENT
 - DEVELOPED AND TAUGHT NEW COURSE FOR ENTRY-LEVEL AI STUDENTS: CSE 4963/6963 INTRO TO MACHINE LEARNING
 - DEVELOPED AND TAUGHT NEW COURSES CSE 8990 GRAPH MACHINE LEARNING RECEIVED POSITIVE FEEDBACK
-

Teaching Philosophy.

At this point in time, AI has a substantial impact on practically every field, and understanding how to unlock the potential of each student could result in enormous social advancement. AI techniques are now breaking down barriers and are accessible to nearly everyone. Current AI developments such as ChatGPT alter perspectives and will likely alter work and life styles in the near future. Thus, as a practitioner and educator in the field of AI, I view teaching as a duty and a privilege. My teaching philosophy primarily comes from my experience in teaching, learning, and communicating with other people in my AI field and who want to use AI. In my opinion, there are four pillars of successful teaching:

Objective - *Education is not the filling of a pail, but the lighting of a fire.* The goal of teaching is to encourage the students to explore what they don't know, rather than repeat what the textbook says. I remember the "Network attack and defense" course in my graduate study. The professor successfully made me interested in the content, even though I was not working in the network security direction. This success originates from (1) his rich industrial and academic experience (e.g., he was serving network security in Olympic games): Numerous real stories were presented, and these interesting case studies aroused my interest, and (2) flexible course project and experienced teaching assistant: every student was allowed to explore possible solutions as long as the task is accomplished. In conclusion, I will follow this rule: motivate students to learn, rather than just push them to learn.

Technique - *If one can't explain it simply, she or he doesn't understand it well enough.* In one course of my second major, my teacher asked everyone to explain part of the textbook to the others, then she gave feedback on our performance and remarked on unclear points. This technique made us think as a teacher, not as a student. The idea behind is that, if one can explain it simply, then he or she should have understood it quite well. This technique was also applied when I was weekly volunteering at a math class of junior high school. I taught the first group about how to solve a set of specific problems, and then I asked the students from the first group to teach the second group. After the students' presentation, I would always encourage these students, and then add comments about unclear points. The result also shows that, when a student thinks like a teacher, he or she will learn better.

Measurement - *One can't manage what is not measurable.* Measure, practice, and then measure is the most simple but effective way to identify whether students learn from the course. Any

teaching technique requires quantitative measurement. Otherwise, all efforts will be wasted. I will set a step by step goal for students, clarifying the milestone of the course and the meaning of each task, such as homework, course project, and presentation. Quantitative teaching also helps the teacher to monitor the progress of teaching and improve specific content that is not well learned by students. In addition to that, quantitative teaching allows the teacher always to attempt potential better content without too many risks.

Meaning - *From everyone knows to nobody knows*. Teaching and research are the two primary jobs of professors, but they are not irrelevant. Instead, they have always been a close integration with interaction. Research is a process that is “from everyone knows to nobody knows,” since research is to explore unknown domains. When writing a paper, giving a talk about our research, one should explain complicated research ideas in simple words, which is “from nobody knows to everyone knows”. Teaching is also another way of connecting “nobody knows” with “everyone knows”: students learn knowledge from “everyone knows” (common sense) to “nobody knows” (new knowledge), while the teacher teaches from “nobody knows” to “everyone knows.” Therefore, teaching and research are two sides of a coin. Quality research provides insightful content for teaching, while teaching offers feedback about how deeply the teacher understands the research, and any question from a student or confusing points might become a problem for research.

Efforts Initially, I developed and introduced machine learning (ML) courses from the first semester, receiving generally favorable feedback. Subsequently, I taught a deep learning module within an AI course, a decision I later questioned due to the significant overlap between ML and AI, despite my primary expertise being in the relatively niche area of new deep learning algorithms applied to graphs. Nevertheless, I embraced the challenge of creating AI course content from the ground up, recognizing this effort as beneficial to the long-term development of the AI curriculum. To further enrich the AI education pathway, I introduced new courses, namely CSE 4963/6963 Introduction to Machine Learning, designed to cater to students at different levels of proficiency. This initiative was aimed at completing the AI learning cycle, ensuring a comprehensive educational journey. In addition, leveraging my specialized knowledge in graph machine learning, I launched a new course titled CSE 8990 Graph Machine Learning. This course was specifically designed to offer students interested in advanced topics the opportunity to explore cutting-edge content in this field. Lastly, I implemented innovative interactive activities within the classroom setting, aiming to enhance the learning experience and foster a more engaging educational environment. Over the past years, I have implemented the following principles in my teaching methodology:

- For each algorithm taught, I developed coding demonstrations to provide practical insights and enhance understanding.
- I introduced a midterm course evaluation process to gather feedback on student needs, allowing for immediate adjustments to the course content and teaching methods.
- Leveraging my research expertise, I developed new course content (such as the new course CSE 8990) and integrated cutting-edge topics into all machine learning courses, enriching the curriculum with the latest advancements in the field.
- I incorporated innovative class interactions through Slido.com, which was highly appreciated by the students. This tool not only facilitated interactive sessions for face-to-face classes but

also significantly engaged online learners, fostering a more inclusive and dynamic learning environment.

Student Evaluation

Students may submit anonymous feedback at the end of each semester in the form of specific scored questions as well as written free-form comments. Here, I discuss my numerical grades and how I incorporate student feedback into my teaching. I selected recent official numeric evaluation by students.

CSE 8673 Machine Learning: my teaching rating is higher than CSE avg, Div Avg and School Avg for all questions on both sections 02/501(see "Question Averages" at page 3/4 and 2/3 below)



There were: 29 possible respondents.

	Question Text	N	Median	Str Disagree	Disagree	Agree	Str Agree	N/A							
1	Student knew course expectations	18	4	0%	6%	33%	61%	0%							
2	Student connected assignments/ exams to course	18	4	0%	0%	33%	67%	0%							
3	Course activities/ assignments helped with learning	18	4	6%	0%	33%	61%	0%							
4	Feedback/ grades helped students to improve	18	4	0%	0%	33%	67%	0%							
5	Student gained knowledge/ skills for future classes/ career	18	4	0%	0%	33%	67%	0%							
				0 - 1	2 - 3	4 - 5	6 - 7	8 - 9	10 - 11	12 - 13	14 - 15	16 - 17	18 - 19	20+	
6	Time spent on course outside of class	18		6%	17%	28%	22%	6%	11%	6%	0%	0%	0%	0%	6%
				Str Disagree	Disagree	Agree	Str Agree	N/A							
10	Student course participation respected by instructor	18	4	0%	6%	28%	67%	0%							
11	Instructor/ teaching assistant available in class/ during office hours	18	4	0%	0%	22%	78%	0%							
12	Instructor presented/ explained course material clearly	18	4	0%	11%	17%	72%	0%							
13	Instructor wants students to succeed	18	4	0%	6%	28%	67%	0%							
14	Overall would recommend instructor	18	4	6%	0%	28%	67%	0%							

Text Responses
Question: What worked well in this class?
His teaching, homeworks and quizzes were good and hes a nice person.
Everything worked well
The way of teaching and modularity.
Everything went good
interactive learning with a gradeless online quiz (Sildo).
I liked the programming assignments.
The in-class, ungraded quizzes on Slido were engaging and helpful.

It was a well planned class. The in-class interaction tool slido worked really good for me. It helped me understand the problems and thus was useful for preparing for the exams.

clear and structured lectures (from easy to hard), The class introduced a broad range of machine learning algorithms and techniques, which provided a comprehensive overview of the field. Using real-world datasets and scenarios to explain machine learning concepts made the material engaging and relevant.

Question: What changes could improve this class? (Specific suggestions will be the most useful.)

His requirements for the term paper is outrageous. I am a first semester PhD student, never written a paper before, and he wants novelty as a requirement? What is he going to do with my novelty, steal it and use it for his own research and not cite it? I mean its already ridiculous enough thats its worth 50% of the overall grade, but the novelty is not fair.

I dont have any recommendations.

Nothing i got what i need.

Nothing

There should be a break (5 minutes) in the middle of the class.

I wish the majority of the class was not dedicated to a group paper.

The heavy mathematics focus was sometimes difficult to follow, but the code examples always helped.

I just wish there were one or two written answers for the quizzes. As MCQs, there is no scope for partial marking, getting one wrong means the whole point is gone. 5 points for one MCQ is scary. May be less points for one MCQ will be great.

Provide more exposure to commonly used machine learning tools and frameworks like TensorFlow and PyTorch through guided tutorials.

Question: Please feel free to say more about your response to any items above or provide any additional feedback.

Please talk to him about the novelty requirement of the term paper, it really stressed me out and the term paper made me feel like dropping out of my PhD, and its not even the end of my first semester.

.

none

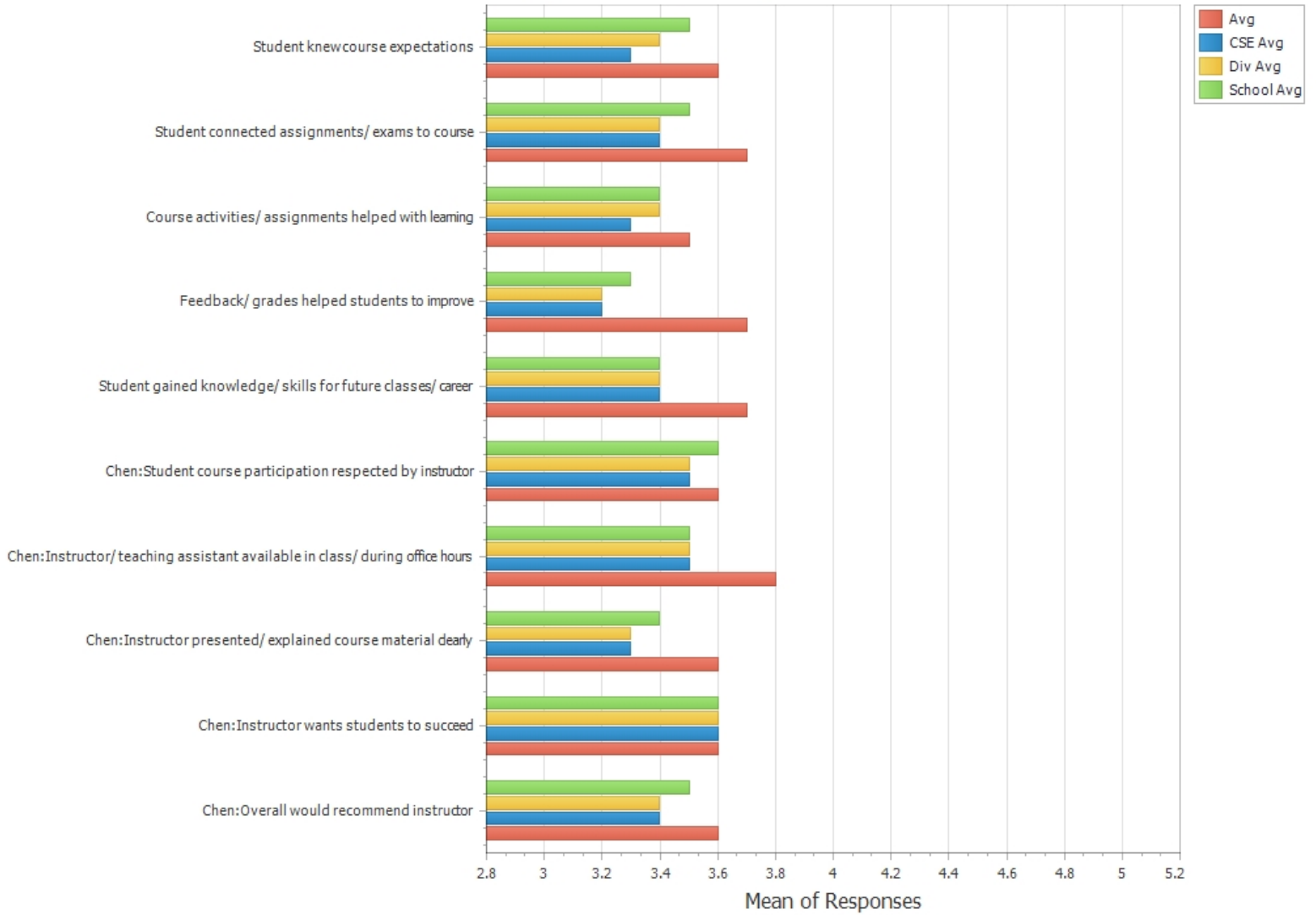
None

I wish there was more practical application.

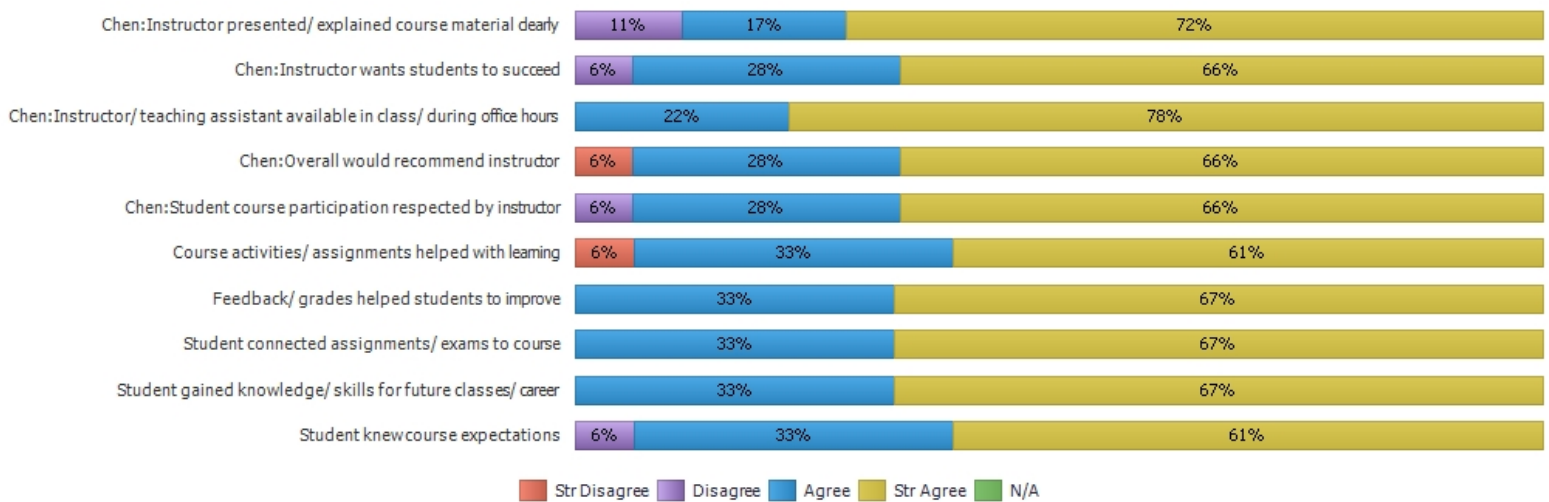
It was a good course. I really enjoyed it. As I already had prior knowledge about machine learning but did not know about them in details, I just knew how they work. This class helped just strengthened my porior knowledge.

Final projects are a great learning opportunity. This class is well structured. I recommend this instructor to every student.

Question Averages



Distribution of Scores



Mississippi State University

Fall 2024, CSE 8673 Machine Learning Section 501
Instructor: Chen, Zhiqian (Primary)

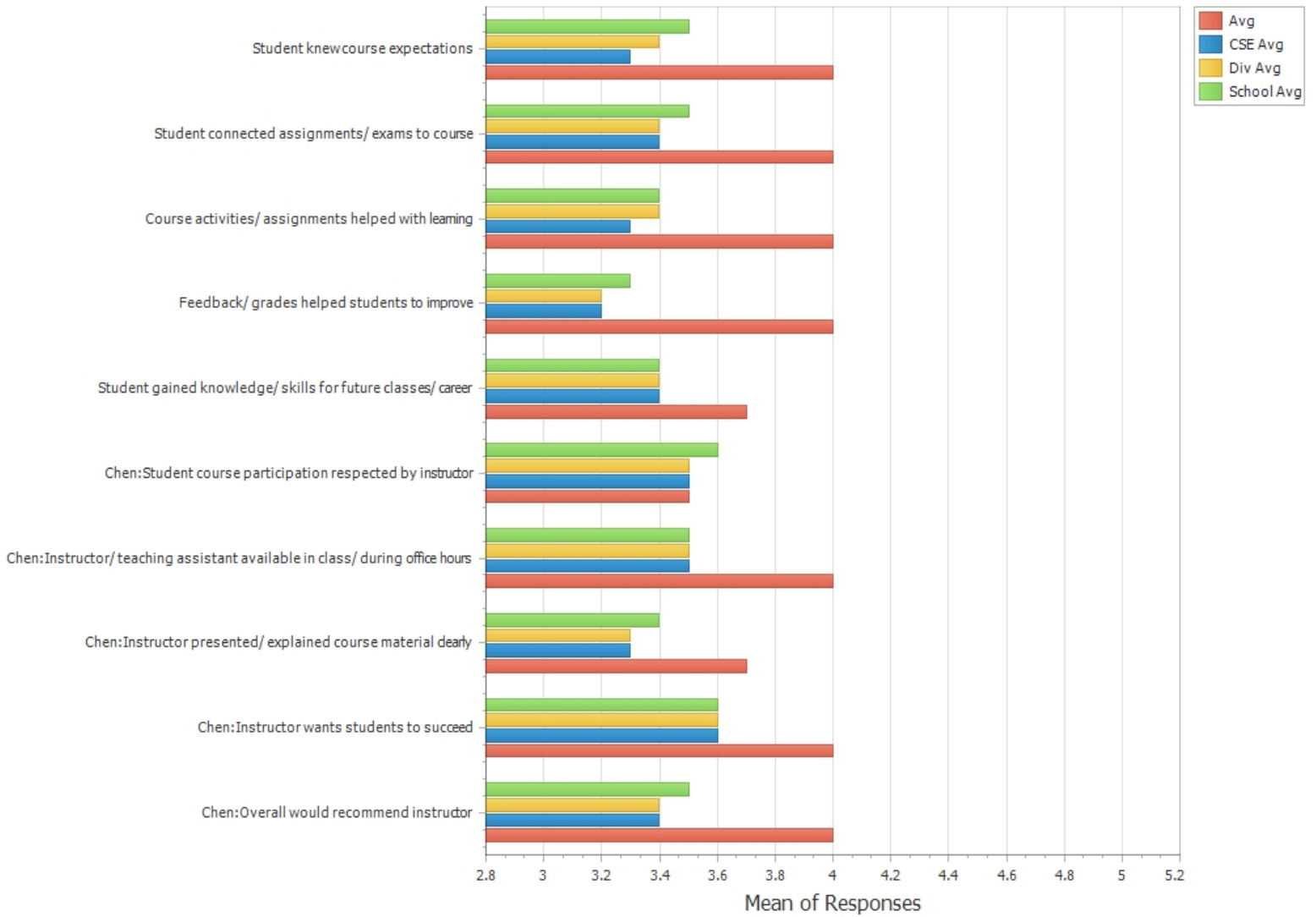


There were: 13 possible respondents.

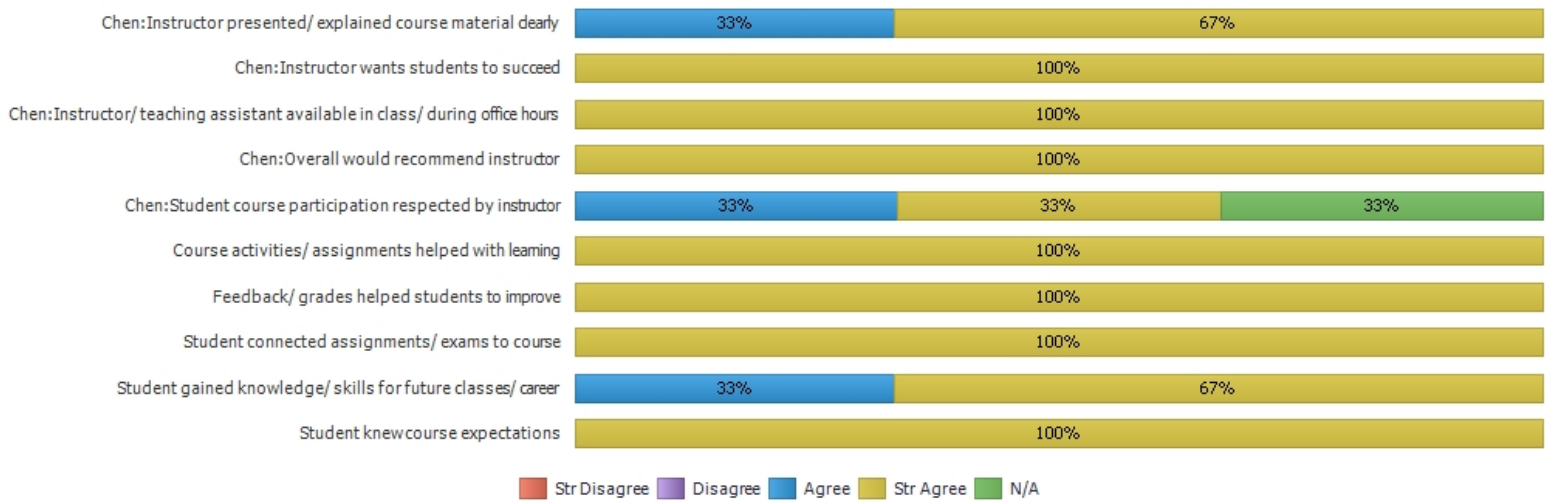
	Question Text	N	Median	Str Disagree	Disagree	Agree	Str Agree	N/A							
1	Student knew course expectations	3	4	0%	0%	0%	100%	0%							
2	Student connected assignments/ exams to course	3	4	0%	0%	0%	100%	0%							
3	Course activities/ assignments helped with learning	3	4	0%	0%	0%	100%	0%							
4	Feedback/ grades helped students to improve	3	4	0%	0%	0%	100%	0%							
5	Student gained knowledge/ skills for future classes/ career	3	4	0%	0%	33%	67%	0%							
				0 - 1	2 - 3	4 - 5	6 - 7	8 - 9	10 - 11	12 - 13	14 - 15	16 - 17	18 - 19	20+	
6	Time spent on course outside of class	3		0%	0%	67%	33%	0%	0%	0%	0%	0%	0%	0%	0%
				Str Disagree	Disagree	Agree	Str Agree	N/A							
10	Student course participation respected by instructor	3	3.5	0%	0%	33%	33%	33%							
11	Instructor/ teaching assistant available in class/ during office hours	3	4	0%	0%	0%	100%	0%							
12	Instructor presented/ explained course material clearly	3	4	0%	0%	33%	67%	0%							
13	Instructor wants students to succeed	3	4	0%	0%	0%	100%	0%							
14	Overall would recommend instructor	3	4	0%	0%	0%	100%	0%							

Text Responses
Question: What worked well in this class?
The term paper works very well and I enjoyed it a lot as I can learn the course and how to write papers at the same time. the homework help us understand the coding and practical aspect of the theories.
Question: What changes could improve this class? (Specific suggestions will be the most useful.)
N/A
Question: Please feel free to say more about your response to any items above or provide any additional feedback.
N/A

Question Averages



Distribution of Scores



**CSE 4693/6693 Intro to
Machine Learning:** my teaching
rating median is 4 for most questions,
which proved effectiveness in my new
course.



Zhiqian Chen

SP24

CSE-6693-501 Intro to Machine Learning

No. of responses = 5

Response Rate = 5 out of 8

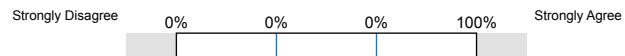
Survey Results

1. Student Course Survey

Mississippi State University welcomes student feedback that can be used to help improve teaching and learning in this course. Meaningful self-reflection on your experience in the course and a focus on your own learning (rather than general critiques or things you have heard) will provide the most useful and constructive feedback. Thank you for your participation.

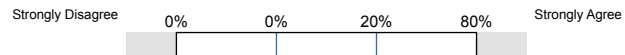
Choose 'Abstain/N.A.' if you do not wish to express an opinion or if the question does not apply to this course.

1.1) I knew what was expected of me in this class.



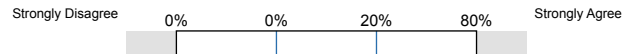
n=5
md=4
cf=100.00%

1.2) I understood how the assignments and/or exams connected with the learning objectives of the class.



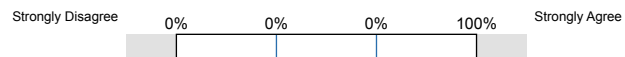
n=5
md=4
cf=100.00%

1.3) Instructional activities and assignments (such as lectures, discussions, demonstrations, tests and exams, papers, field studies, homework, projects, etc.) accomplished inside and outside of class helped me to learn.



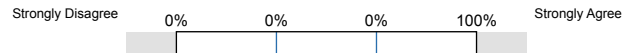
n=5
md=4
cf=100.00%

1.4) My participation in class was welcomed and respected by the instructor.



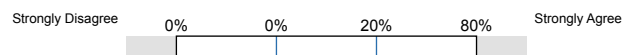
n=4
md=4
ab=1
cf=100.00%

1.5) If I had questions or needed help, the instructor or teaching assistant was available and responsive inside of class or during posted office hours and/or normal University operating hours.



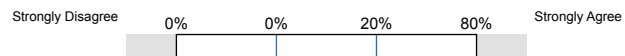
n=5
md=4
cf=100.00%

1.6) Feedback and grades on tests and assignments helped me to improve.



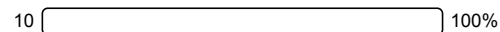
n=5
md=4
cf=100.00%

1.7) In this class, I have gained knowledge and skills that I can use in future classes, a prospective career, or other contexts in my life.



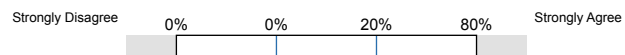
n=5
md=4
cf=100.00%

1.8) Outside of class time, approximately how much time each week did you spend engaging with the course content (reading, studying, completing assignments, etc.)? (hours)



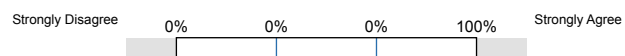
n=3

1.9) I felt that the instructor presented and explained the course material clearly.



n=5
md=4
cf=100.00%

1.10) I felt that the instructor wanted all of us to succeed.



n=5
md=4
cf=100.00%

1.11) Overall, I would recommend this instructor to other students if they wanted to learn this subject.

Strongly Disagree

0%

0%

20%

80%

Strongly Agree

n=5
md=4
cf=100.00%

2. Instructor Optional Questions

Choose 'Abstain/N.A.' if you do not wish to express an opinion or if the question does not apply to this course.

Comments Report

1. Student Course Survey

1.12) What worked well in this class?

■ Group work

■ In the Introduction to Machine Learning class, several aspects stood out positively:

Clarity of Objectives: The course effectively conveyed the main objectives of machine learning, ensuring that students understood the fundamental concepts and goals.
Engaging Assignments: The assignments provided an opportunity to delve deeper into practical aspects, encouraging students to conduct further research and apply what they learned in real-world scenarios. This hands-on approach likely fostered a deeper understanding of the material.

Balanced Approach: The course struck a balance between theoretical and practical components, which is often lacking in many bootcamp-style courses. By incorporating theoretical and mathematical aspects, it provided a comprehensive understanding of machine learning concepts, setting it apart from other programs.

Overall, the combination of clear objectives, engaging assignments, and a balanced approach between theory and practice contributed to a rewarding learning experience in the Introduction to Machine Learning class.

■ Professor had lecture slides, recorded lectures, and demo's in class along with in-class quizzes (slido). The hw's, tests, midterm, quizzes related to the lectures. Professor was accommodating and gave extensions if needed. Professor cares about student success and is accessible outside the regular class meeting times. The project was an opportunity to apply what you learned and also do something more creative and have more freedom vs. just taking a final exam.

■ The homework assignments allowed us to learn the most.

1.13) What changes could improve this class? (Specific suggestions will be the most useful.)

■ Clearer audio equipment for online students. Some Professors (not just Professor Chen) have strong accents and are hard to understand through a crappy microphone system.

■ I don't know if we can do more research tasks or assignments? ex. reading ML or AI papers and writing reviews on them or smaller tasks that build up into a real-world application rather than quizzes (ex. create your own ChatGPT or utilize ChatGPT's API), kinda of like stuff you would see in terms of assignments or projects in a Coursera project or course for AI or ML

■ I wanted to see more about models and less about probabilities. Despite them being an important part of machine learning, the time spent on them isn't as useful vs learning about various modeling techniques in a work or research environment.

■ Improving the class could involve addressing communication challenges. Addressing the challenge posed by communication barriers stemming from the instructor's accent, several strategies could be implemented to enhance the learning experience for all students. Providing prepared lecture materials, including pre-recorded lectures or notes with clear pronunciation of key terms, would offer students supplementary resources for better comprehension. Implementing a peer support system would allow students to collaborate and assist each other in understanding lecture content, leveraging shared experiences to overcome language barriers. Lastly, organizing regular feedback sessions would enable students to express concerns and offer suggestions for improvement, facilitating a proactive approach to addressing communication challenges in the class.

1.14) Please feel free to say more about your response to any items above or provide any additional feedback.

■ Professor was great, highly recommend him to anyone. I thought it was a great intro class for both undergrads and graduate students.



Zhiqian Chen

SP24
CSE-6693-01 Intro to Machine Learning
No. of responses = 11
Response Rate = 11 out of 15

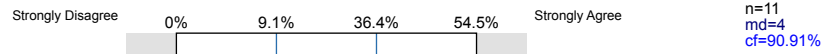
Survey Results

1. Student Course Survey

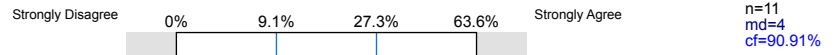
Mississippi State University welcomes student feedback that can be used to help improve teaching and learning in this course. Meaningful self-reflection on your experience in the course and a focus on your own learning (rather than general critiques or things you have heard) will provide the most useful and constructive feedback. Thank you for your participation.

Choose 'Abstain/N.A.' if you do not wish to express an opinion or if the question does not apply to this course.

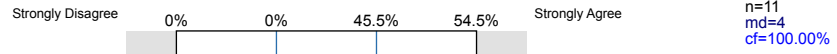
1.1) I knew what was expected of me in this class.



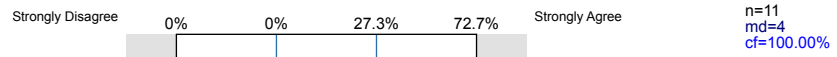
1.2) I understood how the assignments and/or exams connected with the learning objectives of the class.



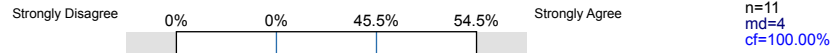
1.3) Instructional activities and assignments (such as lectures, discussions, demonstrations, tests and exams, papers, field studies, homework, projects, etc.) accomplished inside and outside of class helped me to learn.



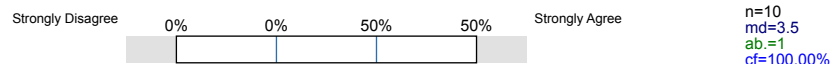
1.4) My participation in class was welcomed and respected by the instructor.



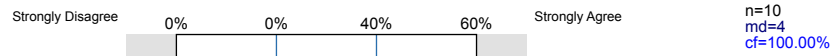
1.5) If I had questions or needed help, the instructor or teaching assistant was available and responsive inside of class or during posted office hours and/or normal University operating hours.



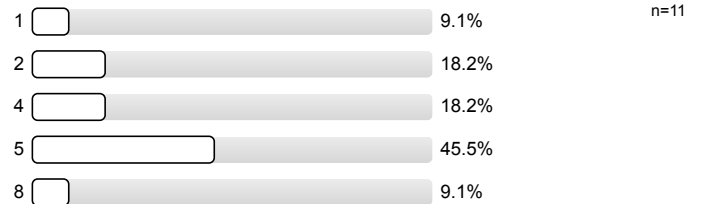
1.6) Feedback and grades on tests and assignments helped me to improve.



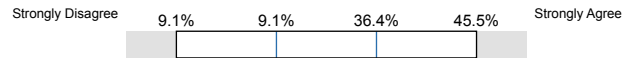
1.7) In this class, I have gained knowledge and skills that I can use in future classes, a prospective career, or other contexts in my life.



1.8) Outside of class time, approximately how much time each week did you spend engaging with the course content (reading, studying, completing assignments, etc.)? (hours)

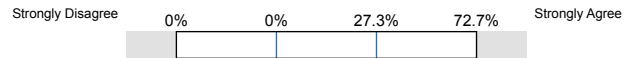


1.9) I felt that the instructor presented and explained the course material clearly.



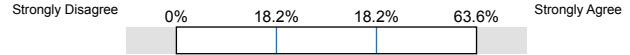
n=11
md=3
cf=81.82%

1.10) I felt that the instructor wanted all of us to succeed.



n=11
md=4
cf=100.00%

1.11) Overall, I would recommend this instructor to other students if they wanted to learn this subject.



n=11
md=4
cf=81.82%

2. Instructor Optional Questions

Choose 'Abstain/N.A.' if you do not wish to express an opinion or if the question does not apply to this course.

Comments Report

1. Student Course Survey

1.12) What worked well in this class?

- Good homework and quizzes. He cares a lot
- Homeworks and quizzes actually helped me learn material.
- Live quiz questions
- Multiple quizzes enhanced my skills
- Slide quizzes, structure of homework, quizzes
- The provided code samples were extremely helpful for tying the concepts we learned in class to genuine programming, and the written explanations and descriptions on the slides in class were very informative.

1.13) What changes could improve this class? (Specific suggestions will be the most useful.)

- I know it's an awkward topic, but his English is not the best. It has improved over the semester but it's still rough
- More engaging lectures, and more organized lecture slides. More homeworks that are manageable but also have us covering more in depth material. Less foundational review and more coverage of advanced topics.
- N/A
- Providing the students with the slides that were annotated with the in class notes would be very valuable for us, as it is often difficult to keep up with them in class.
- The implementation of the project is a little much for an introduction class. The project is a good idea for like a higher level machine learning course. Also, maybe 2 add more coding assignments that ease students into the topic and help in understanding before the high value homework like homework 2
- Uploading content of slides to be covered in class in Canvas, as slide content would keep changing for the same file.

1.14) Please feel free to say more about your response to any items above or provide any additional feedback.

- A thank you to Dr. Chen for a very informative and interesting class on various important machine learning topics.
- Felt like this semester was a lot slower and so we did not cover as much material as I had hoped (we didn't cover CNNs for a ML class) but I feel like I've gained a greater foundation of the math behind classical ML.
- Good



Zhiqian Chen

SP24

CSE-4693-01 Intro to Machine Learning

No. of responses = 4

Response Rate = 4 out of 18

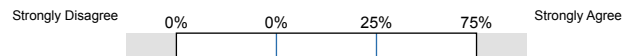
Survey Results

1. Student Course Survey

Mississippi State University welcomes student feedback that can be used to help improve teaching and learning in this course. Meaningful self-reflection on your experience in the course and a focus on your own learning (rather than general critiques or things you have heard) will provide the most useful and constructive feedback. Thank you for your participation.

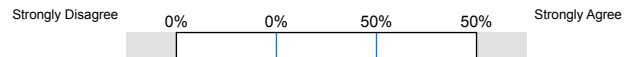
Choose 'Abstain/N.A.' if you do not wish to express an opinion or if the question does not apply to this course.

1.1) I knew what was expected of me in this class.



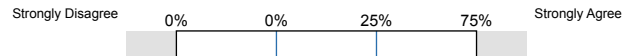
n=4
md=4
cf=100.00%

1.2) I understood how the assignments and/or exams connected with the learning objectives of the class.



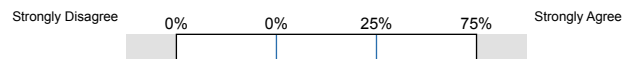
n=4
md=3.5
cf=100.00%

1.3) Instructional activities and assignments (such as lectures, discussions, demonstrations, tests and exams, papers, field studies, homework, projects, etc.) accomplished inside and outside of class helped me to learn.



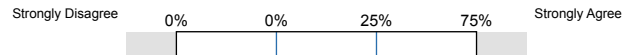
n=4
md=4
cf=100.00%

1.4) My participation in class was welcomed and respected by the instructor.



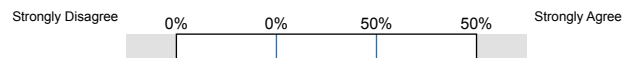
n=4
md=4
cf=100.00%

1.5) If I had questions or needed help, the instructor or teaching assistant was available and responsive inside of class or during posted office hours and/or normal University operating hours.



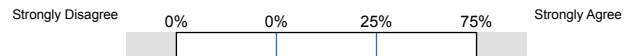
n=4
md=4
cf=100.00%

1.6) Feedback and grades on tests and assignments helped me to improve.



n=4
md=3.5
cf=100.00%

1.7) In this class, I have gained knowledge and skills that I can use in future classes, a prospective career, or other contexts in my life.



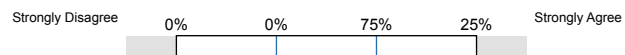
n=4
md=4
cf=100.00%

1.8) Outside of class time, approximately how much time each week did you spend engaging with the course content (reading, studying, completing assignments, etc.)? (hours)



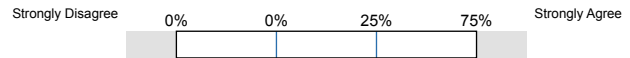
n=3

1.9) I felt that the instructor presented and explained the course material clearly.



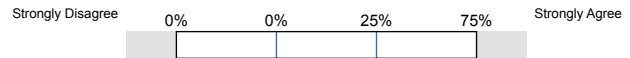
n=4
md=3
cf=100.00%

1.10) I felt that the instructor wanted all of us to succeed.



n=4
md=4
cf=100.00%

1.11) Overall, I would recommend this instructor to other students if they wanted to learn this subject.



n=4
md=4
cf=100.00%

2. Instructor Optional Questions

Choose 'Abstain/N.A.' if you do not wish to express an opinion or if the question does not apply to this course.

Comments Report

1. Student Course Survey

1.12) What worked well in this class?

- Its not what worked well in the course, but Dr.Chen was very considerate to my personal situation at home which was passed on to Dr.Rahimi, in which helped me maintain my grade in this course, and because of his understanding, that is what worked well
- The slides were more useful and informative than most classes I have had. The homework forced me to ensure I truly understood the information without being extremely difficult, although they were time-consuming.
- The slido polling during class worked really well in this class. It helped me check my understanding as we learned the material.

1.13) What changes could improve this class? (Specific suggestions will be the most useful.)

- I would suggest integrating small coding assignments every week or every other week. This assignments do not need to be large and complicated. They just need to be a small challenge that can be done in 30 minutes. It would prevent the huge learning curve for the midterm coding assignment. It will also allow students to translate what they are learning in class to code.

1.14) Please feel free to say more about your response to any items above or provide any additional feedback.

- Abstain/N.A.
- In one instance, my midterm was graded in canvas. Several hours later, my grade was lowered a letter grade because the professor (or grader, I am not sure) noticed an additional error. It was pretty disheartening and I would have much preferred if the grade was not put into canvas until the test was fully graded. Nonetheless, I would still recommend this class and instructor.

Descriptive Student Feedback As this is my most recent and best-prepared course, I will utilize a complete response to the first question (the other two questions are recommendations) from my last teaching in CSE 8673 Machine Learning as my objective evaluation.

What worked well in this class?

- the homework assignments were excellent
- the final project was also a great learning experience
- midterm was tough but good
- the sample code and lecture slides were also useful
- Dr. Chen is understanding (i.e. willing to extend due dates as a result of missed lectures) but has high expectations.
- Discussion to the assignment, test, and homework solutions worked very well.
- Dr. Chen Obviously cares about the class and wanted us to succeed. We did cover a lot of ground, enough that I can have a jumping off point when going further into projects where ML is central component Even though I wouldn't have fully understood the slides by themselves, when he explained them during lectures (especially with the annotations he made) I understood the material well. The assignments were also very helpful to me with understanding the material and being able to use the concepts practically.
- Explanation was very good
- Explanation with demo codes
- Explanation was very clear and easy to understand
- Good use of graphics to explain high-level concepts, I liked the code examples and working through algorithms (such as simplex) by hand before implementing them in code. I loved that you took the time to explain the correct solutions for each homework, quiz, and test after it was complete.
- It is nice that the assignments were open book because I do not have a strong math background. I think that learning some of the coding applications were helpful, too. Especially in a practical situation, being able to learn to use the APIs.
- It's very useful. I like it.
- The instructor was always on ground to assist the students.

and mid-term anonymous feedback for my current teaching in CSE 4693/6693 Intro to Machine Learning

What worked well in this class?

- Being able to look at the code demonstrations was helpful in understanding how to apply the equations in class.
- The quizzes seemed the most effective to me in helping me learn.
- I feel like that I am learning a fair amount from the slides.
- The class flow is generally nice.
- Providing code demos to students is very helpful, as is having the powerpoints available to read. Working through examples in class (via slido or just as a demonstration) is also very helpful for understanding and conceptualizing the topics.
- I really enjoy Slido as it helps clarify my concepts and keeps everyone engaged.
- Dr.Chen is a very nice person and has been understand given my situation.
- I found the project topics to be great for us to learn.
- the assignments as well as the quizzes help me find out what works well.
- The coding examples and related have made the actual lectures make significantly more sense.
- The in-class coding examples were very helpful. I also liked the midterm, I learned more from the midterm than anything else, it was a good assignment. However, I do wish that the notes contained more "step by step" instructions to follow. When I say "step by step" I do not mean giving us code because that would not teach us anything, I mean giving us the equation performed at each step and letting us work though creating the networks from scratch based off the equations.
- The quizzes and homework have been valuable learning experiences, as they provided some more understanding of the topics we've discussed and of the math-based knowledge we've required to move forward. The group project also works well, as it encourages us to cooperate with our group members for researching machine learning topics and models and eventually for designing our own model. Finally, the code examples and demonstrations in class are helpful as they help us to understand and apply the topics we cover to the real world.
- The class is super well structured! Well done!
- As, I am a beginner to ML course. The topics that I thought was interesting and covered the basics well.
- The sequence of topics covered in class, along with Slido questions helped me grasp the contents in an engaging manner. There are some days when there are no in-class Slido quizzes. Keeping them every day will help more.

What worked well in this class? (cont.)

- There are some examples provided in class which I find helpful and the slido do help when we are able to discuss and work the problem out afterwards.
- The quizzes really helped to understand the topics and the course structure is perfectly student friendly.
- I found the live demonstrations and demo code very useful, as well as the proofs for simplified gradient equations and other constant equations.
- Slido is extremely useful, I would suggest using it more for a more interactive approach to learning.
- Instructor demo sessions and math formula explanations with connection to real life applications were very helpful.
- Understanding and applying key machine learning concepts, algorithms, and techniques were the strengths in this class.
- Describing the math behind the machine learning concepts helped me understand them a lot better.
- I think providing code that demonstrated class concepts was exceptionally helpful.
- I like the format of the midterm and how the semester is very project-oriented toward the final project.
- The in-class slido questions work well in the class. I do not always get them right because it helps me check my understanding of the material. It allows me to know what I need to focus on when studying. I would enjoy doing more after every core concept is introduced.
- I think the code demonstrations are the most effective thing in this class.
- The slido has work well in class. It gave us an opportunity to apply the knowledge we just have learned to solve a problem. It helps us to gain deeper understanding of the subject.
- lecture slides with recorded lectures + code demo's quizzes HW's slido's to reinforce learning in-class while content is still fresh from current lecture or previous lectures professor answering questions promptly via e-mail (professor making himself available to his students)

Curriculum Development

During my time at MSU, I have designed and taught one graduate-level course. My goal has been to introduce students to things that may be valuable in the future, regardless of whether they choose jobs in academia or industry, and to impart knowledge that I find personally engaging. I designed the CSE 8990 Graph Machine Learning course. This topic of machine learning encompasses a variety of prominent graph learning models. Additionally, I developed a new course for beginners on machine learning: CSE 4693/6693 Introduction to Machine Learning, which complements the graduate-level course.

Sample Syllabi

In the following pages, I have provided the syllabi for three courses that I designed and taught.

CSE 4693/6693 — Introduction to Machine Learning

Spring 2025

Instructor: Dr. Zhiqian Chen
Email: zchen@cse.msstate.edu (*Do **not** use Canvas inbox*)
Time: Tue & Thu, 3:30–4:45 PM
Location: Rula Civil Eng. Building, Room 3030 (01 section)
Online: WebEx (synchronous) with recording (501 section)
Office Hours: Mon 3–4 PM (Butler 304, walk-in) | Online by appointment

Course Description

The course covers core principles of machine learning and practical applications across supervised and unsupervised learning. Assignments emphasize fundamental concepts and basic applications; a team course project enables deeper exploration of a focused area. By the end of the course, students will be able to understand and implement machine learning models to address common real-world challenges. The primary objective is to strengthen problem-solving skills and build a practical skill set for careers in IT and advanced research.

Tentative Schedule

Note: Schedule subject to change based on course progress.

Day	Date	Topic / Activity	Due / Notes
Thu	Jan 16	Course Introduction	Release: Pre-course Survey
Tue	Jan 21	Fundamental: ML task formulation; Classification/Regression	
Thu	Jan 23	Fundamental: ML task formulation; Classification/Regression	Pre-course Survey due
Tue	Jan 28	Fundamental: ML task formulation; Classification/Regression	
Thu	Jan 30	Fundamental: Analytic Methods for Regression; Quiz 1	Project Phase 1
Tue	Feb 04	Fundamental: Analytic Methods for Regression	
Thu	Feb 06	Fundamental: Gradient Methods for Regression	
Tue	Feb 11	Fundamental: Gradient Methods for Regression	
Thu	Feb 13	Fundamental: Logistic Method for Classification	Release: HW 1

Day	Date	Topic / Activity	Due / Notes
Tue	Feb 18	Fundamental: Logistic Method for Classification; Quiz 2	
Thu	Feb 20	Fundamental: Logistic Method for Classification	Project Phase 2
Tue	Feb 25	Probabilistic: Bayes Rule	
Thu	Feb 27	Probabilistic: Bayes Rule; Quiz 3	
Tue	Mar 04	Mid-term Project Presentations: Groups 1–8	HW 1 due
Thu	Mar 06	Mid-term Project Presentations: Groups 9–16	
Tue	Mar 11	No Class — Spring Break	
Thu	Mar 13	No Class — Spring Break	
Tue	Mar 18	Midterm Review	Project Phase 3
Thu	Mar 20	Midterm Exam	Release: Midterm Course Feedback
Tue	Mar 25	Probabilistic: Bayes Rule	
Thu	Mar 27	Probabilistic: Bayes Rule	Midterm Course Feedback due
Tue	Apr 01	Probabilistic: Naive Bayes Classification	
Thu	Apr 03	Probabilistic: Naive Bayes Classification; Quiz 4	HW 2 releases (Apr 7)
Tue	Apr 08	Probabilistic: Naive Bayes Classification	
Thu	Apr 10	Probabilistic: Gaussian Process / Bayesian Optimization	
Tue	Apr 15	Probabilistic: Gaussian Process / Bayesian Optimization	
Thu	Apr 17	Probabilistic: Gaussian Process / Bayesian Optimization; Quiz 5	HW 2 due (Apr 21)
Tue	Apr 22	Final Project Presentations 2–5	Project Phase 4
Thu	Apr 24	Final Project Presentations 1, 6–9	
Tue	Apr 29	Final Project Presentations 11–15 (prerecorded: 10, 16)	
Thu	May 01	No Class — Instructor travel	
Tue	May 06	No Class — Reserved for Final Report	
Thu	May 08	No Class — Reserved for Final Report; Project Phase 5 (final) due	

Reference Books

- Murphy, Kevin P. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.

- Murphy, Kevin P. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2022.
- Rafatirad, Setareh, Houman Homayoun, **Zhiqian Chen**, and Sai Manoj Pudukotai Dinakarrao. *Machine Learning for Computer Scientists and Data Analysts: From an Applied Perspective*. Springer, 2022.
- Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly, 2022.
- Müller, Andreas C., Sarah Guido. *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly, 2016.

Grading

In-class Quizzes ($5 \times 5\%$)	25%
Pre-course Survey	5%
Mid-term Course Survey	5%
Homework ($2 \times 10\%$)	20%
Mid-term Exam	10%
Term Paper (Phases 1–4: $4 \times 5\%$)	20%
Term Paper (Phase 5: Final)	15%
Novelty	5%
Implementation	5%
Writing	5%
Bonus: $\geq 80\%$ Slido attendance	+5% (final grade)

10 pt Normal Grade Mode

Letter	Range
A	100%–90%
B	<90%–80%
C	<80%–70%
D	<70%–60%
F	<60%

Examinations and Assignments

All submissions must include your name, student ID, and course number. Exams emphasize conceptual understanding. Make-up exams are only for college-approved reasons. Regrade requests must be written and submitted within three days of grade return.

Communication and Grade Returns

Grades are typically returned within two weeks. Quiz results are immediate; feedback follows in one week. Email responses are within 48 hours; include course/section, name, MSU ID, and group ID if applicable.

Late Submission Policy

Late submissions lose 1% per hour, up to a 50% maximum penalty.

AI Policy

Generative AI tools may be used when assigned, with attribution. Include:

“The author acknowledges the use of [Generative AI Tool Name] in the preparation of this assignment for [purpose].”

University Syllabus

The official *Mississippi State University Syllabus* applies to this course: <https://www.provost.msstate.edu/faculty-student-resources/university-syllabus>

CSE 8673 — Machine Learning

Fall 2024

Instructor: Dr. Zhiqian Chen
Email: zchen@cse.msstate.edu (*Do **not** use Canvas inbox*)
Time: Mon & Wed, 2:00–3:15 PM
Location: Rula Civil Eng. Building, Room 3030 (face-to-face section)
Online: WebEx (synchronous) with recording (online section only)
Office Hours: Mon 4–5 PM (walk-in) | Online by appointment

Course Description

The course covers fundamental principles of machine learning, including supervised and unsupervised learning, LDA, SVM, and optimization techniques. Assignments emphasize core concepts and applications; a team course project enables deeper exploration of specialized topics. By the end of the course, students will be able to understand and implement machine learning models for real-world challenges.

Learning Objectives

Students will gain a comprehensive understanding of foundational theories and advanced techniques in machine learning, deep learning, and emerging methods. The course project will provide an opportunity to apply these skills in real-world problem-solving. The aim is to strengthen problem-solving capabilities and prepare students for careers in IT and research.

Tentative Schedule

Note: Schedule subject to change based on course progress.

Day	Date	Topic / Activity	Due / Notes
Mon	Aug 21	Course Introduction	Release: Pre-course Survey
Mon	Aug 26	Supervised Learning: Linear Regression	[Intro] 11, 12, 8.4.2; [Applied] 3
Wed	Aug 28	Supervised Learning: Linear Regression	
Mon	Sep 2	Labor Day — No Class	

Day	Date	Topic / Activity	Due / Notes
Wed	Sep 4	Supervised Learning: Linear Regression	Project Phase 0: Team signup and topic selection
Mon	Sep 9	Supervised Learning: Linear Regression	
Wed	Sep 11	Supervised Learning: Logistic Regression; Quiz 1	[Intro] 10
Mon	Sep 16	Supervised Learning: Logistic Regression	
Wed	Sep 18	Supervised Learning: Logistic Regression	
Mon	Sep 23	Supervised Learning: LDA; Release HW 1	[Intro] 9; Project Phase 1: Literature Survey
Wed	Sep 25	Supervised Learning: LDA	
Mon	Sep 30	Supervised Learning: LDA	
Wed	Oct 2	Supervised Learning: SVM	[Intro] 17.3; [Hands-on] 5
Mon	Oct 7	Supervised Learning: SVM; Quiz 2	HW 1 due
Wed	Oct 9	Supervised Learning: SVM	Project Phase 2: Proposal
Mon	Oct 14	Unsupervised Learning: Clustering	[Intro] 21; [Applied] 4; [Hands-on] 9
Wed	Oct 16	Unsupervised Learning: Clustering	
Mon	Oct 21	Midterm Project Presentations (Groups 1–8, online; some prerecorded)	Instructor travel
Wed	Oct 23	Midterm Project Presentations (Groups 9–18, online; some prerecorded)	Instructor travel
Mon	Oct 28	No Class — Instructor travel; Quiz 3 (online)	
Wed	Oct 30	No Class — Instructor travel; Reading task for Quiz 4/HW 2	Mid-course survey
Mon	Nov 4	Unsupervised Learning: Clustering	
Wed	Nov 6	Unsupervised Learning: Clustering; Quiz 4; HW 2	[Intro] 8.5.3; [AP] 29; Online readings
Mon	Nov 11	Unsupervised Learning: Clustering	
Wed	Nov 13	Optimization: Linear/Integer Programming	[Intro] 8.5.5; <i>Intro to Algorithms</i> , Ch. 29
Mon	Nov 18	Optimization: Linear/Integer Programming; Quiz 5	

Day	Date	Topic / Activity	Due / Notes
Wed	Nov 20	Final Project Presentations (Groups 1–3, 5–6)	Project Phase 3; HW 2 due
Mon	Nov 25	Final Project Presentations (Groups 7–11)	
Wed	Nov 27	Thanksgiving — No Class	
Mon	Dec 2	Final Project Presentations (Groups 4, 12, 14–18)	
Tue	Dec 10	Reserved for Final Report	Project Phase 4: Final

Reference Books

- Murphy, Kevin P. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- Murphy, Kevin P. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2022.
- Rafatirad, Setareh, Houman Homayoun, **Zhiqian Chen**, and Sai Manoj Pudukotai Dinakarrao. *Machine Learning for Computer Scientists and Data Analysts: From an Applied Perspective*. Springer, 2022.
- Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly, 2022.
- Wolsey, Laurence A. *Integer Programming*. 2021.
- Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 2022.

Grading

In-class Quizzes ($5 \times 5\%$)	25%
Mid-term Course Survey	5%
Homework ($2 \times 10\%$)	20%
Term Paper (Phases 0–3)	15%
Phase 0 & 1 (2.5% each), Phase 2 & 3 (5% each)	
Term Paper (Phase 4: Final)	35%
Novelty	15%
Implementation	10%
Writing	10%
<i>10 pt Normal Grade Mode</i>	

Letter	Range
A	100%–90%
B	<90%–80%
C	<80%–70%
D	<70%–60%
F	<60%

Examinations and Assignments

All submissions must include your name, student ID, and course number. Exams emphasize conceptual understanding. Make-up exams are only for college-approved reasons. Regrade requests must be written and submitted within three days of grade return.

Communication and Grade Returns

Grades are typically returned within two weeks. Quiz results are immediate; feedback follows in one week. Email responses are within 48 hours; include course/section, name, MSU ID, and group ID if applicable.

Late Submission Policy

Late submissions lose 1% per hour, up to a 50% maximum penalty.

AI Policy

Generative AI tools may be used when assigned, with attribution. Include:

“The author acknowledges the use of [Generative AI Tool Name] in the preparation of this assignment for [purpose].”

University Syllabus

The official *Mississippi State University Syllabus* applies to this course: <https://www.provost.msstate.edu/faculty-student-resources/university-syllabus>

CSE 8990 — Special Topic: Graph Machine Learning

Fall 2024

Instructor: Dr. Zhiqian Chen
Email: zchen@cse.msstate.edu (*Do **not** use Canvas inbox*)
Time: Tue & Thu, 11:10 AM–12:25 PM (Aug. 19 – Dec. 7)
Location: WALKER 320 (face-to-face section)
Online: WebEx for distance students (recorded sessions)
Office Hours: Tue 3 PM, Wed 3 PM, Fri 10 AM | Online by appointment

Course Description and Learning Objectives

Graphs are used in many domains, including social networks, epidemiology, neuroscience, transportation, chemistry, computer networks, and power grids. With the rise of machine learning, graph-based methods have advanced rapidly. This course introduces concepts from graph signal processing and explores emerging tools in graph machine learning.

Topics include:

1. Graph fundamentals: adjacency and Laplacian matrices, spectral and algebraic analysis, graph signals, filtering, Fourier and wavelet transforms, convolution, modulation, and spectral analysis.
2. Graph applications: graph matching, directed graphs, clustering, and influence maximization.
3. Graph machine learning: spatial- and spectral-domain graph convolution, explainable AI for GNNs, and generative graph models.

Students will review core concepts (linear algebra, signal processing), study advanced methods, and apply these to real-world applications. Projects will be tailored to students' backgrounds. Prior knowledge of calculus, probability, linear algebra, and programming is expected.

Tentative Schedule

Note: Schedule subject to change based on course progress.

Day	Date	Topic / Activity	Due / Notes
Tue	Aug 19	Course Introduction	
Thu	Aug 24	Course Introduction	
Tue	Aug 26	Algebraic Graph Theory: Basic Concepts	
Thu	Aug 31	Algebraic Graph Theory: Basic Concepts	

Day	Date	Topic / Activity	Due / Notes
Tue	Sep 2	Spectral Graph Theory: Basic Concepts	
Thu	Sep 7	Spectral Graph Theory: Basic Concepts	
Tue	Sep 9	Spectral Graph Clustering	
Thu	Sep 14	Spectral Graph Clustering	
Tue	Sep 16	Graph Fourier and Wavelets	
Thu	Sep 21	Graph Fourier and Wavelets	HW 1 released
Tue	Sep 23	Graph Neural Networks: GCN	Project: Group Member & Topic Selection
Thu	Sep 28	Graph Neural Networks: GraphSAGE	
Tue	Sep 30	Graph Neural Networks: DeepWalk, Node2Vec	
Thu	Oct 5	Scientific Computation and Tools for Graph Analysis	HW 1 due
Tue	Oct 7	Fall Break — No Class	
Thu	Oct 12	Midterm; Course Feedback	
Tue	Oct 14	Midterm; Course Feedback	
Thu	Oct 19	HW1 & Midterm Solutions	Project Proposal
Tue	Oct 21	Graph Neural Networks: PageRank, DeepGCN	HW 2 released
Thu	Oct 26	GNNs: Sampling and Oversmoothing	
Tue	Oct 28	Graph Convolution: Function Approximation Perspective	
Thu	Nov 2	Graph Convolution: ODE/PDE Perspective	
Tue	Nov 4	Explainable AI for Graphs: GNNExplainer	HW 2 due
Thu	Nov 9	HW 2 Solutions	
Tue	Nov 11	Influence Maximization	In-class Quiz
Thu	Nov 16	Project Presentations (Part 1)	
Tue	Nov 18	Project Presentations (Part 2)	
Thu	Nov 23	Project Presentations (Part 3)	
Tue	Nov 25	Thanksgiving — No Class	
Thu	Nov 30	Course Review	
Tue	Dec 2	Reading Day — No Class	
Thu	Dec 7	No Class	Final Project Slides & Report Due

Grading

Participation (Quizzes, Attendance, Discussion, Surveys, Feedback)	10%
Homework ($2 \times 10\%$)	20%
Take-home Midterm	20%
Group Project	50%

Bonus: Exceptional project/paper presentations may earn up to an additional 10%.

10 pt Normal Grade Mode

Letter	Range
A	100%–90%
B	<90%–80%
C	<80%–70%
D	<70%–60%
F	<60%

Late Submission Policy

Assignments must be submitted by noon on the due date. Each student has 24 penalty-free hours for the semester. After that: 25% penalty for the first 12 hours, 50% for the first 24 hours. Weekends count toward late time.

Examinations and Assignments

All submissions must include name, student ID, and course number. Exams emphasize conceptual understanding and include multiple-choice and written questions. Make-ups are only for approved reasons. Regrade requests must be submitted in writing within three days.

Distance Students

Distance students attend via WebEx; attendance is recorded. They submit all work via Canvas on the same deadlines. Project presentations may be done remotely. Honor Code applies; no remote proctoring.

University Syllabus

The official *Mississippi State University Syllabus* applies to this course: <https://www.provost.msstate.edu/faculty-student-resources/university-syllabus>

Research Accomplishments

Contents

- Highlights
- Contention Resolution for Modern Systems
- Defending Open Systems
- Wireless Systems: Theory and Practice
- Competitive External Funding

Highlights

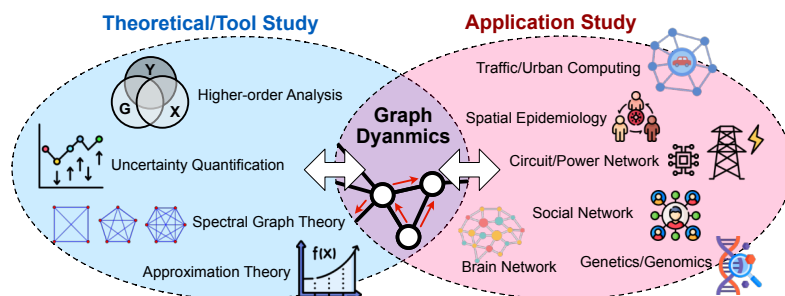
- Fundings - PI or Co-PI on successful competitive external funding, with a total of **\$10,653,443**, and I have secured personal funding credits totaling **\$1,122,628**:
- Publications - While at MSU, over 39 peer-reviewed publications at conference and journal venues that include AAAI, SIAM Data Mining, IEEE ICDM, ACM Computing Surveys, ACM SIG SPATIAL et, with an h-index of 17.

The rapid spread of COVID-19 has resulted in over 6 million fatalities globally since late 2019 and has led to an estimated economic loss of \$12.5 trillion as of 2024 . The pandemic has exposed significant vulnerabilities within modern networked systems, severely disrupting supply chains , inducing financial market volatility, and amplifying the spread of misinformation via social networks. Similarly, the global computer meltdown caused by a faulty CrowdStrike update in July 2024 brought airlines, hospitals, and businesses worldwide to a standstill, underscoring the cascading effects that can occur within interconnected systems. These events underscore the urgent necessity of understanding and managing the complex dynamics inherent in interconnected networks. Global challenges such as climate change, pandemics, and food security inherently involve interdependent networks, including those in transportation, finance, supply chains, ecology, and social networks, which collectively form the backbone of our societal and economic infrastructure. As our world becomes more interconnected, developing resilient and interpretable network models is crucial for addressing these challenges.

Research Interest: My research explores the fundamental principles of networked system dynamics by integrating advances in machine learning, network science, and dynamical systems theory. This work spans both theoretical innovations and practical applications, focusing on creating efficient, interpretable models that improve decision-making in real-world contexts.

1 Current Research and Achievements

Specifically, I study graph dynamics using spectral graph theory, uncertainty quantification, higher-order analysis, and physics-inspired methodologies. Currently, I have a particular interest on the behavior of coupled and interdependent networks, which are prevalent but have received limited development.



Research Outcome: On the (1) Theoretical Study: my work aims to develop new methodologies to better represent and analyze graphs and their dynamics. For instance, I have explored

spectral graph theory as a unified framework for representing complex networks, resulting in several paper , survey , tutorials in prestigious venues , and popular paper collection . Additionally, I investigate higher-order methods, such as Shapley values and Sobol indices , to understand combinatorial interactions among nodes, and I employ Bayesian optimization, Gaussian processes, and Kalman filtering to address efficiency challenges in graph-based problems. In parallel, I actively apply these theoretical advancements to (2) Application Research: My research has touched upon key social infrastructure networks and important biological networks, including transportation/urban system, spatial epidemiology, and genetic networks. Collaborations with ecologists, medical researchers, and engineers have also extended my methods to understand ecological networks, circuit dynamics , and brain signals in conditions such as ADHD and sleep disorders .

My research have successfully garnered support from a variety of esteemed external funding sources, amounting to a total of **\$10,653,443**, and I have secured personal funding credits totaling **\$1,122,628**:

1. From Fragmentation to Integration: Advancing Cross-Graph Dynamics in Interdependent Networks
Role: PI (sole investigator)
Funding Agency: National Science Foundation (NSF)
Program: CAREER
Award Number: 2443266
Amount Awarded: \$517,594 (100% credit)
Duration: 06/2025 – 05/2030
2. RCBP: III: Advancing Speech Detection: A Hybrid Approach Using Large Language Models and Graph Neural Networks
Role: Co-PI / University PI
Funding Agency: National Science Foundation (NSF)
Program: CISE MSI
Award Number: 2431176
Amount Awarded: \$400,000 (33% credit, MS State share \$259,948)
Duration: 01/2025 – 12/2026
3. Planning-C: Synergistic Graph Flow Analytics: An Integrated Infrastructure for Bridging Complexity, Fragmentation, and Interdisciplinary Gaps
Role: PI
Funding Agency: National Science Foundation (NSF)
Program: CIRC
Award Number: 2345921
Amount Awarded: \$99,998 (60% credit)
Duration: 07/2024 – 06/2025
4. Interpretable Influence Propagating and Blocking on Graphs
Role: PI (sole investigator)
Funding Agency: National Science Foundation (NSF)

Program: CRII

Award Number: 2153369

Amount Awarded: \$174,004 (100% credit)

Duration: 05/2022 – 11/2024

5. Developing Detection and Modeling Tools for the Geospatial and Environmental Epidemiology of Animal Disease

Role: Co-PI

Funding Agency: USDA Agricultural Research Service (ARS)

Award Number: 58-6064-3-017

Amount Awarded: \$3,073,602 (5% credit)

Duration: 10/2023 – 08/2028

6. Advancing Agricultural Research through High Performance Computing

Role: Co-PI

Funding Agency: USDA Agricultural Research Service (ARS)

Award Number: 58-0200-0-002

Amount Awarded: \$5,690,689 (5% credit)

Duration: 10/2022 – 08/2024

7. Learning to Create Intelligent Solutions with Machine Learning and Computer Vision: A Pathway to AI Careers for Diverse High School Students

Role: Co-PI

Funding Agency: National Science Foundation (NSF)

Program: EDU ITEST

Award Number: 2342574

Amount Awarded: \$1,192,951 (25% credit)

Duration: 09/2024 – 08/2027

8. REU Supplementary

Role: PI

Funding Agency: National Science Foundation (NSF)

Program: III

Amount Awarded: \$14,000 (100% credit)

Duration: 11/2023 – 11/2024

9. Summer Internship Project

Role: PI

Funding Agency: USDA Agricultural Research Service (ARS)

Amount Awarded: \$8,199 (100% credit)

Duration: 05/2024 – 04/2025

Additional, I have received internal fundings for various sources:

1. Otilie Schillig Special Teaching Award

Role: PI

Funding Agency: Center for Teaching and Learning (CTL)
Amount Awarded: \$2,994.50 (100% credit)
Duration: 2025-2026

2. Global Development Seed Grant Award

Role: PI

Funding Agency: International Institute
Amount Awarded: \$6,000 (100% credit)
Duration: 2024

3. Working Group in Graph AI

Role: PI

Funding Agency: Bagley College of Engineering
Amount Awarded: \$4,100 (100% credit)
Duration: 2023-2024

4. Undergraduate Research Program

Role: PI / Co-PI

Funding Agency: Office of Research and Economic Development (ORED)
Amount Awarded: 2022/2024/2025 (Sole PI, \$2,000), 2023 (Co-PI, \$1,500)
Duration: 2022 – 2025

My primary area of research since joining Mississippi State University focuses on dynamic graphs and their integration with machine learning, alongside their various applications.

Research Topic: Understand Graph Neural Network

Historically, deep learning methodologies like convolutional neural networks (CNNs) and recurrent neural networks (RNNs) have struggled with non-Euclidean challenges presented by graph-structured data. This limitation has spurred a growing interest within the machine learning community towards graph modeling. In response, an extensive array of graph neural network (GNN) models, grounded in diverse theoretical underpinnings, has been developed. One notable challenge, however, is the absence of a unified theoretical framework capable of encompassing a broad spectrum of these methodologies. The objective of this research is to establish a comprehensive theoretical framework for generalizing GNNs. This framework aims to facilitate a direct means of rethinking, comparing, and benchmarking all GNN models from a theoretical standpoint, thereby offering a platform for full comprehension and comparison among existing models. In the preliminary phase of my investigation, I concentrated on analyzing GNNs predicated upon spatial and spectral data, incorporating the breadth of GNNs within these two significant domains, which host the majority of popular GNN models. Leveraging spectral graph theory, I discerned that these domains are analytically comparable, thereby achieving a unification of the two. Furthermore, I engaged in studying approximation theory to elucidate the most prevalent GNNs. My findings reveal that each GNN model can be characterized by linear, polynomial, or rational approximations, each presenting unique theoretical considerations in terms of accuracy and efficiency. Moreover, I demonstrated the utility of the proposed model in elucidating well-established phenomena like over-smoothing and graph sampling.

Graph machine learning is increasingly becoming a pivotal component in the field of artificial intelligence, with applications that span the spreading of epidemic diseases, drug discovery, material science, social network analysis, and neuroscience. This research holds significant value in elucidating the operational mechanisms of both existing and emerging graph machine learning models. The framework introduced in this study lays a robust theoretical foundation that encompasses the principal categories of contemporary graph neural networks (GNNs) and establishes profound linkages among various methodologies. As a result, disparate models can be assimilated within this unified framework, enabling a direct comparison of their characteristics, including computational complexity and accuracy. This comparative capability equips algorithm designers with the insight to predict potential model behaviors and set realistic expectations during empirical data experiments. Furthermore, it provides strategic guidance for algorithm development, particularly in navigating the trade-offs between different computational metrics.

Summary of the Implementation/Influence of the Work:

- Zhiqian Chen, Feng Chen, Rongjie Lai, Xuchao Zhang and Chang-Tien Lu, "Rational Neural Networks for Approximating Graph Convolution Operator on Jump Discontinuities," 2018 IEEE International Conference on Data Mining (ICDM), 2018, pp. 59-68, doi: 10.1109/ICDM.2018.00021. (Combined Citation 24)
- Chen, Zhiqian, Fanglan Chen, Lei Zhang, Taoran Ji, Kaiqun Fu, Liang Zhao, Feng Chen, Lingfei Wu, Charu Aggarwal, and Chang-Tien Lu. "Bridging the gap between spatial and spectral domains: A unified framework for graph neural networks." *ACM Computing Surveys* 56, no. 5 (2023): 1-42. (Combined Citation 55)
- **[Textbook]** Rafatirad, Setareh, Houman Homayoun, Zhiqian Chen, and Sai Manoj Pudukotai Dinakarrao. *Machine Learning for Computer Scientists and Data Analysts: From an Applied Perspective*. Springer, 2022.
- **[Tutorial]** CVPR 24' Tutorial - Unifying Spectral and Spatial Graph Neural Networks

Our work has inspired several works that received over 60 citations:

Our paper serves as an important reference because, as stated in the conclusion section: "Our update is similar to the PageRank and ARMA models proposed in [16] and [18], which are known to approximate a rational filter on the graph [33]." Our research ([33]) provides vital context and identifies the underlying computation of significant studies that are similar to this referencing paper.

- Spinelli, Indro, Simone Scardapane, and Aurelio Uncini. "Adaptive propagation graph convolutional network." *IEEE Transactions on Neural Networks and Learning Systems* 32, no. 10 (2020): 4755-4760. (Citation 75)

This survey paper is published in top 1 survey journal in computer science (i.e., computing survey), and it cites our paper as the ONLY paper to present convolution-based graph neural network. (see [27] in section 3.2):

- Abadal, Sergi, Akshay Jain, Robert Guirado, Jorge López-Alonso, and Eduard Alarcón. "Computing graph neural networks: A survey from algorithms to accelerators." ACM Computing Surveys (CSUR) 54, no. 9 (2021): 1-38. (Citation 99)

This paper cites our paper as a closely related work, since our work directly discusses "rational neural network", which is also the main topic of this. In this paper: "In a closely related work, Chen et al. [8] propose high-degree rational activation functions in a neural network, which have benefits in terms of approximation power."

- Boullé N, Nakatsukasa Y, Townsend A. Rational neural networks. Advances in Neural Information Processing Systems. 2020;33:14243-53. (Citation 67)

Some new papers are also inspired by our papers

As a top journal in power grid, this paper cites our paper as the only paper to introduce graph neural network, and graph neural network is the major technical component to address event identification (see section III, my paper is cited as [21]).

- Yuan, Yuxuan, Zhaoyu Wang, and Yanchao Wang. "Learning Latent Interactions for Event Identification via Graph Neural Networks and PMU Data." IEEE Transactions on Power Systems (2022).

As a top conference paper in machine learning, our paper is cited as one reference to introduce graph neural network.

- Chen, Yuzhou, Baris Coskunuzer, and Yulia Gel. "Topological Relational Learning on Graphs." Advances in Neural Information Processing Systems 34 (2021).

Our paper provide the major reference in introducing graph neural network (see section 4.2, my paper is mentioned two times as [12])

- Shen, Yun, Xinlei He, Yufei Han, and Yang Zhang. "Model Stealing Attacks Against Inductive Graph Neural Networks." In SP 2022-43rd IEEE Symposium on Security and Privacy. 2022.

We have also received email communication from researchers from Editors of IEEE Transactions on Neural Networks and Learning Systems. who have asked for potential collaboration; Researcher from University of Illinois Urbana Champaign who discuss how their work can be fit into our

framework; Researcher from MIT, just express application and point out an author a typographical error.

Research Topic: Dynamic Graph

Given the profound impact this field has on society, the interpretability of network dynamics becomes crucial for earning the trust of practitioners and for integration into their routine operations. Analogous to the trajectory seen in classic machine learning models, network dynamic models are evolving towards complexities that render them nearly inscrutable to human operators. Consequently, the development of interpretable models, especially in the domain of influence modeling and control, is imperative to bridge the widening chasm between data scientists and practitioners. My exploration into dynamics disentangling through the lens of spectral graph theory and graph neural networks unveiled patterns intrinsically linked to network structure, as well as other dynamics within the same network. This inquiry was driven by the essential understanding of network dynamics. It has been determined that consistent dynamics over an extended timeframe serve as a reliable indicator for future predictions within the network's topology.

In my research, I explored optimal representations of network dynamics, as documented in publications for AAAI 22 and SDM 22, focusing on the analysis of traffic flow through graph wavelet and graph Fourier techniques. The theoretical and practical differences between these methods were scrutinized, particularly in terms of prediction capabilities. It was demonstrated that the graph wavelet approach offers theoretical and practical advantages, especially in scenarios involving abrupt signal changes. My investigations have revealed that graph neural networks (GNNs) adeptly model the network structure. Further, my study on Kalman filtering, presented at ACM SIGSPATIAL 20, along with the introduction of a GNN model designed to amalgamate information from multiple network flows, indicates that this novel combination markedly surpasses the performance of any single source model. This research aimed to effectively integrate multiple network dynamics. Following research presented at ASONAM 2021, I discovered that the predictive capacity of two dynamics within the same network largely stems from their mutual influence, affected by the same underlying network structures. Another aspect of my work, featured in AAAI 22, involved employing global sensitivity analysis to unravel the influence of individual seeds and their complex interactions within networks, investigating the nexus between dynamics propagation and its seed nodes. Embarking on a new research avenue, I ventured into graph Bayesian Optimization, a domain introduced in AAAI 24, which adeptly manages uncertainty in target applications such as influence maximization and source localization.

Summary of the Significance of the Work Through the development of neural network models with promising performance and high interpretability, this study bridges the gap between network flow and machine learning. Using machine learning, our research is able to gain a deeper understanding of how a network-wide information flow propagates, as well as identify the diffusion attributes, paths, patterns, and its complex relationship with topology and seed selection. This project will have a wide range of applications, such as information propagation on social networks, viral marketing, epidemiology, transportation, and power networks. Controlling fake news, containing the spread of disease, and detecting anomalies in the power grid are significant applications. In addition, the planned toolset could support a vast array of applications, such as public health and infrastructure security.

Resulting Publication(s) 8 peer-reviewed full papers:

1. Yuan, Zirui, Minglai Shao, and Zhiqian Chen. 2024. "Graph Bayesian Optimization for Multiplex Influence Maximization". Proceedings of the AAAI Conference on Artificial Intelligence 38 (20):22475-83. <https://doi.org/10.1609/aaai.v38i20.30255>.
2. Zhang, Zonghan, Zijian Zhang, and Zhiqian Chen. 2024. "Multiple-Source Localization from a Single-Snapshot Observation Using Graph Bayesian Optimization". Proceedings of the AAAI Conference on Artificial Intelligence 38 (20):22538-46. <https://doi.org/10.1609/aaai.v38i20.30262>.
3. Taoran Ji, Nathan Self, Kaiqun Fu, Zhiqian Chen, Naren Ramakrishnan, and Chang-Tien Lu. 2024. Citation Forecasting with Multi-Context Attention-Aided Dependency Modeling. ACM Trans. Knowl. Discov. Data Just Accepted (February 2024). <https://doi.org/10.1145/3649140>
4. L. Zhang, Q. Zhang, Z. Chen, Y. Sun, C. -T. Lu and L. Zhao, "Infinitely Deep Graph Transformation Networks," 2023 IEEE International Conference on Data Mining (ICDM), Shanghai, China, 2023, pp. 778-787, doi: 10.1109/ICDM58522.2023.00087.
5. Chen, Fanglan, Zhiqian Chen, Subhodip Biswas, Shuo Lei, Naren Ramakrishnan, and Chang-Tien Lu. "Graph convolutional networks with Kalman filtering for traffic prediction." In Proceedings of the 28th International Conference on Advances in Geographic Information Systems, pp. 135-138. 2020.
6. Zhiqian Chen, Gaurav Kolhe, Setareh Rafatirad, Chang-Tien Lu, Sai Manoj PD, Houman Homayoun, and Liang Zhao. "Estimating the circuit de-obfuscation runtime based on graph deep learning." In 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 358-363. IEEE, 2020.
7. Roy, Padmaksha, Shailik Sarkar, Subhodip Biswas, Fanglan Chen, Zhiqian Chen, Naren Ramakrishnan, and Chang-Tien Lu. "Deep diffusion-based forecasting of COVID-19 by incorporating network-level mobility information." In Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM 21), pp. 168-175. 2021.
8. Meng, Guangyu, Qisheng Jiang, Kaiqun Fu, Beiyu Lin, Chang-Tien Lu, and Zhqian Chen. "Early Forecasting of the Impact of Traffic Accidents Using a Single Shot Observation." In Proceedings of the 2022 SIAM International Conference on Data Mining (SDM), pp. 100-108. Society for Industrial and Applied Mathematics, 2022. (Acceptance rate: 27.8%)

Summary of the Implementation/Influence of the Work

One of my works has inspired several works that received 35 citations:

- Chen, Fanglan, Zhiqian Chen, Subhodip Biswas, Shuo Lei, Naren Ramakrishnan, and Chang-Tien Lu. "Graph convolutional networks with Kalman filtering for traffic prediction." In Proceedings of the 28th International Conference on Advances in Geographic Information Systems, pp. 135-138. 2020.

Among citing papers, one paper has been received 80 citations:

- Jiang, Weiwei, and Jiayun Luo. "Graph neural network for traffic forecasting: A survey." arXiv preprint arXiv:2101.11174 (2021). (citation 80)

In this survey paper, our paper is cited as an important example to address heterogeneous data (see first citation in section 6.1 and 6.1.1)

My work is cited as the only work as "combined prediction" with LSTM-CTP (see section 2.2 of the paper below)

- Afrin, Tanzina, and Nita Yodo. "A Long Short-Term Memory-based correlated traffic data prediction framework." Knowledge-Based Systems 237 (2022): 107755.

Our work is used as stat-of-the-art baseline method and is compared in the experiment.

- Park, Jin-Hwi, Young-Jae Park, Junoh Lee, and Hae-Gon Jeon. "DevianceNet: Learning to Predict Deviance from A Large-scale Geo-tagged Dataset." (2022).

Also, two papers used our method as stat-of-the-art baseline and compared in their experiments.

- Park, Jin-Hwi, Young-Jae Park, Junoh Lee, and Hae-Gon Jeon. "DevianceNet: Learning to Predict Deviance from A Large-scale Geo-tagged Dataset." AAAI (2022).
- Yidan, Sun, He Peilan, Jiang Guiyuan, and Siew-Kei Lam. "Learning Congestion Propagation Behaviors for Traffic Prediction." In 2021 IEEE International Intelligent Transportation Systems Conference (ITSC), pp. 2175-2180. IEEE, 2021.

Another work has been accepted by a top conference in mathematics with 27.8% acceptance rate.

- Meng, Guangyu, Qisheng Jiang, Kaiqun Fu, Beiyu Lin, Chang-Tien Lu, and Zhiqian Chen. "Early Forecasting of the Impact of Traffic Accidents Using a Single Shot Observation." In Proceedings of the 2022 SIAM International Conference on Data Mining (SDM), pp. 100-108. Society for Industrial and Applied Mathematics, 2022. (Acceptance rate: 27.8%)

One work in circuit network application published in top conference in hardware security:

- Zhiqian Chen, Gaurav Kolhe, Setareh Rafatirad, Chang-Tien Lu, Sai Manoj PD, Houman Homayoun, and Liang Zhao. "Estimating the circuit de-obfuscation runtime based on graph deep learning." In 2020 Design, Automation Test in Europe Conference Exhibition (DATE), pp. 358-363. IEEE, 2020.

Which has been cited by selected papers:

My paper is cited as one of the two representative works that use graph neural network in circuit application. (see section 7)

- Sisejkovic, Dominik, Farhad Merchant, Lennart M. Reimann, Harshit Srivastava, Ahmed Hallawa, and Rainer Leupers. "Challenging the security of logic locking schemes in the era of deep learning: A neuroevolutionary approach." ACM Journal on Emerging Technologies in Computing Systems (JETC) 17, no. 3 (2021): 1-26. (Citation 39)

My paper is cited as one of the 5 representatives in security analysis. (see table 1)

- Asif, Nurul A., Yeahia Sarker, Ripon K. Chakraborty, Michael J. Ryan, Md Hafiz Ahamed, Dip K. Saha, Faisal R. Badal et al. "Graph neural network: A comprehensive review on non-euclidean space." IEEE Access (2021). (Citation 44)

Mentoring

Mentoring students is an immensely rewarding aspect of my role as a faculty member at Mississippi State University (MSU). The opportunity to contribute to the growth and development of our students has been profoundly fulfilling.

At present, I have the privilege of guiding 3 graduate students through his **doctoral and master** journey at MSU, with all of them are fully funded as research assistant by research projects. The main academic journey is marked by significant achievements, including presenting a comprehensive paper at the prestigious Society for Industrial and Applied Mathematics (SIAM) Data Mining (SDM 2023) conference and at the AAAI 2024 conference. In addition, I mentor 7 online PhD students, fostering our connection through regular meetings and discussions. My mentorship extends to the **undergraduate** level, where I oversee a research initiative funded by the NSF REU (as PI), involving four students. Throughout ORED UG program, I have been also involve several undergraduate students, making me supervise 10 undergraduate students in total. We have another NSF EDU/ITEST project, in collaboration with the College of Education, aims to educate **high school students** about AI. I served as research advisor for a high school student at Columbus High School at Georgia, and I have been serving as a research advisor for a high school student at the Mississippi School for Mathematics and Science.

Service Accomplishments

Contents

- Highlights
- NSF Panel Service
- Workshop Organization
 - Program for the Workshop on Security of Permissionless Systems
 - Program for the Workshop on Competitive Economics of Cybersecurity
- Broader External Service
- High School Involvement
- Student Advising
- National Security Agency NCAE Designation

Highlights

- ACTED AS CO-CHAIR FOR THE IEEE BIG DATA CUP AT IEEE BIG DATA 2024, CO-CHAIR FOR IEEE ICDM POSTER SESSION.
- CONTRIBUTED TO NSF REVIEW PANELS IN THE CISE DIRECTORATE IN 2021, 2023, AND 2024, DEMONSTRATING EXPERTISE AND RECOGNITION IN THE FIELD.
- DELIVERED INFLUENTIAL TUTORIAL TALKS AT SIGNIFICANT CONFERENCES, INCLUDING SIAM DATA MINING 2020 AND CVPR 2024, CONTRIBUTING TO THE BROADER ACADEMIC COMMUNITY.
- ENGAGED IN PEER REVIEW FOR TOP-TIER CONFERENCES AND JOURNALS IN MACHINE LEARNING, AI, AND DATA MINING FROM 2019 TO 2025, UPHOLDING SCIENTIFIC STANDARDS.

Throughout my tenure at Mississippi State University, I have had the esteemed privilege of contributing significantly to both the broader academic community and my department. Below is a delineation of key services rendered, with a comprehensive account available in my curriculum vitae.

Program Committee Leadership I have had the honor of serving as the co-chair for the prestigious Big Data Cup at the IEEE Big Data 2024 conference and IEEE ICDM 2025. This role has allowed me to foster innovation and guide discourse in the realm of big data.

National Science Foundation Panel Service My expertise has been recognized through my service on three National Science Foundation (NSF) review panels within the Computer and Information Science and Engineering (CISE) directorate in the years 2021, 2023, and 2024.

Tutorials at Premier Conferences I have disseminated knowledge through tutorial talks at leading conferences, including a presentation at SIAM Data Mining 2020 alongside esteemed colleagues¹, and a tutorial at CVPR 2024².

Peer Review Contributions My commitment to the advancement of machine learning, artificial intelligence, and data mining has been evident through my rigorous review service for top-tier conferences and journals, adhering to the rankings provided by CSRankings³. My review service includes contributions to:

- International and premier conferences such as ICML, ICLR, NeurIPS, AAI, IJCAI, and KDD between 2020 and 2024.
- Esteemed journals like Knowledge-Based Systems, IEEE TKDE, ACM TKDD, Neurocomputing, and Scientific Reports by Nature.

¹https://beiyulincs.github.io/pub/sdm_tutorial_21.html

²<https://xgraph.team/course/cvpr24/>

³<https://csrankings.org>

Co-Advisor for MSU AI Club Since 2023, I have served as a co-advisor for the MSU AI Club, nurturing a vibrant community passionate about artificial intelligence.

Discovery Day I have participated and hosted a short presentation session for K-12 student in discovery day at MSU in 2023 and 2024.

Departmental Service My service to the department has been multifaceted, including significant contributions to the graduate committee since my arrival in 2020 and participation in the faculty search committee in 2022.

Letters of Recommendation: External Re- viewers

Example Letter to Reviewers

Response/Rebuttal to Recommendation

Selected Publications

Contents

- Overview
- Selected Publication 1
BRIDGING THE GAP BETWEEN SPATIAL AND SPECTRAL DOMAINS: A UNIFIED FRAME-
WORK FOR GRAPH NEURAL NETWORKS
ACM Computing Surveys 2023
A technical survey examining the impact of spectral analysis on graph learning
- Selected Publication 2
UNDERSTANDING INFLUENCE MAXIMIZATION VIA HIGHER-ORDER DECOMPOSITION
SIAM Data Mining 2023
A new method for analyzing hidden higher-order node relationships as seeds in event propagation
- Selected Publication 3
ESTIMATING THE CIRCUIT DE-OBFUSCATION RUNTIME BASED ON GRAPH DEEP LEARN-
ING
DATE 2020
A technical method to help hardware security
- Selected Publication 4
LEARNING TO FUSE MUSIC GENRES WITH GENERATIVE ADVERSARIAL DUAL LEARNING
ICDM 2018
A method for generating new music

Overview

I am especially proud of the following 4 publications, and these are appended to my dossier.

- **Selected Publication 1**

BRIDGING THE GAP BETWEEN SPATIAL AND SPECTRAL DOMAINS: A UNIFIED FRAMEWORK FOR GRAPH NEURAL NETWORKS. ACM Computing Surveys 2023

ACM Computing Surveys is a top-tier, A*-rated journal with a record-high 2025 impact factor of 28.0—ranking first in its field—supported by a strong upward trend from 16.6 in 2022, a Q1 SCImago rank (5.8), and an h-index of 232, reflecting its sustained influence and scholarly prestige. This paper is highest citation (102) among my first-authored papers, which is also one of my important research in understanding graph neural networks.

- **Selected Publication 2**

UNDERSTANDING INFLUENCE MAXIMIZATION VIA HIGHER-ORDER DECOMPOSITION, SIAM Data Mining 2023

SIAM Data Mining is a A-rated conference by CORE with a solid citation profile (h-index $\approx$ 12–14) and impact factor around 2.3–2.6, positioning it just behind premier venues like KDD in scholarly influence. This paper open our research in nodes' higher order interaction in spreading effects, which is one of our important research at Mississippi State University.

- **Selected Publication 3**

ESTIMATING THE CIRCUIT DE-OBFUSCATION RUNTIME BASED ON GRAPH DEEP LEARNING, DATE 2020

DATE (Design, Automation & Test in Europe) is a premier, long-standing (since 1998) European flagship conference in electronic design automation—CORE-ranked B—and draws over 1,200 participants annually, frequently showcasing cutting-edge work in hardware security primitives alongside broader embedded systems and EDA innovations. This paper exemplifies our collaborative efforts at the intersection of artificial intelligence and a specific application domain, namely hardware security. It serves as a model for future interdisciplinary collaborations with additional application domains.

- **Selected Publication 4**

LEARNING TO FUSE MUSIC GENRES WITH GENERATIVE ADVERSARIAL DUAL LEARNING, ICDM 2018

ICDM is a globally premier, flagship-level data mining conference (CORE A*) with highly competitive acceptance rates around 9–10%, an h-index of approximately 133, and a Scopus-based impact score near 2.9 — reflecting sustained quality and influence in the field. This paper exemplifies my personal interest in music and my collaboration with the Music Technology Center at Georgia Tech. This endeavor will be pivotal in pioneering new research directions, even as my

primary focus remains on graph learning.

Selected Publication 1



Bridging the Gap between Spatial and Spectral Domains: A Unified Framework for Graph Neural Networks

ZHIQIAN CHEN, Computer Science and Engineering, Mississippi State University, U.S.A

FANGLAN CHEN and LEI ZHANG, Computer Science, Virginia Tech, U.S.A

TAORAN JI, Computer Science, Texas A&M University - Corpus Christi

KAIQUN FU, Electrical Engineering & Computer Science, South Dakota State University, U.S.A

LIANG ZHAO, Computer Science, Emory University, U.S.A

FENG CHEN, Computer Science, The University of Texas at Dallas, U.S.A

LINGFEI WU, Pinterest, U.S.A

CHARU AGGARWAL, IBM T. J. Watson Research Center, U.S.A

CHANG-TIEN LU, Computer Science, Virginia Tech, U.S.A

Deep learning's performance has been extensively recognized recently. Graph neural networks (GNNs) are designed to deal with graph-structural data that classical deep learning does not easily manage. Since most GNNs were created using distinct theories, direct comparisons are impossible. Prior research has primarily concentrated on categorizing existing models, with little attention paid to their intrinsic connections. The purpose of this study is to establish a unified framework that integrates GNNs based on spectral graph and approximation theory. The framework incorporates a strong integration between spatial- and spectral-based GNNs while tightly associating approaches that exist within each respective domain.

CCS Concepts: • **Mathematics of computing** → **Spectra of graphs**; *Graph algorithms*; **Approximation algorithms**; • **Computing methodologies** → *Spectral methods*;

Additional Key Words and Phrases: Deep learning, graph neural networks, approximation theory, spectral graph theory, graph learning

126

This work was funded by the NSF IIS award # 2153369.

Authors' addresses: Z. Chen, Dept. of Computer Science and Engineering, Mississippi State University, Butler Hall, MS State, MS, 39762; e-mail: zchen@cse.msstate.edu; F. Chen, L. Zhang, and C.-T. Lu, Computer Science, Virginia Tech, 7054 Haycock Road, Falls Church, VA, 22043; e-mails: {[fanglanc](mailto:fanglanc@vt.edu), [zhanglei](mailto:zhanglei@vt.edu)}@vt.edu, ctlu@vt.edu; T. Ji, Texas A&M University- Corpus Christi; e-mail: taoran.ji@tamucc.edu; K. Fu, Electrical Engineering & Computer Science, South Dakota State University; e-mail: kaiqun.fu@sdstate.edu; L. Zhao, Computer Science, Emory University, Fairfax; e-mail: liang.zhao@emory.edu; F. Chen, Computer Science, The University of Texas at Dallas, ECSS 3.901 UTD, Dallas, TX; e-mail: feng.chen@utdallas.edu; L. Wu, Pinterest, Mountain View, CA, 94043; e-mail: lwu@email.wm.edu; C. Aggarwal, IBM T. J. Watson Research Center; e-mail: charu@us.ibm.com.



This work is licensed under a [Creative Commons Attribution International 4.0 License](https://creativecommons.org/licenses/by/4.0/).

© 2023 Copyright held by the owner/author(s).

0360-0300/2023/12-ART126

<https://doi.org/10.1145/3627816>

ACM Reference format:

Zhiqian Chen, Fanglan Chen, Lei Zhang, Taoran Ji, Kaiqun Fu, Liang Zhao, Feng Chen, Lingfei Wu, Charu Aggarwal, and Chang-Tien Lu. 2023. Bridging the Gap between Spatial and Spectral Domains: A Unified Framework for Graph Neural Networks. *ACM Comput. Surv.* 56, 5, Article 126 (December 2023), 42 pages. <https://doi.org/10.1145/3627816>

1 INTRODUCTION

Deep learning’s performance in various machine learning tasks [1–6] has been extensively recognized in recent decades, with amazing success on Euclidean data. In recent decades, a slew of new applications have emerged in which effective information analysis boils down to the non-Euclidean geometry of data represented by a graph, such as social networks, transportation networks, spread of epidemic disease, brain’s neuronal networks, gene data on biological regulatory networks, telecommunication networks, and knowledge graph. Previous deep learning algorithms, such as convolutional and recurrent neural networks, could not handle such non-Euclidean problems on graph-structured data. Modeling data using a graph is difficult, because graph data is irregular, i.e., each graph has a different number of nodes, and each node in a graph has a varied number of neighbors, making some operations like convolutions inapplicable to the network structure.

There has recently been a surge in growing interest in applying deep learning to graph data. Inspired by deep learning’s success, principles from deep learning models are used to handle the graph’s inherent complexity. This growing trend has piqued the interest of the machine learning community, and a huge number of **graph neural networks (GNN)** models have been proposed based on diverse theories [7–12] and grouped into coarse-grained groupings such as the spectral [13–17] and spatial [10–12] domains. GNNs have seen rising popularity recently for learning graph representations and quickly spread out to many application domains, such as physics [18, 19], chemistry [20, 21], knowledge graph [22–25], recommender systems [26–29], computer vision [30–32], natural language processing [33–35], combinatorial optimization [36–38], traffic network [39–42], program representation [43–45], social network [26, 46, 47]. However, current research in methodology has not translated into a clear understanding of the mechanisms involved, nor has it given us insight into GNNs’ effectiveness or physical meaning. As a result, several consequences will occur: (1) There is no underlying principle that connects all GNNs, which also limits their growth. (2) In high-stakes applications such as drug development, GNN models may carry potentially hazardous unknowns, since they are black boxes. Consequently, the necessity of dissecting GNNs is highlighted, thereby driving academics to hunt for a more universal framework. The main problem is that existing GNN models use a variety of techniques, including random walks [48–50], PageRank [51–54], attention models [12, 55, 56], low-pass filters [57, 58], and message forwarding [59, 60]. Some preliminary research can only explain a few GNNs methods [59, 61, 62], leaving the majority of GNN unaccounted for. Previous GNNs surveys have dealt mostly with classifying several existing models into multiple categories and expanding on each category individually, with no regard to the interrelationships between them [13–17].

This research¹ aims to provide a coherent framework for generalizing GNNs by bridging the divide between seemingly unrelated works in the spatial and spectral domains, as well as by linking methods within each domain. The study will build a unified theoretical framework that encompasses diverse GNNs. Our research is novel in that it connects disparate GNN models, allowing for direct rethinking and comparison of all GNN models.

¹A short version is available at Reference [63].

1.1 Graph Neural Networks in Spatial and Spectral Domain

Over the past several years, GNNs have gained a lot of attention. However, the existence of numerous GNNs complicates model selection, because they are not easily understood within the same framework. Specifically, one uses spectral theory to implement early GNNs [9, 64], whereas spatial theory is used to propose others [10, 65]. The mismatch inherent to spectral and spatial approaches means that direct comparisons are difficult. Even in each area, there are numerous models, which makes it difficult to examine their strengths and weaknesses.

To untangle the mess, we present a unified framework that connects the spatial and spectral domains and reveals their intricate relationship. Furthermore, both domains' subcategories are proven to have a hierarchical link. The focus on a unified framework adds to the knowledge of how GNNs operate. The goal of this research is to use a combination of spectral graph theory and approximation theory to investigate the relationship between important categories, such as spatial- and spectral-based approaches. We give a detailed analysis of GNNs' current research findings in this article, as well as a discussion of trending topics such as over-smoothing. Many well-known GNNs will be used to demonstrate the universality of our architecture. This article's main motivation is twofold: **(1) Connecting the spectral and spatial domains.** The fundamental concepts, principles, and physical implications of spectral- and spatial-based GNNs are significantly different due to their distinct features. As a result, we present an overview of the fundamental principles and properties of spectral- and spatial-based GNNs to help people better grasp the problems, potential, and necessity of GNNs. Formally, a rigorous equivalence is established, indicating that spatial connection function is comparable to spectral filtering; **(2) Dissecting spectral and spatial domains, respectively.** In spectral techniques, filtering functions on eigenvalues are examined, and the filtering function choice can be matched with various tactics in approximation theory; while spatial methodologies are used to explore attribute aggregation, which may be understood from the size and direction within a predetermined region.

The structure of the article is summarized as follows: Basic concepts, distinctive principles, and properties of graph neural networks are covered in Section 2, as well as ways for encoding the graph, spectral-based GNNs, spatial-based GNNs, and essential fundamentals. The proposed framework is summarized in Section 3, which emphasizes the relevance of hierarchy. From Section 4 through Section 5, we explore exemplary GNN models in each domain using our proposed taxonomy structure. In Section 6, we go over the advantages and disadvantages of each domain in detail, as well as practical guidance on GNN model selection. Section 7 also includes a case study of our techniques, demonstrating our proposed framework with current and relevant GNNs concerns.

1.2 Related Surveys and Our Contributions

Existing works on GNNs can be categorized into three groups:

- **Group 1 (Comprehensive Surveys):** Most surveys catalog new GNN research into distinct classes, but they generally lack a unified framework [15–17, 66, 67].
- **Group 2 (Theoretical Perspectives):** The surveys undertaken in this context investigate GNNs from certain perspectives, however, their reach is frequently constrained [60, 68–71].
- **Group 3 (Post Hoc Explanation):** This methodology comprehends GNNs through the utilization of post hoc analysis, which provides valuable insights but poses challenges in terms of validation [72–75].

Our research aims to provide a more interpretable and theoretically grounded understanding of GNNs. Previous surveys either categorize several disparate groups or only address a few GNNs using a certain methodology. Following the overall goals of our framework, we want to create one framework that unifies GNNs across the spatial and spectral domains as well as within each domain via theoretical support. It should be noted that the majority of the work presented is related

to Graph Convolution Networks (GCN) [8], which is the most common type of GNNs, and that many other varieties of GNNs are still based on GCN. As a result, we do not differentiate between GNNs and GCN in this context and refer to GNNs in the following sections.

2 PROBLEM SETUP AND PRELIMINARY

We enumerate all the notations utilized in this survey within Table 1. Furthermore, we will discuss the fundamental concepts, necessary preliminaries, and the problem setup for learning node-level representation, which is the major task in the GNN literature. A simple graph is defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a set of n nodes and $\mathcal{E}$ represents edges. An entry $v_i \in \mathcal{V}$ denotes a node, and $e_{i,j} = \{v_i, v_j\} \in \mathcal{E}$ indicates an edge between nodes i and j . The adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ is defined by if there is a link between node i and j , $A_{i,j} = 1$, and else 0. Node features $\mathbf{X} \in \mathbb{R}^{N \times F}$ is a matrix with each entry $x_i \in \mathbf{X}$ representing the feature vector on node i . Another popular graph matrix is the graph Laplacian, which is defined as $\mathbf{L} = \mathbf{D} - \mathbf{A} \in \mathbb{R}^{N \times N}$, where $\mathbf{D}$ is the degree matrix. Due to its generalization ability [76], the symmetric normalized Laplacian is often used, which is defined as $\tilde{\mathbf{L}} = \mathbf{D}^{-\frac{1}{2}} \mathbf{L} \mathbf{D}^{-\frac{1}{2}}$. Another option is random walk normalization: $\tilde{\mathbf{L}} = \mathbf{D}^{-1} \mathbf{L}$. Note that normalization could also be applied to the adjacency matrix, and their relationship is $\tilde{\mathbf{L}} = \mathbf{I} - \tilde{\mathbf{A}}$. Most GNNs focus on node-level embeddings, learning how a graph signal is modified by a graph topology, and outputting a filtered feature as:

Table 1. Commonly Used Notations

Notations	Descriptions
$\mathcal{G}$	A graph.
$\mathbf{V}$	The set of nodes in a graph.
$\mathbf{E}$	The set of edges in a graph.
$\mathbf{A}, \tilde{\mathbf{A}}$	The adjacency matrix and its normalization.
$\mathbf{L}, \tilde{\mathbf{L}}$	The graph Laplacian matrix and its normalization.
v	A node $v \in \mathbf{V}$.
e_{ij}	An edge $e_{ij} \in \mathbf{E}$.
$\lambda_i \in \Lambda$	Eigenvalue(s).
$\mathbf{U}, \mathbf{U}^\top$	Eigenvector matrix and its transpose.
$\mathbf{U}_i \in \mathbf{U}, \mathbf{u}^\top_i \in \mathbf{U}^\top$	Single eigenvector and its transpose.
$\mathbf{D}$	The degree matrix of $\mathbf{A}$ and $D_{ii} = \sum_{j=1}^n A_{ij}$.
$\mathbf{X} \in \mathbb{R}^{N \times d}$	The feature matrix of a graph.
$\mathbf{Z} \in \mathbb{R}^{N \times b}$	New node feature matrix.
$\mathbf{H} \in \mathbb{R}^{N \times b}$	The node hidden feature matrix.
$\mathbf{h}_v \in \mathbb{R}^b$	The hidden feature vector of node v .
N	node number
b	dimension size of hidden feature
$\odot$	Element-wise product.
Θ, θ	Learnable model parameters.
$\mathbf{P}(\cdot), \mathbf{Q}(\cdot)$	Polynomial function.
$\mathcal{N}(v)$	Directed neighbors of node v

$$f : G, \mathbf{X} \rightarrow \mathbf{Z}, \quad (1)$$

where we aim to find a mapping that can integrate graph structure and original node features, generating a update node representation $\mathbf{Z}$. G represents the graph connectivity, and many options are available as listed in Table 2, and most popular are symmetric normalized graph matrices. In this survey, we use the graph Laplacian, adjacency matrix and their modifications described in Table 2 to represent a graph. So, yet, no experimental or theoretical evidence has been shown that any filter has a consistent advantage [77]. This survey is investigating two specific groups of GNNs, namely, spectral- and spatial-based GNNs, which are defined as below:

Definition 2.1 (Spatial Method). By integrating graph connectivity G and node features $\mathbf{X}$, the updated node representations ($\mathbf{Z}$) are defined as:

$$\mathbf{Z} = f(G) \cdot \mathbf{X}, \quad (2)$$

where G is often implemented with $\mathbf{A}$ or $\mathbf{L}$ in existing works. Therefore, spatial methods focus on finding a **node aggregation function** $f(\cdot)$ that learns a proper aggregation with node features $\mathbf{X}$ to obtain a updated node embedding $\mathbf{Z}$.

Before introducing another definition, the necessary preliminary background is offered:

(1) graph Fourier transform: The graph Laplacian L can be diagonalized [78, 79] as $\tilde{L} = U \Lambda U^\top$, where Λ is the diagonal matrix whose diagonal elements are the corresponding eigenvalues (i.e., $\Lambda_{ii} = \lambda_i$), and U represents eigenvectors. Further, the graph Fourier transform of a signal X is defined as $\hat{X} = U^\top X \in \mathbb{R}^{N \times N}$ and its inverse as $X = U \hat{X}$. **(2) spectral convolution:** According to the Convolution Theorem (i.e., Fourier transform of a convolution of two signals is the element-wise product of their Fourier transforms) [80], the convolution is defined in the Fourier domain as:

$$f_1 * f_2 = U[(U^\top f_1) \odot (U^\top f_2)],$$

where $\odot$ is the element-wise product, and f_1/f_2 are two signals defined on the spatial domain.

Definition 2.2 (Spectral Method). A node signal $f_2 = X$ is filtered by spectral function $g = U^\top f_1$ as:

$$g * X = U[g(\Lambda) \odot (U^\top X)] = U \text{diag}(g(\Lambda)) U^\top X, \quad (3)$$

where g is known as **frequency response function**. If g is polynomial, rational, or exponential function, then we can reduce the equation above to:

$$g * X = g(\tilde{L}) X. \quad (4)$$

In short, the objective of spectral methods is to learn a frequency response function $g(\cdot)$.

The goal of both methods is to learn how to approximate node aggregation or a frequency response function using the data. A great deal of approximation techniques are utilized, and thus f or g can be efficiently estimated. Approximation theory is a branch of mathematics dedicated to discovering and quantifying the errors caused when functions are approximated using smaller functions. Despite the fact that polynomials have a more convenient form than rational functions and are more popular due to its efficiency, rational functions are better at approximating functions at singularities and on unbounded domains. The basic characteristics of rational functions are outlined in complex analytic literature [81–91]. As an important polynomial approximation, Chebyshev approximation is first introduced as spectral filtering for graph convolution: A real symmetric graph Laplacian L can be decomposed as $L = U \Lambda U^{-1} = U \Lambda U^\top$. Chebyshev approximation on spectral filter g is applied [9, 64] so it can be approximated with a polynomials with order k :

$$\begin{aligned} g * X &= U g(\Lambda) U^\top X \\ &\approx U \sum_k \theta_k T_k(\tilde{\Lambda}) U^\top X & \left(\tilde{\Lambda} = \frac{2}{\lambda_{\max}} \Lambda - I_N \right) \\ &= \sum_k \theta_k T_k(\tilde{L}) X & (U \Lambda^k U^\top = (U \Lambda U^\top)^k) \end{aligned}$$

Another graph convolutional network [8] further simplifies it by reducing the order to 1:

Table 2. Representations for Graph Topology

Notations	Descriptions
A	Adjacency matrix
L	Graph Laplacian
$\tilde{A} = A + I$	Adjacency with self loop
$D^{-1} A$	Random walk row normalized adjacency
$A D^{-1}$	Random walk column normalized adjacency
$D^{-1/2} A D^{-1/2}$	Symmetric normalized adjacency
$\tilde{D}^{-1} \tilde{A}$	Left renormalized adjacency, $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$
$\tilde{A} \tilde{D}^{-1}$	Right renormalized
$\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$	Symmetric renormalized
$(\tilde{D}^{-1} \tilde{A})^k$	Powers of left renormalized adjacency
$(\tilde{A} \tilde{D}^{-1})^k$	Powers of right renormalized adjacency

Table 3. Chronological List of Notable GNNs in Spatial and Spectral Domains

	Spatial	Spectral
Before 2015	ParWalk [92], DeepWalk [48], LINE [93]	Spectral GNN [7], ISGNN [94], Neural graph fingerprints [95]
2016	DCNN [11], Molecular Graph Convolutions [96], PATCHY-SAN [97]	GCN [8], ChebNet [9]
2017	MPNN [59], PGCN [98], GraphSAGE [10]	MoNet [13]
2018	GIN [61], Adaptive GCN [99], Fast GCN [100] JKNet [101], Large Scale GCN [102]	RationalNet[103], AR [104], CayleyNet [105]
2019	SGCN [106], DeepGCN [107], MixHop [108], PPAP [53]	ARMA [109], GDC [110], EigenPool [111], GWNN [112], Stable GCNN[113]
2020	SIGN [114], Spline GNN [115], UaGGP [116], GraLSP [117], GraphSAINT [118], DropEdge [119], BGNN[120], ALaGNN[121] Continuous GNN [122], GCNII [123], PPRGo [124], DAGNN [125], H2GCN [126]	GraphZoom [127]
2021	ADC [128], UGCN [129], DGC [130], E(n)GNN [131], GRAND [132], C&S [133], LGNN [134]	Interpretable Spectral Filter [135], Expressive Spectral Perspective [136], S2GC [137], BernNet [138]
2022	GINR[139], Adaptive SGC [140], PGGNN [141], DIMP [142]	AGWN [143], ChebNetII [144], JacobiConv [145], SpecGNN [146], G ² CN [147], PGNN [148], ChebGibbsNet [149], SpecFormer[150], SIGN [151], Spectral Density [152]
2023	[153], [154], Low Rank GNN [155], Auto-HeG [156], [157]	DSF [158], F-SEGA [159], [160], [161]

$$\begin{aligned}
\mathbf{g} * \mathbf{X} &\approx \theta_0 \mathbf{I}_N \mathbf{x} + \theta_1 \tilde{\mathbf{L}} \mathbf{X} && \text{(expand to 1st order)} \\
&= \theta_0 \mathbf{I}_N \mathbf{x} + \theta_1 \left(\frac{2}{\lambda_{max}} \mathbf{L} - \mathbf{I}_N \right) \mathbf{X} && \left(\tilde{\mathbf{L}} = \frac{2}{\lambda_{max}} \mathbf{L} - \mathbf{I}_N \right) \\
&= \theta_0 \mathbf{I}_N \mathbf{x} + \theta_1 (\mathbf{L} - \mathbf{I}_N) \mathbf{X} && (\lambda_{max}=2) \\
&= \theta_0 \mathbf{I}_N \mathbf{x} - \theta_1 \tilde{\mathbf{D}} \mathbf{A} \tilde{\mathbf{D}} \mathbf{X} && (\mathbf{L} = \mathbf{I}_N - \tilde{\mathbf{D}} \mathbf{A} \tilde{\mathbf{D}}) \\
&= \theta_0 (\mathbf{I}_N + \tilde{\mathbf{D}} \mathbf{A} \tilde{\mathbf{D}}) \mathbf{X} && (\theta_0 = -\theta_1) \\
&= \theta_0 (\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}) \mathbf{x} && (\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N, \tilde{\mathbf{D}}_{ii} = \sum_j \mathbf{A}_{ij}).
\end{aligned}$$

As a result, θ_0 is the only parameter to learn. The learnable parameters in many different GNNs can vary based on the model design.

3 FRAMEWORK OVERVIEW

The development of GNNs is briefly discussed below with representative studies before we introduce our proposed theoretical framework. Table 3 depicts many sample models that focus on node-level graph convolution. The spectral perspective was previously explored (SGWT [64]),

and it serves as the technical foundation for all subsequent spectral methods, including spectral convolution and approximation. Researchers continue to add to this thread, demonstrating that spectral methods have the ability to handle graphs (SGNN [7], ISGNN [94], ChebNet [9]). Furthermore, GCN [8] and GraphSAGE [10] create effective training methodologies, gaining considerable attention from various communities. After that, spectral techniques development stagnated, with the exception of a few publications on rational filtering (RationalNet [103], AR [104], ARMA [109]). Meanwhile, focus shifts to the spatial domain, which has dominated GNNs to this point. Random walks (ParWalk [92], DeepWalk [48], LINE [93]) and CNN (DCNN [11]) were used in early spatial approaches. Following that, MPNN [59] solidified the message-passing mechanism in spatial techniques. High-order polynomial approximation has been studied [61, 106, 110, 114], but only within the context of ChebNet or DCNN. It is worth noting that while numerous publications described their suggested methods from both spatial and spectral perspectives, only a few GNNs are covered [13, 110]. Until recently, spectral research has demonstrated a resurgence.

In this survey, we provide a framework to fully comprehend spectral methods from a spatial perspective and vice versa. A cross-domain perspective is used to integrate spatial and spectral techniques into a coherent framework. As shown in Figure 1, the proposed framework divides GNNs into two domains: spatial (A-0) and spectral (B-0), each of which is further separated into three subcategories (A-1/A-2/A-3 and B-1/B-2/B-3). A-0 is separated into linear (A-1), polynomial (A-2), and rational (A-3) aggregations based on the types of neighbor aggregation (i.e., f in Definition 2.1). Operations on first-order neighbors only are considered in linear aggregation (A-1), whereas high-order neighbors are incorporated in polynomial aggregation (A-2). In addition, rational aggregation (A-3) includes self-aggregation. According to the types of approximation techniques, the spectral methods are divided into linear (B-1), polynomial (B-2), and rational (B-3) approximation, depending on the types of frequency filtering (i.e., g in Definition 2.2). In Sections 4 and 5, each category and subcategory will be explained in detail with examples.

3.1 Inside the Spatial and Spectral Domain

The hierarchical link between the spatial and spectral domains is depicted in this subsection. Spatial-based techniques can be divided into three types, with specialization and generalization relationships existing:

(A-1) LINEAR AGGREGATION $\rightleftharpoons$ (A-2) POLYNOMIAL AGGREGATION $\rightleftharpoons$ (A-3) RATIONAL AGGREGATION,

where it is a generalization from left to right, and specialization from right to left. Specifically, Linear Aggregation (A-1) comprises all algorithms for learning a linear function among neighbors in the first-order. Higher-order neighbors are included in Polynomial Aggregation (A-2), and the order number is defined by the polynomials. Additionally, Rational Aggregation (A-3) utilizes self-aggregation, since the inclusion of higher-order neighbors causes linear aggregation (A-1) to transform into polynomial aggregation (A-2), and polynomial aggregation (A-2) to transform into rational aggregation (A-3) if self-aggregation is added. The approaches falling under the general category of spectral analysis can be grouped into three distinct groups:

(B-1) LINEAR APPROXIMATION $\rightleftharpoons$ (B-2) POLYNOMIAL APPROXIMATION $\rightleftharpoons$ (B-3) RATIONAL APPROXIMATION,

which includes left-to-right generalization and right-to-left specialization. Concretely, (B-1) outlines all models that aggregate frequency components using a linear function, while (B-2) uses polynomial approximation, and (B-3) applies rational approximation. Therefore, (B-1) can be generalized as (B-2) if replacing linear approximation with polynomial approximation, and (B-2) is generalized as (B-3) if replacing polynomial approximation with rational approximation.

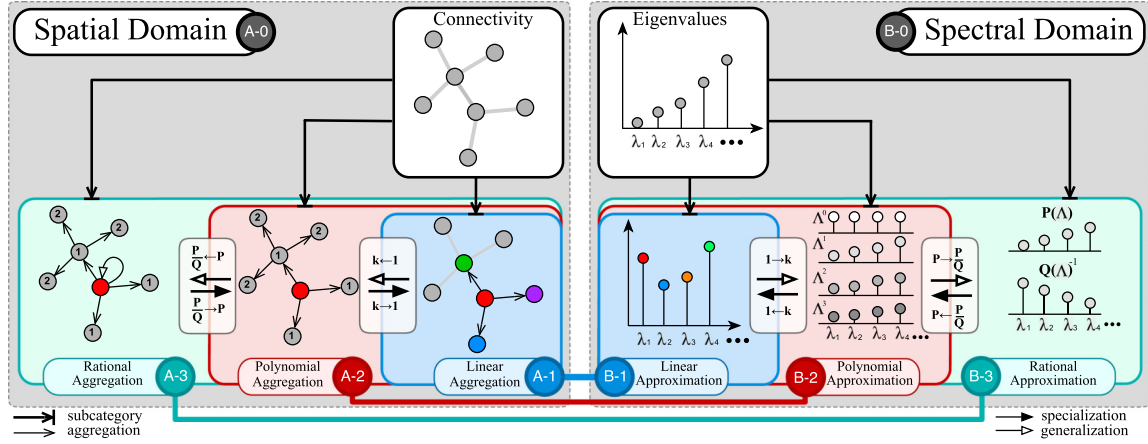


Fig. 1. Illustration of major graph neural operations and their relationship. Spatial and spectral methods are divided into three groups, respectively. Groups A-1, A-2, and A-3 are strongly correlated by generalization and specialization, so are groups B-1, B-3, and B-3. The equivalence among them is marked in the same color.

3.2 Between the Spatial and Spectral Domain

In this section, we go through the link between the spatial and spectral domains over the border. There is also equivalency by modifying the analytical form of these subcategories, as shown below. Linear Aggregation (A-1) and Linear Approximation (B-1) are the initial equivalences:

$$(A-1) \text{ LINEAR AGGREGATION} \Leftrightarrow (B-1) \text{ LINEAR APPROXIMATION},$$

which means that adjusting weights on neighbors in linear aggregation equates to adjusting weights on frequency components in linear approximation using a linear function. Linear Aggregation (A-1) and Linear Approximation (B-1) have the same linear function and can seamlessly convert to each other in closed form. The main difference between them is that Linear Aggregation (A-1) recovers the signal as a linear function of the frequency component, whereas Linear Approximation (B-1) models the target signal as a linear function of neighbor nodes. Both Linear Aggregation (A-1) and Linear Approximation (B-1) aggregate the representations of neighbors by tweaking the weights of each neighbor or use a linear filter on eigenvalues with a negative slope, i.e., $g(\Lambda) = -\Lambda + a$. Because the low-frequency components are given a higher weight by g than their original values, this is referred to as low-pass filtering (i.e., eigenvalues). This group's main advantages are (1) its low computational cost and (2) the large number of real-world scenarios that are subject to the homophily assumption (i.e., neighbors are similar). The fundamental disadvantage is that not every network guarantees homophily. Polynomial Aggregation (A-2) and Polynomial Approximation (B-2) are identical in terms of actual operation, i.e.,

$$(A-2) \text{ POLYNOMIAL AGGREGATION} \Leftrightarrow (B-2) \text{ POLYNOMIAL APPROXIMATION}.$$

This means that in polynomial aggregation, aggregating higher orders of neighbors can be expressed as the sum of different orders of frequency components in polynomial approximation. Both use higher-order neighbors in addition to first-order neighbors, increasing the capacity to simulate a more complex relationship among the neighbors. It is theoretically more powerful than (A-1)/(B-1) from a spectral standpoint, because (A-2)/(B-2) is a polynomial approximation as a spectral filter, whereas (A-1)/(B-1) is linear regression. As a result, one flaw is the cost of border neighborhood, which leads to higher computational complexity than (A-1)/(B-1). Another flaw of

Table 4. Structure of Sections 4 and 5

	Section 4 (A-0) Spatial: function of adjacency matrix	Section 5 (B-0) Spectral: function of eigenvalues
Linear $l(\cdot)$	Section 4.1 (A-1) $l(A) = a_1 A + a_0 A^0 = a_1 A + a_0 I$	Section 5.1 (B-1) $l(\Lambda) = a_1 \Lambda + a_0 \Lambda^0 = a_1 \Lambda + a_0 I$
Polynomial $P(\cdot)$	Section 4.2 (A-2) $P(A) = a_m A^m + \dots + a_k A^k + \dots + a_0 A^0$	Section 5.2 (B-2) $P(\Lambda) = a_m \Lambda^m + \dots + a_k \Lambda^k + \dots + a_0 \Lambda^0$
Rational $\frac{P(\cdot)}{Q(\cdot)}$	Section 4.3 (A-3) $\frac{P(A)}{Q(A)} = \frac{a_m A^m + \dots + a_k A^k + \dots + a_0 A^0}{a_m A^m + \dots + a_k A^k + \dots + a_0 A^0}$	Section 5.3 (B-3) $\frac{P(\Lambda)}{Q(\Lambda)} = \frac{a_m \Lambda^m + \dots + a_k \Lambda^k + \dots + a_0 \Lambda^0}{a_m \Lambda^m + \dots + a_k \Lambda^k + \dots + a_0 \Lambda^0}$

them is that if the order is set too large, then it will over-smooth (i.e., all nodes will look the same), and there is no golden rule for order size because it is based on data. Note that K-layer (A-1) or (B-1) is equivalent to K-order of (A-2)/(B-2), hence stacking K-layer (A-1) or (B-1) causes over-smoothing (B-1). Similarly, the last equivalence relationship is

$$(A-3) \text{ RATIONAL AGGREGATION} \Leftrightarrow (B-3) \text{ RATIONAL APPROXIMATION},$$

in which rational aggregation defines a label aggregation with self-aggregation, while rational approximation adjusts filter function with rational approximation. Both alleviate the over-smoothing issue by introducing self-aggregation, which limits the intensity of uni-directional aggregation in the spatial domain. From a spectral perspective, this advantage can be interpreted as the superiority of rational approximation (A-3/B-3) over polynomial approximation (A-2/B-2). In particular, the rational approximation is more powerful and precise, particularly when estimating some abrupt signals like discontinuity [82, 84–87, 89, 90].

As a result, we may summarize the advantages and disadvantages of each combination as illustrated in Figure 2. The category selection is based on the data complexity and the efficiency required, as there is a tradeoff between computational efficiency and generalization capability.

Table 4 shows the structure of Sections 4 and 5, where we will discuss details of these two threads, respectively, and exemplify using several representative graph neural networks. The second column denotes spatial perspective (Section 4,

A-0), which can treat popular GNNs as learning a function of adjacency matrix, or *node aggregation function*. Similarly, the third column means spectral view (Section 5, B-0), which sees GNN as learning a function of eigenvalues or frequency response functions. The cell at the intersection of the second row and second column means that this category of GNNs can be treated as a linear function of adjacency matrix, or, say, its node aggregation function is linear. The other intersection cells follow a similar schema. Note that categories within the same row have the same format and function. For instance, in the second row, A-1 and B-1 share the format of a linear function, but their parameters need not be identical.

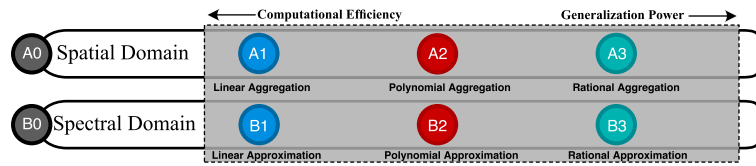


Fig. 2. Category and subcategory comparison.

4 SPATIAL-BASED GNNS (A-0)

In the current literature, spatial approaches such as self-loop, normalization, high-order neighbors, aggregation, and node combination are often explored. We established a new taxonomy for spatial-based GNNS based on these operations, dividing them into three separate groupings as below.

4.1 Linear Aggregation (A-1)

A lot of research has gone into understanding the aggregation among first-order neighbors (i.e., direct neighbors) [10, 12, 48, 59, 61, 162]. The supervisory signal patterns are revealed by altering the weights for the node and its first-order neighbors. The revised node embeddings, $Z(v_i)$, can be represented in the following way:

$$Z(v_i) = \overbrace{\Phi(v_i) \mathbf{h}(v_i)}^{\text{self node}} + \overbrace{\sum_{u_j \in \mathcal{N}(v_i)} \Psi(u_j) \mathbf{h}(u_j)}^{\text{neighbors' aggregation}}, \quad (5)$$

where u_j denotes a neighbor of node v_i , $\mathbf{h}(\cdot)$ is their representations, and Φ/Ψ indicate the weight functions. The first item on the right-hand side denotes the weighted representation of node v_i , while the second represents the update from its neighbors. By applying random walk normalization (i.e., dividing neighbors by degree of the current node), Equation (5) and its symmetric normalization can be written as:

$$Z(v_i) = \Phi(v_i) \mathbf{h}(v_i) + \sum_{u_j \in \mathcal{N}(v_i)} \Psi(u_j) \frac{\mathbf{h}(u_j)}{d_i}, \quad \tilde{Z}(v_i) = \Phi(v_i) \mathbf{h}(v_i) + \sum_{u_j \in \mathcal{N}(v_i)} \Psi(u_j) \frac{\mathbf{h}(u_j)}{\sqrt{d_i d_j}}, \quad (6)$$

where d_i represents the degree of node v_i . Normalization has better generalization capacity, which is not only due to some implicit evidence but also because of a theoretical proof on performance improvement [163]. In a simplified configuration, weights for all the neighbors are the same and is a scalar value ψ , while the weight for self node ϕ is another scalar value. Therefore, they can be rewritten in matrix form as:

$$\mathbf{Z} = \phi \mathbf{X} + \psi \mathbf{D}^{-1} \mathbf{A} \mathbf{X} = (\phi \mathbf{I} + \psi \mathbf{D}^{-1} \mathbf{A}) \mathbf{X}, \quad \tilde{\mathbf{Z}} = \phi \mathbf{X} + \psi \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \mathbf{X} = (\phi \mathbf{I} + \psi \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}) \mathbf{X}. \quad (7)$$

Equation (7) can be generalized as an identical form:

$$\mathbf{Z} = (\phi \mathbf{I} + \psi \tilde{\mathbf{A}}) \mathbf{X}, \quad (8)$$

where $\tilde{\mathbf{A}}$ denotes normalized $\mathbf{A}$, which is implemented by random walk or symmetric normalization. As shown in Figure 3, the new representation of the current node (in red) is updated as the sum of the previous representations of itself and its neighbors. (A-1) may adjust the weights of the neighbors. The following are a few state-of-the-art approaches that were chosen to demonstrate this schema:

4.1.1 Graph Convolutional Network (GCN). As the one state-of-the-art, GCN [8] adds a self-loop to nodes, and applies a *renormalization* trick that changes degree matrix from $\mathbf{D}_{ii} = \sum_j \mathbf{A}_{ij}$ to $\hat{\mathbf{D}}_{ii} = \sum_j (\mathbf{A} + \mathbf{I})_{ij}$. Specifically, GCN can be written as:

$$\mathbf{Z} = \hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{X} = \hat{\mathbf{D}}^{-\frac{1}{2}} (\mathbf{I} + \mathbf{A}) \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{X} = (\mathbf{I} + \tilde{\mathbf{A}}) \mathbf{X}, \quad (9)$$

where $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, and $\tilde{\mathbf{A}}$ is normalized adjacency matrix with self-loop. Therefore, Equation (9) is equivalent to Equation (8) when setting $\phi = 0$ and $\psi = 1$ with the *renormalization* trick. Besides, GCN takes the sum of each node and average of its neighbors as new node embeddings. Note that the normalization of GCN is different from the others, but the physical meaning is the same.

4.1.2 GraphSAGE. Computing intermediate representations of each node and its neighbors, GraphSAGE [10] applies an aggregation among its neighbors. Take mean aggregator as example, it averages a node with its neighbors, i.e.,

$$Z(v_i) = \text{MEAN}(\{\mathbf{h}(v_i)\} \cup \{\mathbf{h}(u_j), \forall u_j \in \mathcal{N}(v_i)\}), \quad (10)$$

where $\mathbf{h}$ indicates the intermediate representation, and $\mathcal{N}$ denotes the neighbor nodes. Equation (10) can be written in matrix form after implementing MEAN using symmetric normalization:

$$\mathbf{Z} = \mathbf{D}^{-\frac{1}{2}}(\mathbf{I} + \mathbf{A})\mathbf{D}^{-\frac{1}{2}}\mathbf{X} = (\mathbf{I} + \tilde{\mathbf{A}})\mathbf{X}, \quad (11)$$

which is equivalent to Equation (8) with $\phi = 1$ and $\psi = 1$. Note that the key difference between GCN and GraphSAGE is the normalization strategy: The former is symmetric normalization with renormalization trick, and the latter is random walk normalization.

4.1.3 Graph Isomorphism Network (GIN). Inspired by the **Weisfeiler-Lehman (WL)** test, GIN [61] develops conditions to maximize the power of GNNs, proposing a simple architecture, Graph Isomorphism Network. With strong theoretical support, GIN generalizes the WL test and updates node representations as:

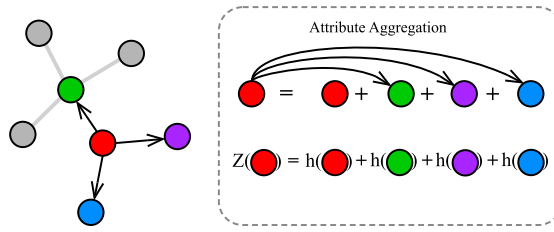


Fig. 3. A-1: the red node updates itself with neighbors.

$$\mathbf{Z} = (1 + \epsilon) \cdot \mathbf{h}(v) + \sum_{u_j \in \mathcal{N}(v_i)} \mathbf{h}(u_j) = [(1 + \epsilon)\mathbf{I} + \mathbf{A}]\mathbf{X}, \quad (12)$$

which is equivalent to Equation (8) with $\phi = 1 + \epsilon$ and $\psi = 1$. Note that GIN does not perform any normalization.

4.1.4 Graph Attention Model (GAT). GAT [12] applies attention mechanism by adjusting neighbors' weights, instead of using uniform weights in many related works:

$$\mathbf{Z} = (W_{att} \otimes \mathbf{A})\mathbf{X}, \quad (13)$$

where $W_{att} \in \mathbb{R}^{N \times N}$ is a matrix, $\otimes$ denotes element-wise multiplication, and calculated by a forward neural network $W_{att}(i, j) = f(\mathbf{h}_i, \mathbf{h}_j)$ with a pair of node representations as input. GAT can be treated as learning dynamic weight on the neighbors, since their weights are not uniform. MoNet [13] is similar to GAT, since its update follows:

$$\mathbf{Z}(v) = \sum_{u \in \mathcal{N}(v)} w_j(\mathbf{u}(\mathbf{h}_i, \mathbf{h}_j)) \mathbf{h}_j, \quad (14)$$

where $\mathbf{u}$ is a d-dimensional vector of pseudo-coordinates $\mathbf{u}(x, y)$, and

$$w_j(\mathbf{u}) = \exp\left(-\frac{1}{2}(\mathbf{u} - \mu_j)^\top \Sigma_j^{-1}(\mathbf{u} - \mu_j)\right), \quad (15)$$

where μ_j are learnable $d \times d$ and $d \times 1$ covariance matrix and mean vector of a Gaussian kernel, respectively. Let $W_{MoNet} = w(u(\cdot))$ as a weight function of a pair of node representations representation, then it is also an attention model:

$$\mathbf{Z} = (W_{MoNet} \otimes \mathbf{A})\mathbf{X}. \quad (16)$$

These works do not consider updating nodes with their original representations, i.e., $\phi = 0$ and ψ is replaced with matrix parameter W in Equation (8). However, it is easy to extend them with self node.

4.2 Polynomial Aggregation (A-2)

Several researches use higher orders of neighbors to integrate deeper structural information [9, 11, 49, 93, 106], because direct neighbors are not always enough to describe a node's surroundings. Excessive order, however, generally averages all node representations, resulting in over-smoothing and a loss of emphasis on the immediate neighborhood [104]. Many models are motivated by this to fine-tune the aggregation strategy based on different orders of neighbors. As a result, adequate constraint and order flexibility are essential for node representation. Challenging signals, such as Gabor-like filters, have been shown to have a high order of neighbors [108].

Define the shortest distance between node i and j as $d_G(i, j)$, and $\partial\mathcal{N}(i, \tau)$ to be the set of nodes j that satisfies $d_G(i, j) = \tau$, i.e., τ -order neighbors. Formally, this type of work can be written as:

$$Z(v_i) = \Phi(v_i) \mathbf{h}(v_i) + \overbrace{\sum_{u_j \in \mathcal{N}(v_i, \tau=1)} \Psi^{(\tau=1)} \mathbf{h}(u_j) + \dots}^{\text{1st-order neighbor}} + \overbrace{\sum_{u_j \in \mathcal{N}(v_i, \tau=k)} \Psi^{(\tau=k)} \mathbf{h}(u_j) + \dots}^{\text{kth order neighbor}}, \quad (17)$$

where $\Psi^{(\tau)}$ indicates a scalar parameter for all τ -order neighbors. Setting the same order neighbors to share the same weights, Equation (17) can be rewritten in matrix form:

$$\mathbf{Z} = \left(\phi \mathbf{I} + \sum_{j=1}^k \psi_j \mathbf{A}^j \right) \mathbf{X} = \mathbf{P}_k(\mathbf{A}) \mathbf{X}, \quad (18)$$

where $\mathbf{P}_k(\cdot)$ is a polynomial function with order number k . Applying symmetric normalization, Equation (18) can be rewritten in matrix form as:

$$\mathbf{Z} = \left(\phi \mathbf{I} + \sum_{j=1}^k \psi_j (\mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}})^j \right) \mathbf{X} = \left(\phi \mathbf{I} + \sum_{i=1}^k \psi_i \tilde{\mathbf{A}}^i \right) \mathbf{X} = \left(\sum_{i=0}^k \psi_i \tilde{\mathbf{A}}^i \right) \mathbf{X} = \mathbf{P}_k(\tilde{\mathbf{A}}) \mathbf{X}, \quad (19)$$

where $\phi = \psi_0$ due to the fact that $\mathbf{I} = \tilde{\mathbf{A}}^0$, and $\mathbf{A}$ could also be normalized by random walk normalization: $\tilde{\mathbf{A}} = \mathbf{D}^{-1} \mathbf{A}$. As shown in Figure 4, the new representation of the current node (in red) is updated as the sum of the previous representations of itself, its first- and second-order neighbors. Note that the weights among those representations are learnable. Several existing works are analyzed below, showing that they are variants of Equation (18) or (19).

4.2.1 ChebNet. To bridge the gap, spectral convolutional operation is generalized, which requires expensive steps of spectral decomposition and matrix multiplication [94, 164]. Introducing truncated Chebyshev polynomial for estimating wavelet in graph signal processing, ChebNet [9] embeds a novel neural network layer for the convolution operator. Specifically, ChebNet can be written as:

$$\sum_{k=0}^{K-1} \theta_k T_k(\tilde{\mathbf{L}}) \mathbf{X} = (\tilde{\theta}_0 \mathbf{I} + \tilde{\theta}_1 \tilde{\mathbf{L}} + \tilde{\theta}_2 \tilde{\mathbf{L}}^2 + \dots) \mathbf{X}, \quad (20)$$

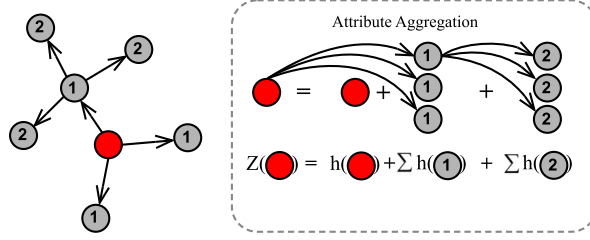


Fig. 4. A-2: This red node updates its representation with its first- and second-order neighbors to update itself.

where $T_k(\cdot)$ denotes the Chebyshev polynomial and θ_k is the Chebyshev coefficient. $\tilde{\theta}$ is the coefficient after expansion and reorganization. Since $\tilde{\mathbf{L}} = \mathbf{I} - \tilde{\mathbf{A}}$, we have:

$$\sum_{k=0}^{K-1} \theta_k T_k(\tilde{\mathbf{L}}) \mathbf{X} = [\tilde{\theta}_0 \mathbf{I} + \tilde{\theta}_1(\mathbf{I} - \tilde{\mathbf{A}}) + \tilde{\theta}_2(\mathbf{I} - \tilde{\mathbf{A}})^2 + \cdots] \mathbf{X} = \left(\phi \mathbf{I} + \sum_{i=1}^k \psi_i \tilde{\mathbf{A}}^i \right) \mathbf{X} = \mathbf{P}_k(\tilde{\mathbf{A}}) \mathbf{X}, \quad (21)$$

which is exactly Equation (19). The predefined K is the order number of Chebyshev polynomial, and a larger K means higher approximation accuracy in estimating the function of eigenvalues. Equation (21) shows that K also can be explained as the highest order of the neighbors.

4.2.2 DeepWalk. Applying random walk, DeepWalk [48] first draws a group of random paths from a graph and applies a skip-gram algorithm to extract node features. Assuming the number of random walks is sufficient, the transition probability on a graph can be represented as:

$$\tilde{\mathbf{A}} = \mathbf{D}^{-1} \mathbf{A}. \quad (22)$$

Let the window size of skip-gram be $2t + 1$ and the current node is the $(t + 1)$ -th one along each sampled random walk path, so the farthest neighbor current node can reach is the first one and the last one. A node and its neighbors are likely to appear in the same random walk path, and the neighbors follow the transition probability (Equation (22)) to appear in the same path. Therefore, the updated representation is as follows:

$$\mathbf{Z} = \frac{1}{t+1} (\mathbf{I} + \tilde{\mathbf{A}} + \tilde{\mathbf{A}}^2 + \cdots + \tilde{\mathbf{A}}^t) \mathbf{X} = \frac{1}{t+1} \mathbf{P}_k(\tilde{\mathbf{A}}) \mathbf{X}, \quad (23)$$

where $\mathbf{I}$ means that DeepWalk always considers previous representation of the current node as one element, $\tilde{\mathbf{A}}$ represents the direct neighbors' transition probability, and $\tilde{\mathbf{A}}^2$ denotes that of the second-order neighbors. It will have at most t -order neighbors, depending on the predefined length of the random walk (i.e., $2t + 1$).

4.2.3 Diffusion Convolutional Neural Networks (DCNN). DCNN [11] uses a degree-normalized transition matrix (i.e., renormalized adjacency matrix: $\tilde{\mathbf{A}} = \tilde{\mathbf{D}} \mathbf{A}$) as graph representation and performs node embedding update as:

$$\mathbf{Z} = \mathbf{W} \odot \tilde{\mathbf{A}}^* \mathbf{X} = [w_1, w_2, w_3, \dots, w_k]^T \odot [\tilde{\mathbf{A}}, \tilde{\mathbf{A}}^2, \tilde{\mathbf{A}}^3, \dots, \tilde{\mathbf{A}}^k]^T \mathbf{X}, \quad (24)$$

where $\tilde{\mathbf{A}}^*$ denotes a tensor containing the power series of $\tilde{\mathbf{A}}$, and the $\odot$ operator represents element-wise multiplication. It can be transformed as:

$$\mathbf{Z} = (w_1 \tilde{\mathbf{A}} + w_2 \tilde{\mathbf{A}}^2 + w_3 \tilde{\mathbf{A}}^3 + \cdots w_k \tilde{\mathbf{A}}^k) \mathbf{X} = \mathbf{P}_k(\tilde{\mathbf{A}}) \mathbf{X}. \quad (25)$$

4.2.4 Scalable Inception Graph Neural Networks (SIGN). By generalizing GCN [8], GAT [12], and SGC [106], SIGN [114] constructs a set of linear diffusion operators along with non-linearity. For node-wise classification tasks, SIGN has the form:

$$\begin{aligned} \mathbf{Z} &= \sigma([\mathbf{X}\Theta_0, \mathbf{A}_1\mathbf{X}\Theta_1, \dots, \mathbf{A}_r\mathbf{X}\Theta_r]), \\ \mathbf{Y} &= \xi(\mathbf{Z}\Omega), \end{aligned} \quad (26)$$

where $[\cdot, \cdot, \dots, \cdot]$ is concatenation, and r denotes the power number. SIGN can be rewritten as:

$$\mathbf{Z} = [\mathbf{X}\Theta_0, \mathbf{A}_1\mathbf{X}\Theta_1, \dots, \mathbf{A}_r\mathbf{X}\Theta_r] \Omega = \omega_0 \sigma(\mathbf{X}\Theta_0) + \omega_1 \sigma(\mathbf{A}_1\mathbf{X}\Theta_1), \dots, \omega_r \sigma(\mathbf{A}_r\mathbf{X}\Theta_r), \quad (27)$$

where $\sigma(A_r \mathbf{X} \Theta_r)$ could be treated as refined representation of each order of label aggregation by non-linear function σ and fully connected layer Ω , i.e., $\widehat{\mathbf{A}^r \mathbf{X}}$. Replacing A with normalized adjacency matrix, it can be rewritten as:

$$\hat{\mathbf{Z}} = \omega_0 \widehat{\tilde{\mathbf{A}}^0 \mathbf{X}} + \omega_1 \widehat{\tilde{\mathbf{A}}^1 \mathbf{X}}, \dots, \omega_r \widehat{\tilde{\mathbf{A}}^r \mathbf{X}} = \sum_r \omega_r \widehat{\tilde{\mathbf{A}}^r \mathbf{X}} = \widehat{\mathbf{P}(\tilde{\mathbf{A}}) \mathbf{X}}. \quad (28)$$

4.2.5 Graph Diffusion Convolution (GDC). GDC [110] defines a generalized graph diffusion via the diffusion matrix:

$$\mathbf{Z} = \sum_{k=0}^{\infty} \theta_k T^k, \quad (29)$$

where θ_k are the weighting coefficients with $\sum_{k=0}^{\infty} \theta_k = 1$, $\theta_k \in [0, 1]$, T is a generalized undirected transition matrix that includes the random walk transition matrix $T_{rw} = \mathbf{A} \mathbf{D}^{-1}$, and the symmetric transition matrix $T_{sym} = \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$. In the general case, it can be written as :

$$\mathbf{Z} = \sum_{k=0}^{\infty} \theta_k \tilde{\mathbf{A}}^k = \mathbf{P}(\tilde{\mathbf{A}}). \quad (30)$$

4.2.6 Node2Vec. Node2Vec [49] defines a second-order random walk to control the balance between **BFS (breadth-first search)** and **DFS (depth-first search)**. Consider a random walk that traversed an edge between node t and v , denoted as (t, v) , and now it resides at node v . Then, the transition probabilities to next stop x from node t is defined as:

$$P(t \rightarrow x) = \begin{cases} \frac{1}{p} & \text{if } d(t, x) = 0, \text{ return to the source} \\ 1 & \text{if } d(t, x) = 1, \text{ BFS} \\ \frac{1}{q} & \text{if } d(t, x) = 2, \text{ DFS,} \end{cases} \quad (31)$$

where $d(t, x)$ denotes the shortest path between nodes t and x . $d(t, x) = 0$ indicates a second-order random walk returns to its source node, (i.e., $t \rightarrow v \rightarrow t$), while $d(t, x) = 1$ means that this walk goes to a BFS node, and $d(t, x) = 2$ to a DFS node. The parameters p and q control the distribution of the three cases. Assuming the random walk is sufficiently sampled, Node2Vec can be rewritten in matrix form:

$$\mathbf{Z} = \left(\frac{1}{p} \cdot \overbrace{\mathbf{I}}^{\text{source}} + \overbrace{\tilde{\mathbf{A}}}^{\text{BFS}} + \frac{1}{q} \overbrace{(\tilde{\mathbf{A}}^2 - \tilde{\mathbf{A}})}^{\text{DFS}} \right) \mathbf{X}, \quad (32)$$

which can be transformed and reorganized as:

$$\mathbf{Z} = \left[\frac{1}{p} \mathbf{I} + \left(1 - \frac{1}{q} \right) \tilde{\mathbf{A}} + \frac{1}{q} \tilde{\mathbf{A}}^2 \right] \mathbf{X} = \mathbf{P}_{k=2}(\tilde{\mathbf{A}}) \mathbf{X}, \quad (33)$$

where transition probabilities $\tilde{\mathbf{A}} = \mathbf{D}^{-1} \mathbf{A}$ is random walk normalized adjacency matrix.

4.2.7 LINE [165]/SDNE [166]. These two models consider first-order and second-order neighbors as the constraints for learning node embeddings. First-order: The nodes representation is forced to be similar to its neighbors, which is equivalent to:

$$\mathbf{Z} = \tilde{\mathbf{A}} \mathbf{X}. \quad (34)$$

Second-order: The pair of nodes is forced to be similar if their neighbors are similar, which is equivalent to make second-order neighbors similar, therefore, we can get the second-order connectivity by taking the power of the original adjacency:

$$\mathbf{Z} = \tilde{\mathbf{A}}^2 \mathbf{X}. \quad (35)$$

Then the final learned node embeddings are formulated as:

$$\mathbf{Z} = \tilde{\mathbf{A}} \mathbf{X} + \alpha \tilde{\mathbf{A}}^2 \mathbf{X} = \mathbf{P}_{k=2}(\tilde{\mathbf{A}}) \mathbf{X}. \quad (36)$$

Since LINE uses concatenation between the representations constrained by first and second-order, $\alpha = 1$; for SDNE, α is pre-defined.

4.2.8 Simple Graph Convolution (SGC). To reduce the computational overhead, SGC [106] removes non-linear function between neighboring graph convolution layers and combines graph aggregation in one single layer:

$$\mathbf{Z} = \tilde{\mathbf{A}}^k \mathbf{X}, \quad (37)$$

where $\tilde{\mathbf{A}}$ is renormalized adjacency matrix, i.e., $\tilde{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{A} \tilde{\mathbf{D}}^{-\frac{1}{2}}$, where $\tilde{\mathbf{D}}^{-\frac{1}{2}}$ is degree matrix with self loop. Therefore, it can be easily rewritten as:

$$\mathbf{Z} = (0 \cdot \mathbf{I} + 0 \cdot \tilde{\mathbf{A}} + 0 \cdot \tilde{\mathbf{A}}^2 + \dots + 1 \cdot \tilde{\mathbf{A}}^k) \mathbf{X} = \mathbf{P}_k(\tilde{\mathbf{A}}) \mathbf{X}, \quad (38)$$

which only has the highest-order term.

4.3 Rational Aggregation (A-3)

Most works merely consider label propagation from the node to its neighbors (i.e., gathering information from its neighbors) but ignore self-aggregation. Self-aggregation means that labels or attributes can be propagated back to themselves or restart propagating with a certain probability. This reverse behavior can avoid over-smoothing issue [53]. Note that Polynomial Aggregation (A-2) may manually change the order number to relieve the over-smoothing issue, but Rational Aggregation (A-3) can do so automatically. Theoretically, rational function approximation is more effective than polynomial and has been researched in machine learning problems [86, 167, 168]. Several works use a rational function on the adjacency matrix to perform self-aggregation, either explicitly or implicitly [53, 103, 105, 109, 169–172]. Because generic label propagation is achieved by multiplying the graph Laplacian, self-aggregation may be achieved by multiplying the inverse graph Laplacian as follows:

$$\mathbf{Z} = \mathbf{P}_m(\tilde{\mathbf{A}}) \mathbf{Q}_n(\tilde{\mathbf{A}})^{-1} \mathbf{X} = \frac{\mathbf{P}_m(\tilde{\mathbf{A}})}{\mathbf{Q}_n(\tilde{\mathbf{A}})} \mathbf{X}, \quad (39)$$

where $\mathbf{P}$ and $\mathbf{Q}$ are two different polynomial functions, and the bias of $\mathbf{Q}$ is often set to 1 to normalize the coefficients. As shown in Figure 5, the new representations of the current node (in red) are updated as the previous one with probability $\mathbf{P}$, and as that of neighbors with probability $(1-\mathbf{P})$. The difference of A-3 beyond A-2 is that A-3 can avoid over-smoothing issue in an automatic manner [104, 173]. Over-smoothing issue happens when GNNs go deep, which would drive node features to a stationary point and average all the information from raw node representations. Graph convolution can be described as an optimization problem [57, 67, 170, 173], e.g., (1) minimizing the supervised loss and (2) keeping the local neighborhood similar:

$$\mathbf{Z} = \arg \min_{\mathbf{Z}} \left\{ \underbrace{\|\mathbf{Z} - \mathbf{Y}\|_2^2}_{(1) \text{ supervised loss}} + \underbrace{\alpha \text{Tr}(\mathbf{Z}^T \mathbf{L} \mathbf{Z})}_{(2) \text{ neighborhood regularization}} \right\}, \quad (40)$$

where α is the controlling weight between the two constraints. The problem has analytical solution:

$$\mathbf{Z} = (\mathbf{I} + \alpha \tilde{\mathbf{L}})^{-1} \mathbf{Y} = ((1 + \alpha) \mathbf{I} - \tilde{\mathbf{A}}) \mathbf{Y}. \quad (41)$$

However, because α increases with the number of times graph convolution is done, it is prone to over-smoothing. Over-smoothing is addressed in a variety of ways [101, 114, 119, 123, 174], including Rational Aggregation (A-3), which does so by retaining a portion of the original representation no matter how many iterations it does, greatly reducing over-smoothing.

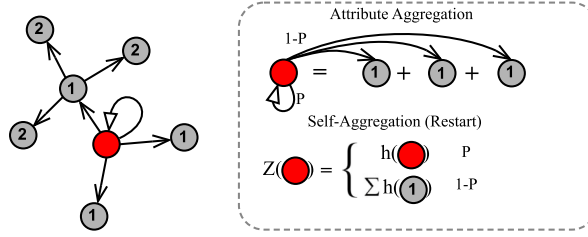


Fig. 5. A-3: The red nodes (red) update its representation with self-loop and neighbors. In A-3, the ratio of the original representation remains stable, whereas A-1 does not control the ratio.

4.3.1 Auto-regressive. Label propagation (LP) [175–177] is a widely used methodology for graph-based learning. The objective of LP is two-fold: one is to extract embeddings that match with the node label, the other is to become similar to neighboring vertices. The label can be treated as part of node attributes, so we have:

$$Z = (I + \alpha \tilde{L})^{-1} X = \frac{I}{I + \alpha(I - \tilde{A})} X = \frac{I}{(1 + \alpha)I - \alpha \tilde{A}} X, \quad (42)$$

which is the closed-form solution and also equivalent to the form of Equation (39), i.e., $P = I$ and $Q = (1 + \alpha)I - \alpha \tilde{A}$.

4.3.2 Personalized PageRank (PPNP). Obtaining node's representation via teleport (restart), PPNP [51, 53, 178] keeps the original representation (self-aggregation) X with probability α . Therefore, $1 - \alpha$ is the probability of performing the normal label propagation:

$$Z = \alpha \left(I - (1 - \alpha) \tilde{A} \right)^{-1} X = \frac{\alpha}{I - (1 - \alpha) \tilde{A}} X, \quad (43)$$

where $\tilde{A} = D^{-1}A$ is random walk normalized adjacency matrix with self-loop. Equation (43) is with a rational function whose numerator is a constant.

4.3.3 ARMA Filter. ARMA [109] filter approximates any desired filter response function with updates as:

$$Z = \frac{b}{I - a \tilde{A}} X. \quad (44)$$

Note that ARMA filter is an unnormalized version of PPNP. When $a + b = 1$, ARMA becomes PPNP.

4.3.4 ParWalks. A partially absorbing random walk is a second-order Markov chain with partial absorption at each state. Reference [92] shows that with proper absorption, the absorption probabilities can well capture the global graph structure. Note that the concept “absorption” in Reference [92] is similar to “teleport” or “restart” in PPNP [53]. ParWalks defines the aggregation as:

$$p_{ij} = \begin{cases} \frac{\alpha_i}{\alpha_i + d_i}, & i = j \\ \frac{w_{ij}}{\alpha_i + d_i}, & i \neq j, \end{cases} \quad (45)$$

where α is defined as a variable to control the level of absorption, w_{ij} and d_i indicate non-negative matrix of pairwise affinities between vertex i and j , and degree of vertex i , respectively.

$$Z = \frac{\alpha I}{\alpha I + \tilde{L}} X = \frac{\alpha}{\alpha I + I - \tilde{A}} X, \quad (46)$$

where α is redefined as a regularizer in the original paper [92]. When $\alpha = 1$, all nodes follow the same absorbing behavior. Otherwise, each node has an independent absorbing policy. Also,

ParWalks model is equivalent to ARMA filter ($a = b = \frac{1}{2}$) when $\alpha = 1$ and with normalized Laplacian:

$$Z = \frac{I}{I + \tilde{L}} X = \frac{I}{I + (I - \tilde{A})} X = \frac{\frac{1}{2} I}{I - \frac{1}{2} \tilde{A}} X. \quad (47)$$

The author also discussed the over-smoothing issue: When $\Lambda = I$ and as $\alpha \rightarrow 0$, a ParWalk would converge to the constant distribution $1/n$, regardless of the starting vertex.

4.3.5 RationalNet. To leverage higher order of neighbors, RationalNet [103] proposes a general rational function with a predefined order number, and it is optimized by Remez algorithm. The analytic form is exactly Equation (39). The major difference beyond PPNP or ARMA filter is that RationalNet generalized them, and the order can be any number.

Remark: The optimization towards rational aggregation (A-3) is exactly the same as the *residual learning* that was first and widely used in image recognition [179]. As shown in the work PPAP [48], the author proposed an iterative algorithm called APPAP, which is

$$Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha Z^0 = \sum_i^k \underbrace{\alpha(1 - \alpha)^i \tilde{A}^i Z^{(0)}}_{F(x) \text{ where } x=Z^{(0)}} + \underbrace{\alpha Z^0}_{\text{identity } x}, \quad (48)$$

where Z^k means the intermediate representations at k th layer. This format is exactly the same as residual learning, as illustrated in Figure 6: sum of multiple graph convolutions serves as $F(x)$, and each time identity input will be added. The difference is that Equation (48) has normalized the weights ($(1 - \alpha) + \alpha = 1$). Because the matrix inversion in Rational Aggregation is computationally costly, iterative methods are commonly used to efficiently determine the matrix inversion [53, 105, 109]. We only demonstrate one iteration of Auto-Regression, PPNP, ARMA, and ParWalks. Multiple iterations will result in a more sophisticated rational function. The main distinction between rational and polynomial aggregation is whether or not the inverse graph Laplacian polynomial exists. In each cycle of rational aggregation, a fixed ratio for the original representation is always reserved, whereas polynomial aggregation does not. However, calculating the inverse of the graph Laplacian is expensive, making iterative fashion a key object in online learning algorithms [180]. Utilizing the concept of conductance, where f represents a heat distribution across the vertices, $L(f)$ signifies the flux induced by f throughout the graph. Consequently, in accordance with the representer theorem [181, 182], $f(\mathcal{V}_i) = L^{-1}(L(f))$ can represent an operation that reconstructs the heat distribution f from this flux. Thus, when L sends a heat distribution f over each node to flux through each vertex, L^{-1} sends some of the fluxes over the graph back to the original heat distribution (i.e., keeps part of fluxes itself). Going back to the graph learning application, we first translate our updated “heat distribution” to flux through all of those nodes by calculating $P(L(f))$. M th degree of $P(\cdot)$ means that each vertex can update M th neighbors at most. Then, using another updated flux in the reverse direction, $Q(L(f))^{-1}$ will adjust or reduce flux within N th neighbors. Polynomial aggregation with more layers or a higher degree tends to involve more neighbors, increasing capacity. When utilizing too many layers or degrees, over-smoothing is almost always unavoidable (e.g., all nodes are

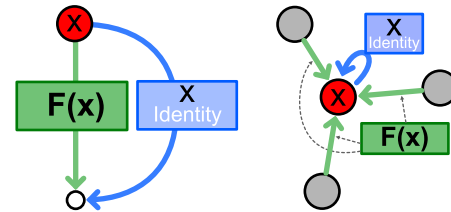


Fig. 6. Left: Residual Learning $x' = F(x) + x$; Right: Rational Aggregation: $x' = F(x) + x$

Table 5. Summary of Representative GNNs

	(A-1) linear function of $\mathbf{A}$	(B-1) linear function of Λ
GCN	$\mathbf{I} + \tilde{\mathbf{A}}$	$2 - \Lambda$
GraphSAGE	$\hat{\mathbf{D}}^{-1} + \tilde{\mathbf{A}}$	$2 - \Lambda$
GIN	$(1 + \epsilon)\mathbf{I} + \mathbf{A}$	$2 + \epsilon - \Lambda$
	(A-2) polynomial function of $\mathbf{A}$	(B-2) polynomial function of Λ
ChebNet	$\phi\mathbf{I} + \sum_{i=1}^k \psi_i \tilde{\mathbf{A}}^i$	$\tilde{\theta}_0 \cdot 1 + \tilde{\theta}_1 \Lambda + \tilde{\theta}_2 \Lambda^2 + \dots$
DeepWalk	$\frac{1}{t+1} \left(\mathbf{I} + \tilde{\mathbf{A}} + \tilde{\mathbf{A}}^2 + \dots + \tilde{\mathbf{A}}^t \right)$	$\frac{1}{t+1} \left[\dots + \left((-1)^{t-1} + \binom{1}{t} (-1)^{t-1} \right) + \dots \right]$
DCNN	$\psi_1 \tilde{\mathbf{A}} + \psi_2 \tilde{\mathbf{A}}^2 + \psi_3 \tilde{\mathbf{A}}^3 + \dots$	$\theta_1 \Lambda + \theta_2 \Lambda^2 + \theta_3 \Lambda^3 + \dots$
GDC	$\psi_1 \tilde{\mathbf{A}} + \psi_2 \tilde{\mathbf{A}}^2 + \psi_3 \tilde{\mathbf{A}}^3 + \dots$	$\theta_1 \Lambda + \theta_2 \Lambda^2 + \theta_3 \Lambda^3 + \dots$
Node2Vec	$\frac{1}{p}\mathbf{I} + \left(1 - \frac{1}{q}\right) \tilde{\mathbf{A}} + \frac{1}{q} \tilde{\mathbf{A}}^2$	$\left(1 + \frac{1}{p}\right) - \left(1 + \frac{1}{q}\right) \Lambda + \frac{1}{q} \Lambda^2$
LINE/SDNE	$\psi_1 \tilde{\mathbf{A}} + \psi_2 \tilde{\mathbf{A}}^2$	$\theta_1 \Lambda + \theta_2 \Lambda^2$
SGC	$0 \cdot \mathbf{I} + 0 \cdot \tilde{\mathbf{A}} + 0 \cdot \tilde{\mathbf{A}}^2 + \dots + 1 \cdot \tilde{\mathbf{A}}^K$	$\binom{K}{0} + \binom{K}{1} \Lambda^1 + \binom{K}{2} \Lambda^2 + \dots + \Lambda^n$
	(A-3) rational function of $\mathbf{A}$	(B-3) rational function of Λ
Auto-Regress	$\frac{1}{(1+\alpha)\mathbf{I} - \alpha \tilde{\mathbf{A}}}$	$\frac{1}{1+\alpha(1-\Lambda)}$
PPNP	$\frac{\alpha}{1 - (1-\alpha) \tilde{\mathbf{A}}}$	$\frac{\alpha}{\alpha\mathbf{I} + (1-\alpha)\Lambda}$
ARMA	$\frac{b}{(1-a\tilde{\mathbf{A}})}$	$\frac{b}{(1-a+a\Lambda)}$
ParWalk	$\frac{\alpha}{\alpha\mathbf{I} + 1 - \tilde{\mathbf{A}}}$	$\frac{\beta}{\beta + \Lambda}$

similar). However, unless all of the layers or degrees are tried, determining the appropriate number of layers or degrees is difficult. The over-smoothing problem is largely overcome by the “sending back” (i.e., teleport) behavior of rational aggregation, in which the out-degree flux is restrained even if excesses of graph convolutional layers or approximation degrees are added [53].

Three groups of spatial methods introduced above (i.e., A-1, A-2, A-3) are strongly connected under *generalization* and *specialization* relationship, as shown in Figure 1. **Generalization:** By adding more neighbors of higher rank, Linear Aggregation (A-1) can be expanded to Polynomial Aggregation (A-2). By adding reverse aggregation, Polynomial Aggregation (A-2) can be advanced to Rational Aggregation (A-3); **Specialization:** Linear Aggregation (A-1) is a special case of Polynomial Aggregation where the order is set to 1. (A-2). Rational Aggregation (A-3) degenerates into Polynomial Aggregation when reverse aggregation is removed (A-2).

5 SPECTRAL-BASED GNNS (B-0)

The use of eigen-decomposition and analysis of the weight-adjusting function (i.e., frequency filter function or frequency response function) on eigenvalues of graph matrices are both parts of graph spectral theory. In spectral-based GNNs (B-0), weights are applied to frequency components (eigenvectors) to recover the target signal using the filter’s output. Accordingly, we propose a new taxonomy for graph neural networks, dividing spectral-based GNNs into three subgroups, depending on the types of response filtering functions. In addition, the same set of representative models discussed in Section 4 will be analyzed under spectral view. To facilitate comprehension of the analysis, their spatial and spectral analytical forms are listed in Table 5. The detailed transformation of equations in category B-0 is deferred to the Appendix.

5.1 Linear Approximation (B-1)

Changing the weights of frequency components in the spectrum domain has been the subject of several research. The filter function's objective is to suit the intended output by adjusting eigenvalues. Many of them have been shown to be low-pass filters [170], which implies that only low-frequency components are highlighted, i.e., the first few eigenvalues are increased, while the rest are decreased. There are several studies that may be understood as changing frequency component weights during aggregation. A linear g function is used in particular:

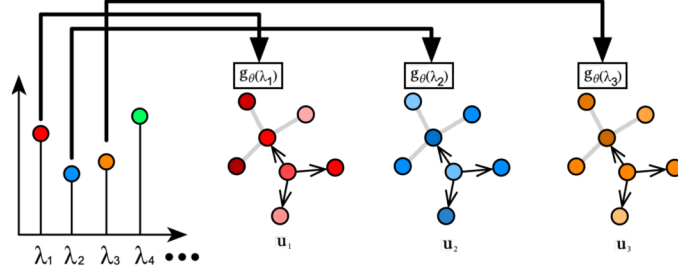


Fig. 7. Illustration of B-1: A linear function g of the eigenvalues.

where $\mathbf{u}_i$ is the i th eigenvector, and g is *frequency filter function* or *frequency response function* controlled by parameters θ , with selected l lowest frequency components. The goal of g is to change the weights of eigenvalues to fit the target output. As shown in Figure 7, B-1 updates the weights of eigenvectors ($\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3 \dots$) as $g_\theta(\lambda)$, which is a linear function. Several state-of-the-art methods introduced in Section 4 are analyzed to provide a better understanding of this scheme.

$$\mathbf{Z} = \left(\sum_{i=0}^l \theta_i \lambda_i \mathbf{u}_i \mathbf{u}_i^T \right) \mathbf{X} = \mathbf{U} \mathbf{g}_\theta(\Lambda) \mathbf{U}^T \mathbf{X}, \quad (49)$$

where $\mathbf{u}_i$ is the i th eigenvector, and g is *frequency filter function* or *frequency response function* controlled by parameters θ , with selected l lowest frequency components. The goal of g is to change the weights of eigenvalues to fit the target output. As shown in Figure 7, B-1 updates the weights of eigenvectors ($\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3 \dots$) as $g_\theta(\lambda)$, which is a linear function. Several state-of-the-art methods introduced in Section 4 are analyzed to provide a better understanding of this scheme.

Remark: The aforementioned methods apply linear low-pass filtering, and the only difference among them is that the bias is different (i.e., 2 for GCN, 2 for GraphSAGE, and $2+\epsilon$ for GIN). Therefore, we study the influence of bias on the filter function and define a metric:

$$w(\lambda_i) = \frac{|bias - \lambda_i|}{\sum_j |bias - \lambda_j|}, \quad (50)$$

which indicates the overall proportion change of each eigenvalue after applying the response function. A large adjusted value means that the filtering will enlarge the influence of the corresponding eigenvector. The range of the eigenvalue is in $[0, 2)$ for the normalized Laplacian matrix [183]. If let $bias$ be larger than or equal to 2, then we have:

$$w(\lambda_i) = \frac{bias - \lambda_i}{N \cdot bias - \sum_j \lambda_j} = \overbrace{\frac{-1}{N \cdot bias - \sum_j \lambda_j}}^{\text{slope}} \lambda_i + \overbrace{\frac{bias}{N \cdot bias - \sum_j \lambda_j}}^{\text{intercept}}, \quad (51)$$

when $bias \geq 2$, the slope is negative, which means that the filter function is low-pass filtering: As the bias increases, the slope becomes larger, and larger weights are assigned to low-frequency spectral components. Therefore, the bias of all studies in this subsection is larger or equal to 2.

5.2 Order of Approximation (B-2)

Considering higher order of frequency, filter function can approximate any smooth filter function, because it is equivalent to applying the polynomial approximation. Therefore, introducing higher-order of frequencies boosts the representation power of filter function in simulating spectral signal.

Formally, this type of work can be written as:

$$\mathbf{Z} = \left(\sum_{i=0}^l \sum_{j=0}^k \theta_j \lambda_i^j \mathbf{u}_i \mathbf{u}_i^\top \right) \mathbf{X} = \mathbf{U} \mathbf{P}_\theta(\Lambda) \mathbf{U}^\top \mathbf{X}, \quad (52)$$

where $\mathbf{g}(\cdot) = \mathbf{P}_\theta(\cdot)$ is a polynomial function. As shown in Figure 8, B-2 updates the weights of eigenvectors $(\mathbf{u}_1, \mathbf{u}_2, \dots)$ as $\mathbf{P}_\theta(\lambda)$, which is a polynomial function.

Remark: Polynomial approximation, in theory, gets more accurate as the order grows [81–85]. It is worth noting that Linear Approximation (B-1) can be thought of as a polynomial approximation of order 1. We look into polynomial approximation on the $\text{sign}(x)$ function, comparing and contrasting all of the cases in Polynomial Approximation (B-2). Because it is difficult for any straight line to suit a jump signal, as shown in Figure 9(a), linear functions cannot accurately approximate $\text{sign}(x)$. The situation improves dramatically when polynomial approximation is used, as demonstrated in Figure 9(b).

The variance will be greatly decreased if the order of the polynomial function is increased (Figure 9(c)). To recapitulate, higher-order polynomial approximation is more accurate than lower-order polynomial approximation, but it comes at the expense of increased computational complexity. Node2Vec/LINE/SDNE

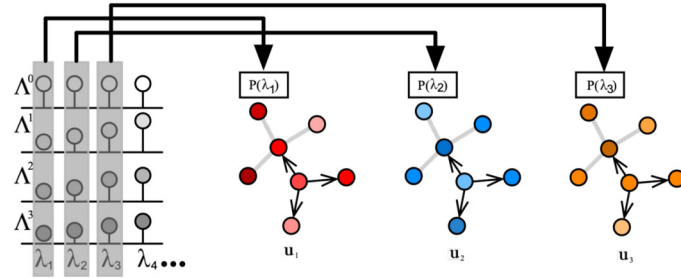


Fig. 8. Illustration of B-2: A polynomial function $\mathbf{P}$ of the eigenvalues.

with an order of 2 have lesser approximation power than those with more than 2 layers/orders, because the latter's order is predefined and can be as large as possible (e.g., ChebNet [9], DeepWalk [48], Diffusion CNN [11], Simple Graph Convolution [106]).

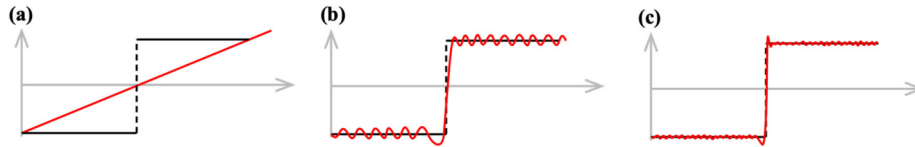


Fig. 9. Approximation for $\text{sign}(x)$ (in black): (a) linear approximation (b) polynomial approximation with low orders, (c) polynomial approximation with high orders.

5.3 Approximation Type (B-3)

Despite its widespread use and experimental success, polynomial approximation only works when applied to a smooth spectral signal. Real-world signals, however, cannot be guaranteed to be smooth. As a result, the rational approximation is employed to improve the accuracy of non-smooth signal modeling. An example of a rational kernel-based technique is as follows:

$$\mathbf{Z} = \left(\sum_i^l \frac{\sum_{j=0}^k \theta_j \lambda_i^j}{\sum_{m=1}^n \phi_m \lambda_i^m + 1} \mathbf{u}_i \mathbf{u}_i^\top \right) \mathbf{X} = \mathbf{U} \frac{\mathbf{P}_\theta(\Lambda)}{\mathbf{Q}_\phi(\Lambda)} \mathbf{U}^\top \mathbf{X}, \quad (53)$$

where $\mathbf{g}(\cdot) = \frac{\mathbf{P}_\theta(\cdot)}{\mathbf{Q}_\phi(\cdot)}$ is a rational function, and $\mathbf{P}, \mathbf{Q}$ are independent polynomial functions. Spectral methods process graph as a signal in the frequency domain. As shown in Figure 10, B-3 updates the weights of eigenvectors $(\mathbf{u}_1, \mathbf{u}_2, \dots)$ as $\mathbf{g}_\theta(\lambda)$, which is a rational function.

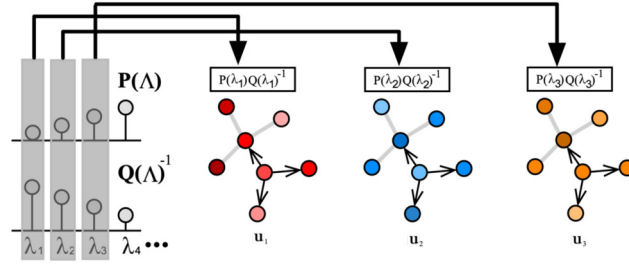
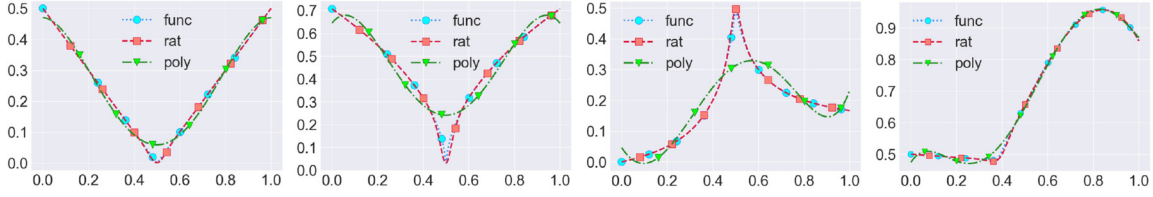


Fig. 10. Illustration of B-3: A rational function of the eigenvalues.

Fig. 11. Rational (rat) and polynomial (poly) approximation for several functions with discontinuity (func). From left to right: $\sqrt{|x - 0.5|}$; $|x - 0.5|$; $\frac{x}{10|x - 0.5| + 1}$; $\max(0.5, \sin(x + x^2)) - \frac{x}{20}$. Figures are from Reference [103].

Remark: When the function to approximate contains discontinuities, rational function has overwhelming advantage over the polynomials or linear functions. Figure 11 illustrates the difference between rational and polynomial approximation. Theoretically, rational approximation only needs exponentially less orders than that of polynomial functions [103].

6 THEORETICAL ANALYSIS

In terms of volume, spatial-based approaches outnumber spectrum-based methods in the literature [15–17, 66, 184], owing to the following reasons: (1) Spectral-based methods have a much higher computing overhead than spatial-based methods, and spectral methods are less intuitive than spatial methods. (2) Spatial-based approaches are convenient for model construction and scalability. However, from a spatial and spectral perspective, there is a tradeoff; neither has a major advantage over the other. This section outlines numerous viewpoints that demonstrate the merits and limitations of such views.

6.1 Uncertainty Principle: Global vs. Local Perspectives

Spectral-based approaches decompose data into orthogonal frequency components and examine graph filtering from the spectral domain with a global perspective. Each frequency indicates a global basis: Low-frequency components emphasize local weights with little variation, whereas high-frequency components are linked to significant variance in neighborhood. In other words, the Laplacian spectrum reflects topological properties: The first few eigenvalues are related with substantial community structure, while the last few eigenvalues indicate the graph’s bipartiteness [185, 186]. A typical low-pass filtering function for eigenvalues is shown on the left of Figure 12, which raises small eigenvalues while decreasing adjusted values for large eigenvalues. Only low-frequency components are maintained in this scenario, and neighbors have little variance.

Filtering patterns from the local neighborhood are characterized by spatial-based approaches. Most GNNs assume homophily among neighbors, so signals traversing across the neighborhood are smooth or with little variation, which is exactly the same concept as low-pass filtering. The relationship between low-pass filtering in the spectral domain (left) and its effects in the spatial domain (middle and right) is depicted in Figure 12 [186]. It appears at first glance that global and

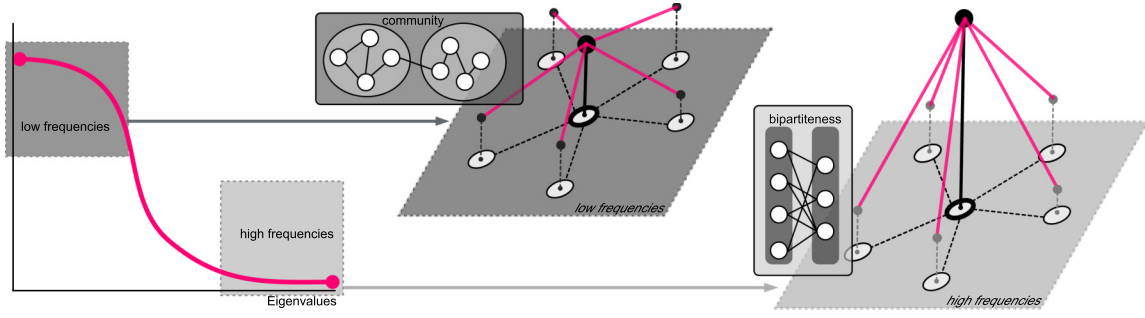


Fig. 12. Connection between the spatial and spectral perspective.

local viewpoints differ greatly, but on closer inspection, they depict the same signal in very different ways: Filtering in the spectral domain that does low-pass/high-pass filtering is analogous to learning which neighbors are similar/dissimilar. While it is true that the two types of observations yield similar results, this does not entail that they are identical. It is impossible to know an unknown quantity's value with absolute confidence in quantum mechanics because of the Heisenberg's uncertainty principle [187]. Specifically,

$$\Delta_t^2 \Delta_\omega^2 \geq \frac{1}{4}, \quad (54)$$

where Δ_t and Δ_ω denote time spread and frequency spread, respectively. Signal concentration can also be impacted by the concentration of time and frequency. A graph representing the tradeoff between a signal's localization on a graph and in its spectral domain is created, which is influenced by the uncertainty principle of quantum mechanics [188]. A lower bound on the product of the two spreads is obtained by quantifying the spreads in the vertex and the spectral domain of a graph signal x . This result suggests that applying spatial-based GNNs results in accuracy loss in the spectral domain, while using spectral-based GNNs results in accuracy loss in the spatial domain. In sum: **(1) Global and local perspectives are strong connected:** Global observation is the process of generalizing details in local settings, while localized understanding provides details of the global picture. **(2) Global and local perspectives outperform each other in their own domains:** Clear local context tends to muddy the big picture, but a focus on global context diminishes the smaller picture. GNNs are able to expand the range of options with the addition of other preexisting works, which bridge the gap between global and local views or between spectral and spatial information, to improve the expressive potential of GNNs [58, 189–193]. According to the information above, we can state that no model can be flawless from a global or local perspective; it is only possible to have a proper tradeoff between. As shown in Table 6, four aspects are compared: **Methodology:** Spatial approaches describe local regions while working bottom-up and identify global patterns using a graph frequency approach. However, spectral methods work top-down, beginning with a graph and ending with a global observation. **Computation:** To use spatial approaches, one has to carry out a number of steps on their local region before convergence is achieved. With spectral approaches, you can get a critical component with a single-step computation. **Space Complexity:** The high space complexity of spectral approaches is associated with the massive memory storage required to load the full graph. If memory is sufficient, then the full graph can be covered by using spatial methods. However, for a smaller graph, you can choose to cover it using samplings such as sampling of regions or paths. **Stability:** To create accurate, consistent results, spatial analysis methods need to apply iterative algorithms, therefore the outcomes will vary. Eigen-decomposition is a unique feature of spectral approaches if no same eigenvalues.

Guidance of Choosing Spatial and Spectral Methods. The user can choose between spatial or spectral GNNs, depending on their previously described properties. **Distributed and Online**

Table 6. Comparison between the Spatial (A-0) and Spectral (B-0) Methods

	Methodology	Computation	Space Complexity	Stability
Spectral	Global	One-step	High	Exact
Spatial	Local	Iterative	Low	Approximate

Table 7. Comparison on Time Complexity and Expressive Power

	Linear (A-1, B-1)	Polynomial (A-2, B-2)	Rational (A-3, B-3)
Time	$O(N^2F)$	$O(N^{K+1}F)$	$O(N^{K+1}F + N^3)$
Expressivity	$O(1)$	$O(1/K)$	$O(\exp^{-\sqrt{K}})$

Learning: Spatial method is easily converted to distributed learning [194, 195], whereas spectral is difficult to transfer. Even if it is possible to approximate spectrum technique using a neural network model [196] and then use distributed learning on a neural network [197], new nodes and edges must be retrained from scratch. Alternatively, the spatial technique can effectively manage online learning with streaming data [198]. **Global View:** The spectral method may provide a global perspective that the spatial method lacks. In situations where the group form is not spherical, the spatial method may disregard this influence and continue to follow the circular shape as a prospective group [199, 200]. This can be remedied by employing clustering before to the spatial technique [201]. This also renders spatial methods more locally interpretable and obscures their global perspective.

6.2 Comparison between Linear, Polynomial, and Rational Methods

Linear methods (A-1 and B-1) have a time complexity of $O(N^2F)$ due to the matrix multiplication of $\mathbf{A}\mathbf{X}$. Accordingly, polynomial and rational method are analyzed in Table 7 where K is the order number. To compare their expressive power, the convergence rate on challenging jump signal is employed as a benchmark [103] (a smooth signal cannot distinguish them). As shown in Table 7, rational methods (A-3 and B-3) converge exponentially faster than linear methods (A-1/B-1), and polynomial methods (A-2/B-2) converge linearly faster than linear methods (A-1/B-1). Therefore, there is a tradeoff between the expressive power and computational efficiency. Linear methods (A-1/B-1) have the best efficiency but only capture the linear relationship. Rational methods (A-3/B-3) consume the most considerable overhead but could tackle more challenging signals.

Experiments are undertaken to highlight our theoretical analysis of spatial and spectral approaches by comparing the differences between the three underlying groups. We chose one typical technique for linear filter [8], polynomial filter [9], and rational filter [109]. Note that we only distinguish them in the function of $\mathbf{A}$ or $\mathbf{\Lambda}$, keeping all the other configurations the same. The dataset includes representative homophily and heterophily datasets [129, 140, 148, 202–204]. The evaluation code is released at <https://github.com/XGraph-Team/Spectral-Graph-Survey>.

As shown in Figure 13 (Left), there is no significant difference in classification accuracy in the homophily dataset between the three models, with the exception of ChebNet, which performs marginally better in *PubMed* and encounters an out-of-memory error in *physics* dataset. ChebNet exhibits inferior accuracy on the *Computers* dataset, whereas GCN is significantly superior. Figure 13 (right) illustrates that ARMA consistently outperforms the others when performing the same task on the heterophily dataset, while ChebNet consistently outperforms GCN. This demonstrates the benefits of the advanced filter function, as theoretically analyzed in Section 3.2. In terms of runtime, as shown in Figure 14, ChebNet is often beyond log 1, while GCN never goes beyond log 1. The Rational method involves a higher volume of matrix operations compared to ChebNet, occasionally exceeding the “log 1” threshold. However, its iterative optimization process enables

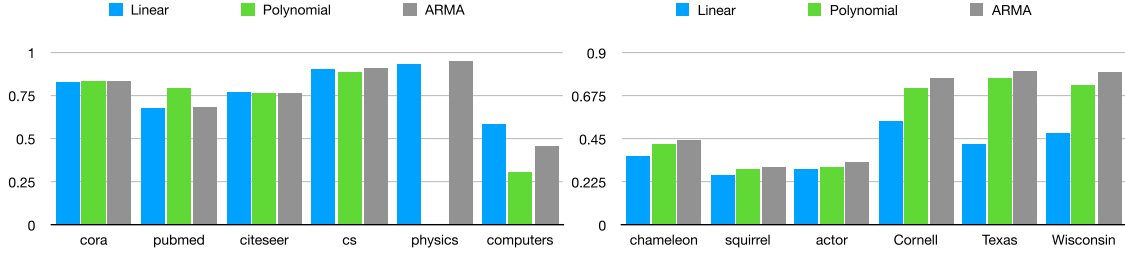


Fig. 13. Performance comparison. **Left:** homophily dataset; **Right:** heterophily dataset.

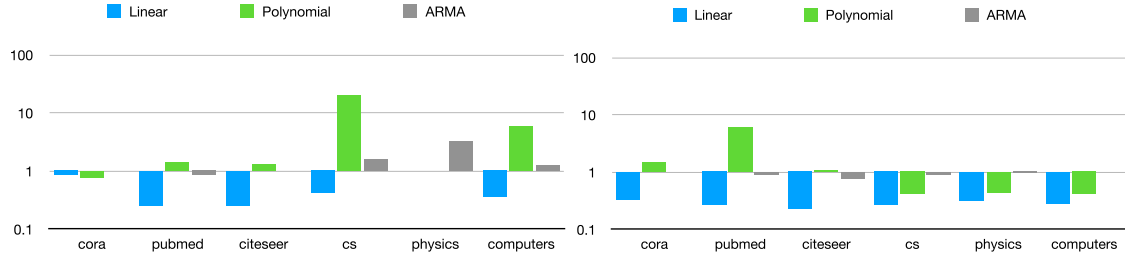


Fig. 14. Runtime comparison. **Left:** homophily dataset; **Right:** heterophily dataset. (Y-axis is log-based).

it to execute slightly faster than ChebNet. Despite this, the Rational method consistently exhibits slower performance than GCN. In terms of runtime, as shown in Figure 14, ChebNet frequently exceeds log 1, whereas GCN never does. Rational method involves more matrix operations than ChebNet, so it sometimes exceeds log 1. However, due to optimization in implementation (iterative algorithm), the rational method is slightly faster than ChebNet but consistently slower than GCN. This verifies our analysis in Section 6.2.

7 EXEMPLIFY THE PROPOSED FRAMEWORK

Over-smoothing and large-scale difficulties are two of the most difficult issues for existing GNNs, and numerous recent publications have proposed various approaches to address them. We will show in this section that all of the enhancements are still covered by our framework.

7.1 Sampling Point of View

The sampling mechanism is used as a spatial method for managing large graphs. Subgraph sampling and random walk are popular approaches.

7.1.1 Subgraph Sampling. For an early work, *GraphSAGE* [10] applies uniform node sampling for each batch, which is equivalent to subgraph sampling. The likelihood of transfer, therefore, follows random normalization (i.e., $\tilde{\mathbf{A}} = \mathbf{D}^{-1} \mathbf{A}$), which makes it part of Linear Aggregation (A-1). In the majority of follow-up works, the same methodology is used [118]: (1) build a local graph convolution for the input graph, (2) sample nodes in each layer, and (3) optimize parameters in graph convolution. Steps (2) and (3) proceed iteratively to update the weights via stochastic gradient descent [65, 99, 118, 205–207].

To avoid the recursive neighborhood expansion, FastGCN [65] treats graph convolutions as integral transformation of embedding functions and proposes Monte Carlo approach to estimate the integral. FastGCN employs importance sampling independently for each layer and reduces variance, cutting down the number of sampling nodes to constant size for all layers, exponentially shrinking the computational cost. FastGCN is proved to be importance sampling, which is better

than uniform sampling, but still suffers from unstable learning when no neighbors are selected for one node and activation is zero. To avoid taxing calculation of activation, Stochastic GCN [205] further uses the historical activation in the previous layer to avoid redundant re-evaluation. With adaptive sampling, nodes on subsequent layers are sampled to speed up GraphSAGE and Fast-GCN [99]. **Learnable graph convolutional layer (LGCL)** [206] selects a fixed number of neighboring nodes for each feature based on value ranking and transforms graph into 1-D data that is compatible with normal convolution networks. Similarly, A scalable GCN samples a fixed number of nodes, with different sampling policy called **frontier sampling (FS)**. FS maintains a constant size frontier set consisting of several vertices, which is randomly popped out with a degree-based probability distribution [208]. Cluster-GCN [209] samples a community of nodes determined by a graph clustering algorithm and computes the graph convolution within each community.

7.1.2 Random Walk. To derive node-level representations with word2vec [210–212], various random walk algorithms are proposed [48, 49, 93, 118, 166, 213]. Paths are viewed as complete sentences, and nodes are viewed as individual words. Transition probability among nodes approximates to a random walk normalized adjacency matrix if enough random walks or uniform sampling has been performed on the paths.

In the preceding section's analysis, it was noted that *DeepWalk* [48] generates multiple random paths from a graph. This results in a transfer probability for the random walk characterized by $\tilde{\mathbf{A}} = \mathbf{D}^{-1} \mathbf{A}$. Let the window size of skip-gram be $2t + 1$ and the index of current node is $t + 1$. Therefore, the updated representation is $\mathbf{Z} = \frac{1}{t+1} \mathbf{P}(\tilde{\mathbf{A}}) \mathbf{X}$ (Equation (23)). One popular word2vec configuration, i.e., **skip-gram with Negative Sampling (SGNS)**, assumes a corpus of words w and their context c . Following the work by Levy and Goldberg [214], SGNS is implicitly factorizing:

$$\log \left(\frac{|(w, c)| \cdot |\mathcal{D}|}{|w| \cdot |c|} \right) - \log b = \mathbf{A} \mathbf{B}^\top,$$

where $\mathbf{A}$ and $\mathbf{B}$ denote matrix of current node and its neighbors, respectively. $|(w, c)|$, $|w|$, $|c|$, and $\mathcal{D}$ denote the number of times word-context pair (w, c) , word w , context c , and corpus size, respectively; b is the number of negative samples. Accordingly, Reference [69] derived an exact format as:

$$\log \left(\mathbf{P}(\tilde{\mathbf{A}}) \right) - \log(b) = \log \left(\frac{|E|}{T} \left(\sum_{r=1}^T (\mathbf{D}^{-1} \mathbf{A})^r \right) \mathbf{D}^{-1} \right) - \log(b),$$

where $|E|$ and T represent edge number and step size, respectively. Consequently, the matrix designated for decomposition remains a polynomial of $\mathbf{A}$. *Node2Vec* [49] defines a second-order random walk to control the balance between **Breadth First Search (BFS)** and **Depth First Search (DFS)**. Assuming the random walk is sufficiently sampled, Node2Vec's second order can be rewritten to decompose matrix [69]:

$$\log \left(\mathbf{P}(\tilde{\mathbf{A}}) \right) - \log(b) = \log \left(\frac{\frac{1}{2T} \sum_{r=1}^T \left(\sum_u X_{w,u} \tilde{\mathbf{A}}_{c,w,u}^r + \sum_u X_{c,u} \tilde{\mathbf{A}}_{w,c,u}^r \right)}{\left(\sum_u X_{w,u} \right) \left(\sum_u X_{c,u} \right)} \right) - \log b,$$

where Node2Vec is demonstrated to be polynomial methods. *LINE* [93] and *SDNE* [166] learn the node representations within the first- and second-order neighbors, which can be treated as unconstrained version of Node2Vec:

$$\log \left(\mathbf{P}(\tilde{\mathbf{A}}) \right) - \log(b) = \log \left(|E| \tilde{\mathbf{A}} \right) - \log b.$$

GraphSAINT [118] employs multiple sampling policies but the best one is random walk. *PinSAGE* [178] improves the efficiency of GraphSAGE [10] by taking the top several neighbors with highest normalized visit counts.

Remark: Sampling methods, as a spatial methodology, seek both variance reduction and efficiency. Subgraph and random walk are equal with enough samples, since they traverse the entire network with the transition probability associated with graph connection. A-3 and B-3 do not, however, include any sample methods, mainly due to their higher computational complexity. Space complexity may be improved when sampling methods are used, but there is no guarantee that the time complexity will be much reduced because the enormous number of steps may be needed before convergence.

7.2 Over-smoothing Point of View

When a node's neighbors are highly dissimilar and when a node's embedding is already similar to its neighbors, neighborhood aggregation is not advantageous. Most GNNs perform poorly when stacking many layers, which is called the over-smoothing issue. Many related works aiming to solve the over-smoothing issue [101, 107, 119, 123, 125, 173, 193, 215–222] can be reduced to one category of our proposed framework. H2GCN [193] proposed a method that combines direct neighbors with higher-order, which is equivalent to Polynomial Aggregation (A-2). Deep GCN [107, 215, 218] developed a model with a residue module, dense connection, and dilated aggregation, which learns the weights of all different orders of neighbors. This is equivalent to Polynomial Aggregation (A-2). GPR [52] generalizes PageRank and found the equivalence of GPR and Polynomial Approximation (B-2). JKNet [101] also follows the same residue methodology as Deep GCN. DAGNN [125] stacks multiple layers, which uses different orders of propagation with the learnable weights, which makes it belong to Polynomial Aggregation (A-2). PairNorm [173] presents a two-step method that includes centering and re-scaling, which mitigates the over-smoothing from graph convolution. Therefore, PairNorm is equivalent to Rational Propagation (A-3), since re-scaling is similar to do propagation and restart at the same time. Reference [216] designed an adaptive method to dynamically adjust the weights between low-frequency and high-frequency components, resulting in two peaks in the spectral domain. This could also be modeled by Rational Aggregation (A-3) with its accuracy in jump signals. DropEdge [119, 219] randomly drops a certain number of edges to avoid over-smoothing, which can be categorized as Rational Aggregation (A-3), since dropping edge prevents the propagation and thereby provides a probability of keeping the original values of nodes. GCNII [123] applies *initial residual*, which combines the smoothed representation with an initial residual connection to the first layer and *identity mapping*, which adds an identity matrix to the weight matrix. *Initial residual* is a trick that PPAP [53] uses, which enables itself to retard the over-smoothing by keeping partial previous representations. *Identity mapping* further remains one original representation to slow down the spreading of over-smoothing propagation.

Remark: A-1 lacks state-of-the-art methods, implying it is vulnerable to over-smoothing. Applying A-1 many times with learnable weights for different ordering equates to A-2. So, A-2 might balance low (raw representations) and high orders (smoothed representations). However, too many A-1 operations may result in over-smoothing. So, for A-2, precise order configuration is required. No matter the number of orders or layers, A-3 reserves a proportion of the final representation as raw representation, making itself robust to over-smoothing.

7.3 Inverse Problem Point of View

Let us consider a link represented by the variable e , which establishes a connection between two nodes, denoted as i and j . **Graph neural networks (GNNs)** can be formally expressed as a function $f : (v_i, e) \rightarrow v_j$. The function denoted as f , which is equivalent to Equation (19), incorporates the application of graph convolution and neural networks:

$$f : (A, X_i) \rightarrow X_j, \quad (55)$$

which implies that the features of node i , denoted as X_i , undergo a smoothing process to become part of the features of node j , represented as X_j .

The reconstruction of graphs [223] poses an inverse problem to that of Equation (55). The issue at hand is frequently illustrated through a task within the realm of **knowledge graphs (KGs)**, which is commonly referred to as link completion. The objective can be articulated in the following manner:

$$f : (X_i, X_j) \rightarrow A, \quad \text{or in single-instance format} \quad f : (h, t) \rightarrow r, \quad (56)$$

where relation between node i and j is denoted by A , while the head and tail entity vertices connected by relation r are represented by h/t . The semantic similarity-based scoring function is a prevalent approach in KGs for assessing the veracity of facts by means of semantic matching, as reported by Reference [224]. The conventional method involves utilizing a broad multiplicative structure, denoted as $h^\top A \approx t^\top$, in which the function A is dependent on the relationship r . This formulation exhibits a high degree of consistency with the graph convolution operator. It is worth noting that empirical evidence indicates that multiplicative models ($h^\top A \approx t^\top$) outperform additive models ($h + r \approx t$) in the context of KGs [225, 226].

Instead of allowing the usage of arbitrary linear maps represented by random matrices A to depict relations, a particular collection of matrices has been investigated for linear maps that demonstrate favorable characteristics. A theoretical analysis posits that normal matrices possess desirable theoretical properties that are commonly sought in relational modeling [227]. The diagonalization of these entities can be achieved through unitary means, thereby enabling their analysis through the spectral theorem as described in Reference [228]. The Graph Laplacian matrix is a normal matrix that is utilized to characterize the feature-wise relationship or covariance of entities, as per its definition. This matrix can elucidate the physical significance of the learned A . The aim of link completion is to establish the feature-wise relationship (A) between training pairs of (h, t) by utilizing fundamental statistical measures such as covariance. Following this, an objective function is optimized to precisely match the training data, i.e., $\min \Delta(h^\top P(A), t)$, where Δ denotes a difference metric function between $h^\top P(A)$ and t . The degree of inconsistency can be mathematically represented as $\langle h^\top P(A), t \rangle = h^\top P(A)t$, which bears a resemblance to the notion of Dirichlet energy of a signal across a graph framework. Assuming similarity between h and t , and their smoothness over a function of feature-wise graph Laplacian $P(A)$, it is expected that the value of $h^\top P(A)t$ will be minimized. Otherwise, the magnitude of the value will be significant. As demonstrated in Reference [226], the diagonal matrix A can be utilized to detect features that display no covariance, thereby signifying the independence of all features during the learning phase. Rewrite the above-mentioned link completion task in matrix format (suppose multiple pairs of (h, t)), and we have *feature-level* graph convolution:

$$\underbrace{H}_{N \times d} \underbrace{P(A)}_{d \times d} \rightarrow \underbrace{T}_{N \times d}, \quad (57)$$

where N is used to denote the number of instances, whereas d is used to represent the number of features. Thus, the aforementioned equation, denoted as Equation (57), converts the given dimension d of H into d dimensions of T . Given that matrix A is normal, it is possible to consider matrix P as a filter function applied to its eigenvalues, as previously discussed. Existing studies on knowledge graphs have observed that P is modeled as a linear function (e.g., References [229, 230]) with the form of $(A-1/B-1)$, or as a polynomial function (e.g., Reference [231]) with the form of $(A-2/B-2)$. Furthermore, GNN functions as a nascent encoder for entity graph and employs prevalent GNN models in KGs [230, 232–234]. Performing graph convolution at the *entity-level*

can be accomplished through a standard GNN scheme, which is straightforward to implement:

$$\underbrace{N \times N}_{P(A)} \underbrace{N \times d}_H \rightarrow \underbrace{N \times d}_T, \quad (58)$$

where the convolution of H is performed by $P(A)$ at the entity level, whereby each node is averaged by its neighbors. It is worth noting that Equations (57) and (58) operate at distinct levels, with the latter typically utilized for node classification as opposed to link prediction.

8 LIMIT, OPEN CHALLENGES, AND CONCLUSION

Theoretical Understanding - Most recent research has examined spatial and spectral approaches individually, and even from a theoretical standpoint. [61, 145, 235]. However, it is unclear how these two distinct interpretations can be connected. This survey demonstrates that the theoretical advantage of rational function over the others may be proved. It is still unclear how the learning approach can be optimally structured to execute this advantage. Using the partial differential equation [236], in which the diffusion equation and wave equation are analogous to polynomial and rational filters, provides an alternative viewpoint to integrate spatial and spectral views.

Directed Graph - Most contemporary work, especially spectral methods, only handle undirected graphs, due to symmetric graph matrix is readily available with off-the-shelf techniques. Many related works integrate or sum up asymmetric adjacency matrices from bi-direction into symmetric matrix, which avoids decomposing asymmetric matrix directly. It is possible to decompose asymmetric matrix by some techniques such as the Jordan norm [237]; asymmetric matrices can be used to express properties such as graph and filter complexity. Directed graphs and their decomposition can also be achievable in a certain type of geometric space called a graph manifold. As shown in Section 2.4 of Reference [238], spectral filters can be defined on directed graphs represented by non-symmetric adjacency matrices with Hermitian transpose.

Dynamic Graph - The existing work model with GNNs and RNNs is built using graph convolutional networks and recurrent neural networks, which lack transparency. Due to the limited expressive power of RNNs, the task is constrained to prediction, and it is unable to perform long-term sequence processing well. Graph dynamics has a large number of valuable tasks, such as inferring the structure of a graph, constructing a joint dynamic of structure and attributes, and exploring the connection between structure and mass flow. It is possible to use graph spectra to detect the patterns in dynamics of graph [239–241]. Due to the challenge of the long-dependency of a path, the spectral method can also provide a potential way to model trajectory prediction, as the spectra of trajectory implicitly reveal its relationship with the whole graph [242].

Higher-order Interaction and Combinatorial Optimization - Most current work falls into the first-order relationship at the node-level, but higher-order interaction is either ignored or implicitly included. Existing explainable learning also focuses on the neighbors' identification [243]. Also, the existing work pays the most attention to neighbors, but remote connection or higher-order relationship with the other nodes receives little attention. Hypergraphs provide a possibility to model the combinatorial effect [244–246] Hodge Laplacian [247, 248] and simplicial complex [249–251] provide more theoretical tools for modeling this combinatorial effect.

Multilayer Network - In real-world biological and engineering systems, units are often interconnected across multiple networks. Failure in one network can trigger cascading failures across systems. Likewise, misinformation spread via social media in cyber-physical systems can lead to group-level risks like epidemic outbreaks. Such interconnected networks can be represented by *multilayer networks* that produce new degrees of freedom via coupling interactions. Such “new

physics” is prevalent in multilayer systems, but they are still poorly understood [252–255]. Graph neural network research in this crucial area is scarce [256].

APPENDICES

Details of equation transformation for Section 5 are listed below.

A B-1

A.1 Graph Convolutional Network (GCN)

Rewriting GCN [8] in spectral domain, we have:

$$\mathbf{Z} = \tilde{\mathbf{A}} \mathbf{X} = \mathbf{D}^{-\frac{1}{2}} (\mathbf{A} + \mathbf{I}) \mathbf{D}^{-\frac{1}{2}} \mathbf{X} = \mathbf{D}^{-\frac{1}{2}} (\mathbf{D} - \mathbf{L} + \mathbf{I}) \mathbf{D}^{-\frac{1}{2}} \mathbf{X} = (\mathbf{I} - \mathbf{D}^{-\frac{1}{2}} \mathbf{L} \mathbf{D}^{-\frac{1}{2}} + \mathbf{I}) \mathbf{X} = \mathbf{U}(2 - \Lambda) \mathbf{U}^T \mathbf{X}, \quad (59)$$

where $\tilde{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}} (\mathbf{A} + \mathbf{I}) \mathbf{D}^{-\frac{1}{2}}$ is renormalization of $\tilde{\mathbf{A}}$. Therefore, the frequency response function is $g(\Lambda) = 1 - \Lambda$, which is a low-pass filter, i.e., smaller eigenvalues that correspond to low frequency components are assigned with a larger value.

A.2 GraphSAGE

Considering the MEAN aggregation as example, we can rewrite GraphSAGE [10] in matrix form:

$$\mathbf{Z} = \mathbf{D}^{-\frac{1}{2}} (\mathbf{I} + \mathbf{A}) \mathbf{D}^{-\frac{1}{2}} \mathbf{X} = (\mathbf{I} + \tilde{\mathbf{A}}) \mathbf{X} = (2\mathbf{I} - \tilde{\mathbf{L}}) \mathbf{X} = \mathbf{U}(2 - \Lambda) \mathbf{U}^T \mathbf{X}. \quad (60)$$

Hence, the frequency response function is $g(\Lambda) = 2 - \Lambda$, which is a low-pass filtering. Note that GraphSAGE’s normalization is different from GCN, which utilizes the renormalization trick.

$$(\mathbf{I} + \mathbf{D}^{-1} \mathbf{A}) \mathbf{X} \quad (61) \quad \mathbf{U} (1 + \Lambda_P) \mathbf{U}^T \mathbf{X}, \quad (62)$$

where $P = \mathbf{D}^{-1} \mathbf{A}$

A.3 Graph Isomorphism Network (GIN)

A multi-layer neural network has the capability to conform to the normalization scale (i.e., normalization) [8], so GIN [61] can be rewritten as:

$$\mathbf{Z} = \mathbf{D}^{-\frac{1}{2}} [(1 + \epsilon) \mathbf{I} + \mathbf{A}] \mathbf{D}^{-\frac{1}{2}} \mathbf{X} = \mathbf{D}^{-\frac{1}{2}} [(2 + \epsilon) \mathbf{I} - \tilde{\mathbf{L}}] \mathbf{D}^{-\frac{1}{2}} \mathbf{X} = \mathbf{U}(2 + \epsilon - \Lambda) \mathbf{U}^T \mathbf{X}. \quad (63)$$

GIN can be seen as a generalization of GCN or GraphSAGE without normalized adjacency matrix $\mathbf{A}$. The frequency response function is $g(\Lambda) = 2 + \epsilon - \Lambda$, which is low-pass.

B B-2

B.1 ChebNet

As analyzed in Equation (21), ChebNet [64] can be written as:

$$\sum_{k=0}^{K-1} \theta_k T_k(\tilde{\mathbf{L}}) \mathbf{X} = (\tilde{\theta}_0 \mathbf{I} + \tilde{\theta}_1 \tilde{\mathbf{L}} + \tilde{\theta}_2 \tilde{\mathbf{L}}^2 + \dots) \mathbf{X}, \quad (64)$$

where $T_k(\cdot)$ is the Chebyshev polynomial and θ_k is the Chebyshev coefficient. $\tilde{\theta}$ is the coefficient after expansion and reorganization. Therefore, we can rewrite it as:

$$\sum_{k=0}^{K-1} \theta_k T_k(\tilde{\mathbf{L}}) \mathbf{X} = \mathbf{U}(\tilde{\theta}_0 \cdot 1 + \tilde{\theta}_1 \Lambda + \tilde{\theta}_2 \Lambda^2 + \dots) \mathbf{U}^T \mathbf{X}, \quad (65)$$

where spectral response function is $g(\Lambda) = \tilde{\theta}_0 \cdot 1 + \tilde{\theta}_1 \Lambda + \tilde{\theta}_2 \Lambda^2 + \dots = \mathbf{P}(\Lambda)$.

B.2 DeepWalk

Starting from Equation (23), DeepWalk [48] can be rewritten as:

$$\begin{aligned}
 \mathbf{Z} &= \frac{1}{t+1} (\mathbf{I} + \tilde{\mathbf{A}} + \tilde{\mathbf{A}}^2 + \dots + \tilde{\mathbf{A}}^t) \mathbf{X} \\
 &= \frac{1}{t+1} (\mathbf{I} + (\mathbf{I} - \tilde{\mathbf{L}}) + (\mathbf{I} - \tilde{\mathbf{L}})^2 + \dots + (\mathbf{I} - \tilde{\mathbf{L}})^t) \mathbf{X} \\
 &= \frac{1}{t+1} \left(2\mathbf{I} + (-1 - 2 - 3 - \dots) \tilde{\mathbf{L}} + (1 + 3 + 6 + \dots) \tilde{\mathbf{L}}^2 + \dots \left((-1)^{t-1} + \binom{1}{t} (-1)^{t-1} \right) \tilde{\mathbf{L}}^{t-1} + (-1)^t \tilde{\mathbf{L}}^t \right) \mathbf{X} \\
 &= (\theta_0 \mathbf{I} + \theta_1 \tilde{\mathbf{L}} + \theta_2 \tilde{\mathbf{L}}^2 + \dots + \theta_t \tilde{\mathbf{L}}^t) \mathbf{X} \\
 &= \mathbf{U}(\theta_0 + \theta_1 \Lambda + \theta_2 \Lambda^2 + \dots + \theta_t \Lambda^t) \mathbf{U}^\top \mathbf{X} \\
 &= \mathbf{U} \mathbf{P}_{k=r}(\Lambda) \mathbf{U}^\top \mathbf{X},
 \end{aligned}$$

where $\mathbf{g}(\Lambda) = \theta_0 + \theta_1 \Lambda + \theta_2 \Lambda^2 + \dots + \theta_t \Lambda^t$, and all parameters θ_i are determined by the predefined step size t .

B.3 Scalable Inception Graph Neural Networks (SIGN)

Substituting $\tilde{\mathbf{A}} = \mathbf{I} - \tilde{\mathbf{L}}$ in Equation (28), it can be rewritten as:

$$\hat{\mathbf{Z}} = \sum_r \omega_r \widehat{(\mathbf{I} - \tilde{\mathbf{L}})^r} \mathbf{X} = \mathbf{U} \widehat{\mathbf{P}_{k=r}(\Lambda)} \mathbf{X} \mathbf{U}^\top. \quad (66)$$

B.4 Graph Diffusion Convolution (GDC)

Substituting $\tilde{\mathbf{A}} = \mathbf{I} - \tilde{\mathbf{L}}$, general case in Equation (30) can be written as:

$$\mathbf{Z} = \sum_{k=0}^{\infty} \theta_k (\mathbf{I} - \tilde{\mathbf{L}}) = \mathbf{U} \sum_{k=0}^{\infty} \theta_k (1 - \Lambda)^k \mathbf{U}^\top = \mathbf{U} \mathbf{P}(\Lambda) \mathbf{X} \mathbf{U}^\top. \quad (67)$$

B.5 Diffusion Convolutional Neural Networks (DCNN)

As analyzed in Equation (24), DCNN [11] can be transformed with $\tilde{\mathbf{A}} = \mathbf{I} - \tilde{\mathbf{L}}$ as:

$$\mathbf{Z} = \mathbf{P}(\mathbf{I} - \tilde{\mathbf{L}}) \mathbf{X} = \mathbf{U} \mathbf{P}(\Lambda) \mathbf{X} \mathbf{U}^\top, \quad (68)$$

which is equivalent to ChebNet, and parameters θ_i are learnable.

B.6 Node2Vec

Node2Vec [49] can be rewritten in matrix form as Equation (32). Then it can be transformed and reorganized after substituting $\tilde{\mathbf{A}} = \mathbf{I} - \tilde{\mathbf{L}}$:

$$\mathbf{Z} = \left[\left(1 + \frac{1}{p} \right) \mathbf{I} - \left(1 + \frac{1}{q} \right) \tilde{\mathbf{L}} + \frac{1}{q} \tilde{\mathbf{L}}^2 \right] \mathbf{X} = \mathbf{U} \left[\left(1 + \frac{1}{p} \right) - \left(1 + \frac{1}{q} \right) \Lambda + \frac{1}{q} \Lambda^2 \right] \mathbf{U}^\top \mathbf{X}. \quad (69)$$

Therefore, Node2Vec's frequency response function is:

$$\mathbf{g}(\Lambda) = \left(1 + \frac{1}{p} \right) - \left(1 + \frac{1}{q} \right) \Lambda + \frac{1}{q} \Lambda^2, \quad (70)$$

which integrates a second-order function of Λ with predefined parameters, i.e., p and q .

B.7 LINE/SDNE

As described in Equation (36), LINE [165] and SDNE [166] can be rewritten as:

$$\mathbf{Z} = \tilde{\mathbf{A}} \mathbf{X} + \alpha \tilde{\mathbf{A}}^2 \mathbf{X} = (\mathbf{I} - \tilde{\mathbf{L}}) \mathbf{X} + \alpha (\mathbf{I} - \tilde{\mathbf{L}})^2 \mathbf{X} = \mathbf{U}[(\mathbf{I} - \Lambda) + \alpha(1 - \Lambda)^2] \mathbf{X} \mathbf{U}^\top = \mathbf{U} \mathbf{P}_{k=2}(\Lambda) \mathbf{U}^\top \mathbf{X}, \quad (71)$$

where response function $\mathbf{g}(\Lambda) = \Lambda + \alpha \Lambda^2$ is a polynomial function with order 2.

B.8 Simple Graph Convolution (SGC)

As analyzed in Equation (38), SGC can be transformed as:

$$\begin{aligned} Z &= (\mathbf{I} - \tilde{\mathbf{L}})^k \mathbf{X} = \left[\begin{pmatrix} k \\ 0 \end{pmatrix} \mathbf{I} + \begin{pmatrix} k \\ 1 \end{pmatrix} \tilde{\mathbf{L}}^1 + \begin{pmatrix} k \\ 2 \end{pmatrix} \tilde{\mathbf{L}}^2 + \cdots + \tilde{\mathbf{L}}^k \right] \mathbf{X} \\ &= \mathbf{U} \left[\begin{pmatrix} k \\ 0 \end{pmatrix} + \begin{pmatrix} k \\ 1 \end{pmatrix} \Lambda^1 + \begin{pmatrix} k \\ 2 \end{pmatrix} \Lambda^2 + \cdots + \Lambda^k \right] \mathbf{U}^\top \mathbf{X}, \end{aligned}$$

where spectral response function is a polynomial function of order k :

$$g(\Lambda) = \begin{pmatrix} k \\ 0 \end{pmatrix} + \begin{pmatrix} k \\ 1 \end{pmatrix} \Lambda^1 + \begin{pmatrix} k \\ 2 \end{pmatrix} \Lambda^2 + \cdots + \Lambda^k.$$

B.9 Improved GCN (IGCN)

By stacking multiple layers, IGCN [170] is proposed as:

$$\mathbf{Z} = \tilde{\mathbf{L}}^k \mathbf{X} = \mathbf{U} \Lambda^k \mathbf{U}^\top \mathbf{X}, \quad (72)$$

where the spectral response function is a polynomial function with order k .

C B-3

C.1 Auto-regressive Filter

Label propagation (LP) [175–177] is a prevailing methodology for graph-based learning. The objective of LP is two-fold: one is to extract embeddings that match with the label, the other is to be similar with neighboring vertices. Label can be treated as part of node attributes, so we can have:

$$\mathbf{Z} = (\mathbf{I} + \alpha \tilde{\mathbf{L}})^{-1} \mathbf{X} = \mathbf{U} \frac{1}{1 + \alpha(1 - \Lambda)} \mathbf{U}^\top \mathbf{X}. \quad (73)$$

C.2 PPNP

Personalized PageRank (PPNP) [53] can obtain node's representation via teleport (restart) probability α , which indicates the ratio of keeping the original representation:

$$\mathbf{Z} = \frac{\alpha}{\mathbf{I} - (1 - \alpha)(\mathbf{I} - \tilde{\mathbf{L}})} \mathbf{X} = \mathbf{U} \frac{\alpha}{\alpha + (1 - \alpha)\Lambda} \mathbf{U}^\top \mathbf{X}, \quad (74)$$

where $\tilde{\mathbf{A}} = \mathbf{D}^{-1} \mathbf{A}$ is random-walk normalized adjacency matrix with self-loop. Equation (74) is with a rational function whose numerator is a constant.

C.3 ARMA Filter

Substituting $\tilde{\mathbf{A}} = \mathbf{I} - \tilde{\mathbf{L}}$, Equation (44) can be rewritten as:

$$\mathbf{Z} = \frac{b}{\mathbf{I} - a(\mathbf{I} - \tilde{\mathbf{L}})} \mathbf{X} = \mathbf{U} \frac{b}{(1 - a) + a\Lambda} \mathbf{U}^\top \mathbf{X}. \quad (75)$$

Note that ARMA filter is an unnormalized version of PPNP. When $a + b = 1$, ARMA filter becomes PPNP. Therefore, ARMA filter is more generalized than PPNP due to its unnormalization.

C.4 ParWalks

Upon decomposing the graph Laplacian, ParWalks can be expressed as follows:

$$\mathbf{Z} = \mathbf{U} \frac{\beta}{\beta + \Lambda} \mathbf{X} \mathbf{U}^\top, \quad (76)$$

when setting $\beta = \frac{\alpha}{1-\alpha}$, it becomes PPNP:

$$\mathbf{Z} = \mathbf{U} \frac{\frac{\alpha}{1-\alpha}}{\frac{\alpha}{1-\alpha} + \Lambda} \mathbf{X} \mathbf{U}^\top = \mathbf{U} \frac{\alpha}{\alpha + (1-\alpha)\Lambda} \mathbf{X} \mathbf{U}^\top. \quad (77)$$

C.5 RationalNet

Substituting $\tilde{\mathbf{A}} = \mathbf{I} - \tilde{\mathbf{L}}$, Equation (39) can be transformed to Equation (53). The frequency response function is a generalized rational function.

REFERENCES

- [1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *Nature* 521, 7553 (2015), 436.
- [2] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 779–788.
- [3] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. 2015. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 91–99.
- [4] Geoffrey Hinton, Li Deng, Dong Yu, George Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara Sainath, and Brian Kingsbury. 2012. Deep neural networks for acoustic modeling in speech recognition. *IEEE Sig. Process. Mag.* 29 (2012).
- [5] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Lukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144* (2016).
- [6] Thang Luong, Hieu Pham, and Christopher D. Manning. 2015. Effective approaches to attention-based neural machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 1412–1421.
- [7] Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann Lecun. 2014. Spectral networks and locally connected networks on graphs. In *Proceedings of the International Conference on Learning Representations (ICLR’14)*. Retrieved from: <http://openreview>
- [8] Thomas N. Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations* (2017).
- [9] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 3844–3852.
- [10] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 1024–1034.
- [11] James Atwood and Don Towsley. 2016. Diffusion-convolutional neural networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 1993–2001.
- [12] Petar Veličković, Guillem Cucurull, Arantxa Csanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph attention networks. In *International Conference on Learning Representations*.
- [13] Federico Monti, Davide Boscaini, Jonathan Masci, Emanuele Rodola, Jan Svoboda, and Michael M. Bronstein. 2017. Geometric deep learning on graphs and manifolds using mixture model CNNs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 5115–5124.
- [14] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Representation learning on graphs: Methods and applications. *arXiv preprint arXiv:1709.05584* (2017).
- [15] Ziwei Zhang, Peng Cui, and Wenwu Zhu. 2018. Deep learning on graphs: A survey. *arXiv preprint arXiv:1812.04202* (2018).
- [16] Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, and Maosong Sun. 2018. Graph neural networks: A review of methods and applications. *arXiv preprint arXiv:1812.08434* (2018).
- [17] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2019. A comprehensive survey on graph neural networks. *arXiv preprint arXiv:1901.00596* (2019).
- [18] Ferran Alet, Adarsh Keshav Jeewajee, Maria Bauza Villalonga, Alberto Rodriguez, Tomas Lozano-Perez, and Leslie Kaelbling. 2019. Graph element networks: Adaptive, structured computation and memory. In *Proceedings of the International Conference on Machine Learning*. PMLR, 212–222.

- [19] Thomas Kipf, Ethan Fetaya, Kuan-Chieh Wang, Max Welling, and Richard Zemel. 2018. Neural relational inference for interacting systems. In *Proceedings of the International Conference on Machine Learning*. PMLR, 2688–2697.
- [20] Hanjun Dai, Chengtao Li, Connor Coley, Bo Dai, and Le Song. 2019. Retrosynthesis prediction with conditional graph logic network. *Proceedings of the International Conference on Advances in Neural Information Processing Systems*.
- [21] John Bradshaw, Matt J. Kusner, Brooks Paige, Marwin H. S. Segler, and José Miguel Hernández-Lobato. 2018. A generative model for electron paths. *arXiv preprint arXiv:1805.10970* (2018).
- [22] Siddhant Arora. 2020. A survey on graph neural networks for knowledge graph completion. *arXiv preprint arXiv:2007.12374* (2020).
- [23] Zi Ye, Yogan Jaya Kumar, Goh Ong Sing, Fengyan Song, and Junsong Wang. 2022. A comprehensive survey of graph neural networks for knowledge graphs. *IEEE Access* 10 (2022), 75729–75741.
- [24] Yuyu Zhang, Xinshi Chen, Yuan Yang, Arun Ramamurthy, Bo Li, Yuan Qi, and Le Song. 2020. Efficient probabilistic logic reasoning with graph neural networks. *arXiv preprint arXiv:2001.11850* (2020).
- [25] Xiaoran Xu, Wei Feng, Yunsheng Jiang, Xiaohui Xie, Zhiqing Sun, and Zhi-Hong Deng. 2019. Dynamically pruned message passing networks for large-scale knowledge graph reasoning. *arXiv preprint arXiv:1909.11334* (2019).
- [26] Yongji Wu, Defu Lian, Yiheng Xu, Le Wu, and Enhong Chen. 2020. Graph convolutional networks with Markov random field reasoning for social spammer detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 1054–1061.
- [27] Lei Chen, Le Wu, Richang Hong, Kun Zhang, and Meng Wang. 2020. Revisiting graph based collaborative filtering: A linear residual graph convolutional network approach. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 27–34.
- [28] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph neural networks for social recommendation. In *Proceedings of the World Wide Web Conference*. 417–426.
- [29] Shoujin Wang, Liang Hu, Yan Wang, Xiangnan He, Quan Z. Sheng, Mehmet A. Orgun, Longbing Cao, Francesco Ricci, and Philip S. Yu. 2021. Graph learning based recommender systems: A review. *arXiv preprint arXiv:2105.06339* (2021).
- [30] Licheng Jiao, Jie Chen, Fang Liu, Shuyuan Yang, Chao You, Xu Liu, Lingling Li, and Biao Hou. 2022. Graph representation learning meets computer vision: A survey. *IEEE Trans. Artif. Intell.* 4, 1 (2022), 2–22.
- [31] Junyu Gao, Tianzhu Zhang, and Changsheng Xu. 2019. I know the relationships: Zero-shot action recognition via two-stream graph convolutional networks and knowledge graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 8303–8311.
- [32] Ruiyu Li, Makarand Tapaswi, Renjie Liao, Jiaya Jia, Raquel Urtasun, and Sanja Fidler. 2017. Situation recognition with graph neural networks. In *Proceedings of the IEEE International Conference on Computer Vision*. 4173–4182.
- [33] Lingfei Wu, Yu Chen, Kai Shen, Xiaojie Guo, Hanning Gao, Shucheng Li, Jian Pei, and Bo Long. 2021. Graph neural networks for natural language processing: A survey. *arXiv preprint arXiv:2106.06090* (2021).
- [34] Daniil Sorokin and Iryna Gurevych. 2018. Modeling semantics with gated graph neural networks for knowledge base question answering. *arXiv preprint arXiv:1808.04126* (2018).
- [35] Daniel Beck, Gholamreza Haffari, and Trevor Cohn. 2018. Graph-to-sequence learning using gated graph neural networks. *arXiv preprint arXiv:1806.09835* (2018).
- [36] Ryoma Sato, Makoto Yamada, and Hisashi Kashima. 2019. Approximation ratios of graph neural networks for combinatorial problems. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*.
- [37] Quentin Cappart, Didier Chételat, Elias Khalil, Andrea Lodi, Christopher Morris, and Petar Veličković. 2021. Combinatorial optimization and reasoning with graph neural networks. *arXiv preprint arXiv:2102.09544* (2021).
- [38] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. 2019. Exact combinatorial optimization with graph convolutional neural networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*.
- [39] Weiwei Jiang and Jiayun Luo. 2022. Graph neural network for traffic forecasting: A survey. *Expert Syst. Applic.* 207 (2022), 117921.
- [40] Khac-Hoai Nam Bui, Jiho Cho, and Hongsuk Yi. 2022. Spatial-temporal graph neural network for traffic forecasting: An overview and open research issues. *Appl. Intell.* 52, 3 (2022), 2763–2774.
- [41] Jiexia Ye, Juanjuan Zhao, Kejiang Ye, and Chengzhong Xu. 2020. How to build a graph-based deep learning architecture in traffic domain: A survey. *IEEE Trans. Intell. Transport. Syst.* 23, 5 (2020), 3904–3924.
- [42] Fanglan Chen, Zhiqian Chen, Subhodip Biswas, Shuo Lei, Naren Ramakrishnan, and Chang-Tien Lu. 2020. Graph convolutional networks with Kalman filtering for traffic prediction. In *Proceedings of the 28th International Conference on Advances in Geographic Information Systems*. 135–138.
- [43] Elizabeth Dinella, Hanjun Dai, Ziyang Li, Mayur Naik, Le Song, and Ke Wang. 2020. Hoppity: Learning graph transformations to detect and fix bugs in programs. In *Proceedings of the International Conference on Learning Representations (ICLR’20)*.

- [44] Jiayi Wei, Maruth Goyal, Greg Durrett, and Isil Dillig. 2020. LambdaNet: Probabilistic type inference using graph neural networks. *arXiv preprint arXiv:2005.02161* (2020).
- [45] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*.
- [46] Tian Bian, Xi Xiao, Tingyang Xu, Peilin Zhao, Wenbing Huang, Yu Rong, and Junzhou Huang. 2020. Rumor detection on social media with bi-directional graph convolutional networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 549–556.
- [47] Hao Peng, Jianxin Li, Qiran Gong, Yangqiu Song, Yuanxing Ning, Kunfeng Lai, and Philip S. Yu. 2019. Fine-grained event categorization with heterogeneous graph convolutional networks. *arXiv preprint arXiv:1906.04580* (2019).
- [48] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 701–710.
- [49] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 855–864.
- [50] Wenzheng Feng, Jie Zhang, Yuxiao Dong, Yu Han, Huanbo Luan, Qian Xu, Qiang Yang, Evgeny Kharlamov, and Jie Tang. 2020. Graph random neural network for semi-supervised learning on graphs. *arXiv preprint arXiv:2005.11079* (2020).
- [51] Aleksandar Bojchevski, Johannes Klicpera, Bryan Perozzi, Amol Kapoor, Martin Blais, Benedek Rózsemberczki, Michal Lukasik, and Stephan Günnemann. 2020. Scaling graph neural networks with approximate PageRank. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2464–2473.
- [52] Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. 2020. Adaptive universal generalized PageRank graph neural network. In *Proceedings of the International Conference on Learning Representations*.
- [53] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Predict then propagate: Graph neural networks meet personalized PageRank. In *International Conference on Learning Representations*.
- [54] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. 2019. Combining neural networks with personalized PageRank for classification on graphs. In *Proceedings of the International Conference on Learning Representations*. Retrieved from <https://openreview.net/forum?id=H1gL-2A9Ym>
- [55] Boris Knyazev, Graham W. Taylor, and Mohamed Amer. 2019. Understanding attention and generalization in graph neural networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 4202–4212.
- [56] Dongkwan Kim and Alice Oh. 2020. How to find your friendly neighborhood: Graph attention design with self-supervision. In *Proceedings of the International Conference on Learning Representations*.
- [57] Hoang N. T. and Takanori Maehara. 2019. Revisiting graph neural networks: All we have is low-pass filters. *arXiv preprint arXiv:1905.09550* (2019).
- [58] Deyu Bo, Xiao Wang, Chuan Shi, and Huawei Shen. 2021. Beyond low-frequency information in graph convolutional networks. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence*.
- [59] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*. JMLR. org, 1263–1272.
- [60] Floris Geerts, Filip Mazowiecki, and Guillermo Perez. 2021. Let’s agree to degree: Comparing graph convolutional networks in the message-passing framework. In *Proceedings of the International Conference on Machine Learning*. PMLR, 3640–3649.
- [61] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How powerful are graph neural networks? In *Proceedings of the International Conference on Learning Representations*. Retrieved from <https://openreview.net/forum?id=ryGs6iA5Km>
- [62] Rex Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. GNN explainer: A tool for post-hoc explanation of graph neural networks. *CoRR*. arXiv preprint arXiv:1903.03894 (2019).
- [63] Zhiqian Chen, Fanglan Chen, Lei Zhang, Taoran Ji, Kaiqun Fu, Liang Zhao, Feng Chen, and Chang-Tien Lu. 2020. Bridging the gap between spatial and spectral domains: A survey on graph neural networks. *arXiv preprint arXiv:2002.11867* (2020).
- [64] David K. Hammond, Pierre Vandergheynst, and Rémi Gribonval. 2011. Wavelets on graphs via spectral graph theory. *Appl. Comput. Harmon. Anal.* 30, 2 (2011), 129–150.
- [65] Jie Chen, Tengfei Ma, and Cao Xiao. 2018. FastGCN: Fast learning with graph convolutional networks via importance sampling. In *Proceedings of the International Conference on Learning Representations*.
- [66] Michael M. Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. 2017. Geometric deep learning: Going beyond Euclidean data. *IEEE Sig. Process. Mag.* 34, 4 (2017), 18–42.
- [67] Meiqi Zhu, Xiao Wang, Chuan Shi, Houye Ji, and Peng Cui. 2021. Interpreting and unifying graph neural networks with an optimization framework. *arXiv preprint arXiv:2101.11859* (2021).

- [68] John Boaz Lee, Ryan A. Rossi, Sungchul Kim, Nesreen K. Ahmed, and Eunyee Koh. 2019. Attention models in graphs: A survey. *ACM Trans. Knowl. Discov. Data* 13, 6, Article 62 (Nov. 2019), 25 pages. DOI:<http://dx.doi.org/10.1145/3363574>
- [69] Jiezhong Qiu, Yuxiao Dong, Hao Ma, Jian Li, Kuansan Wang, and Jie Tang. 2018. Network embedding as matrix factorization: Unifying DeepWalk, LINE, PTE, and node2vec. In *Proceedings of the 11th ACM International Conference on Web Search and Data Mining*. 459–467.
- [70] Xin Liu, Tsuyoshi Murata, Kyoung-Sook Kim, Chatchawan Kotarasu, and Chenyi Zhuang. 2019. A general view for network embedding as matrix factorization. In *Proceedings of the 12th ACM International Conference on Web Search and Data Mining*. 375–383.
- [71] Carl Yang, Yuxin Xiao, Yu Zhang, Yizhou Sun, and Jiawei Han. 2020. Heterogeneous network representation learning: Survey, benchmark, evaluation, and beyond. *arXiv preprint arXiv:2004.00216* (2020).
- [72] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. Gnnexplainer: Generating explanations for graph neural networks. *Advances in Neural Information Processing Systems* 32 (2019).
- [73] Federico Baldassarre and Hossein Azizpour. 2019. Explainability techniques for graph convolutional networks. *arXiv preprint arXiv:1905.13686* (2019).
- [74] Hao Yuan, Haiyang Yu, Shurui Gui, and Shuiwang Ji. 2020. Explainability in graph neural networks: A taxonomic survey. *arXiv preprint arXiv:2012.15445* (2020).
- [75] Wanyu Lin, Hao Lan, and Baochun Li. 2021. Generative causal explanations for graph neural networks. In *Proceedings of the International Conference on Machine Learning*.
- [76] Béla Bollobás. 2004. *Extremal Graph Theory*. Courier Corporation.
- [77] Yewen Wang, Ziniu Hu, Yusong Ye, and Yizhou Sun. 2020. Demystifying Graph Neural Network Via Graph Filter Assessment. Retrieved from <https://openreview.net/forum?id=r1erNxBtwr>
- [78] David I. Shuman, Sunil K. Narang, Pascal Frossard, Antonio Ortega, and Pierre Vandergheynst. 2013. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. *IEEE Sig. Process. Mag.* 30, 3 (2013), 83–98.
- [79] Xiaofan Zhu and Michael Rabbat. 2012. Approximating signals supported on graphs. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP'12)*. IEEE, 3921–3924.
- [80] Alan V. Oppenheim, John R. Buck, and Ronald W. Schaffer. 2001. *Discrete-time Signal Processing*. Vol. 2. Prentice Hall, Upper Saddle River, NJ.
- [81] Lars V. Ahlfors. 1981. *Complex Analysis: An Introduction to Theory of Analytic Functions of One Complex Variable*. McGraw Hill International Book Company. https://www.google.com/books/edition/Complex_Analysis_an_Introduction_to_Theo/HGPrzAEACAAJ?hl=en
- [82] Lloyd N. Trefethen. 2013. *Approximation Theory and Approximation Practice*. Vol. 128. Siam.
- [83] Ricardo Pachon. 2010. *Algorithms for Polynomial and Rational Approximation*. Ph.D. Dissertation. University of Oxford.
- [84] Michael James David Powell. 1981. *Approximation Theory and Methods*. Cambridge University Press.
- [85] Harold Cohen. 2011. *Numerical Approximation Methods*. Springer.
- [86] Penco Petrov Petrushev and Vasil Atanasov Popov. 2011. *Rational Approximation of Real Functions*. Vol. 28. Cambridge University Press.
- [87] Naum I. Achieser. 2013. *Theory of Approximation*. Courier Corporation.
- [88] Eric Ziegel. 1987. Numerical recipes: The art of scientific computing. *Technometrics* 29, 4 (November 1, 1987), 501–502. <https://doi.org/10.1080/00401706.1987.10488304>
- [89] John P. Boyd. 2001. *Chebyshev and Fourier Spectral Methods*. Courier Corporation.
- [90] John C. Mason and David C. Handscomb. 2002. *Chebyshev Polynomials*. CRC Press.
- [91] Eugene Y. Remez. 1934. Sur la détermination des polynômes d'approximation de degré donnée. *Comm. Soc. Math. Kharkov* 10 (1934), 41–63.
- [92] Xiao-Ming Wu, Zhenguo Li, Anthony M. So, John Wright, and Shih-Fu Chang. 2012. Learning with partially absorbing random walks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 3077–3085.
- [93] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. 2015. LINE: Large-scale information network embedding. In *Proceedings of the 24th International Conference on World Wide Web*.
- [94] Mikael Henaff, Joan Bruna, and Yann LeCun. 2015. Deep convolutional networks on graph-structured data. *arXiv preprint arXiv:1506.05163* (2015).
- [95] David K. Duvenaud, Dougal Maclaurin, Jorge Iparraguirre, Rafael Bombarell, Timothy Hirzel, Alan Aspuru-Guzik, and Ryan P. Adams. 2015. Convolutional networks on graphs for learning molecular fingerprints. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS'15)*. Curran Associates, Inc., 2224–2232.

- [96] Steven Kearnes, Kevin McCloskey, Marc Berndl, Vijay Pande, and Patrick Riley. 2016. Molecular graph convolutions: Moving beyond fingerprints. *J. Comput.-aid. Molec. Des.* 30, 8 (2016), 595–608.
- [97] Mathias Niepert, Mohamed Ahmed, and Konstantin Kutzkov. 2016. Learning convolutional neural networks for graphs. In *Proceedings of the 33rd International Conference on Machine Learning (ICML'16)*. 2014–2023.
- [98] Alex Fout, Jonathon Byrd, Basir Shariat, and Asa Ben-Hur. 2017. Protein interface prediction using graph convolutional networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS'17)*. Curran Associates, Inc., 6530–6539.
- [99] Wenbing Huang, Tong Zhang, Yu Rong, and Junzhou Huang. 2018. Adaptive sampling towards fast graph representation learning. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 4558–4567.
- [100] Jie Chen, Tengfei Ma, and Cao Xiao. 2018. FastGCN: Fast learning with graph convolutional networks via importance sampling. In *Proceedings of the International Conference on Learning Representations (ICLR'18)*.
- [101] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation learning on graphs with jumping knowledge networks. *arXiv preprint arXiv:1806.03536* (2018).
- [102] Hongyang Gao, Zhengyang Wang, and Shuiwang Ji. 2018. Large-scale learnable graph convolutional networks. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD'18)*. 1416–1424.
- [103] Zhiqian Chen, Feng Chen, Rongjie Lai, Xuchao Zhang, and Chang-Tien Lu. 2018. Rational neural networks for approximating graph convolution operator on jump discontinuities. In *2018 IEEE International Conference on Data Mining (ICDM)*, IEEE, 59–68.
- [104] Qimai Li, Zhichao Han, and Xiao-Ming Wu. 2018. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*.
- [105] Ron Levie, Federico Monti, Xavier Bresson, and Michael M. Bronstein. 2018. CayleyNets: Graph convolutional neural networks with complex rational spectral filters. *IEEE Trans. Sig. Process.* 67, 1 (2018), 97–109.
- [106] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *Proceedings of the International Conference on Machine Learning*. 6861–6871.
- [107] Guohao Li, Matthias Muller, Ali Thabet, and Bernard Ghanem. 2019. DeepGCNs: Can GCNs go as deep as CNNs? In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 9267–9276.
- [108] Sami Abu-El-Haija, Bryan Perozzi, Amol Kapoor, Nazanin Alipourfard, Kristina Lerman, Hrayr Harutyunyan, Greg Ver Steeg, and Aram Galstyan. 2019. MixHop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. *arXiv preprint arXiv:1905.00067* (2019).
- [109] Filippo Maria Bianchi, Daniele Grattarola, Lorenzo Livi, and Cesare Alippi. 2021. Graph neural networks with convolutional ARMA filters. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 7 (2021), 3496–3507.
- [110] Johannes Klicpera, Stefan Weißenberger, and Stephan Günnemann. 2019. Diffusion improves graph learning. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 13354–13366.
- [111] Yao Ma, Suhang Wang, Charu C. Aggarwal, and Jiliang Tang. 2019. Graph convolutional networks with EigenPooling. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD'19)*. 723–731.
- [112] Bingbing Xu, Huawei Shen, Qi Cao, Yunqi Qiu, and Xueqi Cheng. 2019. Graph wavelet neural network. In *Proceedings of the International Conference on Learning Representations (ICLR'19)*.
- [113] Saurabh Verma and Zhi-Li Zhang. 2019. Stability and generalization of graph convolutional neural networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD'19)*. 1539–1548.
- [114] Emanuele Rossi, Fabrizio Frasca, Ben Chamberlain, Davide Eynard, Michael Bronstein, and Federico Monti. 2020. SIGN: Scalable inception graph neural networks. *arXiv preprint arXiv:2004.11198* (2020).
- [115] Shichao Zhu, Lewei Zhou, Shirui Pan, Chuan Zhou, Guiying Yan, and Bin Wang. 2020. GSSNN: Graph smoothing splines neural network. In *Proceedings of the 34th Conference on Association for the Advancement of Artificial Intelligence (AAAI'20)*. 7007–7014.
- [116] Zhao-Yang Liu, Shao-Yuan Li, Songcan Chen Yao Hu, and Sheng-Jun Huang. 2020. Uncertainty aware graph Gaussian process for semi-supervised learning. In *Proceedings of the 34th Conference on Association for the Advancement of Artificial Intelligence (AAAI'20)*. 4957–4964.
- [117] Yilun Jin, Guojie Song, and Chuan Shi. 2020. GraLSP: Graph neural networks with local structural patterns. In *Proceedings of the 34th Conference on Association for the Advancement of Artificial Intelligence (AAAI'20)*. 4361–4368.
- [118] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2019. GraphSaint: Graph sampling based inductive learning method. In *Proceedings of the International Conference on Learning Representations*.
- [119] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. 2019. DropEdge: Towards deep graph convolutional networks on node classification. In *Proceedings of the International Conference on Learning Representations*.

- [120] Hongmin Zhu, Fuli Feng, Xiangnan He, Xiang Wang, Yan Li, Kai Zheng, and Yongdong Zhang. 2020. Bilinear graph neural network with neighbor interactions. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI'20)*. 1452–1458.
- [121] Yiqing Xie, Sha Li, Carl Yang, Raymond Chi-Wing Wong, and Jiawei Han. 2020. When do GNNs Work: Understanding and improving neighborhood aggregation. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI'20)*. 1303–1309.
- [122] Louis-Pascal Xhonneux, Meng Qu, and Jian Tang. 2020. Continuous graph neural networks. In *Proceedings of the 37th International Conference on Machine Learning (ICML'20)*. 10432–10441.
- [123] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. 2020. Simple and deep graph convolutional networks. In *International Conference on Machine Learning*, PMLR, 1725–1735.
- [124] Aleksandar Bojchevski, Johannes Klicpera, Bryan Perozzi, Amol Kapoor, Martin Blais, Benedek Rózemberczki, Michal Lukasik, and Stephan Günnemann. 2020. Scaling graph neural networks with approximate PageRank. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD'20)*. 2464–2473.
- [125] Meng Liu, Hongyang Gao, and Shuiwang Ji. 2020. Towards deeper graph neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 338–348.
- [126] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond homophily in graph neural networks: Current limitations and effective designs. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS'20)*. Curran Associates, Inc., 7793–7804.
- [127] Chenhui Deng, Zhiqiang Zhao, Yongyu Wang, Zhiru Zhang, and Zhuo Feng. 2020. GraphZoom: A multi-level spectral approach for accurate and scalable graph embedding. In *Proceedings of the International Conference on Learning Representations (ICLR'20)*.
- [128] Jialin Zhao, Yuxiao Dong, Ming Ding, Evgeny Kharlamov, and Jie Tang. 2021. Adaptive diffusion in graph neural networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS'21)*. Curran Associates, Inc., 23321–23333.
- [129] Di Jin, Zhizhi Yu, Cuiying Huo, Rui Wang, Xiao Wang, Dongxiao He, and Jiawei Han. 2021. Universal graph convolutional networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS'21)*. Curran Associates, Inc., 10654–10664.
- [130] Yifei Wang, Yisen Wang, Jiansheng Yang, and Zhouchen Lin. 2021. Dissecting the diffusion process in linear graph convolutional networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS'21)*. Curran Associates, Inc., 5758–5769.
- [131] Victor Garcia Satorras, Emiel Hoogeboom, and Max Welling. 2021. E(n) equivariant graph neural networks. In *Proceedings of the 38th International Conference on Machine Learning (ICML'21)*. 9323–9332.
- [132] Benjamin Paul Chamberlain, James Rowbottom, Maria Gorinova, Stefan Webb, Emanuele Rossi, and Michael M. Bronstein. 2021. GRAND: Graph neural diffusion. *arXiv preprint arXiv:2106.10934* (2021).
- [133] Qian Huang, Horace He, Abhay Singh, Ser-Nam Lim, and Austin Benson. 2021. Combining label propagation and simple models out-performs graph neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR'21)*.
- [134] Zemin Liu, Yuan Fang, Chenghao Liu, and Steven C. H. Hoi. 2021. Node-wise localization of graph neural networks. In *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI'21)*. 1520–1526.
- [135] Henry Kenlay, Dorina Thanou, and Xiaowen Dong. 2021. Interpretable stability bounds for spectral graph filters. In *Proceedings of the 38th International Conference on Machine Learning (ICML'21)*. 5388–5397.
- [136] Muhammet Balcilar, Guillaume Renton, Pierre Héroux, Benoit Gaüzère, Sébastien Adam, and Paul Honeine. 2021. Analyzing the expressive power of graph neural networks in a spectral perspective. In *Proceedings of the International Conference on Learning Representations (ICLR'21)*.
- [137] Hao Zhu and Piotr Koniusz. 2021. Simple spectral graph convolution. In *Proceedings of the International Conference on Learning Representations (ICLR'21)*.
- [138] Mingguo He, Zhewei Wei, Zengfeng Huang, and Hongteng Xu. 2021. BernNet: Learning arbitrary graph spectral filters via Bernstein approximation. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 14239–14251.
- [139] Daniele Grattarola and Pierre Vandergheynst. 2022. Generalised implicit neural representations. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS'22)*. Curran Associates, Inc.
- [140] Sudhanshu Chanpuriya and Cameron Musco. 2022. Simplified graph convolution with heterophily. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS'22)*. Curran Associates, Inc.

- [141] Zhongyu Huang, Yingheng Wang, Chaozhuo Li, and Huiguang He. 2022. Going deeper into permutation-sensitive graph neural networks. In *Proceedings of the 39th International Conference on Machine Learning (ICML'22)*. 9377–9409.
- [142] Liang Yang, Chuan Wang, Xiaochun Cao, Junhua Gu, Bingxin Niu, Yuanfang Guo, Weixun Li, Cheng Chen, and Dongxiao He. 2022. Self-supervised graph neural networks via diverse and interactive message passing. In *Proceedings of the 36th Conference on Association for the Advancement of Artificial Intelligence (AAAI'22)*. 4327–4336.
- [143] Guangyu Meng, Qisheng Jiang, Kaiqun Fu, Beiyu Lin, Chang-Tien Lu, and Zhqian Chen. 2022. Early forecasting of the impact of traffic accidents using a single shot observation. In *Proceedings of the SIAM International Conference on Data Mining (SDM'22)*. SIAM, 100–108.
- [144] Mingguo He, Zhewei Wei, and Ji-Rong Wen. 2022. Convolutional neural networks on graphs with Chebyshev approximation, revisited. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS'22)*. Curran Associates, Inc.
- [145] Xiyuan Wang and Muhan Zhang. 2022. How powerful are spectral graph neural networks. In *Proceedings of the 39th International Conference on Machine Learning (ICML'22)*. 23341–23362.
- [146] Mingqi Yang, Yanming Shen, Rui Li, Heng Qi, Qiang Zhang, and Baocai Yin. 2022. A new perspective on the effects of spectrum in graph neural networks. In *Proceedings of the 39th International Conference on Machine Learning (ICML'22)*. 25261–25279.
- [147] Mingjie Li, Xiaojun Guo, Yifei Wang, Yisen Wang, and Zhouchen Lin. 2022. G²CN: Graph Gaussian convolution networks with concentrated graph filters. In *Proceedings of the 39th International Conference on Machine Learning (ICML'22)*. 12782–12796.
- [148] Guoji Fu, Peilin Zhao, and Yatao Bian. 2022. *p*-Laplacian based graph neural networks. In *Proceedings of the 39th International Conference on Machine Learning (ICML'22)*. 6878–6917.
- [149] Jie Zhang, Bo Hui, Po-wei Harn, Min-Te Sun, and Wei-Shinn Ku. 2023. From ChebNet to ChebGibbsNet. <https://openreview.net/forum?id=2a5Ru3JtNe0>
- [150] Deyu Bo, Chuan Shi, Lele Wang, and Renjie Liao. 2023. *Specformer: Spectral Graph Neural Networks Meet Transformers*. arXiv preprint arXiv:2303.01028 (2023).
- [151] Derek Lim, Joshua Robinson, Lingxiao Zhao, Tess Smidt, Suvrit Sra, Haggai Maron, and Stefanie Jegelka. 2022. Sign and basis invariant networks for spectral graph representation learning. arXiv preprint arXiv:2202.13013 (2022).
- [152] Yongyu Wang, Zhiqiang Zhao, and Zhuo Feng. 2022. Scalable graph topology learning via spectral densification. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM'22)*. Association for Computing Machinery, New York, NY, 1099–1108. DOI : <http://dx.doi.org/10.1145/3488560.3498480>
- [153] Zeyu Zhang, Jiamou Liu, Xianda Zheng, Yifei Wang, Pengqian Han, Yupan Wang, Kaiqi Zhao, and Zijian Zhang. 2023. RSGNN: A model-agnostic approach for enhancing the robustness of signed graph neural networks. In *Proceedings of the ACM Web Conference*. 60–70.
- [154] Yu Wang, Yuying Zhao, Yi Zhang, and Tyler Derr. 2023. Collaboration-aware graph convolutional network for recommender systems. In *Proceedings of the ACM Web Conference*. 91–101.
- [155] Liang Yang, Qiuliang Zhang, Runjie Shi, Wenmiao Zhou, Bingxin Niu, Chuan Wang, Xiaochun Cao, Dongxiao He, Zhen Wang, and Yuanfang Guo. 2023. Graph neural networks without propagation. In *Proceedings of the ACM Web Conference*. 469–477.
- [156] Xin Zheng, Miao Zhang, Chunyang Chen, Qin Zhang, Chuan Zhou, and Shirui Pan. 2023. Auto-HeG: Automated graph neural network on heterophilic graphs. *arXiv preprint arXiv:2302.12357* (2023).
- [157] Taoran Fang, Zhiqing Xiao, Chunping Wang, Jiarong Xu, Xuan Yang, and Yang Yang. 2022. DropMessage: Unifying random dropping for graph neural networks. *arXiv preprint arXiv:2204.10037* (2022).
- [158] Jingwei Guo, Kaizhu Huang, Xinping Yi, and Rui Zhang. 2023. Graph neural networks with diverse spectral filtering. In *Proceedings of the ACM Web Conference*. 306–316.
- [159] Dongqi Fu, Dawei Zhou, Ross Maciejewski, Arie Croitoru, Marcus Boyd, and Jingrui He. 2023. Fairness-aware clique-preserving spectral clustering of temporal graphs. In *Proceedings of the ACM Web Conference*. 3755–3765.
- [160] Jincheng Huang, Lun Du, Xu Chen, Qiang Fu, Shi Han, and Dongmei Zhang. 2023. Robust mid-pass filtering graph convolutional networks. *arXiv preprint arXiv:2302.08048* (2023).
- [161] Yuan Gao, Xiang Wang, Xiangnan He, Zhenguang Liu, Huamin Feng, and Yongdong Zhang. 2023. Addressing heterophily in graph anomaly detection: A perspective of graph spectrum. In *Proceedings of the ACM Web Conference*. 1528–1538.
- [162] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826* (2018).
- [163] Rie Johnson and Tong Zhang. 2007. On the effectiveness of Laplacian normalization for graph semi-supervised learning. *J. Mach. Learn. Res.* 8, July (2007), 1489–1517.
- [164] Joan Bruna Estrach, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. 2014. Spectral networks and locally connected networks on graphs. In *Proceedings of the 2nd International Conference on Learning Representations (ICLR'14)*.

- [165] Yankai Lin, Zhiyuan Liu, Maosong Sun, Yang Liu, and Xuan Zhu. 2015. Learning entity and relation embeddings for knowledge graph completion. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 2181–2187.
- [166] Daixin Wang, Peng Cui, and Wenwu Zhu. 2016. Structural deep network embedding. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1225–1234.
- [167] Matus Telgarsky. 2017. Neural networks and rational functions. In *Proceedings of the International Conference on Machine Learning*. PMLR, 3387–3393.
- [168] Nicolas Boullé, Yuji Nakatsukasa, and Alex Townsend. 2020. Rational neural networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 14243–14253.
- [169] Martin Trimmel, Mihai Zanfir, Richard Hartley, and Cristian Sminchisescu. 2022. ERA: Enhanced rational activations. In *Proceedings of the European Conference on Computer Vision*. Springer, 722–738.
- [170] Qimai Li, Xiao-Ming Wu, Han Liu, Xiaotong Zhang, and Zhichao Guan. 2019. Label efficient semi-supervised learning via graph filtering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR'19)*.
- [171] A. Loukas, A. Simonetto, and G. Leus. 2015. Distributed autoregressive moving average graph filters. *IEEE Sig. Process. Lett.* 22, 11 (Nov. 2015), 1931–1935. DOI : <http://dx.doi.org/10.1109/LSP.2015.2448655>
- [172] E. Isufi, A. Loukas, A. Simonetto, and G. Leus. 2017. Autoregressive moving average graph filtering. *IEEE Trans. Sig. Process.* 65, 2 (Jan. 2017), 274–288. DOI : <http://dx.doi.org/10.1109/TSP.2016.2614793>
- [173] Lingxiao Zhao and Leman Akoglu. 2019. PairNorm: Tackling oversmoothing in GNNs. In *Proceedings of the International Conference on Learning Representations*.
- [174] Qian Huang, Horace He, Abhay Singh, Ser-Nam Lim, and Austin R. Benson. 2020. Combining label propagation and simple models out-performs graph neural networks. *arXiv preprint arXiv:2010.13993* (2020).
- [175] Xiaojin Zhu, Zoubin Ghahramani, and John D. Lafferty. 2003. Semi-supervised learning using Gaussian fields and harmonic functions. In *Proceedings of the 20th International Conference on Machine Learning*. 912–919.
- [176] Dengyong Zhou, Olivier Bousquet, Thomas N. Lal, Jason Weston, and Bernhard Schölkopf. 2004. Learning with local and global consistency. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 321–328.
- [177] Yoshua Bengio, Olivier Delalleau, and Nicolas Le Roux. 2006. 11 label propagation and quadratic criterion. located at: https://www.researchgate.net/profile/Y_Bengio/publication/238675708_Label_Propagation_and_Quadratic_Criterion/links/0f3175320aae4ada34000000/Label-Propagation-and-Quadratic-Criterion.pdf
- [178] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 974–983.
- [179] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 770–778.
- [180] Mark Herbster, Massimiliano Pontil, and Lisa Wainer. 2005. Online learning over graphs. In *Proceedings of the 22nd International Conference on Machine Learning*. ACM, 305–312.
- [181] Andreas Argyriou, Charles A. Micchelli, and Massimiliano Pontil. 2009. When is there a representer theorem? Vector versus matrix regularizers. *J. Mach. Learn. Res.* 10, Nov. (2009), 2507–2529.
- [182] B. Schölkopf, R. Herbrich, and A. J. Smola. 2001. A generalized representer theorem. In *International Conference on Computational Learning Theory*, Springer Berlin Heidelberg, Berlin, Heidelberg, 416–426.
- [183] Fan R. K. Chung. 1997. *Spectral Graph Theory*. Number 92. American Mathematical Society.
- [184] Sergi Abadal, Akshay Jain, Robert Guirado, Jorge López-Alonso, and Eduard Alarcón. 2020. Computing Graph Neural Networks: A Survey from Algorithms to Accelerators. *arXiv:cs.LG/2010.00130*
- [185] Siemon C. de Lange, Martijn P. van den Heuvel, and Marcel A. de Reus. 2016. The role of symmetry in neural networks and their Laplacian spectra. *NeuroImage* 141 (2016), 357–365.
- [186] Siemon de Lange, Marcel de Reus, and Martijn Van Den Heuvel. 2014. The Laplacian spectrum of neural networks. *Front. Comput. Neurosci.* 7 (2014), 189.
- [187] Gerald B. Folland and Alladi Sitaram. 1997. The uncertainty principle: A mathematical survey. *J. Fourier Anal. Applic.* 3, 3 (1997), 207–238.
- [188] Ameya Agaskar and Yue M. Lu. 2013. A spectral graph uncertainty principle. *IEEE Trans. Inf. Theor.* 59, 7 (2013), 4338–4356.
- [189] Petar Veličković, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R. Devon Hjelm. 2018. Deep graph infomax. In *Proceedings of the International Conference on Learning Representations*.
- [190] Xiaomin Liang, Daifeng Li, and Andrew Madden. 2020. Attributed network embedding based on mutual information estimation. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 835–844.
- [191] Chenyi Zhuang and Qiang Ma. 2018. Dual graph convolutional networks for graph-based semi-supervised classification. In *Proceedings of the World Wide Web Conference*. 499–508.

- [192] Songyang Zhang, Xuming He, and Shipeng Yan. 2019. LatentGNN: Learning efficient non-local relations for visual recognition. In *Proceedings of the International Conference on Machine Learning*. PMLR, 7374–7383.
- [193] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond homophily in graph neural networks: Current limitations and effective designs. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*.
- [194] Haiyang Lin, Mingyu Yan, Xiaochun Ye, Dongrui Fan, Shirui Pan, Wenguang Chen, and Yuan Xie. 2022. A comprehensive survey on distributed training of graph neural networks. *arXiv preprint arXiv:2211.05368* (2022).
- [195] Yingxia Shao, Hongzheng Li, Xizhi Gu, Hongbo Yin, Yawen Li, Xupeng Miao, Wentao Zhang, Bin Cui, and Lei Chen. 2022. Distributed graph neural network training: A survey. *arXiv preprint arXiv:2211.00216* (2022).
- [196] Uri Shaham, Kelly Stanton, Henry Li, Boaz Nadler, Ronen Basri, and Yuval Kluger. 2018. SpectralNet: Spectral clustering using deep neural networks. *arXiv preprint arXiv:1801.01587* (2018).
- [197] Joost Verbraeken, Matthijs Wolting, Jonathan Katzy, Jeroen Kloppenburg, Tim Verbelen, and Jan S. Rellermeyer. 2020. A survey on distributed machine learning. *ACM Comput. Surv.* 53, 2 (2020), 1–33.
- [198] Zhan Gao, Fernando Gama, and Alejandro Ribeiro. 2022. Wide and deep graph neural network with distributed online learning. *IEEE Trans. Sig. Process.* 70 (2022), 3862–3877.
- [199] Honglei He, Yuxuan He, Fang Wang, and Wenming Zhu. 2022. Improved K-means algorithm for clustering non-spherical data. *Expert Syst.* 39, 9 (2022), e13062.
- [200] Martin Azizyan, Aarti Singh, and Larry Wasserman. 2015. Efficient sparse clustering of high-dimensional non-spherical Gaussian mixtures. In *Artificial Intelligence and Statistics*. PMLR, 37–45.
- [201] Yan Shi, Jun-Xiong Cai, Yoli Shavit, Tai-Jiang Mu, Wensen Feng, and Kai Zhang. 2022. ClusterGNN: Cluster-based coarse-to-fine graph neural network for efficient feature matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12517–12526.
- [202] Sitao Luan, Chenqing Hua, Qincheng Lu, Jiaqi Zhu, Mingde Zhao, Shuyuan Zhang, Xiao-Wen Chang, and Doina Precup. 2022. Revisiting heterophily for graph neural networks. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS’22)*. Curran Associates, Inc.
- [203] Liang Yang, Mengzhe Li, Liyang Liu, Bingxin Niu, Chuan Wang, Xiaochun Cao, and Yuanfang Guo. 2021. Diverse message passing for attribute with heterophily. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems (NeurIPS’21)*. Curran Associates, Inc., 4751–4763.
- [204] Xiang Li, Renyu Zhu, Yao Cheng, Caihua Shan, Siqiang Luo, Dongsheng Li, and Weining Qian. 2022. Finding global homophily in graph neural networks when meeting heterophily. In *Proceedings of the 39th International Conference on Machine Learning (ICML’22)*. 13242–13256.
- [205] Jianfei Chen, Jun Zhu, and Le Song. 2018. Stochastic training of graph convolutional networks with variance reduction. In *Proceedings of the International Conference on Machine Learning*. 942–950.
- [206] Hongyang Gao, Zhengyang Wang, and Shuiwang Ji. 2018. Large-scale learnable graph convolutional networks. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1416–1424.
- [207] Kai-Lang Yao and Wu-Jun Li. 2021. Blocking-based neighbor sampling for large-scale graph neural networks. In *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI’21)*. 3307–3313.
- [208] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2019. Accurate, efficient and scalable graph embedding. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS’19)*. IEEE, 462–471.
- [209] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*.
- [210] Tomáš Mikolov, Wen-Tau Yih, and Geoffrey Zweig. 2013. Linguistic regularities in continuous space word representations. In *Proceedings of the Annual Conference of the North American Chapter of the ACL (NAACL’13)*. 746–751.
- [211] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. In *Proceedings of the Workshop at the International Conference on Learning Representations*.
- [212] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 3111–3119.
- [213] Tianmeng Yang, Yujing Wang, Zhihan Yue, Yaming Yang, Yunhai Tong, and Jing Bai. 2022. Graph pointer neural networks. In *Proceedings of the 36th Conference of the Association for the Advancement of Artificial Intelligence (AAAI’22)*. 8832–8839.
- [214] Omer Levy and Yoav Goldberg. 2014. Neural word embedding as implicit matrix factorization. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*. 2177–2185.
- [215] Chaoqi Yang, Ruijie Wang, Shuochao Yao, Shengzhong Liu, and Tarek Abdelzaher. 2020. Revisiting “over-smoothing” in deep GCNs. *arXiv preprint arXiv:2003.13663* (2020).

- [216] Deyu Bo, Xiao Wang, Chuan Shi, and Huawei Shen. 2021. Beyond low-frequency information in graph convolutional networks. In *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 5 (2021), 3950–3957.
- [217] Kenta Oono and Taiji Suzuki. 2019. Graph neural networks exponentially lose expressive power for node classification. In *Proceedings of the International Conference on Learning Representations*.
- [218] Guohao Li, Chenxin Xiong, Ali Thabet, and Bernard Ghanem. 2020. DeeperGCN: All you need to train deeper GCNs. *arXiv preprint arXiv:2006.07739* (2020).
- [219] Wenbing Huang, Yu Rong, Tingyang Xu, Fuchun Sun, and Junzhou Huang. 2020. Tackling Over-smoothing for General Graph Convolutional Networks. *arXiv:cs.LG/2008.09864*
- [220] Kaixiong Zhou, Xiao Huang, Yuening Li, Daochen Zha, Rui Chen, and Xia Hu. 2020. Towards Deeper Graph Neural Networks with Differentiable Group Normalization. *arXiv:cs.LG/2006.06972*
- [221] Deli Chen, Yankai Lin, Wei Li, Peng Li, Jie Zhou, and Xu Sun. 2020. Measuring and relieving the over-smoothing problem for graph neural networks from the topological view. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 3438–3445.
- [222] Yimeng Min, Frederik Wenkel, and Guy Wolf. 2020. Scattering GCN: Overcoming oversmoothness in graph convolutional networks. *arXiv preprint arXiv:2003.08414* (2020).
- [223] Antonio G. Marques, Santiago Segarra, and Gonzalo Mateos. 2020. Signal processing on directed graphs: The role of edge directionality when processing and learning from network data. *IEEE Sig. Process. Mag.* 37, 6 (2020), 99–116.
- [224] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S. Yu. 2022. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Trans. Neural Netw. Learn. Syst.* 33, 2 (Feb. 2022), 494–514. DOI : <http://dx.doi.org/10.1109/TNNLS.2021.3070843>
- [225] Maximilian Nickel, Volker Tresp, and Hans-Peter Kriegel. 2011. A three-way model for collective learning on multi-relational data. *ICML*, 11, 10.5555 (2011), 3104482–3104584.
- [226] Bishan Yang, Wen-Tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. 2014. Embedding entities and relations for learning and inference in knowledge bases. *arXiv preprint arXiv:1412.6575* (2014).
- [227] Hanxiao Liu, Yuexin Wu, and Yiming Yang. 2017. Analogical inference for multi-relational embeddings. In *Proceedings of the International Conference on Machine Learning*. PMLR, 2168–2178.
- [228] Nelson Dunford and Jacob T. Schwartz. 1988. *Linear Operators, Part 1: General Theory*. Vol. 10. John Wiley & Sons.
- [229] Canran Xu and Ruijiang Li. 2019. Relation embedding with dihedral group in knowledge graph. *arXiv preprint arXiv:1906.00687* (2019).
- [230] Chao Shang, Yun Tang, Jing Huang, Jinbo Bi, Xiaodong He, and Bowen Zhou. 2019. End-to-end structure-aware convolutional networks for knowledge base completion. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 3060–3067.
- [231] Richard Socher, Danqi Chen, Christopher D. Manning, and Andrew Ng. 2013. Reasoning with neural tensor networks for knowledge base completion. In *Proceedings of the International Conference on Advances in Neural Information Processing Systems*.
- [232] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *Proceedings of the 15th International Conference on the Semantic Web (ESWC'18)*. Springer, 593–607.
- [233] Shikhar Vashishth, Soumya Sanyal, Vikram Nitin, and Partha Talukdar. 2019. Composition-based multi-relational graph convolutional networks. *arXiv preprint arXiv:1911.03082* (2019).
- [234] Deepak Nathani, Jatin Chauhan, Charu Sharma, and Manohar Kaul. 2019. Learning attention-based embeddings for relation prediction in knowledge graphs. *arXiv preprint arXiv:1906.01195* (2019).
- [235] Henry Kenlay, Dorina Thanou, and Xiaowen Dong. 2021. Interpretable stability bounds for spectral graph filters. In *Proceedings of the International Conference on Machine Learning*. PMLR, 5388–5397.
- [236] Walter A. Strauss. 2007. *Partial Differential Equations: An Introduction*. John Wiley & Sons.
- [237] Bo Kågström and Axel Ruhe. 1980. An algorithm for numerical computation of the Jordan normal form of a complex matrix. *ACM Trans. Math. Softw.* 6, 3 (1980), 398–419.
- [238] Ron Levie, Wei Huang, Lorenzo Bucci, Michael M. Bronstein, and Gitta Kutyniok. 2021. Transferability of spectral graph convolutional neural networks. *J. Mach. Learn. Res.* 22 (2021), 272–1.
- [239] Muhammad Zubair Malik and Sarfraz Khurshid. 2012. Dynamic shape analysis using spectral graph properties. In *Proceedings of the IEEE 5th International Conference on Software Testing, Verification and Validation*. 211–220. DOI : <http://dx.doi.org/10.1109/ICST.2012.101>
- [240] Arlei Silva, Ambuj Singh, and Ananthram Swami. 2018. Spectral algorithms for temporal graph cuts. In *Proceedings of the World Wide Web Conference (WWW'18)*. International World Wide Web Conferences Steering Committee, 519–528. DOI : <http://dx.doi.org/10.1145/3178876.3186118>
- [241] Jared C. Bronski and Lee DeVille. 2014. Spectral theory for dynamics on graphs containing attractive and repulsive interactions. *SIAM J. Appl. Math.* 74, 1 (2014), 83–105.

- [242] Defu Cao, Jiachen Li, Hengbo Ma, and Masayoshi Tomizuka. 2021. Spectral temporal graph neural network for trajectory prediction. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA'21)*. IEEE, 1839–1845.
- [243] Hao Yuan, Haiyang Yu, Shurui Gui, and Shuiwang Ji. 2022. Explainability in graph neural networks: A taxonomic survey. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 5 (2022), 5782–5799.
- [244] Song Bai, Feihu Zhang, and Philip H. S. Torr. 2021. Hypergraph convolution and hypergraph attention. *Pattern Recog.* 110 (2021), 107637.
- [245] Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. 2019. Hypergraph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 3558–3565.
- [246] Jaehyuk Yi and Jinkyoo Park. 2020. Hypergraph convolutional recurrent neural network. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 3366–3376.
- [247] Emily Ribando-Gros, Rui Wang, Jiahui Chen, Yiyong Tong, and Guo-Wei Wei. 2022. Graph and Hodge Laplacians: Similarity and difference. *arXiv preprint arXiv:2204.12218* (2022).
- [248] Lek-Heng Lim. 2020. Hodge Laplacians on graphs. *Siam Rev.* 62, 3 (2020), 685–715.
- [249] Ginestra Bianconi. 2021. *Higher-order Networks*. Cambridge University Press.
- [250] Austin R. Benson, Rediet Abebe, Michael T. Schaub, Ali Jadbabaie, and Jon Kleinberg. 2018. Simplicial closure and higher-order link prediction. *Proc. Nat. Acad. Sci.* 115, 48 (2018), E11221–E11230.
- [251] Federico Battiston, Giulia Cencetti, Iacopo Iacopini, Vito Latora, Maxime Lucas, Alice Patania, Jean-Gabriel Young, and Giovanni Petri. 2020. Networks beyond pairwise interactions: Structure and dynamics. *Phys. Rep.* 874 (2020), 1–92.
- [252] Manlio De Domenico, Clara Granell, Mason A. Porter, and Alex Arenas. 2016. The physics of spreading processes in multilayer networks. *Nat. Phys.* 12, 10 (2016), 901–906.
- [253] Ginestra Bianconi. 2018. *Multilayer Networks: Structure and Function*. Oxford University Press.
- [254] Alberto Aleta and Yamir Moreno. 2018. Multilayer networks in a nutshell. *arXiv preprint arXiv:1804.03488* (2018).
- [255] Gaogao Dong, Dongli Duan, and Yongxiang Xia. 2021. A briefing survey on advances of coupled networks with various patterns. *Front. Phys.* 9 (2021), 795279.
- [256] Marco Grassia, Manlio De Domenico, and Giuseppe Mangioni. 2021. mGNN: Generalizing the graph neural networks to the multilayer case. *arXiv preprint arXiv:2109.10119* (2021).

Received 5 August 2021; revised 15 May 2023; accepted 18 September 2023

Selected Publication 2

Understanding Influence Maximization via Higher-Order Decomposition

Zonghan Zhang

Zhiqian Chen

Department of Computer Science and Engineering, Mississippi State University
zz239@msstate.edu, zchen@cse.msstate.edu

Abstract

Given its vast application on online social networks, Influence Maximization (IM) has garnered considerable attention over the last couple of decades. Due to the intricacy of IM, most current research concentrates on estimating the first-order contribution of the nodes to select a seed set, disregarding the higher-order interplay between different seeds. Consequently, the actual influence spread frequently deviates from expectations, and it remains unclear how the seed set quantitatively contributes to this deviation. To address this deficiency, this work dissects the influence exerted on individual seeds and their higher-order interactions utilizing the Sobol index, a variance-based sensitivity analysis. To adapt to IM contexts, seed selection is phrased as binary variables and split into distributions of varying orders. Based on our analysis with various Sobol indices, an IM algorithm dubbed **SIM** is proposed to improve the performance of current IM algorithms by over-selecting nodes followed by strategic pruning. A case study is carried out to demonstrate that the explanation of the impact effect can dependably identify the key higher-order interactions among seeds. **SIM** is empirically proved to be superior in effectiveness and competitive in efficiency by experiments on synthetic and real-world graphs.

1 Introduction

As online social networks have drawn and maintained hundreds of millions of users during the past few decades, influence maximization (IM) attracts great attention [8, 16]. It is widely applied to viral marketing [4], rumor control [11], social recommendation [32], and infectious disease containment [17]. IM is an NP-hard task in which a k -sized seed set is selected to maximize the number of influenced nodes which is also called influence spread [13]. The ineffectiveness or unreliability of IM methods could result in the loss of millions of dollars or even human lives, so their performance and dependability are of the utmost importance.

Current IM research suffers two major flaws. **(1) Lack of tools for influence spread decomposition.** Most current research focuses on identifying a set of seeds that maximizes the expected influence spread. Other

than that, which seed or seeds account for the greatest contribution to the final influence has not been discussed. Although some of the heuristics select seeds according to transparent measures such as centrality metrics, the process through which the seed set results in the final influence spread is still hidden. Nonetheless, due to the high stake in the quality of the solution, this explanation of the selected seed set is necessary. Meanwhile, existing tools that can disentangle and quantitatively evaluate the contributions of the input variables in a nonlinear function have various drawbacks: *Local interpretation methods*, including LIME [20] and Grad-cam [26], focus only on local neighborhoods and disregard the global impact of the seeds and their higher-order interactions. *Marginal contribution* [3] does not separate the higher-order interaction effects from the main effect thus fails to quantify the interactions among seeds. *The Shapley value* [22] distributes the higher-order interaction among the seeds and tends to underestimate the contribution of the nodes with larger influence overlaps with others [18]. Beyond that, none handle higher-order interactions in the IM scenario well.

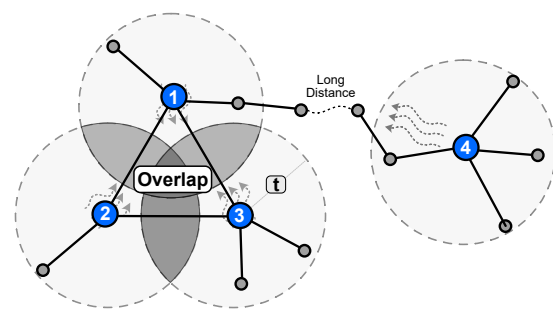


Figure 1: Interaction among the seeds.

(2) The imbalance of effectiveness and efficiency. Current seed selection methods are typically divided into two categories, simulation-based and proxy-based [16]. Unfortunately, neither can offer consistently trustworthy solutions in a timely manner. *The simulation-based methods* are theoretically guaranteed to have a high approximation performance by greedily adding the node

with the highest marginal spread in each step [13]. However, multiple rounds of simulations are required to evaluate the marginal spread for each candidate node in each round, making the heuristic extremely slow when the graph size is large [1]. For the sake of efficiency, the *proxy-based IM algorithms* emphasize the first-order contribution while disregarding the interaction efficacy among the seeds [5, 31, 33]. Nevertheless, due to the combinatorial impact in the seed set, the actual influence spread is heavily influenced by this interaction [13, 5, 16]. Thus, there is no assurance that the proxy-based algorithms can reach a good approximation ratio. Figure 1 illustrates an example of this interaction by depicting the influence distribution of a four-node seed set. Three of them (nodes 1, 2, and 3) are close to one another, whereas the fourth is far away. During t stages of the propagation, the influence flows propagated from the first three seeds may overlap (gray areas), canceling a portion of their overall influence spread and failing to maximize the joint coverage. In IM community, most research still focuses on searching for faster but less accurate simulation methods [6, 2] or designing better proxies that usually come with higher computational loads [31, 33]. Still, the gap between effectiveness and efficiency is yet to be filled.

To address these issues, we introduce a global sensitivity analysis method, namely the Sobol indices [27, 29], as an effective method to decompose a seed set's influence spread. In particular, the global contribution of a seed node is measured using the Sobol total index, which reflects its overall contribution to the influence spread. The contribution of the node is then explained using first- and higher-order Sobol indices. Notably, the interactions between seed nodes are identified quantitatively with a higher-order index, which explains why most first-order approaches fail to identify the contributions of nodes appropriately. The Sobol indices can overcome the shortcomings of the alternative tools mentioned above: **(1)** It is a global sensitivity method that considers a seed's global impact within the seed set. **(2)** The first-order effect and the higher-order interactions can be separated with a clear cut utilizing the corresponding Sobol indices. **(3)** A seed's Sobol total index includes the complete contribution of all of its higher-order interactions with the rest of the seed set. Hence it never overestimates the influence like Shapley value does [18].

After decomposing the influence spread towards each node, we rank the nodes according to their relative importance defined by their contributions. With this information, we propose a light-weighted method named **SIM** to find a balance between effectiveness and efficiency. Specifically, **SIM** selects excessive candidates

and uses the Sobol indices to identify the ones that need to be excluded. Our primary contributions include:

- **Explain and decompose the influence spread:** Given any seed set and the corresponding graph, each seed's contribution to the influence spread can be measured with the proposed explanatory method regardless of the IM algorithm that generates the seed set. Thus, this method can serve as a universal post-hoc explanation of an IM algorithm. To the best of our knowledge, this paper serves as the first discussion on quantifying the influence overlap among the seeds.
- **Provide a new IM schema:** By combining the simulation with the proxies, **SIM** overcomes the scalability and dependability issues in the literature. It suggests a new direction for IM research and provides a new schema for designing IM methods.
- **Conduct extensive empirical experiments:** we prove that the Sobol indices can reliably evaluate the contribution of the nodes in the seed set with a case study. Real-world and synthetic datasets are employed to demonstrate that **SIM** achieves superior performance in a reasonable amount of time.

2 Related Work

Influence Maximization. Given that IM is NP-hard, researchers attempt to find a feasible solution with the best performance. As the first approximation attempt, a simulation-based greedy algorithm was proposed in [13], which is not scalable. Similarly, a thread of simulation-based methods is developed to increase the performance or reduce the complexity [15, 10]. Although great effort has been made to accelerate the process of the simulation-based methods, the complexity is still unacceptably high for the enormous online social networks [1, 28]. Most importantly, the opaqueness of the simulation makes this thread of methods impossible to be explained or improved by evaluating the diffusion process [16]. To alleviate the heavy computational burden of simulations, researchers turned to proxy-based methods in which the spreading power of the nodes is estimated by specific proxies. They started from simple heuristic measures such as degree, PageRank [19], eigen-centrality [34] and later turned to influence-aware or diffusion model-aware proxies [5, 14, 31, 33] to better estimate the influence spread brought by the seeds.

Variance-Based Sensitivity Analysis. Sensitive analysis studies the proportion based on which the uncertainty lying within the output of a model is distributed to the multiple sources of uncertainty in the input variables [25]. Variance-based methods take a major part in

sensitive analysis and have been dated back to 1970s [7]. Since then, these methods have been well studied and widely adopted by researchers and practitioners [24]. Among them, the Sobol indices [27] is considered a significant milestone. However, variance-based methods cannot be directly utilized on IM problems since the input variables are not clearly defined and the influence spread has not been represented as a nonlinear function. In the following section, we will adapt the Sobol indices and a few of its important inheritors to the IM context and utilize them to provide a human-understandable explanation for a given seed set.

3 Problem Setup

A network is denoted by a bi-directional graph $G = (\mathcal{V}, \mathcal{E})$ where $\mathcal{V}$ and $\mathcal{E}$ represent vertices and edges respectively. Given the graph G , a seed budget $k \in \mathbb{N}^+$, and an influence maximization method M , a k -size seed set $\Omega = M(G, k)$ can be generated to approximately maximize the expected influence spread.

(1) Our first objective is to develop *an explanatory method* that can explain the total contribution of a single candidate seed node from a higher-order perspective. The Total contribution $f_i^{(T)}$ of seed i towards the expected influence spread $\sigma(\Omega)$ can be decomposed to different orders of i such that:

$$(3.1) \quad f_i^{(T)} = \sum_{\Psi \subseteq \Omega} f_{\Psi},$$

where Ψ is all the possible subsets containing node i , and f_{Ψ} is the contribution of the interaction involving all nodes within a seed set Ψ . Suppose $\Omega = \{i, j, k\}$, then $\Psi \in \{\{i\}, \{i, j\}, \{i, k\}, \{j, k\}, \{i, j, k\}\}$. Note that $\Psi = \{i\}$ means the first-order contribution. f_{Ψ} is the contribution of the highest-order interaction in Ψ . For example, when $\Psi = \{i, j, k\}$, f_{Ψ} represent the interaction among i, j, k together, with no pairwise interaction such as the interaction between i and j .

(2) Our second objective is to design *an efficient and effective IM schema* to identify the most influential seed set (Ω^*) of size k in terms of higher-order interactions, such that:

$$(3.2) \quad \Omega^* = \underset{\Omega}{\operatorname{argmax}} \left(\sum_i f_{\{\Omega_i\}} + \sum_i \sum_{j>i} f_{\{\Omega_i, \Omega_j\}} + \cdots + f_{\Omega} \right)$$

in which Ω_i is the i -th node of Ω .

4 Preliminary: Functional Analysis of Variance

The objective of the functional analysis of variance (ANOVA) is to assess the significance of input to output based on the variance relationship between them. In this section, we will introduce ANOVA with *first-order effect*, *higher-order effects* and *total effect*, which is

defined by Sobol indices [23] and represent significance from different perspectives. Specifically, variance-based functional decomposition is employed to analyze the impact of each variable. Given any model of the form $Y = f(X_1, X_2, \dots, X_n)$, with Y a scalar, the steps of a variance-based framework are as follows:

$$(4.3) \quad Y = f_0 + \overbrace{\sum_i f_i}^{\text{first-order}} + \overbrace{\sum_i \sum_{j>i} f_{ij} + \cdots + f_{12\dots n}}^{\text{higher-order}},$$

where

$$\begin{aligned} f_0 &= \mathbb{E}(Y), \\ f_i &= \mathbb{E}_{\mathbf{X}_{\sim i}}(Y | X_i) - \mathbb{E}(Y), \\ f_{ij} &= \mathbb{E}_{\mathbf{X}_{\sim ij}}(Y | X_i, X_j) - f_i - f_j - \mathbb{E}(Y), \end{aligned}$$

and similarly for the higher-order terms. X_i is the i -th variable, and $\mathbf{X}_{\sim i}$ denotes the variable set $\mathbf{X} = \{X_1, X_2, \dots, X_n\}$ that excludes X_i . $\mathbb{E}$ denotes the expectation. So f_i can be explained as the expected value change of Y taken over $\mathbf{X}_{\sim i}$ before and after X_i is fixed, deriving the contribution of X_i to Y . Similarly, f_{ij} means the expected value change of Y before and after fixing X_i and X_j . We will demonstrate below how to calculate the variance of a variable in various orders. **(1) First-Order Effect**, denoted as $\mathbf{V}_i$, is the impact through one single variable only, which is represented by variance $\mathbb{V}$ accounted for X_i and it can be written as:

$$(4.4) \quad \mathbf{V}_i := \mathbb{V}(f_i) = \mathbb{V}_{X_i}[\overbrace{\mathbb{E}_{\mathbf{X}_{\sim i}}(Y | X_i)}^{\text{inner expectation}}],$$

since $\mathbb{V}_{X_i}[\mathbb{E}(Y)] = 0$. $\mathbb{V}_{X_i}(\cdot)$ represents the variance of argument $(\cdot)$ taken over X_i . The meaning of the *inner expectation* is that the mean of Y is taken over all possible values of $\mathbf{X}_{\sim i}$ while keeping X_i fixed. The outer variance ($\mathbb{V}_{X_i}$) is taken over all possible values of X_i . The associated first-order sensitivity is obtained by normalization:

$$(4.5) \quad S_i = \frac{\mathbf{V}_i}{\mathbb{V}(Y)}.$$

(2) Higher-Order Effect, using a second-order interaction effect between X_i and X_j as an example, can be expressed as:

$$(4.6) \quad \mathbf{V}_{ij}^{(H)} := \mathbb{V}[f_{ij}(X_i, X_j)] = \mathbf{V}_{ij} - \mathbf{V}_i - \mathbf{V}_j,$$

which means the variance brought by the interactions between variables i and j . $\mathbf{V}_{ij}$ means the first-order variance of the variable set $\{i, j\}$, and it considers the variance of $\{i\}$, $\{j\}$, or $\{i, j\}$. Extending Equation 4.4 from an individual variable to a variable set, we have:

$$(4.7) \quad \mathbf{V}_{ij} = \mathbb{V}_{X_i X_j}[\mathbb{E}_{\mathbf{X}_{\sim ij}}(Y | X_i, X_j)].$$

Further, Equation 4.6 can be normalized by $\mathbb{V}(Y)$:

$$(4.8) \quad S_{ij}^{(H)} = \frac{\mathbf{V}_{ij}^{(H)}}{\mathbb{V}(Y)} = S_{ij} - S_i - S_j,$$

which means that the second-order effect is the first-order effect of the variable set minus the first-order effects of both individual variables. Similarly, we can calculate the third-order interaction among variables X_i, X_j , and X_h as

$$(4.9) \quad S_{hij}^{(H)} = S_{hij} - S_h - S_i - S_j - S_{hi}^{(H)} - S_{hj}^{(H)} - S_{ij}^{(H)},$$

and so on. Given there are totally n variables in the model, we have

$$(4.10) \quad \sum_i S_i + \sum_i \sum_{j>i} S_{ij}^{(H)} + \cdots + S_{12\dots n}^{(H)} = 1.$$

(3) Total Effect is the impact through one variable and all the interactions with the other variables, i.e., first- and higher-order effects. It can be calculated with [12]:

$$(4.11) \quad S_i^{(T)} = \frac{\mathbb{E}_{\mathbf{x}_{\sim i}}(\mathbb{V}_{X_i}(Y | \mathbf{X}_{\sim i}))}{\mathbb{V}(Y)},$$

where $S_i^{(T)}$ measures the total effect, i.e. sum of all first- and higher-order effects involving X_i .

5 Higher-Order Influence Decomposition

We will use the Sobol indices described in the preliminary to examine the current IM algorithms. Specifically, we will discuss how to (1) **explain** how higher-order effect characterizes the contribution of which first-order methods fail to do; and (2) **estimate** the contribution of node selection with total effect.

As discussed in the last section, sensitivity analysis is based on continuous variables of a nonlinear function. Nevertheless, the node selection problem in IM has not been described as such a function before. To bridge the gap, we define node selection as binary variables in the IM problem, representing whether a seed is included in the optimal seed set since a node can be either selected or not while it can not be partially selected. Specifically, a uniform distribution is employed such that each variable can be either 0 or 1 with a 50%/50% probability. Similar to Sobol notation, Ω_i denotes the selection decision for the i -th seed (1 for selected, 0 for not). This adaptation would greatly simplify the estimation of the Sobol indices since the variables in the model are bounded to be binary, limiting the number of combinations to 2^k . Comparatively, the time complexity of calculating the Sobol indices of a model including not only binary variables is very high. For discrete variables, the number of combinations increases exponentially along with the number of categories. And continuous

variables can be considered as categorical variables that have infinite categories.

With the binary variables representing the selection decisions of the nodes, the randomness in the model is brought by the uncertainty in the influence process. In our research, we adopt both the IC model and the LT model [13] to mimic the influence process. The propagation spreads in multiple discrete time steps t_i . At t_0 , some initial nodes are active as seeds. The uncertainty is brought by the propagation probability for the IC model and the random assignment of the node thresholds in each round of simulation for the LT model. Details are discussed in Appendix.

5.1 Explain with First- and Higher-order Effects. To explain the significance of the seed, its influence spread is decomposed into first- and higher-order effects. We will illustrate how to calculate first-order and higher-order impacts in an IM setting and identify which higher-order relationship contributes so substantially that a first-order analysis alone is insufficient.

(1) First-Order Effect in IM. The first-order Sobol index of the node i evaluates the influence spread it brings with no overlaps with other nodes. This is the area activated by the node i as the seed, while no other seeds could successfully activate it. Representing the selection of the node i with a binary variable, we can calculate the first-order Sobol index inspired by Equation 4.5:

$$(5.12) \quad S_i = \frac{\mathbb{V}_{\Omega_i}(\mathbb{E}_{\Omega_{\sim i}}(Y | \Omega_i))}{\mathbb{V}(Y)} = \frac{1}{4^k \cdot \mathbb{V}(Y)} [\sum_{\Omega_{\sim i}} Y_{\Omega_i=1} - Y_{\Omega_i=0}]^2,$$

where $\Omega_{\sim i}$ represents a subset of the seed set that excludes the i -th seed. See Appendix for transformation details of Equation 5.12. Define $\Delta_{\Omega_{\sim i}} := |Y_{\Omega_i=1} - Y_{\Omega_i=0}|$ as the difference of influence spreads of Ω_i given seed sets $\Omega_{\sim i}$, we can rewrite Equations 5.12 as:

$$(5.13) \quad S_i = \frac{(\sum_{\Omega_{\sim i}} \Delta_{\Omega_{\sim i}})^2}{4^k \cdot \mathbb{V}(Y)},$$

which will be used to quantify the first-order effect of certain nodes and to compare the higher-order effects.

(2) Higher-Order Effect in IM. In the IM scenario, the higher-order Sobol indices can quantify the influence overlaps among the nodes and help with identifying the critical overlaps that make the IM algorithms misjudge the value of selecting a certain node as one of the seeds. The larger a node's total higher-order Sobol indices are, the more the actual influence spread it results in would shrink from its value identified by the IM algorithms. These indices can be calculated either by summing the higher-order Sobol indices from all orders together or by subtracting the first-order Sobol index from the Sobol

total index, The calculation of the higher-order Sobol indices can also be adapted to the IM settings and simplified. Based on 4.4, the first-order Sobol index of a size- s subset $\Psi \subseteq \Omega$ can be calculated by:

$$(5.14) \quad S_{\Psi} = \frac{\mathbb{V}_{\Psi}(\mathbb{E}_{\Omega \sim \Psi}(Y|\Psi))}{\mathbb{V}(Y)} = \frac{\sum_{\Psi} (\sum_{\Omega \sim \Psi} (Y|\Psi) \cdot 2^{s-k} - \mathbb{E}(Y))^2}{2^s \cdot \mathbb{V}(Y)}.$$

See Appendix for transformation details of Equation 5.14. The higher-order Sobol indices can be calculated in an iterative manner, starting from the second order to the k -th order. Specifically, the s -th order Sobol indices of Ψ whose $|\Psi| = s$ is:

$$(5.15) \quad S_{\Psi}^{(H)} = S_{\Psi} - \sum_{i=1}^s S_i - \sum_{|\zeta|=2}^{|\zeta|=(s-1)} \sum_{\zeta} S_{\zeta}^{(H)}, \quad (5.18)$$

where $\zeta \subseteq \Psi$. Note that there exist multiple subsets for each order. For example, suppose $\Psi = \{1, 2, 3\}$ as in Figure 2, the implementation of Equation 5.15 is:

$$(5.16) \quad S_{123}^{(H)} = S_{123} - S_1 - S_2 - S_3 - S_{12}^{(H)} - S_{23}^{(H)} - S_{13}^{(H)}$$

Similarly, it is possible to compute the fourth-order and even higher-order effects. Intuitively, though, higher-order relationships have fewer effects on average, such as $S_{123}^{(H)}$ being smaller than $S_{12}^{(H)}$, yet incur exponentially rising computing costs. In light of this, this study will investigate the second-order contributions to influence propagation, which, as proved by our experiments, are sufficient to explain why first-order is insufficient. Various higher-order selections can be expanded as necessary.

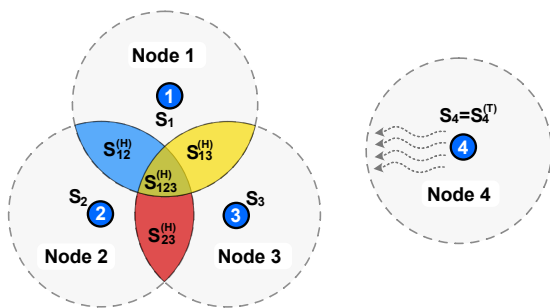


Figure 2: Sobol-based Influence decomposition.

5.2 Estimate influence spread of Node Set with Total Effect. Given a seed set Ω and the graph G representing the network where the propagation happens, we can estimate the influence spread within t time steps or until the propagation fully stops. Intuitively, we can write the relationship between the influence spread and the seed set as:

$$(5.17) \quad g : \Omega \rightarrow \sigma,$$

where g is a function of seed set Ω whose size is smaller than or equal to k , and g predicts the influence spread σ based on the seed set Ω . To evaluate the contribution of a single seed within the set, we need to quantify the variance brought by including the seed in the seed set.

To facilitate the evaluation, an analysis of the total effect will be applied to node set. As discussed, total effect measures the amount of influence variance lost if node i is not included in the seed set. It includes not only the part of the network activated by node i , but also the part that could be activated by this node and some other nodes at the same time. Following Equation 4.11, the Sobol total index for a specific node in IM can be written as:

$$S_i^{(T)} = \frac{\mathbb{E}_{\Omega \sim i} (\mathbb{V}_{\Omega_i} (Y | \Omega_{\sim i}))}{\mathbb{V}(Y)} = \frac{\sum_{\Omega \sim i} (Y_{\Omega_i=1} - Y_{\Omega_i=0})^2}{2^{k+1} \cdot \mathbb{V}(Y)}.$$

Detailed transformation is shown in Appendix. Similar to Equation 5.13, Equations 5.18 can be rewritten as:

$$(5.19) \quad S_i^{(T)} = \frac{\sum_{\Omega \sim i} (\Delta_{\Omega \sim i})^2}{2^{k+1} \cdot \mathbb{V}(Y)},$$

This will be used to evaluate a specific node's global contribution to IM. Note that the numerator in Equation 5.19 is the sum of the squares while the numerator in Equation 5.13 is the square of the sum.

5.3 SIM: The Proposed Algorithm. Now that the Sobol total index can be used to evaluate the significance of a seed within the seed set, we can deduce that deleting the node with the lowest Sobol total index will result in the most negligible influence loss when the budget restriction is reduced from k to $k - 1$. Based on this finding, we propose a light-weighted plug-in approach named **SIM** as described in Algorithm 1. It can improve the performance of the mainstream IM method in two steps, namely **collecting** of seed candidates and **pruning** of the nodes with less impact on the influence spread while maintaining a good balance between effectiveness and efficiency.

(1) Collecting (line 1-3). In the first step of SIM, $\lceil ak \rceil$ candidates are selected with state-of-the-art IM algorithms (line 3), leading to extra $\lceil ak \rceil - k$ nodes beyond the budget. IM algorithms would select overall good seed nodes ignoring higher-order interaction among them, but there may be over- or under-estimated effects, which need to be corrected.

(2) Pruning (line 4-11). To satisfy the budget constraint, $\lceil (a-1)k \rceil$ seeds need to be removed from the $\lceil ak \rceil$ candidates. This selection process is accomplished by the Sobol total indices (line 7). Note that the Sobol total index is to measure the variance lost when holding one variable stable with all other variables still in the

Algorithm 1 Sobol Influence Maximization (SIM)

Input: Graph $G = (\mathcal{V}, \mathcal{E}, A)$, budget constraint k , number of propagation steps t , IM algorithm M

Output: A k -sized seed set Ω , i.e., $|\Omega| = k$

```

1: /* 1, Collecting */
2: Set a over-selection parameter  $a \in \mathbb{R}^+, a > 1$ 
3:  $\Omega \leftarrow M(G, \lceil ak \rceil, t)$ 
4: /* 2, Pruning */
5: while  $|\Omega| > k$  do
6:   for seed  $i$  in  $\Omega$  do
7:     Calculate  $S_i^{(T)}$  (with Equation 5.19)
8:   end for
9:    $i \leftarrow \arg \min_i S_i^{(T)}$ 
10:   $\Omega \leftarrow \Omega_{\sim i}$ 
11: end while

```

model. Thus, removing the node with the lowest index results in the least influence loss. The pruning of the extra nodes must be done one at a time iteratively until exactly k nodes are in the set. The Sobol total index for each seed left would be updated after each iteration, and the rank of the seeds might change. Thus, selecting the nodes with the largest value of indices from the candidates or removing the $\lceil ak \rceil - k$ lowest-ranking nodes at one time does not guarantee the best result.

Time Complexity Analysis. For heuristic algorithms with no updates to the proxies during the selection procedure, such as degree and Eigen-centrality, the time cost of selecting ak nodes is the same as selecting k . For those who update their proxies, the time for each node selection and proxy update iteration remains the same while the number of iterations rises from k to ak . Thus, the time complexity of the Collecting process is at most $a * \mathcal{O}(M)$ where $\mathcal{O}(M)$ is the time complexity of M . This complexity is equal to $\mathcal{O}(M)$ when we consider a as a constant. Assuming that it takes r rounds of simulations to estimate a seed's Sobol total index (line 7 in Algorithm 1), the total number of simulation rounds required for calculating the indices throughout the pruning process is $2^{ak} \cdot r + 2^{ak-1} \cdot r + \dots + 2^{k+1} \cdot r = (2^{ak+k} - 2^{k+1}) \cdot r = \mathcal{O}(2^{ak}r)$. The time complexity concerning ak seems exponentially huge. However, a is a small constant, and k is the budget restriction that does not grow with the graph size. Besides, r is a linear factor that will not expand as the size of the problem grows and will be set to an acceptable value. Therefore the extra time cost is constant regardless of the graph size as long as the budget restriction stays the same. The overall time complexity of the Algorithm 1 is $a * \mathcal{O}(M) + \mathcal{O}(2^{ak}r) \sim \mathcal{O}(M)$ unless the budget constraint grows with the graph size.

6 Experiment

Synthetic and real-world datasets are used to test the proposed explanatory method and SIM. Code for the experiments is available at <https://github.com/oates9895/SIM>

6.1 Configurations. All experiments are carried out on a server with 32 AMD EPYC 7302P 16-Core processors and 32GB RAM. Simulations are performed by NDLlib [21] which is an open-source tool for investigating diffusion processes and dynamics.

Datasets. The framework is tested on seven datasets. (1) Five real-world datasets¹ including *Cora*, *CiteSeer*, *PubMed*, *Amazon Computers*, and *Amazon Photo* are employed to mimic the sophisticated online social network structure. Since the IM problems traditionally focus on connected networks, the largest connected component of each graph is utilized as the network from which the seed set is selected. For *Cora*, *CiteSeer*, and *PubMed*, the edges are uniformly and randomly weighted between 0.40 to 0.80 for the IC model to reflect the varied activation probabilities between nodes. For *Amazon Computers* and *Amazon Photo* graphs, the edges are between 0.05 to 0.20 since those two graphs are denser, and the propagation can pass on with lower activation probabilities. For the LT model, the nodes are randomly assigned thresholds uniformly distributed between 0.01 to 0.20 in each round of simulation for each graph. (2) Two synthetic graphs representing pseudo social networks are generated using NetworkX². The graphs include *connected Watts-Strogatz small-world graphs* (SW) [30] and *Erdős-Rényi random graphs* (ER) [9]. Each graph has 5000 nodes, and the average degree is approximately 10. The edges and nodes are weighted in a similar manner to the *CiteSeer* graph.

Baselines. To evaluate the legitimacy of our evaluation of the selected seed set and the corresponding IM method, we select a few popular IM algorithms as our candidates. SIM is compared with these baselines to evaluate its performance: (1) *Degree Centrality (DEG)*: The first k nodes with the highest degree centrality from the target graph are selected. Before each iteration, the degrees are updated such that the selected seeds are removed from the original graph. This heuristic is also known as SingleDiscount [5]. (2) *Eigenvector Centrality (EIG)*: Similarly, the first k nodes are selected based on their eigenvector centrality. (3) *Simulation-based greedy algorithm (GRD)* [13]: In each iteration, the node with the highest marginal influence spread, which is measured by the average of 1000 rounds of simulations, is added to

¹Datasets at <https://pytorch-geometric.readthedocs.io>

²<https://networkx.org>

the seed set. (4) *Degree discount (DD)* [5]: The degree of each candidate node is discounted according to the likelihood of it being activated by the nodes that have been already selected. (5) *Sigma* [31]: The spreading powers of the nodes are estimated by $\sum_t I \cdot A^t$ where I is a unit column vector. (6) *Pi* [33]: The nodes spreading powers are estimated by $I \cdot (J - \prod_{r=1}^r (1 - A^r))$ where J is an all-one matrix and $\prod$ is the element-wise product of matrices. Among the six, GRD is run on Cora graph only with both the IC model and the LT model to show its lack of scalability, DEG and EIG are applied to both the IC model and the LT model, while DD, Sigma, and Pi are only applied to the IC model.

6.2 Results. The empirical study generally consists of three major parts: (1) **case study**: illustrate the effectiveness of the node contribution decomposition with a 5-node seed scenario. (2) **effectiveness verification**: demonstrate the effectiveness of the proposed algorithm against current IM algorithms. (3) **runtime analysis**: compare the SIM with the baseline IM methods to evaluate its time efficiency.

6.2.1 Case Study. To determine if the Sobol indices are capable of decomposing influence, we conduct a case study using a seed set created by DEG on the most linked component of CiteSeer as an illustration. Five seeds are chosen based on their degree centralities, and the IC model is utilized to estimate the Sobol indices. DEG is chosen as the sampling technique due to its simplicity and readability. And CiteSeer is chosen for computational efficiency because it is the smallest of the five graphs. The contribution of

Node Index	Sobol Total	Marginal Contribution
1422	0.305 ± 0.001	47.782 ± 5.960
582	0.462 ± 0.004	194.512 ± 4.495
1214	0.175 ± 0.002	6.916 ± 5.014
2782	0.182 ± 0.005	7.220 ± 5.751
1943	0.154 ± 0.001	5.354 ± 4.434

Table 1: The Legitimacy of Each Seed’s Relative Contribution.

each seed inside the set is measured by the difference between the estimated influence spreads of the node before and after its inclusion in the seed set (represented as “marginal contribution”). The total Sobol indices for the five seeds are shown in Table 1. The substantial positive association between a node’s Sobol total index and its marginal contribution demonstrates that this index is a reliable indicator of the node’s contribution. In addition, we analyzed the overlap amount and the distance between the nodes. When all the nodes involved

are closer to one another, the influence overlaps tend to be larger because the influence flows emanating from the seeds converge early in the propagation process. If at least one set of nodes are separated by a great distance, the overlap will be small. As shown in Table 2, the second-order Sobol indices are proportional to the distance between any two nodes.

Pairs (node index)	1422	1422	1422	1422	2782
	582	1214	2782	1943	1943
Second-order Sobol	0.00	591.60	619.46	518.00	340.83
Distance	6	2	2	2	2
Pairs (node index)	582	582	582	1214	1214
	1214	2782	1943	2782	1943
Second-order Sobol	0.01	0.00	0.08	424.81	329.21
Distance	7	7	7	1	2

Table 2: The second-order Sobol indices vs. distance.

6.2.2 Effectiveness Verification. Under the IC model, it takes the simulation-based greedy algorithm (GRD) 17758.43 seconds to find a 5-seed set on the largest connected component of the Cora graph, who has 2485 nodes. While the running times of SIM and the other baselines are at most 113.62 seconds, we could observe that the expected influence spreads of the generated seed sets are almost the same. The IM performances of the six heuristics with and without SIM are compared with GRD in Figure 3. It shows that after combined with SIM, the heuristics could achieve a performance almost as good as GRD. The experiment under the LT models shows a similar result. DEG with SIM achieves an influence spread of 1936.11 within 965.03 seconds while a similar performance of 1949.15 takes GRD 34871.65 seconds.

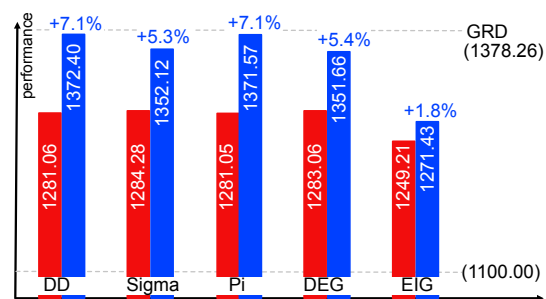


Figure 3: Performances on Cora with IC comparing to GRD. Red: Without SIM. Blue: with SIM.

Figure 3 also demonstrates that applying SIM enhances the IM performance with a ratio up to 7%. On the other six graphs, we compare the IM performance of the five baseline IM methods before and after they are combined with SIM. Without SIM, each IM algorithm generates a seed set consisting of 5 nodes. With SIM,

		CiteSeer (n=2120)	PubMed (n=19717)	Computers (n=13471)	Photo (n=7535)	ER (n=5000)	SW (n=5000)
DEG	W/O	618.90 $\pm$ 34.37	10948.96 $\pm$ 164.77	9221.05 $\pm$ 101.55	3523.68 $\pm$ 81.76	4608.97 $\pm$ 68.80	1284.44 $\pm$ 125.10
	W	698.20 $\pm$ 37.62	11258.95 $\pm$ 156.70	9254.33 $\pm$ 118.61	3581.44 $\pm$ 80.87	4619.67 $\pm$ 67.84	1386.88 $\pm$ 131.43
EIG	W/O	423.41 $\pm$ 20.34	5039.46 $\pm$ 411.31	9216.60 $\pm$ 105.91	3456.62 $\pm$ 77.64	4601.65 $\pm$ 69.25	1084.60 $\pm$ 115.95
	W	485.83 $\pm$ 24.79	7639.77 $\pm$ 439.76	9246.57 $\pm$ 98.83	3568.40 $\pm$ 79.86	4628.89 $\pm$ 63.24	1227.93 $\pm$ 126.65
DD	W/O	672.80 $\pm$ 37.99	11067.78 $\pm$ 172.69	9219.99 $\pm$ 94.94	3523.22 $\pm$ 78.76	4593.98 $\pm$ 72.22	1311.43 $\pm$ 121.18
	W	698.79 $\pm$ 36.01	11334.28 $\pm$ 178.05	9263.77 $\pm$ 99.24	3612.01 $\pm$ 87.54	4625.59 $\pm$ 65.79	1416.00 $\pm$ 117.92
Sigma	W/O	423.82 $\pm$ 20.89	11084.95 $\pm$ 162.37	9214.13 $\pm$ 100.97	3522.06 $\pm$ 83.07	4592.14 $\pm$ 67.82	1142.68 $\pm$ 105.22
	W	694.06 $\pm$ 36.18	11311.20 $\pm$ 144.72	9214.29 $\pm$ 98.72	3578.63 $\pm$ 82.87	4613.20 $\pm$ 67.19	1226.70 $\pm$ 113.71
Pi	W/O	619.03 $\pm$ 33.78	10960.65 $\pm$ 174.90	9212.89 $\pm$ 97.13	3524.33 $\pm$ 81.99	4597.28 $\pm$ 65.79	1163.93 $\pm$ 121.87
	W	697.63 $\pm$ 35.81	11345.50 $\pm$ 151.77	9251.23 $\pm$ 117.30	3583.76 $\pm$ 83.45	4624.04 $\pm$ 62.64	1355.83 $\pm$ 120.89

Table 3: IM performance (i.e., influence spread) with (W) and without (W/O) SIM under IC model. *Italic bold* for the best performance, **bold** for the second best.

each IM algorithm selects 10 candidate nodes during the Collecting procedure and then prunes 5 of them to meet the budget constraint. The final influence spread is measured by the mean and standard deviation of 1000 simulations to eliminate the uncertainty. The detailed result under the IC model is presented in Table 3. See Appendix for the result under the LT model.

We observe a significant increase on influence spread after SIM is applied in most scenarios, with a ratio up to 60%. The performance enhancement effect is greater when the original IM algorithm fails to generate an effective seed set. In most cases, the framework can achieve a boost of around 5% on the influence spread. Another significant finding is that combining the degree discount heuristic with SIM always achieves the best or the second best performance among the algorithms.

6.2.3 Runtime Analysis. It is also proved empirically that the time cost of generating the seed set increases as predicted. We observe that the increase ratios are smaller on larger graphs. This is because the time cost of pruning extra nodes is not related to the graph size, while the time cost of collecting candidate nodes scales with the graph size. Also, SIM runs faster on more sparse graphs since each round of simulations during the Sobol total indices estimation is faster. The details are demonstrated in Table 5 in Appendix.

7 Conclusion

This study presents a way of universally explaining the global effect of a seed set under the IM issue. Sobol's total index is used to assess the contribution of the seeds. The influence overlaps among seeds are quantified for the first time as an explanation using higher-order Sobol indices, which provide insight into the interaction effect and also demonstrate why the first-order method fails to accurately locate the ideal node set. The Sobol indices are calculated within the context of IM. Experiments indicate that the explanation we offered is applicable to graphs and IM algorithms. A revolutionary framework,

SIM, is provided for improving the performance of any proxy-based IM algorithm. Experiments conducted on synthetic and real-world datasets demonstrated that our algorithm offers greater performance at an acceptable cost in terms of time.

Acknowledgement

This work was funded by the NSF IIS award # 2153369.

References

- [1] A. ARORA, S. GALHOTRA, AND S. RANU, *Debunking the myths of influence maximization: An in-depth benchmarking study*, in Proceedings of the 2017 ACM international conference on management of data, 2017, pp. 651–666.
- [2] C. BORGS, M. BRAUTBAR, J. CHAYES, AND B. LUCIER, *Maximizing social influence in nearly optimal time*, in Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms, SIAM, 2014, pp. 946–957.
- [3] A. CATAV, B. FU, Y. ZOABI, A. L. W. MEILIK, N. SHOMRON, J. ERNST, S. SANKARARAMAN, AND R. GILAD-BACHRACH, *Marginal contribution feature importance—an axiomatic approach for explaining data*, in International Conference on Machine Learning, PMLR, 2021, pp. 1324–1335.
- [4] W. CHEN, C. WANG, AND Y. WANG, *Scalable influence maximization for prevalent viral marketing in large-scale social networks*, in Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining, 2010, pp. 1029–1038.
- [5] W. CHEN, Y. WANG, AND S. YANG, *Efficient influence maximization in social networks*, in Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining, 2009, pp. 199–208.
- [6] S. CHENG, H. SHEN, J. HUANG, G. ZHANG, AND X. CHENG, *Staticgreedy: solving the scalability-accuracy dilemma in influence maximization*, in Proceedings of the 22nd ACM international conference on Information & Knowledge Management, 2013, pp. 509–518.

- [7] R. CUKIER, C. FORTUIN, K. E. SHULER, A. PETSCHKE, AND J. H. SCHAIBLY, *Study of the sensitivity of coupled reaction systems to uncertainties in rate coefficients. i theory*, The Journal of chemical physics, 59 (1973), pp. 3873–3878.
- [8] P. DOMINGOS AND M. RICHARDSON, *Mining the network value of customers*, in Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining, 2001, pp. 57–66.
- [9] E. N. GILBERT, *Random graphs*, The Annals of Mathematical Statistics, 30 (1959), pp. 1141–1144.
- [10] A. GOYAL, W. LU, AND L. V. LAKSHMANAN, *Celf++ optimizing the greedy algorithm for influence maximization in social networks*, in Proceedings of the 20th international conference companion on World wide web, 2011, pp. 47–48.
- [11] X. HE, G. SONG, W. CHEN, AND Q. JIANG, *Influence blocking maximization in social networks under the competitive linear threshold model*, in Proceedings of the 2012 siam international conference on data mining, SIAM, 2012, pp. 463–474.
- [12] T. HOMMA AND A. SALTELLI, *Importance measures in global sensitivity analysis of nonlinear models*, Reliability Engineering & System Safety, 52 (1996), pp. 1–17.
- [13] D. KEMPE, J. KLEINBERG, AND É. TARDOS, *Maximizing the spread of influence through a social network*, in Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining, 2003, pp. 137–146.
- [14] M. KIMURA, K. SAITO, AND H. MOTODA, *Blocking links to minimize contamination spread in a social network*, ACM Transactions on Knowledge Discovery from Data (TKDD), 3 (2009), pp. 1–23.
- [15] J. LESKOVEC, A. KRAUSE, C. GUESTRIN, C. FALOUTSOS, J. VANBRIESEN, AND N. GLANCE, *Cost-effective outbreak detection in networks*, in Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining, 2007, pp. 420–429.
- [16] Y. LI, J. FAN, Y. WANG, AND K.-L. TAN, *Influence maximization on social graphs: A survey*, IEEE Transactions on Knowledge and Data Engineering, 30 (2018), pp. 1852–1872.
- [17] M. E. NEWMAN, *Spread of epidemic disease on networks*, Physical review E, 66 (2002), p. 016128.
- [18] A. B. OWEN, *Sobol’indices and shapley value*, SIAM/ASA Journal on Uncertainty Quantification, 2 (2014), pp. 245–251.
- [19] L. PAGE, S. BRIN, R. MOTWANI, AND T. WINOGRAD, *The pagerank citation ranking: Bringing order to the web.*, tech. report, Stanford InfoLab, 1999.
- [20] M. T. RIBEIRO, S. SINGH, AND C. GUESTRIN, *“why should i trust you?” explaining the predictions of any classifier*, in Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, 2016, pp. 1135–1144.
- [21] G. ROSSETTI, L. MILLI, S. RINZIVILLO, A. SIRBU, D. PEDRESCHI, AND F. GIANNOTTI, *Ndlib: Studying network diffusion dynamics*, in 2017 IEEE International Conference on Data Science and Advanced Analytics (DSAA), IEEE, 2017, pp. 155–164.
- [22] A. E. ROTH, *The Shapley value: essays in honor of Lloyd S. Shapley*, Cambridge University Press, 1988.
- [23] A. SALTELLI, P. ANNONI, I. AZZINI, F. CAMPOLONGO, M. RATTO, AND S. TARANTOLA, *Variance based sensitivity analysis of model output. design and estimator for the total sensitivity index*, Computer Physics Communications, 181 (2010), pp. 259–270.
- [24] A. SALTELLI, M. RATTO, T. ANDRES, F. CAMPOLONGO, J. CARIBONI, D. GATELLI, M. SAISANA, AND S. TARANTOLA, *Global sensitivity analysis: the primer*, John Wiley & Sons, 2008.
- [25] A. SALTELLI AND I. M. SOBOLOV, *About the use of rank transformation in sensitivity analysis of model output*, Reliability Engineering & System Safety, 50 (1995), pp. 225–239.
- [26] R. R. SELVARAJU, M. COGSWELL, A. DAS, R. VEDANTAM, D. PARIKH, AND D. BATRA, *Grad-cam: Visual explanations from deep networks via gradient-based localization*, in Proceedings of the IEEE international conference on computer vision, 2017, pp. 618–626.
- [27] I. M. SOBOLOV, *Sensitivity analysis for non-linear mathematical models*, Mathematical modelling and computational experiment, 1 (1993), pp. 407–414.
- [28] Y. TANG, X. XIAO, AND Y. SHI, *Influence maximization: Near-optimal time complexity meets practical efficiency*, in Proceedings of the 2014 ACM SIGMOD international conference on Management of data, 2014, pp. 75–86.
- [29] T. WAGENER AND F. PIANOSI, *What has global sensitivity analysis ever done for us? a systematic review to support scientific advancement and to inform policy-making in earth system modelling*, Earth-science reviews, 194 (2019), pp. 1–18.
- [30] D. J. WATTS AND S. H. STROGATZ, *Collective dynamics of ‘small-world’ networks*, nature, 393 (1998), pp. 440–442.
- [31] R. YAN, D. LI, W. WU, D.-Z. DU, AND Y. WANG, *Minimizing influence of rumors by blockers on social networks: algorithms and analysis*, IEEE Transactions on Network Science and Engineering, 7 (2019), pp. 1067–1078.
- [32] M. YE, X. LIU, AND W.-C. LEE, *Exploring social influence for recommendation: a generative model approach*, in Proceedings of the 35th international ACM SIGIR conference on Research and development in information retrieval, 2012, pp. 671–680.
- [33] Z. ZHANG, S. BISWAS, F. CHEN, K. FU, T. JI, C.-T. LU, N. RAMAKRISHNAN, AND Z. CHEN, *Blocking influence at collective level with hard constraints (student abstract)*, (2022).
- [34] L.-F. ZHONG, M.-S. SHANG, X.-L. CHEN, AND S.-M. CAI, *Identifying the influential nodes via eigen-centrality from the differences and similarities of structure*, Physica A: Statistical Mechanics and its Applications, 510 (2018), pp. 77–82.

Selected Publication 3

Estimating the Circuit De-obfuscation Runtime based on Graph Deep Learning

Zhiqian Chen*, Gaurav Kolhe[†], Setareh Rafatirad[§], Chang-Tien Lu*, Sai Manoj P D[‡],
Houman Homayoun[†], Liang Zhao[§]

* Department of Computer Science, Virginia Tech, Blacksburg, VA, USA

[†] Department of Electrical and Computer Engineering, University of California, Davis, CA, USA

[‡] Department of Electrical and Computer Engineering, George Mason University, VA, USA

[§] Department of Information Science and Technology, George Mason University, VA, USA

*{czq, ctlu}@vt.edu [†]{gskolhe, hhomayoun}@ucdavis.edu {[‡]spudukot, [§]srafatir, [§]lzhao9}@gmu.edu

Abstract—Circuit obfuscation has been proposed to protect digital integrated circuits (ICs) from different security threats such as reverse engineering by introducing ambiguity in the circuit, i.e., the addition of the logic gates whose functionality cannot be determined easily by the attacker. In order to conquer such defenses, techniques such as Boolean satisfiability-checking (SAT)-based attacks were introduced. SAT-attack can potentially decrypt the obfuscated circuits. However, the deobfuscation runtime could have a large span ranging from few milliseconds to a few years or more, depending on the number and location of obfuscated gates, the topology of the obfuscated circuit and obfuscation technique used. To ensure the security of the deployed obfuscation mechanism, it is essential to accurately pre-estimate the deobfuscation time. Thereby one can optimize the deployed defense in order to maximize the deobfuscation runtime.

However, estimating the deobfuscation runtime is a challenging task due to 1) the complexity and heterogeneity of the graph-structured circuit, 2) the unknown and sophisticated mechanisms of the attackers for deobfuscation, 3) efficiency and scalability requirement in practice. To address the challenges mentioned above, this work proposes the first machine-learning framework that predicts the deobfuscation runtime based on graph deep learning. Specifically, we design a new model, ICNet with new input and convolution layers to characterize the circuit's topology, which is then integrated by composite deep fully-connected layers to obtain the deobfuscation runtime. The proposed ICNet is an end-to-end framework that can automatically extract the determinant features required for deobfuscation runtime prediction. Extensive experiments on standard benchmarks demonstrate its effectiveness and efficiency beyond many competitive baselines.

I. INTRODUCTION

The considerable high capital and operational costs on semiconductor fabrication have motivated most semiconductor companies to outsource it is a fabrication to off-shore foundries. Despite the reduced cost and other benefits, this trend has led to ever-increasing security risks such as IC counterfeiting, piracy and unauthorized overproduction by the contract foundries [23]. The overall financial risk caused by such counterfeit and unauthorized ICs was estimated to be over \$169 billion per year [16]. The major threats from the attackers arise from reverse engineering (RE) an IC and fully identifying its functionality through brute force approaches by applying test inputs and obtaining outputs. To prevent such reverse engineering, *Hardware obfuscation* techniques have been extensively researched in recent years [30]. The general idea is to introduce the ambiguity in the functionality of the IC through obfuscation so that the while preserving the original functionality. Such techniques were highly effective until the advent of advanced attacking techniques. This is based on the fact that there are limited types of gates (e.g., AND, OR, XOR) in IC, so the attackers can just brute force all the possible combinations of types for all obfuscated gates

to find out the one that functions identically to the targeted IC to be deobfuscated. As brute force is usually prohibitively expensive, more recently, efficient methods such as Boolean satisfiability problem (SAT)-based attacks have been proposed, which have attracted enormous attention [14].

The runtime of the SAT attack to reverse engineer the IC highly depends on the complexity of the obfuscated IC, which can vary from milliseconds to years or more depending on the number and location of obfuscated gates. Therefore, a successful obfuscation defence is to increase the amount of time (i.e., many years) required to reverse engineer the design. However, obfuscation comes at a substantial cost in finance, power, and space, and such trade-off requires us to search for optimal positions instead of purely increasing their quantity. The obfuscation policy tries to select a set of gates such that maximum obfuscation can be achieved while incurring minimal overheads. Although such selection can significantly influence the deobfuscation runtime, however, until now it is still generally based on human heuristics or experience, which is seriously arbitrary and sub-optimal [7]. This is major because it is unable to “try and error” all the different ways of obfuscation, as there are millions of combinations to try and the runtime for each execution (i.e., to run the attacker) can be days, weeks, or years.

To address this issue, this paper focuses on efficient and scalable ways to estimate the runtime of an attacker to reverse engineer an obfuscated IC. This research topic is highly under-explored because of its significant challenges: **1) Difficulty in characterizing the hidden and sophisticated algorithmic mechanism of attackers.** Over the recent years, a large number of deobfuscation methods have been proposed with various techniques [7]. In order to practically defeat the obfuscation schemes, methods with more and more sophisticated theories, rules, and heuristics have been proposed and adopted. The behaviour of such highly nonlinear and strongly-coupling systems is prohibitive for conventional simple models (e.g., linear regression and support vector machine [1]) to characterize. **2) Challenge in extracting determinant features from discrete and graph-structured IC.** The inputs of the runtime estimation problem is the IC and the selected gates for obfuscation, where the first input is a heterogeneous graph while the second is a vector with discrete values. Conventional feature extraction methods are not intuitive to be applied to such type of data without significant information loss. Hence, it is highly challenging to instantly formulate and seamlessly integrate them as mathematical forms that can be input to conventional computational and machine learning models. **3) Requirement on high efficiency and scalability for deobfuscation runtime estimation.** The key to the defence against deobfuscation is the

speed. The faster the defender can estimate the deobfuscation runtime for each candidate set of obfuscated gates, the more candidate sets the defender can evaluate, and hence the better the obfuscation effect will be. Moreover, the estimation speed of deobfuscation runtime must not be sensitive to different obfuscation strategies in order to make the defender strategy controllable.

This work addresses all the above challenges and proposes the first generic framework for deobfuscation runtime prediction, based on graph deep learning techniques. In the recent years, deep learning methods in complex cognitive tasks such as object recognition and machine translation have achieved immense success [27], [12], which motivates the generalization of it into graph-structured data [8]. By concretely formulating ICs and the obfuscated gates as multi-attributed graphs, this work innovatively leverages and extends the state-of-the-art graph deep learning methods such as Graph Convolutional Neural Networks (GCN) [8] to instantiate a graph regressor. Such end-to-end deep graph regressor can characterize the underlying and sophisticated cognitive process of the attacker for deobfuscating the ICs. To adopt the powerfulness of GCN and handle the aforementioned issues, we extend it by adjusting the connectivity representation inspired by domain facts. Our enhanced GCN can automatically extract the discriminative features that are determinants to the estimation of the deobfuscation runtime to achieve accurate runtime prediction. After being trained, the prediction based on this deobfuscation runtime estimator just runs instantly fast by simply performing a feed-forward propagation process. The major contributions of this paper are:

- Proposing a new framework, ICNet, for deobfuscation runtime estimation based on graph deep learning.
- Developing a new multi-attributed graph convolutional neural network for graph regression.
- Conducting systematical experimental evaluations and analyses on real-world datasets (ISCAS-85 benchmark).

The rest of the paper is organized as follows. Section II reviews the existing work. Section III elaborates proposed graph learning model for SAT runtime prediction. In Section IV, experiments on real-world data are presented. This paper concludes by summarizing the study's important findings in Section V.

II. BACKGROUND AND RELATED WORK

We discuss the logic obfuscation and SAT attacks followed by graph convolutional networks and the relevant works.

A. Logic Obfuscation and SAT Attacks

Logic obfuscation often referred to as logic locking [29] is a hardware security solution that facilitates to hide the IP using key-programmable logic gates. The activation of the obfuscated IP is accomplished in a trusted regime before releasing the product into the market, thereby reducing the probability to obtain the secret configuration keys by the attacker. During the activation phase, the correct key is applied to these key-programmable gates to recover the correct functionality of the IC/IP. Besides, the correct key will be stored in the IC in a tamper-proof memory. Although obfuscation schemes try to minimize the probability of determining the correct key by an attacker, thereby curbing the ongoing piracy of the legitimate IPs. However, SAT attack shows that the contemporary obfuscation schemes can be broken [24] to retrieve the correct key. In order to perform SAT attack, the attacker is required to have The SAT attack first tries to find the Distinguishing Input Patterns (DIP) X_i , which when applied as the input can produce different outputs (Y_i) such that ($Y_1 \neq Y_2$) when

different key values are applied (K_1, K_2). This DIP can then be used to distinguish the correct and incorrect keys. The number of DIPs discovered during the SAT-based attack is the same as the number of iterations needed to unlock the obfuscated design. In each iteration, a constraint is added to SAT solver, until SAT solver cannot find a satisfying assignment. This results in finding the correct key.

Different SAT-hard schemes such as [9], [10] are proposed. Furthermore, new obfuscation schemes that focus on non-Boolean Behaviour of circuits [28], that are not convertible to an SAT circuit is proposed for SAT resilience. Some of such defences include adding cycles into the design [18]. By adding cycles into the design may cause that the SAT attack gets stuck in the infinite loop, however, advanced SAT-based attacks such as cycSAT [32] can extract the correct key despite employing such defences. To ensure that the proposed defence ensures robustness against SAT attacks, the defenders need to run the rigorous simulations which could range from few minutes up to a few days. Furthermore, this can be exacerbated when the defender verifies for large and real-world circuits. This work proposes the use of graph convolutional networks (GCNs) to alleviate the need to run the attack to verify whether the defence is strong enough or not. The work in [19] utilizes neural network with single-bit supervision to predict whether a given circuit in Conjunctive Normal Form (CNF) can be decrypted or not. However, this is limited to determining for few kinds of SAT-solvers, but cannot be applied to SAT-hard solutions such as SMT-SAT [31], a superset of SAT attacks. However, with the proposed graph convolutional network (GCN) based predictor, the defender can determine the deobfuscation time in a single run of GCN, which consumes a few seconds. We introduce the GCN below.

B. Graph Convolutional Networks

Spectral graph theory is the study of the properties of a graph in relationship to the characteristic polynomial, eigenvalues, and eigenvectors of matrices associated with the graph. Many graphs and geometric convolution methods have been proposed recently. The spectral convolution methods [4], [8] are the mainstream algorithms developed as the graph convolution methods. Their theory is based on the graph Fourier analysis [20]. The polynomial approximation is firstly proposed by [6]. Inspired by this, graph convolutional neural networks (GCNs) ([4]) is a successful attempt at generalizing the powerful convolutional neural networks (CNNs) in dealing with Euclidean data to modelling graph-structured data. Kipf and Welling proposed a simplified type of GCNs [8], called graph convolutional networks (GCNs). The GCN model naturally integrates the connectivity patterns and feature attributes of graph-structured data and outperforms many state-of-the-art methods significantly.

Therefore, it is promising to apply GNNs for the circuit problem, since ICs can be naturally represented using a graph with connectivity among gates.

III. PROPOSED MODEL FOR RUNTIME PREDICTION

This section introduces the problem setting, and we present the deobfuscation time prediction through the proposed ICNet.

A. Problem Setting

First, a circuit is modeled as a graph network: $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{W})$, where $\mathcal{V}$ is a set of n vertexes (gates), $\mathcal{E}$ represents links among gates and $\mathcal{W} = [w_{ij}] \in \{0, 1\}^{n \times n}$ is an unweighted adjacency matrix. A signal $\mathbf{X}$ defined on the nodes is regarded as a vector $\mathbf{X} \in \mathbb{R}^{n \times F}$. A graph structure representation, i.e., combinatorial graph Laplacian,

is defined as $\mathbf{L} = \mathbf{D} - \mathbf{W} \in \mathbb{R}^{n \times n}$ where $\mathbf{D}$ is degree matrix. Accordingly, we formulate the estimation of running time on IC as a regression task. Specifically, the model accepts graph structure along with gate features as input, and predict the running time:

$$Y = f(\mathcal{G}, \mathbf{X})\Theta, \quad (1)$$

where f is a function integrating graph structure $\mathcal{G}$ and gate feature $\mathbf{X}$. $\mathcal{G}$ is often represented by graph Laplacian $\mathbf{L}$ in graph theory [2] Θ indicates the parameters of fully neural network layers connecting the actual runtime Y and f . The purpose of Θ is (1) fitting dimension with Y and (2) generalizing the logic pattern between Y and f . The goal of 1 is to learn f and Θ so that the difference between Y and $f(\mathcal{G}, \mathbf{X})\Theta$ is minimized. However, there exists no straightforward relationship among the number of obfuscated gates, type of obfuscated gates and other factors to determine the deobfuscation time. This makes it harder to efficiently estimate the SAT-attack runtime with traditional machine learning models. Hence, we survey a thread of works called graph neural networks or geometric deep learning to address the problem of deobfuscation estimation, as the netlist can be perceived as the graph representation of various logical elements.

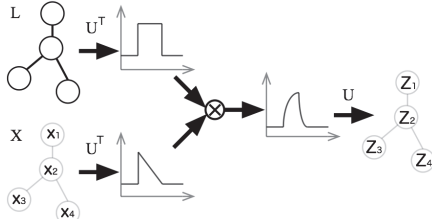


Figure 1: GCN workflow.

Graph convolutional network (GCN) is a recently emerging technique that integrate graph structure and node attributes, and its general process is performed as follows: it determines a vertex complete set of orthonormal Eigen vectors (frequency components) and their associated ordered real non-negative eigenvalues identified as the weights of these frequencies components. Specifically, the Laplacian is first diagonalized by the Fourier basis $\mathbf{U}^T$: $\mathbf{L} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T$ where $\mathbf{\Lambda}$ is the diagonal matrix whose diagonal elements are the corresponding eigenvalues, i.e., $\Lambda_{ii} = \lambda_i$. The graph Fourier transform of a signal $\mathbf{X} \in \mathbb{R}^{n \times F}$ is defined as $\hat{\mathbf{X}} = \mathbf{U}^T \mathbf{X} \in \mathbb{R}^n$ and its inverse as $\mathbf{X} = \mathbf{U} \hat{\mathbf{X}}$ [20], [21]. To enable the formulation of fundamental operations such as filtering in the vertex domain, the convolution operator on graph is defined in the Fourier domain such that $f_1 * f_2 = \mathbf{U}[(\mathbf{U}^T f_1) \otimes (\mathbf{U}^T f_2)]$, where $\otimes$ is the element-wise product, and f_1/f_2 are two signals defined on vertex domain. The intuitive workflow of GCN is shown in Figure 1. It follows that a vertex signal $f_2 = \mathbf{X}$ (gate features) is filtered by spectral signal $f_1 = \mathbf{U}^T f_1 = \mathbf{g}$ (graph structure) as:

$$\mathbf{g} * \mathbf{X} = \mathbf{U}[\mathbf{g}(\mathbf{\Lambda}) \odot (\mathbf{U}^T f_2)] = \mathbf{U} \mathbf{g}(\mathbf{\Lambda}) \mathbf{U}^T \mathbf{X}.$$

B. Proposed Model: ICNet

Our proposed method, namely ICNet, is a neural network that is based on graph convolution operator. As shown in Figure 2, ICNet encodes the obfuscated circuit into two components:

- **Graph Structure $\mathcal{G}$:** Complete set of local connections are often used to represent the graph structure [2]. Typically, a graph Laplacian is employed, in this work since it contains gate-wise connections.

- **gate features $\mathbf{X}$:** Gate-level information is encoded as numerical vector as input feature. Such information could include gate type, whether it is obfuscated and so on.

By applying the GCN, we can easily build a model to learn the relationship between the circuit and deobfuscation time automatically. However, GCN suffers from several issues: (1) the original graph convolutional operator is not suitable for the circuit since the graph Laplacian will make the graph convolutional operator behaviour as label propagation, i.e., the attributes of each gate are similar to its neighbours. This is called the smoothness assumption [13], and it does not fit the fact that gate type or encryption location of each gate does not determine its neighbours' related attributes in theory. This issue is due to that graph Laplacian matrix is used during graph convolution operation, which counts each node as $-N_i$ (i is the index of the row in graph Laplacian), and counts the weighted sum of its neighbours as N_i . Consequently, they are cancelled out when gate representation are aggregated using sum, and the model can hardly learn the relationship between their sum (residues) and actual runtime. (2) The default setting of GCN aggregate gates and their features using the mean function, which is not supported by any domain knowledge, and not likely to cover the actual pattern on features or gates. To solve these issues, our model employs several policies to enhance the traditional GCN for circuit learning.

- **Graph Representation $\mathcal{G} = A$:** Our model uses adjacency matrix A instead of graph Laplacian. This representation can avoid intrinsic smoothness assumption which is not compatible with ICs.
- **Feature Aggregation(Θ_{feat}):** The mean function is a typical methods for aggregating node feature. However, the mean function does not consider the quantity of sum. A more flexible way is to learn feature aggregation by a neural network automatically.
- **Gate Aggregation(Θ_{gate}):** similarly, mean function can also be used to aggregate gate representation. Due to the complicated real-world aggregation, another neural network is designed to learn the gate aggregation function for more flexibility.

Our model is based on GCN which simplify the layer parameters of graph convolutional operator and applies an approximate technique to boost the efficiency. GCNs, as a state-of-the-art deep learning method for the graph, focus on processing graph signals defined on undirected graphs. According to the analysis above, graph Laplacian is replaced with adjacency matrix. To fit whole-graph level regression task, the proposed method designs two aggregation neural networks. Formally, it is denoted as:

$$\begin{aligned} Y &= f(\mathcal{G}, \mathbf{X})\Theta && \text{(GCN definition Eq. 1)} \\ &= \underbrace{\mathbf{GCN}(\mathcal{W}, \mathbf{X})}_{f=\mathbf{GCN}} \Theta && \text{(apply GCN with adjacency matrix)} \\ &= \mathbf{GCN}(\mathcal{W}, \mathbf{X})\Theta_{feat}\Theta_{gate} && (\Theta \rightarrow \{\Theta_{feat}, \Theta_{gate}\}) \\ &= \underbrace{\sigma(\mathcal{W} \mathbf{X} \Theta_{GCN})}_{\mathbf{GCN}} \Theta_{feat}\Theta_{gate}, && \text{(rewrite GCN in matrix form)} \end{aligned} \quad (2)$$

where activation σ is implemented by ReLU function. The running time tends to grow at an exponential rate as the number of encrypted gates increase. Therefore, the model is modified as:

$$Y = \exp(\mathcal{W} \mathbf{X} \Theta_{GCN} \Theta_{feat} \Theta_{gate}) \quad (3)$$

As illustrated in Fig. 2, the proposed ICNet conducts two graph convolutional operations (GCN) to fuse the information

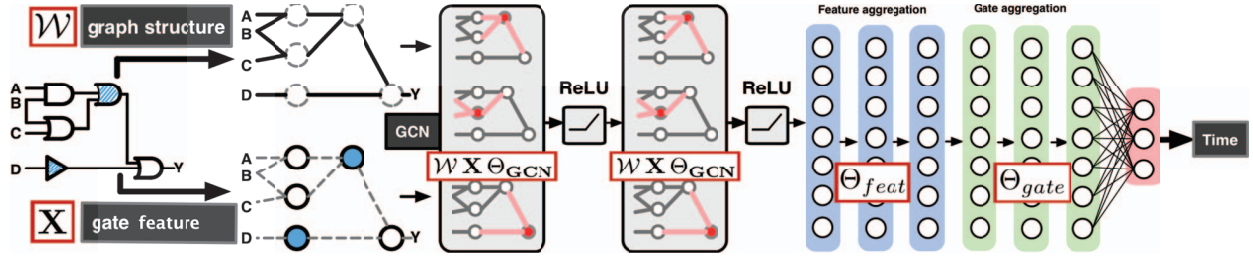


Figure 2: Illustration of ICNet structure: Two graph convolutions ($\mathcal{W} \mathbf{X} \Theta_{\text{GCN}}$) followed by ReLU activation, and attention layers for features (Θ_{feat}) and gate (Θ_{gate}) respectively.

from graph structure and gate features. Then two sets of neural networks are performed for the feature and gate aggregation. To further increase the model's interpretability, we replace fully connected layers Θ_{feat} and Θ_{gate} as follows: Generally, *sum* or *mean* function is a typical method for aggregating node attribute into lower dimensional vector. This treats voting from each gate equally, which is not fit in theory. For example, the encrypted gate should be weighed higher, since it impose more difficulty on obfuscation task; gate types also have a significant impact on runtime [23]. Therefore, a more flexible way is to build a neural network to automatically learn attribute aggregation. To fit the whole-graph level regression task, the proposed method designs two aggregation neural components based on soft attention mechanism [26] for feature-level and gate-level. Formally, the feature based attention is calculated as:

$$a_i = \frac{\exp(e_i)}{\sum_i \exp(e_i)}, e_i = \sum_i \theta_i \mathbf{F}_i, \quad (4)$$

where $\mathbf{F}_i$ represents i th feature after GCN, θ_i is the weight parameter for $\mathbf{F}_i$, a_i is the corresponding attention and thereby the output of this layer is $\sum_i a_i \mathbf{F}_i$. This attention shows which feature contributes more to the obfuscation time. Similarly, gate-wise attention is utilized for gate-level aggregation by setting $\mathbf{F}_i$ to i th gate in (4).

Algorithm 1: ICNet

Input: An integrated circuit graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, gate features set: $x_j(i)$, $i \in 1, 2, \dots, |\mathcal{V}|$ for each encryption instance D_j , the real runtime Y_j for instance D_j
Output: A neural network function with parameters Θ_{GCN} , Θ_{feat} and Θ_{gate}

- 1 // Data preparing
- 2 Calculate $\mathcal{W}$ which is the adjacency matrix of $\mathcal{G}$
- 3 Split encryption instances D into training set D_{train} and testing set D_{test}
- 4 Split both D_{train} and testing set D_{test} into batch set d_{train} and testing set d_{test}
- 5 // Update ICNet
- 6 $\theta = \{\Theta_{\text{GCN}}, \Theta_{\text{feat}}, \Theta_{\text{gate}}\}$
- 7 Initialize θ with Gaussian or uniform distribution.
- 8 **repeat**
- 9 Randomly select one $d_{\text{train}} = x_{d1}, x_{d2}, \dots$
- 10 Calculate predicted runtime $\hat{Y}$ ▷ Eq. 3 and 4
- 11 Calculate residues $\delta = Y - \hat{Y}$
- 12 Compute derivatives to update parameters: $\theta \leftarrow \theta + \beta \nabla_{\theta} \delta$, where β is learning rate
- 13 **until** δ convergence;

C. Algorithm description

The Algorithm 1 first prepare graph adjacency as circuit connection representation (line 2). To fit the machine learning schema, the whole dataset is split into training and testing dataset. Each dataset is then split into small batch size to improve learning efficiency (line 3-4). ICNet training is an iterative process which updates the model until the residues

are small enough or converged (line 6-13). First, the model parameters are initialized by Gaussian or uniform distribution. In each iteration, a batch of the training set is selected randomly. By equation 3, the model computes the predicted runtime (line 10) and then calculates the residues between real runtime and prediction (line 11). Following normal deep learning schema, the model update parameters by the derivatives regarding the parameters themselves with learning rate (line 12).

IV. EVALUATION

This section elaborates evaluation of the proposed method ICNet with competitive baselines including: Graph deep learning methods: GCN [8], ChebNet [4]. The input of these models above is exactly same as our model. We also compare against several state-of-the-art regression models¹: Linear Regression (LR), LASSO [25], Epsilon-Support Vector Regression(SVR) (Two kernels were applied: polynomial (P) and RBF (R)), [22], Ridge Regression (RR) [17], Elastic Net (EN) [33], Orthogonal Matching Pursuit (OMP) [15], SGD Regression, Least Angle Regression (LARS) [5], Theil-Sen Estimators (Theil) [3]. These regression models does not model graph using Laplacian or adjacency matrix, since they can only accept feature vector. Therefore, the input are encoded as mean or sum on concatenation of Laplacian or adjacency matrix and gate features.

A. Data processing

The datasets are obtained by running SAT algorithm [24], [23] on real-world ISCAS-85 benchmark: First, we take a circuit and select a random gate and replace it with LUT of fixed size (LUT size 4 in current work). To deobfuscate, we implement SAT attack algorithm [24], [23] with the obfuscated circuit netlist as input. We monitor the time that SAT-attack takes to decode the key, which is the deobfuscation time. The proposed model is evaluated on two datasets: **Dataset 1**: the total number of the encryption location ranges from 1 to 350, this is for testing if the model is sensitive to the number of encrypted quantity of gates. **Dataset 2**: the total number of the encryption location ranges from 1 to 3, this is for testing if the model can handle very small value.

The circuit in the experiments is the same, and the total gate number of the circuit is 1529. For graph deep learning methods, the graph is represented using Laplacian matrix or adjacency matrix, while for general regression baselines, the graph Laplacian or adjacency matrix is summed or averaged across gates. Though the evaluations showed here are mere proof-of-concept of how powerful the proposed GCN based deobfuscation runtime prediction is, it can be applied to an SAT-hardening solution utilizing any replacement policy, LUT size and other SAT parameters, by retraining GCN.

¹https://scikit-learn.org/stable/modules/linear_model.html

B. Experiment configuration

The features of gate used in experiments include: **gate mask**: if the gate is encrypted, the value is set to 1, otherwise 0. and **gate type**: the gate type include {AND, NOR, NOT, NAND, OR, XOR}, they are encoded using one-hot coding, such as [1,0,0,0,0] for AND and [0,1,0,0,0] for NOR gate.

For graph deep learning model (ChebNet and ICNet), the graph structure is represented using graph Laplacian matrix or adjacency matrix. These model employ ADAM [11] optimizer and will stop learning when the learning loss is converged. The implementation of our model will be available online. All the baselines and the proposed model are tested on two different feature set, since gate type is useful or not is unknown.: **Location**: Only the gate mask is included. **All features**: Besides gate mask, gate type is also included.

For node aggregation, we apply *sum*, and *mean* since they are popular. Deep learning model can have another node aggregation method, i.e., learning by a neural network automatically. Therefore, in the results, ChebNet-NN and ICNet-NN denote the automatic version. It is expected that a deep neural network can learn an optimal aggregation which is not worse than our assumption, i.e., sum or mean.

Table I: Regression Performance (MSE) on Dataset 1

Method	Location		All feat	
	Sum	Mean	Sum	Mean
SVR RBF	1.6791	0.6784	1.6675	0.6739
SVR Poly	0.1913	2.1890	0.1696	2.2091
SGD	2.1450e+25	2.1823	1.0430e+26	2.2072
LR	0.2839	0.2284	0.2449	0.2253
RR	0.2309	2.1508	0.2058	2.1738
LASSO	0.9213	2.1843	1.0127	2.2083
EN	0.5763	2.1843	0.6409	2.2083
OMP	1.8182	1.9192	1.8651	2.0337
LARS	1.9968	2.1277	2.0434	2.1833
Theil	0.2948	0.2238	0.2385	0.2277
ChebNet	0.1484	8.8370e+33	0.1761	0.1760
ChebNet-NN		0.17858	3.8549e+27	
GCN	0.3364	0.4149	0.2496	0.3290
GCN-NN		0.1811	0.1606	
ICNet	0.1534	0.1256	0.2390	0.1902
ICNet-NN		0.0843	0.1367	

C. Regression Results

In the dataset 1 experiment (Table I, all methods achieved acceptable mean square error (MSE) except SGD (sum) which did not learn a reasonable model to predict the runtime, since the value is tremendous (at e+25/+26 scale). Most regression methods are sensitive to the aggregation method. For example, only using location feature, MSE of RR is 0.2309 when using sum, but it got 2.1508 when using the mean function. Sensitive models include SVR, LASSO, and EN. The best of the regression baselines is SVR (poly), which achieved MSE of 0.1913. On the other hand, ChebNet is slightly better than the best regression model. However, ChebNet is not stable and sensitive to the aggregation method and feature set since it may yield a substantial error. Our proposed ICNet-NN is stable to the feature and aggregation setting and outperformed all the other methods, i.e., 0.0843 of MSE. Note that ICNet-NN is better than ICNet with sum or mean function, which demonstrates that there exists a better aggregation method, and graph neural network can learn it automatically. ICNet is always better than GCN under any settings, which shows that our improvement based on GCN works on circuit scenario. While in the dataset 2 (Table II), it is more challenging, since all the runtime are small and the model has to be very precise to achieve low MSE. All methods at almost the same level of MSE. Once again, some of the regression models are not stable such as SGD and LR. Graph deep learning method includes ChebNet and ICNet still at the best error level. ChebNet can achieve the best level but sensitive to the

Table II: Regression Performance (MSE) on Dataset 2

Method	Location		All feat	
	Sum	Mean	Sum	Mean
SVR RBF	0.0051	0.0048	0.0050	0.0051
SVR Poly	0.0048	0.0048	0.0048	0.0051
SGD	7.6301e+25	0.0045	2.0675e+26	0.0049
LR	6.9063e+23	4.6521e+20	7.2916e+25	5.8600e+23
RR	0.0070	0.0045	0.0065	0.0049
LASSO	0.0047	0.0045	0.0046	0.0049
EN	0.0047	0.0045	0.0046	0.0049
OMP	0.0047	0.0045	0.0045	0.0049
PAR	0.0054	0.1918	0.0051	0.3143
LARS	0.0047	0.0045	0.0046	0.0049
Theil	N/A	N/A	N/A	N/A
ChebNet	0.0047	0.0045	4.3570e+28	0.0048
ChebNet-NN		0.0043	0.0047	
GCN	0.0061	0.0046	0.0048	0.0050
GCN-NN		0.0050	0.1606	
ICNet	0.0049	0.0047	0.0040	0.0043
ICNet-NN		0.0051	0.0048	

settings (i.e., location or all feature), while ICNet is insensitive to this setting. Therefore, ICNet is more stable than GCN and ChebNet, since the difference of MSE between location and all feature is smaller than that of ChebNet or GCN. Under all feature setting, ICNet-NN is still the best method, and it outperformed its mean and sum version.

The proposed method, ICNet, not only predicted the value very precisely but also with small variance. The runtime of ICNet is 1.1336 seconds on average, ranging from 1 to 2 seconds on dataset 1 and 2. This is because runtime of ICNet only depends on its parameter number. The instance with the largest runtime on dataset 1 and 2 spends 2411.11 seconds by actual solver. Therefore, ICNet can save 99.95% of solver's time to get accurate runtime.

Next, Fig. 3 illustrates several predicted value along with real value to analyze the prediction characterization. Since there is little difference in dataset 2, we choose several competitive baselines in dataset 1 experiments under all feature setting. Several baselines performed very badly such as OMP and SGD which only output values around a constant level. SVR (RBF) is also bad and yield constant value when the real runtime is larger than a threshold. The results of EN and LASSO is positively related to the real values, but the correlation parameters are significantly different from the truth. Linear, RR, SVR (POLY) and Theil predicted the values that are relatively closer than that of the other baselines, but with high variance.

D. Case Study: Attentions on Attributes

The subsection studies the attention mechanism quantitatively. Several circuits are evaluated, as shown in Table III. Gate number consistently attracted greater attention than the gate type by 9.64% on average. This motivates us to study the correlation between actual runtime and gate number. The Pearson (P) and Spearman (S) correlation are 0.8238 and 0.9722 on average in Table III. Take circuit c7553 as an example, the runtime is 2.37% of gate number. Different circuits show different linear parameters, which gives us a convenient message that can accurately predict the deobfuscation time and could serve circuit obfuscation task.

Table III: Case study: attributes and extracted rules.

circuit	gate #	gate type	corr(P/S)	linear param
c7553	56.40%	43.59%	0.8754 / 0.9345	0.0237
c499	54.39%	47.05%	0.8149 / 0.9965	0.1300
c2670	52.94%	47.05%	0.7769 / 0.9753	0.0559
c1335	56.27%	43.72%	0.8282 / 0.9846	0.0599

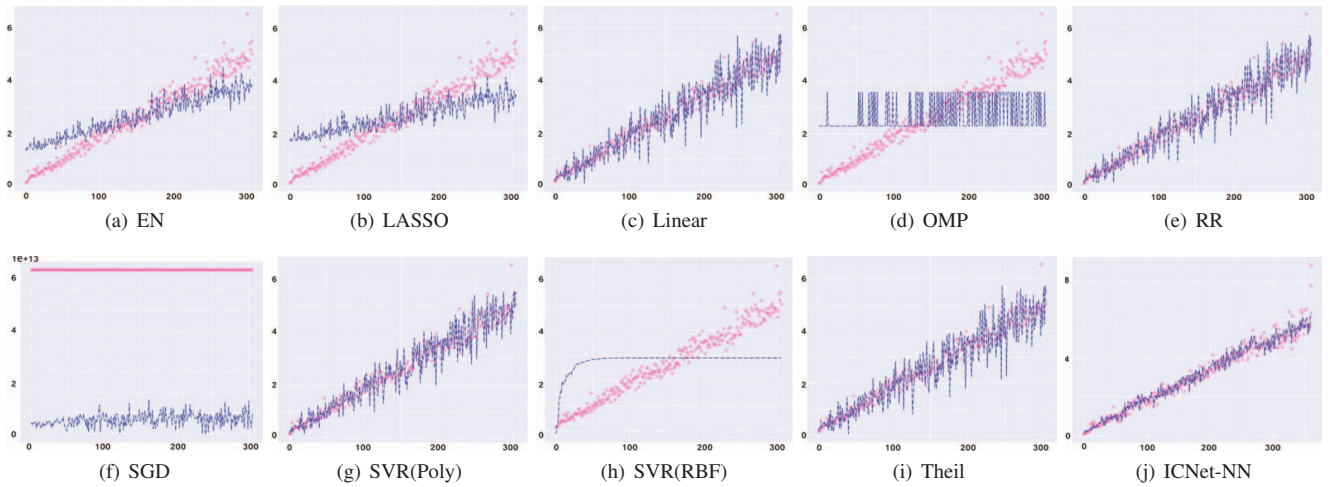


Figure 3: Comparison between predictions and real values: Pink dot are real values, blue lines are the predictions. x-axis is data index in testing data while y-axis is runtime value in log scale. Note that only SGD has different y-axis scale, i.e., $1e+13$.

V. CONCLUSION

In this work, we have introduced a neural network model for recovering SAT runtime on ICs, which expedites the evaluation on the hardness of obfuscated instances and therefore boosts the efficiency of developing obfuscation policy. To properly fuse graph structure and gate features, an enhanced graph convolutional operator is introduced. The proposed ICNet can avoid attribute propagation which is in the original GCN but not suitable for ICs. ICNet automatically extracts determinant features and aggregates gate representation regarding the runtime. Experiments on real-world datasets suggest that the proposed model is capable of modelling the runtime regarding the circuit graph accurately and stably, improving the baselines by a significant margin.

REFERENCES

- [1] C. M. Bishop and T. M. Mitchell. Pattern recognition and machine learning. 2014.
- [2] F. R. Chung. *Spectral graph theory*. Number 92. American Mathematical Soc., 1997.
- [3] X. Dang, H. Peng, X. Wang, and H. Zhang. Theil-sen estimators in a multiple linear regression model. *Olemiss. edu*, 2008.
- [4] M. Defferrard, X. Bresson, and P. Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *NIPS*, pages 3844–3852, 2016.
- [5] B. Efron, T. Hastie, I. Johnstone, R. Tibshirani, et al. Least angle regression. *The Annals of statistics*, 32(2):407–499, 2004.
- [6] D. K. Hammond, P. Vandergheynst, and R. Gribonval. Wavelets on graphs via spectral graph theory. *Applied and Computational Harmonic Analysis*, 30(2):129–150, 2011.
- [7] S. Khaleghi and W. Rao. Hardware obfuscation using strong pufs. In *2018 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pages 321–326. IEEE, 2018.
- [8] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.
- [9] G. Kolhe and et.al. On custom lut-based obfuscation. In *Great Lakes Symposium on VLSI*, 2019.
- [10] G. Kolhe and et.al. Security and complexity analysis of lut-based obfuscation: From blueprint to reality. In *Int. Conference On Computer Aided Design*, 2019.
- [11] Q. V. Le, J. Ngiam, A. Coates, A. Lahiri, B. Prochnow, and A. Y. Ng. On optimization methods for deep learning. In *ICML*, pages 265–272. Omnipress, 2011.
- [12] M. Lechner and et al. ResCoNN: Resource-efficient FPGA-accelerated CNN for traffic sign classification. In *IEEE Int. Green and Sustainable Computing Conference (IGSC)*, 2019.
- [13] Q. Li, Z. Han, and X.-M. Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
- [14] D. Liu, C. Yu, X. Zhang, and D. Holcomb. Oracle-guided incremental sat solving to reverse engineer camouflaged logic circuits. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2016, pages 433–438. IEEE, 2016.
- [15] S. G. Mallat and Z. Zhang. Matching pursuits with time-frequency dictionaries. *IEEE Transactions on signal processing*, 41(12):3397–3415, 1993.
- [16] I. T. P. R. T. . most counterfeited parts represent a \$169 billion potential challenge for global semiconductor industry. <https://technology.ihc.com/405654/top5-most-counterfeited-parts-represent-a-169-billion-potentialchallenge-for-global-semiconductor-market,2>.
- [17] A. Y. Ng. Feature selection, l_1 vs. l_2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on Machine learning*, page 78. ACM, 2004.
- [18] S. Roshanifard, H. Mardani Kamali, and A. Sasan. Srclock: Sat-resistant cyclic logic locking for protecting the hardware. In *Proceedings of the 2018 on Great Lakes Symposium on VLSI, GLSVLSI '18*, 2018.
- [19] D. Selsam, M. Lamm, B. Bünz, P. Liang, L. de Moura, and D. L. Dill. Learning a SAT solver from single-bit supervision. *ArXiv*, abs/1802.03685, 2018.
- [20] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, and P. Vandergheynst. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. *IEEE Signal Processing Magazine*, 30(3):83–98, 2013.
- [21] D. I. Shuman, B. Ricaud, and P. Vandergheynst. Vertex-frequency analysis on graphs. *Applied and Computational Harmonic Analysis*, 40(2):260–291, 2016.
- [22] A. J. Smola and B. Schölkopf. A tutorial on support vector regression. *Statistics and computing*, 14(3):199–222, 2004.
- [23] P. Subramanyan, S. Ray, and S. Malik. Evaluating the security of logic encryption algorithms. In *IEEE Int. Symp. on Hardware Oriented Security and Trust (HOST)*, 2015.
- [24] P. Subramanyan, S. Ray, and S. Malik. Evaluating the security of logic encryption algorithms. In *IEEE Int. Symp. on Hardware Oriented Security and Trust (HOST)*, 2015.
- [25] R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society. Series B (Methodological)*, pages 267–288, 1996.
- [26] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. In *ICLR*, 2018.
- [27] S. Wang, G. Ananthanarayanan, Y. Zeng, N. Goel, A. Pathania, and T. Mitra. High-throughput CNN inference on embedded ARM big.little multi-core processors. *CoRR*, abs/1903.05898, 2019.
- [28] Y. Xie and A. Srivastava. Delay locking: Security enhancement of logic locking against ic counterfeiting and overproduction. In *ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2017.
- [29] M. Yasin, J. J. Rajendran, O. Sinanoglu, and R. Karri. On improving the security of logic locking. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 35(9):1411–1424, Sept 2016.
- [30] M. Yasin, A. Sengupta, M. T. Nabeel, M. Ashraf, J. J. Rajendran, and O. Sinanoglu. Provably-secure logic locking: From theory to practice. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1601–1618. ACM, 2017.
- [31] K. Zamiri Azar, H. Mardani Kamali, H. Homayoun, and A. Sasan. SMT-attack: Next generation attack on obfuscated circuits with capabilities and performance beyond the SAT attacks. In *Transaction of Cryptography Hardware and Embedded Systems*, 2019.
- [32] H. Zhou, R. Jiang, and S. Kong. Cyclesat: Sat-based attack on cyclic logic encryptions. In *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 49–56, Nov 2017.
- [33] H. Zou and T. Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67(2):301–320, 2005.

Selected Publication 4

Learning to Fuse Music Genres with Generative Adversarial Dual Learning

Zhiqian Chen*, Chih-Wei Wu†, Yen-Cheng Lu*, Alexander Lerch† and Chang-Tien Lu*

*Computer Science Department, Virginia Tech

Email: {czq, kevinlu, ctlu}@vt.edu

†Center for Music Technology, Georgia Institute of Technology

Email: {cwu307, alexander.lerch}@gatech.edu

Abstract—FusionGAN is a novel genre fusion framework for music generation that integrates the strengths of generative adversarial networks and dual learning. In particular, the proposed method offers a dual learning extension that can effectively integrate the styles of the given domains. To efficiently quantify the difference among diverse domains and avoid the vanishing gradient issue, FusionGAN provides a Wasserstein based metric to approximate the distance between the target domain and the existing domains. Adopting the Wasserstein distance, a new domain is created by combining the patterns of the existing domains using adversarial learning. Experimental results on public music datasets demonstrated that our approach could effectively merge two genres.

I. INTRODUCTION

Computational creativity is a lively research area that focuses on understanding and facilitating human creativity through the implementation of software programs [2]. With the rapid advances in data-driven algorithms such as deep learning [7], the exploration into computational creativity via machine learning approaches has become increasingly popular. Examples showing the potential of deep learning for creative approaches are the artistic style transfer on images [4] and videos [12]. Music, with its complex hierarchical and sequential structure and its inherent emotional and aesthetic subjectivity, is an intriguing research subject at the core of human creativity. While there has been some work on generative models for music, research that investigates the capabilities of deep learning for creative applications such as style transfer is limited. This paper aims to fill this space by exploring the idea of style fusion in music with generative adversarial dual learning.

In the field of unsupervised generative learning, generative adversarial networks (GAN) [6] have recently gained considerable attention. It is important to note, however, that GANs are designed to learn from a single domain and cannot discover cross-domain knowledge. Inspired by dual learning in machine translation, several recent publications propose methods to pair unlabeled data points in different domains by optimizing bi-directional reconstruction errors [14], [8], [16]. Although these methods are designed to address the challenge of multimodal learning, they do not address the problem of domain fusion.

The goal of this paper is to provide a solution for fusing two or more groups of sequential patterns using unsupervised methods. Specifically, we focus on the problem of generating music with a fused music genre. To combine genres using generative adversarial learning, we propose a framework for integrating multiple GANs. Unlike previous

work, our study targets the creation of an unknown domain by mixing two given domains; a multi-way GAN-based model is proposed to absorb existing patterns and yield a new mixture. With the utilization of the Wasserstein measure [1], we pose a distance constraint on the new domain that automatically balances its relation to the given domains. As a result, our model is capable of generating a mixed pattern after convergence. The main contributions of this paper are:

- **A novel framework for unsupervised music fusion:** The dual learning scheme is extended to involve multiple domains by leveraging mutual regularization. In this way, the proposed framework enables the new domain to keep equal similarities with the given domains and produce a mixture of existing sequential patterns.
- **A sequence fusion method using GANs:** To apply unsupervised fusion learning, we extend the generative adversarial model so that multiple generators and discriminators can be updated by one another.
- **Formulation of an objective function indicating fusion progress:** An objective function based on Wasserstein distance is designed to integrate the information from multiple domains and represent learning progress.

II. RELATED WORK

Music Genre Fusion: Fusion is mostly known as the sub-genre of jazz that emerged in the late '60s, and that combines several musical styles such as funk, rock, and blues with the jazz harmony and improvisation [5]. The term can be generalized, however, to any combinations of music genres. The creative combination of two or more genres obviously requires not only intimate knowledge of the stylistic characteristics of the involved genres but also extensive compositional experience and skill to combine genres in a meaningful and satisfying way. Recently, Engel et al. [3] proposed a new approach of generating new musical sounds through interpolating the latent space learned by WaveNet [11] autoencoders. Additionally, Gatys et al. demonstrated the Deep Neural Networks' (DNNs) capabilities of learning the artistic styles by blending the style of the training materials to the testing images using ConvNet [4]. Both examples show the great potential of using DNNs for merging two abstract concepts without the need for domain knowledge and explicitly defined rules.

Cross-domain GAN: GAN [6], which has quickly risen to one of the most popular generative approaches, learns patterns without requiring adversarial examples or labels. In CGAN [10], the generator G has a conditional parameter c and will learn the conditional distribution of the data.

However, those conditions need to be provided manually, somewhat similar to supervised learning. CoGAN [9] is proposed to learn a joint distribution with only samples drawn from the marginal distributions. DiscoGAN [8] solves the cross-domain pairing by minimizing bi-directional reconstruction loss, while DualGAN [14] takes advantage of the dual learning paradigm in machine translation. CycleGAN [16] presents a method whose mapping functions are cycle-consistent and proposes a cycle consistency loss function to further reduce the space of possible mapping functions.

Different from the previous studies, our work raises a new problem, i.e., merging two sequential patterns into one. We propose a GAN-based framework that merges two domains without manually tuning the distance between the given domains and the target domain.

III. LEARNING TO BLEND MUSIC WITH FUSIONGAN

A. Problem Definition

This paper aims to characterize music fusion across different genres in an unsupervised manner. Based on the data $\mathbf{X}_A$ and $\mathbf{X}_B$ from existing music domains $\mathcal{D}_A, \mathcal{D}_B$ respectively, our goal is to create a new domain $\mathcal{D}_F$ which resembles both $\mathcal{D}_A$ and $\mathcal{D}_B$. In the first phase, the new domain $\mathcal{D}_F$ is expected to learn from $\mathcal{D}_A, \mathcal{D}_B$ and keep an equal distance from both. Specifically, the generator $\mathbf{G}_F$ of $\mathcal{D}_F$ obtains feedback from the discriminators $\mathbf{D}_A, \mathbf{D}_B$ of existing domains $\mathcal{D}_A, \mathcal{D}_B$, while the discriminator $\mathbf{D}_F$ collects data from $\mathbf{G}_F$ and $\mathcal{D}_A, \mathcal{D}_B$ for iterative updating. The $\mathbf{G}_F$ from the new domain minimizes the distance from the domains $\mathcal{D}_A, \mathcal{D}_B$, respectively, and keeps the same distance from $\mathcal{D}_A$ and $\mathcal{D}_B$.

B. FusionGAN

First, the framework initializes individual GAN models for existing domains. Maximum likelihood estimation using Long Short Term Memory (LSTM) is applied to initialize the sequence generators $\mathbf{G}_A(\mathbf{G}_B)$. Then $\mathbf{D}_A(\mathbf{D}_B)$ are trained given the domain data and sequence sampled from $\mathbf{G}_A(\mathbf{G}_B)$ and $\mathbf{D}_A(\mathbf{D}_B)$. After pre-training, we build a GAN model for the new domain using the feedback from the models constructed in the pre-training procedure. First, a new domain $\mathcal{D}_F$ is initialized randomly and trained by both $\mathcal{D}_A$ and $\mathcal{D}_B$. Following the dual learning strategy, $\mathcal{D}_A$ is enhanced by $\mathcal{D}_B$ and $\mathcal{D}_F$ in the second phase. Similarly, $\mathcal{D}_B$ is improved by $\mathcal{D}_A$ and $\mathcal{D}_F$. Such learning proceeds until convergence. The framework overview is shown in Figure 1.

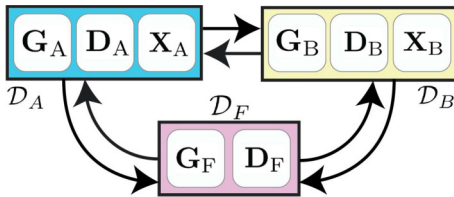


Figure 1: FusionGAN framework

In FusionGAN, we employ the Wasserstein-1 distance for all discriminators because it is sensible when the learning distributions are supported by low dimensional manifolds

[1]. To avoid the intractable infimum in Wasserstein-1 distance, we resort to its Kantorovich-Rubinstein duality [13]: $W(\mathbb{P}_r, \mathbb{P}_\theta) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim \mathbb{P}_r} [f(x)] - \mathbb{E}_{x \sim \mathbb{P}_\theta} [f(x)]$. For the vanilla GAN, the goal is to find the optimal configuration of the parameters ϕ of discriminator ($f = \mathbf{D}$). When the discriminator is optimized, the maximized Wasserstein distance can be used as reward in the policy gradient process of the generator. Similarly, we define a three-way Wasserstein distance as the objective function: $W(\mathbb{P}_{\mathbf{X}_A}, \mathbb{P}_{\mathbf{X}_B}, \mathbb{P}_\theta) = \mathcal{L}$ to measure the integrated distance between $\mathbb{P}_\theta$ and $\mathbb{P}_{\mathbf{X}_A}, \mathbb{P}_{\mathbf{X}_B}$. This measure should consider the relationship between every pair of domains. The framework consists of three pairs of generators and discriminators, i.e., $\mathbf{G}_A - \mathbf{D}_A, \mathbf{G}_B - \mathbf{D}_B, \mathbf{G}_F - \mathbf{D}_F$, and two input data, i.e., $\mathbf{X}_A, \mathbf{X}_B$. For each discriminator, there are five possible inputs, i.e., $\mathbf{X}_A, \mathbf{X}_B, \mathbf{G}_A, \mathbf{G}_B, \mathbf{G}_F$. Collecting all possible inputs w.r.t. the three discriminators, the objective function to maximize is defined as:

$$\begin{aligned} \mathcal{L} = & W(\mathbb{P}_{\mathbf{X}_A}, \mathbb{P}_{\mathbf{X}_B}, \mathbb{P}_\theta) \\ & \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_A}} [\mathbf{D}_A(x)] - \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_B}} [\mathbf{D}_A(x)] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_A(\mathbf{G}_A(z))] - \mathbb{E}_{z \sim p(z)} [\mathbf{D}_A(\mathbf{G}_B(z))] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_A(\mathbf{G}_F(z))] - \\ & \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_A}} [\mathbf{D}_B(x)] + \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_B}} [\mathbf{D}_B(x)] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_B(\mathbf{G}_A(z))] - \mathbb{E}_{z \sim p(z)} [\mathbf{D}_B(\mathbf{G}_B(z))] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_B(\mathbf{G}_F(z))] + \\ & \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_A}} [\mathbf{D}_F(x)] + \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_B}} [\mathbf{D}_F(x)] + \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_A(z))] + \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_B(z))] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_F(z))], \end{aligned} \quad (1)$$

where the first five terms are for $\mathbf{D}_A$, the second five terms are for $\mathbf{D}_B$, and the last five terms are for $\mathbf{D}_F$. This loss is used to update all the generators and discriminators. The following two subsections, III-C and III-D, will present the update rules using Eq. 1.

C. $\mathcal{D}_F$ Update of FusionGAN

Removing the terms that are unrelated to $\mathcal{D}_F$ in Eq. 1 (zero derivative w.r.t. $\mathbf{D}_F$ and $\mathbf{G}_F$), we have the objective function for updating $\mathbf{D}_F$ and $\mathbf{G}_F$:

$$\begin{aligned} \mathcal{L}_F = & \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_A}} [\mathbf{D}_F(x)] + \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_B}} [\mathbf{D}_F(x)] + \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_A(z))] + \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_B(z))] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_F(z))] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_A(\mathbf{G}_F(z))] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_B(\mathbf{G}_F(z))]. \end{aligned} \quad (2)$$

First, $\mathcal{L}_F$ is calculated w.r.t. $\mathbf{G}_F$ so as to update $\mathbf{G}_F$. Considering differentiating $\nabla_\theta \mathcal{L}_F$, the optimal $\mathbf{G}_F$ is approximated as its convergence. The proof is provided below to show that this derivative is principled under the optimality assumption.

Theorem III.1 (FusionGAN Optimality Theorem). Let $\mathbb{P}_{\mathbf{X}_A}$ and $\mathbb{P}_{\mathbf{X}_B}$ be any distribution, Let $\mathbb{P}_\theta$ be the distribution of $\mathbf{G}_{F\theta}(Z)$ with Z , a random variable with density p and $\mathbf{G}_{F\theta}$ a function satisfying $\mathbb{E}_{z \sim p} [L(\theta, z)] < +\infty$, where L is Lipschitz constants. All functions are well-defined. Then there is a solution $\mathbf{D}_F : \mathcal{X} \rightarrow \mathbb{R}$ to the problem:

$$W(\mathbb{P}_{\mathbf{X}_A}, \mathbb{P}_{\mathbf{X}_B}, \mathbb{P}_\theta) = \max_{\|\mathbf{D}_F\|_L \leq 1} \mathcal{L}_F. \quad (3)$$

and we have:

$$\begin{aligned} \nabla_\theta \mathcal{L}_{\mathbf{G}_F} = & \nabla_\theta W(\mathbb{P}_{\mathbf{X}_A}, \mathbb{P}_{\mathbf{X}_B}, \mathbb{P}_\theta) \\ = & -\mathbb{E}_{z \sim p(z)} \nabla_\theta \sum_{i \in \{A, B, F\}} \mathbf{D}_i(\mathbf{G}_F(z)) \end{aligned} \quad (4)$$

Proof of Theorem III.1: Let us define $\mathcal{L}_F = V(\tilde{\mathbf{D}}_F, \theta)$ where $\tilde{\mathbf{D}}_F$ lies in $\mathcal{F} = \{\tilde{\mathbf{D}}_F : \mathcal{X} \rightarrow \mathbb{R}, \tilde{\mathbf{D}}_F \in C_b(\mathcal{X}), \|\tilde{\mathbf{D}}_F\|_L \leq 1\}$ and $\theta \in \mathbb{R}_d$. By the Kantorovich-Rubinstein duality, there is an $f \in \mathcal{F}$ that satisfies:

$$W(\mathbb{P}_{\mathbf{X}_A}, \mathbb{P}_{\mathbf{X}_B}, \mathbb{P}_\theta) = \sup_{\tilde{\mathbf{D}}_F \in \mathcal{F}} V(\tilde{\mathbf{D}}_F, \theta) = V(\mathbf{D}_F, \theta).$$

Let us define $X^*(\theta) = \{\mathbf{D}_F \in \mathcal{F} : V(\mathbf{D}_F, \theta) = W(\mathbb{P}_{\mathbf{X}_A}, \mathbb{P}_{\mathbf{X}_B}, \mathbb{P}_\theta)\}$, which shows that $X^*(\theta)$ is non-empty. According to Theorem 1 in [1]: (1) if $\mathbf{G}_F$ is continuous in θ , so is $\tilde{W}(\mathbb{P}_{\mathbf{X}_A}, \mathbb{P}_{\mathbf{X}_B}, \mathbb{P}_\theta)$, (2) If $\mathbf{G}_F$ is locally Lipschitz and satisfies $\mathbb{E}_{z \sim p} [L(\theta, z)] < +\infty$, then $W(\mathbb{P}_{\mathbf{X}_A}, \mathbb{P}_{\mathbf{X}_B}, \mathbb{P}_\theta)$ is continuous and differentiable for any $\mathbf{D}_F \in X^*(\theta)$ when both terms are well-defined. Let $\mathbf{D}_F \in X^*(\theta)$, which we knows exists since $X^*(\theta)$ is non-empty for all θ . Then:

$$\begin{aligned} & \nabla_\theta W(\mathbb{P}_{\mathbf{X}_A}, \mathbb{P}_{\mathbf{X}_B}, \mathbb{P}_\theta) \\ &= \nabla_\theta V(\mathbf{D}_F, \theta) = \nabla_\theta \mathcal{L}_F \\ &= -\nabla_\theta \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_F(z)) + \mathbf{D}_A(\mathbf{G}_F(z)) + \mathbf{D}_B(\mathbf{G}_F(z))], \end{aligned} \quad (5)$$

since $\mathbf{D}_F$ is 1-Lipschitz. Furthermore, $\mathbf{G}_{F\theta}(z)$ is locally Lipschitz as a function of (θ, z) . Therefore, $\mathbf{G}_{F\theta}(z)$ is locally Lipschitz on (θ, z) with constants $L(\theta, z)$. By Radamachers Theorem, $\mathbf{D}_F(\mathbf{G}_{F\theta}(z))$ has to be differentiable almost everywhere for (θ, z) jointly. Rewriting this, the set $A = \{(\theta, z) : \mathbf{D}_F \circ \mathbf{G}_F \text{ is not differentiable}\}$ has measure 0. By Fubini's Theorem, this implies that for almost every θ the section $A_\theta = \{z : (\theta, z) \in A\}$ has measure 0. Let's fix a θ_0 such that the measure of A_{θ_0} is null. For this θ_0 we have $\nabla_\theta \mathbf{D}_F(\mathbf{G}_{F\theta}(z))|_{\theta_0}$ is well-defined for almost any z , and since $p(z)$ has a density, it is defined $p(z)$ almost everywhere. Given the condition $\mathbb{E}_{z \sim p} [L(\theta, z)] < +\infty$, we have

$$\mathbb{E}_{z \sim p(z)} [\|\nabla_\theta \mathbf{D}_F(\mathbf{G}_{F\theta}(z))\|] \leq \mathbb{E}_{z \sim p(z)} [L(\theta_0, z)] < +\infty, \quad (6)$$

so $\mathbb{E}_{z \sim p(z)} [\nabla_\theta f(g_\theta(z))]$ is well-defined for almost every θ_0 . To keep the notation simple, we leave it implicit in the following equation that the $\mathbb{E}$ subjects to $z \sim p(z)$:

$$\begin{aligned} & \frac{1}{\|\theta - \theta_0\|} \sum_i \left[\mathbb{E} [\mathbf{D}_i(\mathbf{G}_{F\theta}(z))] - \mathbb{E} [\mathbf{D}_i(\mathbf{G}_{F\theta_0}(z))] - \right. \\ & \quad \left. \langle (\theta - \theta_0), \mathbb{E} [\nabla_\theta \mathbf{D}_i(\mathbf{G}_{F\theta}(z))] |_{\theta_0} \rangle \right] \\ &= \frac{1}{\|\theta - \theta_0\|} \sum_i \mathbb{E} \left[\mathbf{D}_i(\mathbf{G}_{F\theta}(z)) - \mathbf{D}_i(\mathbf{G}_{F\theta_0}(z)) - \right. \\ & \quad \left. \langle (\theta - \theta_0), \nabla_\theta \mathbf{D}_i(\mathbf{G}_{F\theta}(z)) |_{\theta_0} \rangle \right] \\ &\leq \frac{1}{\|\theta - \theta_0\|} \sum_i \|\theta - \theta_0\| L(\theta_0, z) + \|\theta - \theta_0\| \cdot \|\nabla_\theta \mathbf{D}_i(\mathbf{G}_{F\theta}(z))\| \\ &\leq \sum_i 2L(\theta_0, z) = 6L(\theta_0, z), \end{aligned} \quad (7)$$

where $i \in \{A, B, F\}$. By differentiability, the term inside the integral converges $p(z)$ to 0 as $\theta \rightarrow \theta_0$. Furthermore, and since $\mathbb{E}_{z \sim p(z)} [6L(\theta_0, z)] < +\infty$, we get by dominated convergence that Eq. 5 converges to 0 as $\theta \rightarrow \theta_0$. So the result of Eq. 5 equals to:

$$-\mathbb{E}_{z \sim p(z)} [\nabla_\theta \mathbf{D}_F(\mathbf{G}_F(z)) + \nabla_\theta \mathbf{D}_A(\mathbf{G}_F(z)) + \nabla_\theta \mathbf{D}_B(\mathbf{G}_F(z))].$$

Therefore, the update rules for the parameters θ of $\mathbf{G}_F$ is Eq. 4. $\blacksquare$

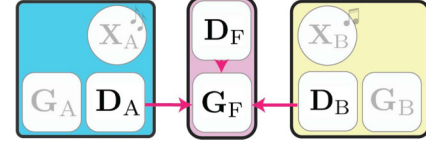


Figure 2: FusionGAN: update $\mathbf{G}_F$

The physical meaning is that all three discriminators can offer feedback to generator $\mathbf{G}_F$ as shown in Fig. 2. The left cyan rectangle indicates $\mathcal{D}_A$, the middle pink one is $\mathcal{D}_F$, and the right yellow one $\mathcal{D}_B$. Similarly, the gradient w.r.t. the parameters ϕ of $\mathbf{D}_F$ is:

$$\begin{aligned} \nabla_\phi \mathcal{L}_F = & \nabla_\phi [\mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_A}} [\mathbf{D}_F(x)] + \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_B}} [\mathbf{D}_F(x)] + \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_A(z))] + \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_B(z))] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_F(z))]]. \end{aligned} \quad (8)$$

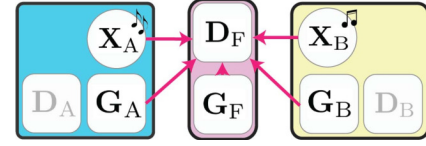


Figure 3: FusionGAN: update $\mathbf{D}_F$

As shown in Fig. 3, the update for $\mathbf{D}_F$ actually implicitly balances itself between $\mathcal{D}_A$ and $\mathcal{D}_B$. However, this update does not cover the degree of blending which may lead to an unbalanced mixture. Specifically, a good learning process for the discriminator $\mathbf{D}_F$ should consider two factors: (1) minimizing the distance with $\mathcal{D}_A$, $\mathcal{D}_B$ and $\mathcal{D}_F$ simultaneously and (2) maintaining a equal distance from $\mathcal{D}_A$ and $\mathcal{D}_B$ so as to satisfy both domains. To satisfy (2) and push the discriminator $\mathbf{D}_F$ to have a well-proportioned blending, an additional constraint is employed to further balance the ratio of existing domains, i.e., $\mathcal{D}_A, \mathcal{D}_B$:

$$\begin{aligned} \mathcal{L}_{F-bal} = & \|\mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_A(z))] - \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_B(z))]\| + \\ & \|\mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_A}} [\mathbf{D}_F(x)] - \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_B}} [\mathbf{D}_F(x)]\|. \end{aligned} \quad (9)$$

D. $\mathcal{D}_A$ and $\mathcal{D}_B$ Update of FusionGAN

After updating $\mathcal{D}_F$, FusionGAN will improve $\mathcal{D}_A$ and $\mathcal{D}_B$. Since $\mathcal{D}_A$ and $\mathcal{D}_B$ are symmetric in the framework, we only discuss $\mathcal{D}_A$ as shown in Fig. 9. Keeping related terms in Eq. 1, we get the loss function for $\mathcal{D}_A$:

$$\begin{aligned} \mathcal{L}_A = & \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_A}} [\mathbf{D}_A(x)] - \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_B}} [\mathbf{D}_A(x)] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_A(\mathbf{G}_A(z))] - \mathbb{E}_{z \sim p(z)} [\mathbf{D}_A(\mathbf{G}_B(z))] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_A(\mathbf{G}_F(z))] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_B(\mathbf{G}_A(z))] + \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_F(\mathbf{G}_A(z))]. \end{aligned} \quad (10)$$

Taking the derivative of $\mathcal{L}_A$ w.r.t. $\mathbf{D}_A$, we have

$$\begin{aligned} \nabla_{\mathbf{D}_A} \mathcal{L}_A = & \nabla_{\mathbf{D}_A} [\mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_A}} [\mathbf{D}_A(x)] - \mathbb{E}_{x \sim \mathbb{P}_{\mathbf{X}_B}} [\mathbf{D}_A(x)] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_A(\mathbf{G}_A(z))] - \mathbb{E}_{z \sim p(z)} [\mathbf{D}_A(\mathbf{G}_B(z))] - \\ & \mathbb{E}_{z \sim p(z)} [\mathbf{D}_A(\mathbf{G}_F(z))]]. \end{aligned} \quad (11)$$

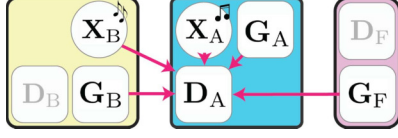


Figure 4: FusionGAN: update D_A

There are four negative terms and one positive term in Eq. 11 w.r.t. D_A . The training implicitly increases the positive terms and decreases the value of the negative terms. However, the inequalities in the two groups are not controlled. To further control the distance among the three domains, we propose an inequality constraint for D_A :

$$D_A(X_A) \geq D_A(G_F(z)) \geq D_A(X_B) \quad (12)$$

Then combining Eq. 12 into loss function Eq. 11, it can be rewritten as:

$$\mathcal{L}_{A-bal} = \|D_A(X_A) - D_A(G_F(z))\| + \|D_A(G_F(z)) - D_A(X_B)\|. \quad (13)$$

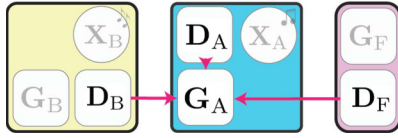


Figure 5: FusionGAN: update G_A

Some inequalities that have been implicitly included in GAN training are not covered in Eq. 12, such as $D_A(X_A) \geq D_A(G_A)$. Similar to the G_F update in Fig. 5, the derivative of $\mathcal{L}_A$ w.r.t. G_A is:

$$\nabla_{G_A} \mathcal{L}_A = \mathbb{E}_{z \sim p(z)} \nabla_{G_A} [-D_A(G_A(z)) - D_B(G_A(z)) + D_F(G_A(z))]. \quad (14)$$

E. Algorithm Description

The Algorithm 1¹ initializes the parameters of the discriminators and generators (line 1). TextCNN [15] and LSTM are employed to build the discriminators and generators respectively. Given X_A , the generator is updated by maximum likelihood estimation (line 3). From lines 4 to 5, D_A receives the sequences generated by G_A as negative data, and real data as positive samples. Between lines 6 and 16, a GAN procedure is applied to optimize D_A and G_A . D_A returns regression scores that are treated as rewards for the policy gradient of G_A . Similarly, D_A improves itself by the real world data X_A and the data generated by G_A . Our method proceeds to update D_B and G_B by using the same process from lines 3 to 16. D_F repeats the same pre-training procedure from 3 to 16 with $D_F, G_F, X_F = X_A + X_B$. From line 21 to 28, the framework iteratively updates the generators and discriminators from D_F, D_A, D_B . Lines 21 to 24 are for D_F , while lines 25 to 28 are for D_A and D_B . The update rules for the parameters of D_F and G_F have been covered in subsection (III-C), while those for the parameters of G_A and D_A are described in subsection

¹https://github.com/aquastar/fusion_gan

Algorithm 1: FusionGAN

Input: Input data X_A, X_B from $\mathcal{D}_A, \mathcal{D}_B$
Output: a generator and a discriminator: G_F and D_F

- 1 Randomly initialize $G_A, D_A, G_B, D_B, G_F, D_F$, generators/discriminators employ LSTM/TextCNN;
- 2 // Pre-training
- 3 Train G_A using maximum likelihood estimation on real data X_A ;
- 4 Generate negative samples N_A using G_A ;
- 5 Train binary classifier D_A with N_A and X_A ;
- 6 **repeat**
- 7 **for** G_A training **do**
- 8 Sample a sequence $S_{1:T} = (s_1, \dots, s_T) \sim G_A$;
- 9 Derive Q value as rewards R from D_A ;
- 10 Update parameters θ of G_A by policy gradient: $\theta \leftarrow \theta + \alpha \nabla_{\theta} R$;
- 11 **end**
- 12 **for** D_A training **do**
- 13 Generate negative examples $\sim G_A$;
- 14 Train D_A with negative samples and X_A ;
- 15 **end**
- 16 **until** D_A converges;
- 17 Repeat line 3 to 16 with $\mathcal{D}_B$, i.e., D_B, G_B, X_B ;
- 18 Repeat line 3 to 5 with $\mathcal{D}_F$, i.e., $D_F, G_F, X_F = X_A + X_B$;
- 19 // FusionGAN training
- 20 **repeat**
- 21 **for** D_F training **do**
- 22 Update parameters of D_F using the sum of Eq. 8 and Eq. 9 : $\theta_{D_F} \leftarrow \theta_{D_F} + \alpha \nabla_{D_F} (\mathcal{L}_F + \mathcal{L}_{F-bal})$;
- 23 Update parameters of G_F using Eq. 4 : $\theta_{G_F} \leftarrow \theta_{G_F} + \alpha \nabla_{G_F} \mathcal{L}_F$;
- 24 **end**
- 25 **for** D_A training **do**
- 26 Update parameters of D_A using the sum of Eq. 11 and Eq. 13 : $\theta_{D_A} \leftarrow \theta_{D_A} + \alpha \nabla_{D_A} (\mathcal{L}_A + \mathcal{L}_{A-bal})$;
- 27 Update parameters of G_A using Eq. 14: $\theta_{G_A} \leftarrow \theta_{G_A} + \alpha \nabla_{G_A} \mathcal{L}_A$;
- 28 **end**
- 29 Repeat line 25 to 28 with $\mathcal{D}_B$, i.e., D_B, G_B, X_B ;
- 30 **until** D_A, D_B, D_F converges;

(III-D). Again, D_B follows the same process as that of D_A . The algorithm stops when the discriminators converge.

IV. EVALUATION

Training data includes two datasets called the Weimar jazz dataset² and the Essen Associative Code (EsAC) folk dataset³. Both of these datasets contain symbolic data of single-voiced tracks, which support our objective of generating solo melodies. The baseline methods include:

- 1) **Random Mixing (RM):** RM randomly selects notes and duration from the pitch and duration distribution

²<http://jazzomat.hfm-weimar.de>

³<http://www.esac-data.org>

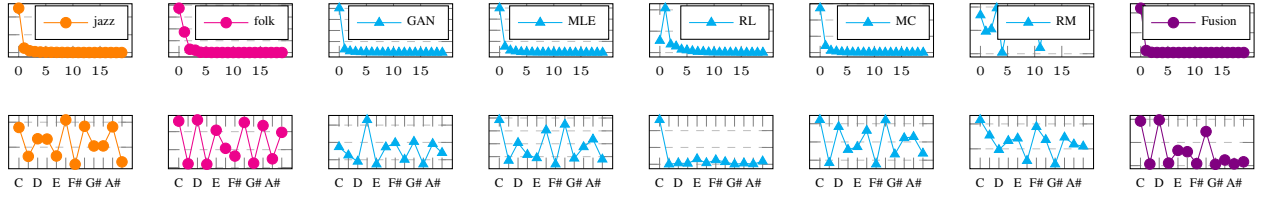


Figure 6: upper line: DD (x-axis: duration length; y-axis: percentage); lower line: NPD (x-axis: pitch class; y-axis: percentage)

- on the combination of $\mathbf{X}_A$ and $\mathbf{X}_B$.
- 2) **Monte Carlo Sampling (MC):** After calculating the note pitch and duration distributions on the combination of $\mathbf{X}_A$ and $\mathbf{X}_B$, **MC** samples notes and duration subject to these distributions.
 - 3) **Maximum Likelihood Estimation (MLE):** Employing LSTM, **MLE** directly learns a pattern from the mixture of $\mathbf{X}_A$ and $\mathbf{X}_B$ by predicting the next element.
 - 4) **GAN:** The model directly learns a pattern from the mixture of $\mathbf{X}_A$ and $\mathbf{X}_B$ using GAN.
 - 5) **Reinforcement Learning (RL):** After applying GAN on $\mathbf{D}_A - \mathbf{G}_A, \mathbf{D}_B - \mathbf{G}_B$, we exchange the discriminators $\mathbf{D}_A$ and $\mathbf{D}_B$ which are treated as rewards function to $\mathbf{G}_B$ and $\mathbf{G}_A$ respectively.

A. Quantitative Study

Due to the aesthetic nature of the task, a quantitative evaluation can only give a first impression of the power of the generative system. Here, we investigate the distance between pitch and note duration distributions, knowingly discarding all sequential information of the melodies.

The ideal output should keep equally close to the given domains and minimize this distance. Two distributions of properties are selected for the study.

- 1) **Duration Distribution (DD):** Notes are categorized by their durations.
- 2) **Normalized Pitch Distribution (NPD):** All tokens are categorized into C/C#/D/D#/E/F/F#/G/G#/A/A#/B.

To quantify the similarity between distributions, two symmetric metrics, the Euclidean distance (**EUD**) and the Wasserstein-1 metric (Earth Mover distance (**EM**)), are employed to calculate the distance between the distributions of generated sequences and those of domains $\mathbf{X}_A$ and $\mathbf{X}_B$. Intuitively, the distance between $\mathbf{X}_A$ and $\mathbf{X}_B$, between $\mathbf{X}_A, \mathcal{D}_F$, and between $\mathbf{X}_B, \mathcal{D}_F$ should satisfy the triangle inequality, and the optimal configuration should minimize the inequality, i.e.,

$$\min \text{Diff} = \|\mathbf{X}_A - \mathbf{G}_F(z)\| + \|\mathbf{X}_B - \mathbf{G}_F(z)\| - \|\mathbf{X}_A - \mathbf{X}_B\|,$$

$$\text{Ratio} = \frac{\|\mathbf{X}_A - \mathbf{G}_F(z)\| + \|\mathbf{X}_B - \mathbf{G}_F(z)\|}{\|\mathbf{X}_A - \mathbf{X}_B\|}.$$

Figure 6 (top) shows the note duration distributions (DD) and Table I (top) gives the corresponding distance measures. The note duration distribution of jazz and folk have shapes that resemble the power law distribution. The shape of **RM** is very different from jazz or folk. Visually, **RL**'s duration distribution does not match that of jazz and folk because the portion of notes at the length of 0 and 1 (the peak) were relatively higher than that of jazz or folk. The other baselines

Table I: Distance between Distributions (the lower the better)

	EUD		EM	
	Diff	Ratio	Diff	Ratio
DD				
RM	39742.2	1.375	2757.6	1.461
MLE	24546.4	1.231	2005.2	1.335
GAN	24765.2	1.233	2064.3	1.345
RL	37971.2	1.358	2629.0	1.439
MC	13988.7	1.132	1289.2	1.215
Fusion	19452.6	1.183	1831.9	1.306
NPD				
RM	19586.6	1.647	5231.0	1.643
MLE	15921.4	1.526	4147.5	1.510
GAN	16807.1	1.555	4098.0	1.504
RL	16927.1	1.559	4399.2	1.541
MC	11175.0	1.369	2660.2	1.327
Fusion	11564.6	1.382	3182.5	1.391

(**GAN**, **MLE**, **MC**) appear similar to the given domains because they conform to the power law distributions. **Fusion** is roughly at the same level of **MC**. Note that, given our evaluation metrics, **MC** has the best performance in this experiment since it directly samples notes from the true distribution. This disregard of sequential information, however, results in unnatural rhythmic qualities of the music clips generated by **MC**.

Figure 6 (bottom) shows the pitch distributions (NPD) and Table I (bottom) gives the corresponding distance measures. **RL**'s pitch distribution is —due to overfitting— different from that of jazz and folk, which shows its low capacity in modeling pitch distribution. The single peak at D# as generated by **GAN** does not match either the jazz or folk distributions. The distributions of the other baselines are difficult to categorize as a good or bad match. Since **MC** is directly sampled from real distributions, its configurations should be exactly the same as that of the optimal fusion. **MC** shows two valleys at C# and F#, and two peaks at C and G, which should be expected. **RM** satisfies one of them (F# is a valley), while **RM** meets only one standard (C is the peak). **GAN** fails in all the characteristic pitch classes, and both **MLE** and **Fusion** meet all of them. Quantitatively, the ratio of **RM** in NPD is the highest which means that **RM** is the worst in modeling pitch. **GAN** and **RL** improve **RM** by around 10%. As one of the two distributions that look most similar to the original distributions, **MLE** outperforms **RM**, **RL**, and **GAN**. Our method **Fusion** has the second lowest ratio after **MC**. However, as mentioned above, the **MC** does not consider the sequential consistence, resulting in bad evaluations in the listening test described in Sect. IV-B.

B. User Study via Listening Test

To evaluate the subjective quality of the generated sequences, a user study was conducted via Amazon Mechanical Turk. The listening test was divided into three steps:

- 1) **Qualification test:** “Listen to the music, and choose one genre below that most matches the music”. This is to test the evaluators’ qualification level. The provided choices are (A) *jazz*, (B) *folk*, (C) *neither*.
- 2) **Fusion recognition:** “Listen to the attached music, and then choose the choice that most matches the music” Possible answers are (A) *pure jazz*, (B) *pure folk*, (C) *mixture of jazz and folk*, (D) *neither*. Each of the pieces of music generated by our proposed method and the baselines are assigned to one such question.
- 3) **Musicality:** “Listen to the attached music in 2nd step, who do you think its composer”, the allowed answers are (A) *expert*, (B) *newbie*, (C) *robot*. Similarly, each generated sample is associated with one such question.

We randomly drew from the training data and prepared 800 sets for the first question. In the first step, 66.5% of candidates chose the correct answer, which is an acceptable rate given the inherent difficulties of genre identification for average listeners. The statistics resulting from the valid answers are shown in Table II. Each of the methods generated 500 samples, which were randomly selected for the second and third questions.

As shown in Table II, the best system based on the percentage of *mixture* is **MLE**. A closer look at **MLE**, however, reveals the unbalanced distribution between *jazz* and *folk*, which implies its bias towards the jazz genre. On the other hand, **RM** is the best system based on the balance between *jazz* and *folk*, but the high percentage of *neither* suggests the confusion of the listeners. It is clear that a single criterion is insufficient for determining the best system. Therefore, we design a metric to summarize the results and represent the fusion level (**FL**) of the evaluated systems with the following equation:

$$\text{maximize } \mathbf{FL} = 1 - \frac{\|C_{jazz} - C_{folk}\| + C_{neither}}{C_{jazz} + C_{folk} + C_{mixture} + C_{neither}},$$

where C_i indicates the count of i . $\|C_{jazz} - C_{folk}\|$ means the unbalanced error, and $C_{neither}$ can be treated as another type of error. A higher **FL** value implies better fusion. In the *Musicality* test, **MLE** (45.5%), **GAN** (42.0%), and **Fusion** (43.8%) were voted by the majority as *expert*. Overall, **Fusion** shows a balanced performance in both *Fusion recognition* and *Musicality* tests, which demonstrates the effectiveness of our proposed method.

V. CONCLUSION

In this paper, we proposed a three-way GAN-based learning framework to integrate multiple domains. A Wasserstein distance based metric is introduced to indicate the blending progress. The evaluation investigated objective metrics looking at the pitch and note duration distributions of the generated data. A user study explicitly illustrates the validity of the proposed method as compared to the baselines, as the melodies generated by our model were preferred by the majority of users. While the listening test results are encouraging, future work will benefit from removing key dependence of the training data, a careful look

Table II: Listening test results

Fusion recognition					
	<i>jazz</i>	<i>folk</i>	<i>mixture</i>	<i>neither</i>	FL
RM	25.0%	22.5%	12.5%	40%	57.5%
MLE	43.6%	9.1%	30.9%	16.4%	49.1%
GAN	34.0%	17.0%	26.0%	14%	69%
RL	20.1%	28.3%	20.8%	30.8%	61%
MC	32.0%	2.0%	14.0%	52%	16%
Fusion	35.9%	25.0%	20.0%	19.1%	70%

Musicality			
	<i>expert</i>	<i>newbie</i>	<i>robot</i>
RM	22.5%	50.0%	27.5%
MLE	45.5%	21.8%	32.7%
GAN	42.0%	32.0%	26.0%
RL	32.1%	37.7%	30.2%
MC	30.0%	36.0%	34.0%
Fusion	43.8%	28.1%	28.1%

into perceptually meaningful evaluation metrics that take into account the sequential nature of music, and a careful listening test design, including human-composed melodies for comparison.

REFERENCES

- [1] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein gan,” *arXiv:1701.07875*, 2017.
- [2] S. Colton, R. L. de Mántaras, and O. Stock, “Computational creativity: Coming of age,” *AI Magazine*, vol. 30, no. 3, p. 11, 2009.
- [3] J. Engel, C. Resnick, A. Roberts, S. Dieleman, D. Eck, K. Simonyan, and M. Norouzi, “Neural audio synthesis of musical notes with wavenet autoencoders,” *arXiv:1704.01279*, 2017.
- [4] L. A. Gatys, A. S. Ecker, and M. Bethge, “A Neural Algorithm of Artistic Style,” *arXiv preprint*, pp. 3–7, 2015.
- [5] J. Gerald David, *Encyclopedia of African American Society*. London: SAGE Publications, Inc., 2005.
- [6] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *NIPS*, 2014, pp. 2672–2680.
- [7] G. E. Hinton, S. Osindero, and Y.-W. Teh, “A fast learning algorithm for deep belief nets,” *Neural computation*, vol. 18, no. 7, pp. 1527–1554, 2006.
- [8] T. Kim, M. Cha, H. Kim, J. Lee, and J. Kim, “Learning to discover cross-domain relations with generative adversarial networks,” *arXiv:1703.05192*, 2017.
- [9] M.-Y. Liu and O. Tuzel, “Coupled generative adversarial networks,” in *NIPS*, 2016, pp. 469–477.
- [10] M. Mirza and S. Osindero, “Conditional Generative Adversarial Nets,” *arXiv preprint arXiv:1411.1784*, 2014.
- [11] A. v. d. Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, “Wavenet: A generative model for raw audio,” *arXiv:1609.03499*, 2016.
- [12] M. Ruder, A. Dosovitskiy, and T. Brox, “Artistic style transfer for videos,” in *German Conference on Pattern Recognition*. Springer, 2016, pp. 26–36.
- [13] C. Villani, *Optimal transport: old and new*. Springer Science & Business Media, 2008, vol. 338.
- [14] Z. Yi, H. Zhang, P. Tan, and M. Gong, “Dualgan: Unsupervised dual learning for image-to-image translation,” *arXiv preprint arXiv:1704.02510*, 2017.
- [15] X. Zhang and Y. LeCun, “Text understanding from scratch,” *arXiv preprint arXiv:1502.01710*, 2015.
- [16] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired image-to-image translation using cycle-consistent adversarial networks,” *arXiv:1703.10593*, 2017.